MultiCharts .NET Engines — User Manual (v4)

For _CustomSignal strategies on dxFeed @ES (Portfolio Trader, IBKR execution)

Source: origin/main @ a07257a

MultiCharts .NET Engines — User Manual (v4)

Source: `origin/main` at commit `a07257a` (a07257a8eada45a4ab2413b67f8bdb693e5cac94, merge of PR #49, 2026-10-30 09:30 EDT). Member names, signatures and file:line references in §4, §5 and Appendix E were extracted from that tree (Appendix E regenerated by script); all 42 examples, the 30 §5 snippets and the 14 §4 snippets type-check against it with 0 errors (see `engines-manual-v4-verification.md`). Earlier v4 drafts were based on `628542f` and then `b2685dd`; since 628542f only `_CustomSignal`, `_DvpEngine`, their tests and the new `_InstrProbe`/`_InstrProbeCore` files changed. Anything I could not confirm from the code is marked **VERIFY**. No section of this manual reports or implies trading results.

What is new in v4 (full list in Appendix H). §3 now has **42 worked examples** (the 10 from v3.2 plus 32 new ones), ordered easy → hard, each a complete strategy file. A new **§5 Feature reference** documents every row of the trading-idea catalogue's §2.4 base-feature map (31 rows): status on main, signatures, snippets placed by lifecycle method, which examples use the feature, and pitfalls. Appendix E is regenerated from `a07257a` and adds E2, a feature × example coverage matrix.

Contents. §1 Quick start · §2 Gate 0 · §3 Forty-two worked examples (easy → hard) · §4 Engines and helpers · §5 Feature reference (catalogue §2.4) · Appendices A–D (trace levels, files, tests, glossary) · E (signatures; E2 coverage matrix) · F (VERIFY items and discrepancies) · G (EV-4.6b event coverage) · H (changelog).

---

1 — Quick start

1.1 Architecture

```
                        MultiCharts .NET (Chart / Portfolio Trader)
                                       │  CalcBar() per tick / bar close
                                       ▼
┌──────────────────────────────────────────────────────────────────────────┐
│ PortfolioSignalObject                                                    │
│ └─ _CustomSignal  (TraceLevel [Input], journal, sizing, time helpers, M-PORT) │
│      │  StartCalc → startEngines()  : create ONLY the engines you enabled      │
│      │  CalcBar   → stepEngines()   : step them on CLOSED bars                 │
│      │                                                                         │
│      │   RollDetector → VpHelpers(rings/FVG) → PriceAction → ExtraPA → Tick →  │
│      │   Xmkt → Daily → Regime → Event → Tod → Avwap                           │
│      │   then LevelStudy → portfolioBar (pm.* keys) → journalScan → CS-8 events│
│      │                                                                         │
│      ├─ YourStrategy : _CustomSignal        (e.g. _Test6, your signals)        │
│      └─ _VolumeProfileBase : _CustomSignal  (VP, VWAP slots, confluence, P3)   │
│           └─ _VolumeProfileFuturesBase       (CME session clock, IB/RTH stats,  │
│                                               order flow, CME holiday hooks)    │
└──────────────────────────────────────────────────────────────────────────┘
   Engines (internal sealed classes, MC-free, unit-tested in tests/*):
   _DailyContext  _RegimeEngine(+_Gmm,_Rng)  _EventCalendar  _AvwapManager
   _PriceActionEngine  _TodNormaliser  _XmktLeg  _TickStream  _FvgDetector
   _BarRing  _LevelStudy  _MtfMath  _CmeSessionClock  _CmeHolidays/_NyseHolidays
   Helpers: RegimeGate/A263Core/AtrTercileTracker (_RegimeGate), _PortMM (+ _PortfolioRiskManager)
```

Some rules hold for every engine:

1. **Lazy.** An engine is created in `StartCalc` only if you enabled it through its **protected config** (never an `[Input]`). If you didn't, its accessor returns `null` (or a neutral state) and logs a one-time Warning.
2. **Closed bars only.** Engines commit on `Bars.Status == EBarState.Close`. An intrabar tick never changes their state.
3. **As-of discipline.** Daily values describe completed prior sessions. `RegimeState.AsOf` is the day whose close produced the value, and that value is usable from the **next** session. AVWAP anchors have a `UsableFrom` time. HTF reads use `HtfClosedIndex`/`htfMapped`.
4. **Journaling.** Engines write facts through `engineSink(ETraceLevel.Journal)`, which is `null` when filtered out, so they format no strings. The base never blocks orders on your behalf. For example, `EventBlackout` is only exposed.
5. **Same assembly.** The engine types and accessors are `internal`, which works because MC compiles every study into one assembly. Use them as locals (`_DailyContext d = DailyContext;`). Don't expose them in public or protected signatures of a public class.

1.2 Minimal `_CustomSignal` skeleton

```
using System;
using System.Collections.Generic;
using PowerLanguage.Function;

namespace PowerLanguage.Strategy
{
    public class _MySignal : _CustomSignal
    {
        private const string EntryName = "MyLE", ExitName = "MyLX";
        private IOrderPriced m_le;
        private IOrderMarket m_lx;

        public _MySignal(object _ctx) : base(_ctx) { }

        protected override void Create()
        {
            base.Create();                      // NYSE holiday function (TraceLevel default None is set in
```

```
the constructor, PR #49)
            m_le = OrderCreator.Stop(new SOrderParameters(Contracts.UserSpecified, EntryName, EOrderAction.Bu
y));
            m_lx = OrderCreator.MarketNextBar(new SOrderParameters(Contracts.Default, ExitName,
EOrderAction.Sell));
            // engine configs may be set here (pattern A) ...
        }

        protected override void StartCalc()
        {
            // ... or here, BEFORE base.StartCalc() (pattern B) — engines are created inside base.StartCalc()
            base.StartCalc();
        }

        protected override void CalcBar()
        {
            base.CalcBar();                      // steps all enabled engines first
            if (Bars.Status != EBarState.Close) return;
            // read engines (values as of this closed bar), decide, Send() orders for the next bar
        }
    }
}
```

`TraceLevel` is the only `[Input]` that `_CustomSignal` defines (`[Input] public virtual ETraceLevel TraceLevel`). Set it to **Journal** in Format Signals. The default `None` is set in the `_CustomSignal` constructor (PR #49), so `Create()` no longer overwrites a Format Signals value; a code default belongs in the derived class's constructor. **VERIFY:** that MC applies the saved dialog value after the constructor is MC behaviour I have not confirmed in this repo.

1.3 Enabling an engine lazily and setting its config

There are two real patterns in the repo:

```
// Pattern A — in Create() (as _EventCalendarTest does)
protected override void Create()
{
    base.Create();
    EventConfig.enabled = true;
    EventConfig.minTier = EEventTier.T3;
    RollDetection = ERollDetection.Series;
}

// Pattern B — in StartCalc() before base.StartCalc() (as _Test6 / _MtfProbe do)
protected override void StartCalc()
{
    RegimeGate.ApplyArm(RegimeGateConfig, RegimeConfig);   // must precede base.StartCalc
    NeedsDaily = true;
    DailyConfig.atrLength = 20;
    HtfStreams = new[] { 2 };
    base.StartCalc();
}
```

Engine	Switch (protected, not an Input)	Config object	Accessor
M-DAILY	`NeedsDaily = true`	`DailyConfig`	`DailyContext`, `DailyGapPct`
G-R Regime	`RegimeConfig.enabled = true` (forces Daily + HV{20,63})	`RegimeConfig`	`Regime` (RegimeState copy)
M-EVENT	`EventConfig.enabled = true`	`EventConfig`	`EventCalendar`, `EventBlackout`, `EventFlattenNow`
M-AVWAP	`AvwapEvents = EAvwapAnchor.…` (≠ None)	`AvwapLimits`	`AvwapManager`
PRICEACTION	`PriceActionConfig.enabled = true` / `ExtraPriceAction.Add(...)`	`PriceActionConfig`	`PriceActionEngine`, `PriceActionFor(ds, role)`
M-TOD	`TodMetrics = ETodMetric.…`		`TodNormaliser`

Engine	Switch (protected, not an Input)	Config object	Accessor
		`TodSessions`, `TodMinSessions`, `TodBucketMinutes`	
M-XMKT	`XmktLegs.Add(new XmktLegConfig{…})`	`XmktLegConfig`	`XmktLeg(role)`, `XmktLeaderSwing(role)`
M-TICK	`TickConfig.enabled = true`	`TickConfig`	`TickStats`
M-MTF	`HtfStreams = new[]{2}`	—	`HtfClosedIndex`, `HtfClosed`, `HtfClosedBar`, `htfMapped`, `HtfForming`
Rolls	`RollDetection = ERollDetection.Series`	`RollGapTicks`, `RollDates`	`rollOn(ds)`, `RollBars`, `IsRollWindow(day)`
FVG	`Fvgs.enabled = true`	`Fvgs` (FvgConfig)	`FvgDetector`, `fvgLevel(f, edge)`
Bar rings	`requireRing(ds, bars, consumer)` (after base.StartCalc)	`BarRings`	`Ring(ds)`, `ringBarOf(ds, ago)`
A2-33 level study	`LevelStudyConfig.enabled = true`	`LevelStudyConfig`	`levelStudyAdd(...)`
M-PORT	automatic in Portfolio Trader (`PortfolioPublish`)	virtual `Portfolio*` props	`portfolioArm`, `portfolioStop`

Dependencies are enabled for you. Regime forces `NeedsDaily`. AVWAP `Swing`/`Breakout` anchors force a PriceAction instance. AVWAP `VolumeClimax`, or a P2 placebo on Data1, forces TOD `Volume`. `XmktLegConfig.leaderSwing` creates a PA on the leg.

1.4 Reading outputs without lookahead

```
protected override void CalcBar()
{
    base.CalcBar();
    if (Bars.Status != EBarState.Close) return;            // 1. closed bars only

    _DailyContext d = DailyContext;                        // 2. null if NeedsDaily == false
    DateTime today = tradingDayKey(Bars.Time[0]);
    bool atrOk = d != null && d.Valid && d.PrevDate < today; // 3. t-1: completed PRIOR session
    double atr = atrOk ? d.Atr20d : double.NaN;

    RegimeState r = Regime;                                // 4. usable iff AsOf < today
    bool regimeOk = r.Valid && !r.UsingFallback && r.AsOf < today;

    int i = HtfClosedIndex(2);                             // 5. HTF: last CLOSED bar of Data2
    double htfClose = BarsOfData(2).Close[i];
}
```

Orders sent on bar t's close execute on bar t+1 (`MarketNextBar`, or a `Stop`/`Limit` working from the next bar). Every value above is known at t's close, so there is no lookahead. Never read `BarsOfData(n).Close[0]` of a forming HTF bar. Use `HtfForming(ds)` only when you mean it (it is journaled once).

1.5 Journaling

- Set `TraceLevel = Journal`. That enables Error/Warning/Info, the engines' Journal facts, the per-trade CSV `MCJournal_<runId>.csv`, and CS-8 signal events `MCEvents_<runId>.csv`.
- Guard expensive strings with `if (journalOn()) trace(ETraceLevel.Journal, $"...")`, or use `traceOn(level)`.
- Call `journalArm(entryName, plannedEntry, initialStop, target, context)` next to `Send()` so the trade row carries R and context.

- Call `logSignal(tag, dir, price, stop, context, taken)` to record taken and untaken signals. The base tracks forward MFE/MAE in R at `SignalHorizons` (default {5,15,30,60} bars).
- To find the files: each signal prints `{SignalClass} {Symbol}: trace log {path}` to its own Output window the first time it traces (PR #49). Lines are appended one at a time with no open handle, so you can copy or delete the CSVs while MultiCharts runs; while a CSV is locked (e.g. open in Excel) its new lines are dropped and the error is printed to Output (Appendix B).

```
if (journalOn())
{
    var ctx = new Dictionary<string, string> { { "atr", atr.ToString("0.00") }, { "regime", r.Label.ToString(
) } };
    journalArm(EntryName, stopPrice, initialStop, double.NaN, ctx);
    logSignal("ORB", +1, stopPrice, initialStop, ctx, true);
}
m_le.Send(stopPrice, qty);
```

1.6 Important for @ES on a plain `_CustomSignal`: the CME session adapter

On a plain `_CustomSignal`, `tradingDayKey()` defaults to the **calendar date** (`ETradingDayRoll.Calendar`). `sessionClock()` defaults to the bar's own time of day, and `isRthBar()` returns **true** for every bar. With a 24-hour Globex @ES chart, that means:

- the Sunday/evening 18:00 ET session is split at midnight;
- M-DAILY aggregates calendar days, not Globex days, and its RTH-close basis becomes "last bar of the day";
- `NoEntryBefore`/`NoEntryAfter`/`FlatBy` are compared in chart time.

`_VolumeProfileFuturesBase` fixes all of this by overriding these virtuals with a `_CmeSessionClock`. A plain `_CustomSignal` can do the same, because every member used here is a real protected virtual or public member:

```
private _CmeSessionClock m_clock;
private _CmeSessionClock Clock
{
    get { if (m_clock == null) m_clock = new _CmeSessionClock(resolveBarTimeZone(Bars.Info)); return
m_clock; }
}
protected override void StartCalc() { m_clock = null; /* engine configs … */ base.StartCalc(); }

protected override DateTime tradingDayKey(DateTime barTime) { return Clock.GlobexDate(barTime); }
protected override TimeSpan sessionClock(DateTime barTime) { return Clock.ToEastern(barTime).TimeOfDay; }
protected override bool isRthBar(DateTime barTime) { return
Clock.IsRthValueBarEt(Clock.ToEastern(barTime)); }
protected override int todMinuteOfSession(DateTime barTime)
{
    int m = (int)Clock.ToEastern(barTime).TimeOfDay.TotalMinutes - (int)Clock.GlobexOpen.TotalMinutes;
    return m < 0 ? m + 1440 : m;
}
protected override bool todHalfDay(DateTime d)
{
    return Clock.IsNoRthDay(d) || Clock.HolidayHaltEt(d) != null || Clock.RthEarlyCloseEt(d) != null;
}
```

These bodies mirror `_VolumeProfileFuturesBase` lines 356–512. Since PR #46 `_Test6` overrides `tradingDayKey` with the static Globex key, `_CmeSessionClock.GlobexTradingDayEt(ToZone(barTime, Et))` (an ET time at or after 18:00 belongs to the next date), so its M-DAILY days are Globex days. A plain `_CustomSignal` that does not override it still gets calendar days (Appendix F). Also, on a plain `_CustomSignal`, `PriceActionConfig` keeps the **Library** preset. Call `PriceActionConfig.ApplyPreset(EPriceActionPreset.Futures)` in `Create()` to match the futures bases (LeftInclusive ties, `volumeIncludesCurrent = false`).

1.7 Pitfalls checklist

- [ ] **Lookahead.**
- Read engines only after `base.CalcBar()` and on `EBarState.Close`.
- Daily values require `PrevDate < today`; Regime requires `AsOf < today`; AVWAP requires `UsableFrom <= Bars.Time[0]`; HTF requires `HtfClosedIndex`/`htfMapped`, never `[0]` of a forming bar.
- TOD norms use prior completed sessions only.
- [ ] **Rolls (continuous unadjusted @ES).**
- Set `RollDetection = ERollDetection.Series` (or `Table` + `RollDates`, as `_Test6` does since PR #46). It is **Off** by default, and then nothing is roll-adjusted.
- `Series` on an intraday chart whose `tradingDayKey` is the calendar date is blind to the 18:00 ET splice; the base traces a one-time Warning (`_RollDetector.SeriesKeyWarning`). Use a Globex key or `Table`. Even with a Globex key, `Series` currently over-detects rolls on intraday charts; use `Table` until PR #51 merges (App F, Known issues).
- `rollOn(1)` is true on the first bar of the new contract. Engines reset or adjust there: Daily `OnRoll` (history difference-adjusted; ATR **reseeded**, `Atr20d` NaN for `atrLength` sessions, unless `DailyConfig.atrRestartOnRoll = false`, PR #46), PA `roll reset`, AVWAP `invalidated (roll)`, and a Regime refit deferred on the roll day.
- Exclude roll ±1 via `IsRollWindow(day)`. This works only when rings/FVG or RollDetection are active.
- [ ] **Half-days and holidays.**
- Use `_CmeHolidays.Rules.Get(globexDate)`: `.Closed`, `.NoRth`, `.HaltEt`, `.RthCloseEt`.
- TOD excludes half-days only if `todHalfDay` is overridden (the futures base does this, a plain `_CustomSignal` returns false).
- The Event calendar flags `EventDay.isHalfDay`.
- [ ] **Trading day.** On a plain `_CustomSignal` with a Globex template, override `tradingDayKey`/`sessionClock`/`isRthBar` (see 1.6).
- [ ] **Multi-data streams.**
- Orders always go to Data1.
- `HtfStreams` must carry the Data1 root; other markets go through `XmktLegs`.
- A leg with another root never inherits Data1 rolls.
- `requireDataStream` throws at StartCalc if a stream is missing.
- A leg is `Available` only if not stale (`maxStaleSec`, or 2× the leg bar).
- [ ] **Portfolio Trader.**
- `PortfolioPublish` is true only in PT.
- `sizeForRisk` arms M-PORT for Data1. Otherwise call `portfolioArm(side, entry, stop, qty)` next to `Send`, and `portfolioStop(stop)` when you trail.
- Apply `_PortfolioRiskManager` as the PT money-management signal. It sizes via `EntryContracts` and never blocks exits.
- `equityAvailable` in PT uses `Portfolio.MaxRiskEquityPerPosPercent`. The *Percent parameters are fractions (0.01 = 1%).
- [ ] **Validation data.**
- Engines step on closed Data1 bars, so the bar size changes results (`_Test6` journals a bar-size deviation).
- Validate on 1-min with Bar Magnifier, or on tick (`TickConfig` uses a DataLoader 1-tick request; its size outputs stay parked until `sizeOutputsVerified`).
- [ ] **Config timing.** Configs may be set in `Create()` or in `StartCalc()` before `base.StartCalc()`; engines are always constructed by the base inside `StartCalc` (lazily, only if enabled). Per-run helper objects may be (re)built in `StartCalc` so a recalculation starts clean. The exception is roll settings, which are read live.
- [ ] **Event table coverage (EV-4.6b, PRs #28/#29).** The engine disables itself (Error line ending `; engine disabled`, neutral answers) only when the event table is missing or its hash does not match `EventRules.ExpectedTableSha`. Coverage holes no longer disable it: each kind and day is `EEventCoverage.NoEvent/Event/Unknown`. Call `CoverageBlocks(day, mask, minTier, out kind)` and treat true as "no trade" (unless `allowPartialCoverage`). See Appendix G.
- [ ] **Time windows on a Globex chart.** ET 18:00–23:59 bars belong to the next trading day. A test like `tod >= 15:55` also needs `tod < 17:00`.

- [ ] **Status Bar numbers.** `asOfBar` arguments (VP, naked levels) take `Bars.FullSymbolData.Current` (1-based), not `Bars.CurrentBar`.
- [ ] **Cost floor (catalogue §2.1).** The minimum stop for @ES is **14 ticks** (costs ≤ 0.10 R at $17.00 per round trip ($16.98 in catalogue §2.1)). The stop distance is **max(structural stop, 14-tick floor)**: a short structural stop is widened to the floor. The trade is **skipped and logged** only if that widened stop breaches the per-trade risk budget (`riskBudgetFraction` = 0.01, 1% of equity, decided by Ettienne 2026-10-23; PF-04's 0.5% cap is not the default for the floor rule); size is never shrunk. The §3 examples that place a protective stop, 2, 5–7, 10–20, 22, 23, 25–34 and 36–38, apply the floor and the risk check, measure R from the planned entry and journal a skip line (Appendix F #9, resolved). Exception: example 3 uses a fixed 16-tick stop (above the floor) with the risk check only. Examples 1 and 9 exit without a protective stop, 24 only flattens, and 4, 8, 39, 41 and 42 send no orders; the module examples 21, 35 and 40 inherit the floor from their hosts (15, 32, 37). The rule is Ettienne's decided rule implemented in each strategy; the library has no floor yet (see §3 house rules).
- [ ] **Intraday rules (catalogue §2.1).** No entries 09:30–09:45 unless the idea is an opening idea; no new entries after 15:30; flat by 15:55.
- **Half-days:** on a CME early-close day the intraday examples (3, 5, 7, 9) go flat on the first bar closing at or after `RthCloseEt` − 5 min, the same rule as `A263Config.halfDayExitBefore` (example 9 gets it from `A263Core`).
- **News blackout:** −2/+10 min around tier-1 releases. M-EVENT's `EventBlackout` does this only if you set `EventConfig.blackoutPreMin`/`blackoutPostMin` (both default 0 = no blackout; `blackoutTier` defaults to T1). None of the §3 examples applies it.
- **One live position per instrument; ES and NQ count as one position.** Nothing in the base enforces this. M-PORT's `clusterCap` limits risk per correlation cluster (2% by default); it is a heat cap, not a one-position rule.
- [ ] **Exit orders start on the bar after the fill.** Examples 5, 6, 7 and 10 send their stop and target from the first bar close at which the position exists, so the fill bar itself has no working stop or target (on a 1m chart, one minute). Examples 2 and 3 send the stop together with the entry.

---

2 — Gate 0: the level baseline and the canaries

Gate 0 is a **measurement**, not a trading idea. It answers two questions before any level-based idea in the catalogue is read:

1. **G0-d (level baseline):** does price react to a given level type (prior POC, IB high, weekly VAH, …) more than to matched random prices at the same distance? Scored by the A2-33 matched-control study (`_LevelStudy`).
2. **G0-b (canaries):** do deliberately naive "known loser" rules lose after costs? A canary that shows a **net profit factor > 1.0 is treated as a bug** in fills, costs, lookahead or session handling, not as an edge.

The other Gate-0 items are conventions the examples follow rather than code you run: **G0-a** is the cost model (A2-40 + A2-59; the canary commission/slippage below is the part `_TestGate0` documents), and **G0-c** is roll honesty (roll window excluded, multi-day levels reset at rolls).

> Sources: `_Gate0.Function.CS` (engine), `_TestGate0.Strategy.CS` (host and the setup header quoted below), `tests/Gate0Tests`, PR #20 and its review `specs/pr20-review-analyst2.md`. Catalogue §2.6 defines G0-a…G0-d. **Discrepancy:** the catalogue text names the host `_Test1`; the code on `main` hosts Gate 0 in `_TestGate0` (`_Test1` is a plain `SignalObject`). **KD-02** is listed as a G0-b canary in the catalogue (v2.3) but is **not** in `EGate0Canary`, so it cannot be run yet.

2.1 The pieces

Piece	File · class	Role
Engine	`_Gate0.Function.CS` · `Gate0Engine` (sealed, MC-free)	Builds the level families per RTH session, feeds `_LevelStudy`, runs canaries, bootstrap, verdicts
Bar	`Gate0Bar` (struct)	one closed Data1 bar: ET time, Globex day, OHLCV, roll flag
Host	`_TestGate0.Strategy.CS` · `_TestGate0 : _VolumeProfileFuturesBase`	Converts bars, routes canary orders, journals trades and totals
Study	`_LevelStudy.Function.CS` · `_LevelStudy`	A2-33 matched controls, t, Holm, half-life, terciles
RNG	`_Rng.Function.CS` · `_Rng`	SplitMix64; seed 33
Tests	`tests/Gate0Tests`	unit tests of the engine (run with `bash tests/Gate0Tests/run.sh`)

The engine's public surface (full list in Appendix E under `_Gate0.Function.CS`):

```
// _Gate0.Function.CS (namespace PowerLanguage.Function)
public enum EGate0Mode   { Strategy = 0, EventStudy = 1, ControlRT = 2, ControlFL = 3, Canary = 4 }
public enum EGate0Family { All = 0, PriorProfile = 1, NakedPoc = 2, IbEdge = 3, Overnight = 4, Avwap = 5, Vwa
p = 6,
                           Weekly = 7, CompositeVa = 8, SinglePrint = 9, DvSource = 10, HvnLvn = 11, Mixture
= 12,
                           SigmaMixture = 13 }
public enum EGate0Canary { VW08 = 0, A2_24_i = 1, A2_24_ii = 2, A2_24_iii = 3, DV10 = 4, A2_53 = 5, DV01 =
6, PS03 = 7 }

// class Gate0Engine
public EGate0Mode mode = EGate0Mode.EventStudy;
public int variant;
public double tick = 0.25;
public long seed = 33;
public double vaPct = 0.70;
public Action<string> journal;
public string Configure();                      // null/"" = OK, else an error text (the host throws it)
public void Bar(Gate0Bar b, int mp = 0);        // one CLOSED Data1 bar; mp = market position sign
public List<Gate0Result> Finish();              // scores and journals; call once
public bool InRollWindow(DateTime day);
public static string Verdict(int n, double t, string a2);
```

```
public static string PreRegistration();
public static readonly DateTime DevEnd = new DateTime(2025, 1, 1);
public const int BootstrapDraws = 500, BootstrapBlock = 5;
public const int Controls = 20, ReactionBars5m = 12, MaxAge = 15;
public const int CompositeSessions = 5;
public const double KdeBandwidthTicks = 6, KdeProminence = 0.10;
```

`Configure()` returns these errors (the host turns them into an `ArgumentException` at StartCalc): `tick must be > 0`; `Canary Variant must be 0..7 or 100..123 (A2-53 sweep)`; `level-study Variant must be 0..13`; or the `_LevelStudy.Configure()` error. Mode **Strategy** is not runnable: `_TestGate0.StartCalc` throws `Mode Strategy does not run Gate 0 - use EventStudy/Control*/Canary`.

2.2 Inputs and pre-registered constants

`_TestGate0` has exactly two `[Input]`s, plus the inherited `TraceLevel`:

- **Mode** (`EGate0Mode`, default `EventStudy`):
- `EventStudy`: the real level family `Variant`. Each level registered at the RTH open gets 20 matched controls at 0.9–1.1× its distance from the RTH open price, random side.
- `ControlRT` (negative control, "random time"): each level is replaced by the 1m close at a random bar of the prior RTH session; labels are prefixed `RT:`. Expected: PARK. A PASS is a bug.
- `ControlFL` (negative control, "fake level"): each level shifted by ±{0.5, 1, 2} × ATR14(5m); labels prefixed `FL:`. Expected: PARK. A PASS is a bug.
- `Canary`: G0-b canary number `Variant`, 1 contract.
- **Variant** (int):
- level-study modes: 0 = all families (**overview only, not a verdict run**: controls drop to 5 and every line is prefixed `NOT A VERDICT RUN`); 1–13 = one `EGate0Family` each. Run 1…13 one by one for G0-d.
- Canary mode: 0–7 = `EGate0Canary`; 100–123 = the A2-53 sweep (idx = Variant − 100; gap {0.3, 0.5, 0.8}% [idx / 8]; j = 1 + (idx / 4) % 2; bar 30m if (idx / 2) % 2 == 0 else 15m; vol filter on if idx is even). Canary 5 (A2-53 default) equals 108.
- **TraceLevel**: set it to **Journal**, or nothing is printed.

Everything else is a pre-registered constant in `_Gate0.Function.CS`: seed 33, value area 70%, 500 bootstrap draws in blocks of 5 sessions, 20 controls per level, reaction window 12 × 5m bars after the first touch, sigma = ATR14(5m) frozen at the RTH open, maximum level age 15, composite of 5 sessions, KDE bandwidth 6 ticks and prominence 10%. There is no config object to edit; changing a constant is a new pre-registration.

The level families (Variant 1–13) and their journaled types:

V	Family	Types
1	Prior profile	`POC`, `VAH`, `VAL` (age 1–15)
2	Naked POCs	`NPOC` (age 2–15)
3	Initial balance	`IBH`, `IBL` (prior 09:30–10:30)
4	Overnight	`ONH`, `ONL` (18:00 → 09:30)
5	AVWAP anchors	`AV_PH`, `AV_PL`, `AV_WK`, `AV_MO`, `AV_PC` (`AV_OPEN` retired by PR #44: the prior-RTH-open AVWAP is the prior session VWAP, already type `VWAP` in Variant 6; the name is not reused)
6	VWAP	`VWAP`, `VWAP+1`, `VWAP-1` (prior session, at its close)
7	Weekly profile	`WPOC`, `WVAH`, `WVAL`
8	Composite VA	`CVAH`, `CVAL` (last 5 RTH sessions)
9	Single prints	`SP_HI`, `SP_LO` (primary: interior runs ≥ 4 rows, TPO count 1); `SPV_HI`, `SPV_LO` (sensitivity: volume ≤ 5% of the POC row)
10	DV sources	`DV_ROT_POC_R6`, `DV_ROT_POC_R12`, `DV_TREND_HVN`
11	HVN / LVN	`HVN`, `LVN` (KDE 6 ticks, primary); `HVN_HCV`, `LVN_HCV` (bandwidth h_cv, sensitivity)

V	Family	Types
12	Mixture (A2-27)	`GMM_MU`, `GMM_EDGE`
13	Sigma mixture (A2-32)	`SIG_MU`, `SIG_EDGE`

The canaries (Variant 0–7 in Canary mode; all flat by 15:55 ET, RTH only):

V	Canary	Rule (as documented in the `_TestGate0` header)
0	VW-08	long on a 1m close above RTH VWAP, short below, reverse on the opposite close
1	A2-24 (i)	fade the first VWAP touch after 09:35; stop/target 1R = ATR14(5m)
2	A2-24 (ii)	fade a 1m close beyond VWAP ±2σ; target VWAP frozen at entry; stop 1:1
3	A2-24 (iii)	fade a 1m close ≥ 2 × ATR14(5m) from VWAP; target VWAP frozen; stop 1:1
4	DV-10	weekly VWAP ±1σ fade when both bands moved < 0.5 × ATR30m over 8 × 30m bars; limit at the band; TP weekly VWAP; SL 0.20 × ATR20d
5	A2-53	gap ≥ 0.5%, vol filter on; fade after the 1st 30m bar if the prior close was not retraced; stop-and-reverse at the session extreme ±1 tick (one reversal)
6	DV-01	prior RTH POC after a move-away ≥ 1 × ATR14(5m); limit at the POC, first touch only; SL = TP = 0.15 × ATR20d
7	PS-03	naive 80% rule: open outside the prior VA, enter on the first 30m close back inside; target the prior POC; stop 0.25 × VA width beyond the entry edge

Roll honesty. In level-study modes all multi-session sources restart at the roll, and events on the session **before** a roll, the roll session and the session after are dropped before scoring. In Canary mode the roll session and the session after are skipped live (journal `G0-b roll-skip {day} total(n)`), and the session before a roll is removed from the "ex-roll" totals afterwards. The intraday ATR14(5m) and ATR(30m) also restart at the splice (PR #43): the first 5m/30m true range on the new contract is that bucket's own range, not the cross-contract gap.

2.3 Outputs and journal lines

All lines go to the PowerLanguage Output window and the `_CustomSignal` trace log (Appendix B) when `TraceLevel = Journal`.

- At StartCalc (level-study modes): `G0 PRE-REGISTERED: …` (the DV rotation types and the HVN/LVN bandwidth arms).
- Once, at the first bar of 2025 or the last bar on the chart, whichever comes first:
- `A2-33 …` summary lines from `_LevelStudy` per type;
- one `G0 type(T) levels(n) touched(t) pairs(p) edgeR(e) t() p() holm() ci95[lo,hi] sessions(s) halfLife(h) tTercile(lo/mid/hi) tByAge[age:t/n …] A2-33(rule) => PASS | PARK (reason)` line per type;
- `G0 done mode() family() types() rollDays() sessions()`.
- Canary mode: one `G0-b trade {canary} L|S {entry time} {price} -> {exit time} {price} gross() net() cumGross() cumNet()` line per closed trade, then `G0-b canary(...) entries() rollSkips() cumGross() cumNet() - read NET profit factor in the Strategy Performance Report; > 1.0 = BUG` and four splits: **all**, **ex-roll**, **2016-2024** and **2019-2024**. The two dated splits also exclude the roll window (code: `printTotals`).

Definitions (from the `_TestGate0` header):

- **R** = max over the 12 5m bars after the first touch of |close − level| / sigma, sigma = ATR14(5m) at the RTH open.
- **edgeR** = mean R of touched levels minus the mean R of their touched controls. t and p come from `_LevelStudy`.

- **holm** = Holm step-down across all Gate-0 level types, with m fixed at the full type count, **m = 36** (`_Gate0.AllTypes.Length`; 37 before PR #44 retired `AV_OPEN`), so per-family runs are conservative. Changing m is a new pre-registration; `tests/Gate0Tests` checks the count.
- **tByAge** = t per age 1–15; an `x` marks |t| < 2 (that age is not traded).
- **tTercile** = t in the low / mid / top quality tercile (HVN/LVN prominence, GMM weight; other types have no quality).
- **ci95** = 2.5/97.5 percentiles of edgeR over 500 moving-block bootstrap draws (blocks of 5 sessions).

2.4 How to read the verdict

The verdict is `Gate0Engine.Verdict(n, t, a2)` with n = pairs. In code order:

1. n < 30 → `PARK (no evaluation: n<30)`.
2. 30 ≤ n < 150 → `PARK (inconclusive: 30<=n<150)`.
3. A2-33's rule starts with "trade top" → `PASS (trade top-quality tercile only …)`. This check comes **before** the |t| test, so a top-tercile PASS can carry an overall |t| < 2.
4. t is NaN or |t| < 2 → `PARK (kill: edge ~ 0, |t| < 2 - level type not traded)`.
5. A2-33's rule starts with "keep" → `PASS (keep …)`.
6. Otherwise → `PARK (kill: <A2-33 rule or "A2-33 drop">)`.

How to use it:

- A level type is usable by a G0-d-gated idea only with a **PASS from a single-family run (Variant 1–13) in EventStudy mode**. Variant 0 lines are never verdicts.
- `ControlRT` and `ControlFL` must come back PARK for the same family. A control PASS means the harness, not the market, produced the edge: stop and investigate.
- Keep only the ages without an `x` in `tByAge`; if t decays with age, keep ages up to the half-life.
- For **canaries**, the verdict is the **net** profit factor in the Strategy Performance Report over the development window: > 1.0 = bug to investigate, ≤ 1.0 = the plumbing behaves. Compare the "ex-roll" split with "all" to see whether roll sessions are driving anything.
- Gate 0 says nothing about whether any idea makes money. A PASS only says a level type reacts more than its matched controls.

2.5 Step by step: an @ES Gate-0 baseline

1. **Import** (PowerLanguage .NET Editor → File → Import, then compile), in the header's order: `_Rng.Function.CS`; `_LevelStudy.Function.CS`; `_BarRing`, `_ProfileMath`, `_ProfileStats`; `_Gmm`; `_LevelRef`, `_RollDetector`; the `_CustomSignal` / `_VolumeProfileBase` / `_VolumeProfileFuturesBase` chain and their helper functions; then `_Gate0.Function.CS` and `_TestGate0.Strategy.CS`.
2. **Chart:** @ES from dxFeed, continuous and **not back-adjusted** (the levels and roll logic assume raw prices). Data1 = **1-minute** bars (recommended; the 2016–2024 window exists only as 1m history), 1-tick bars, or N-second bars with N dividing 60, all of chart type Regular. The engine re-aggregates Data1 into 1m/5m/15m/30m itself, so since PR #45 `StartCalc` **throws** (`Gate0Data1.Check`) on anything else: minute bars > 1, multi-tick, volume, range, Renko and other variable-duration bars. No Data2.
3. **Session:** the "CME Globex" template (18:00–17:00 ET). RTH is derived internally. Any chart time zone works.
4. **Date range:** 2016-01-01 (or the first 1m bar) to 2024-12-31. The engine hard-stops at 2025-01-01 (`DevEnd`), so the holdout stays unseen even if more data is loaded. Allow about 5 months of warm-up (ATR20d needs 20 RTH sessions, HV100 about 100).
5. **Insert `_TestGate0`.** Format Signals → Inputs: `TraceLevel = Journal`, `Mode = EventStudy`, `Variant = 1`. Properties: Max bars back 1 is enough (all state is internal).
6. **Run** and read the Output window or the trace log. Record the `G0 PRE-REGISTERED:` line and the `G0 type(...)` lines.

7. **Repeat** for Variant 2…13 one by one, then each family again with `Mode = ControlRT` and `Mode = ControlFL`.
8. **Canaries:** `Mode = Canary`, Variant 0–7 (and 100–123 for the A2-53 sweep). In Format Signals → Properties set: - commission **$2.25 per contract per side**; slippage **0.5 tick ($6.25) per contract per side** ($17.00 per ES round trip with the commission); fixed **1 contract**; - **Bar Magnifier ON at 1 tick** where tick history exists (2024+). Before that MC uses its intrabar O-H-L-C assumption. Canaries that rest stop/limit orders (A2-24 i–iii, DV-10, DV-01, PS-03, the A2-53 reversal) need it for honest stop-versus-target ordering; - if your MC version offers it, "fill limit order when the trade price goes beyond the limit price" (1 tick): DV-01 and DV-10 require a 1-tick trade-through.
9. **Read** the canary's net profit factor in the Strategy Performance Report and the journal splits.

Pitfalls:

- Leaving `Variant = 0` and reading its lines as verdicts.
- A back-adjusted series: the levels are wrong and roll detection misfires.
- Data1 longer than 1 minute (or of variable duration): the 5m/30m re-aggregation and the 12 × 5m reaction window would break, so `StartCalc` now stops with `ArgumentException("_TestGate0: Data1 is …")`.
- TraceLevel left at None: the run completes and prints nothing.
- Loading data past 2024: harmless (hard stop), but the "last bar" trigger fires at the first 2025 bar, so the results appear there, not at the chart's end.
- Bar Magnifier off on canaries with resting orders: stop-versus-target ordering inside a 1m bar is then MC's assumption.
- **Cost per round trip.** The header's MC settings (`_TestGate0.Strategy.CS` lines 128–130 on `origin/main`, PR #30) are $2.25 commission and 0.5 tick = $6.25 slippage, each **per side**, so an ES round trip costs $4.50 + $12.50 = **$17.00**. That matches the catalogue's cost floor assumption of $16.98 (IBKR round trip + 1 tick in total); at $17.00 the 14-tick floor ($175) keeps costs at about 0.097 R (≤ 0.10 R). These are MC settings (Format Signals → Properties), not code. Canary runs made with the old 1-tick-per-side setting ($29.50) were over-costed and should be rerun.
- **Cost floor.** @ES stops under 14 ticks are widened to 14 ticks; the trade is skipped and logged only if the widened stop breaches the per-trade risk budget, and size is never shrunk (catalogue §2.1; §1.7). (This is a rule for strategy code; `_TestGate0` canaries do not apply it, and the library has no floor — see §3 house rules.)
- **VERIFY:** whether G0-a's full A2-40 cost model (including the 2-tick slippage at 09:30–09:35) is applied anywhere; MC's fixed per-side slippage cannot model a time-of-day slippage.

2.6 Which examples in §3 depend on Gate 0

Ideas that trade a **level type** are G0-d-gated: they are not read until that level type has a Gate-0 PASS. From the v3.2 set: examples 6 (PS-21: naked POC/VAH/VAL), 7 (PS-13/DV-03: rotation POC, family 10), 10 (MT-02: weekly VAH, family 7), and the AVWAP arms of example 3 (AV-08, family 5). Example 8 (A2-29) is a **conflict in the catalogue**: G0-d's table lists A2-25…A2-32 as gated, but A2-29's own entry says "Gate (v2): none (classifier)" (Appendix F).

Of the new examples, the catalogue's Gate lines name a level verdict for: example 21 (MT-03: "G0-d for DVP level types"; DVP rails are on main since 628542f (confluence and acceptance since b2685dd, the A2-33 hook since 800bce7), but no DVP level type has an A2-33 verdict yet (it needs a study run; M-DVP PR-D8 pending), so the example uses classic composite value), example 32 and its module example 35 (DV-02: "A2-33 verdict for the source level type first"), example 33 (DV-08: "parents must pass first"), and example 36 (A2-27: "A2-33 verdict for its level type must be positive"). Example 41 is the A2-33 study itself. For every other example, read the Gate line of its catalogue entry before reading any result; this manual does not restate it. Every example inherits G0-a (costs) and G0-b (canaries must still pass after any engine change).

---

3 — Forty-two worked examples, easy to hard

Each example implements one catalogue entry (or a module on a host) as a complete strategy you can import. They are written for @ES (full-size, $12.50/tick) on dxFeed continuous **unadjusted** data with the CME Globex session template; IBKR is the broker only.

Read these first:

- **Compile status.** Every example is **type-checked with Roslyn against the repo sources (`origin/main` a07257a) plus MultiCharts API stubs** (0 errors for all 42 files). The stubs are mine and are not authoritative: a member MC names differently would only show in the PowerLanguage editor. **None of the examples has been run in MultiCharts or backtested.** They carry no results, and nothing here says any of them makes money.
- **Rules** are the catalogue's. Where the catalogue leaves something open, the code makes a choice and labels it in a comment ("assumption", "coding choice", "illustrative" or "PLACEHOLDER").
- **House rules.** Settings live in a protected config class, never in `[Input]`s (examples 2, 9 and 10 still expose one `[Input]` each from v3: `Variant`, `GateArm`, `AcceptanceDays`; VP-base `[Input]`s are set in code). Journaling uses `ETraceLevel.Journal`. Stop = max(structural stop, 14-tick cost floor); a trade is **skipped** if that risk breaches `riskBudgetFraction = 0.01` of equity, and size is **never** shrunk. Costs: $17.00 per ES round trip (MC Commission/Slippage settings).
- **The floor is example code, not library behaviour.** Stop = max(structural, 14 ticks), skip if q × widened stop × $50 > 0.01 × equity, never shrink q: this is Ettienne's decided rule (2026-10-23), implemented **per strategy**. Main (a07257a) has no stop floor or widening; each example implements it locally (`costFloorTicks = 14`, `riskBudgetFraction = 0.01`, a local `overRiskBudget` or an inline check on `equityAvailable(fraction)`). The library part is **pending**: a library `CostConfig` is only specified in `instrument-profile-spec.md` IP-3.2/IP-8.1 (not built). The only library cost rule is `CostFilterMaxFractionOfR` (default 0 = off), which skips a trade whose round-trip cost exceeds that fraction of R; it never widens. Consequence: when sizing at exactly 1% risk, any stop under 14 ticks is skipped (widening it breaches the budget); widening without a skip happens only for fixed size or sizing below 1%. The examples trade a fixed 1 contract, so a 14-tick stop ($175) is skipped only when equity < $17,500 (14 × $12.50 / 0.01).
- **Orders** go to Data1. HTF streams are read through `HtfClosedBar` (closed bars only).
- **Numbering.** An example's number is its file number (`_Ex07…` is example 7). The table gives the reading order.
- **Modules.** Examples 21, 24, 35 and 40 subclass a host example and inherit its features; compile them together with the host file.

Order	Example	ID	Idea	Difficulty	Features (direct; + inherited)	File
1	1	A2-69	Kaufman 3-day trade	easy	F-ROLL, F-CAL, F-JRNL, F-EVT	`_Ex01KaufmanThreeDay.Strategy.CS`
2	2	TS-01	European-open overnight drift	easy	F-CAL, F-JRNL, F-EVT, M-MTF, M-EVENT	`_Ex02EuropeanOpenDrift.Strategy.CS`
3	13	TS-03	close regime, variant A: at 15:50 ET trade the 15:30-15:50 drift, exit at 16:00. Stop 1 x ATR5, floored.	easy	F-ROLL, F-CAL, F-TM, F-JRNL, F-EVT, M-MTF	`_Ex13CloseDrift.Strategy.CS`

Order	Example	ID	Idea	Difficulty	Features (direct; + inherited)	File
4	12	A2-49	Kaufman afternoon breakout of the morning range	easy	F-ROLL, F-CAL, F-TM, F-JRNL, F-EVT	`_Ex12MiddayBreakout.Strategy.CS`
5	16	SW-02	Connors RSI	easy	F-ROLL, F-JRNL, F-EVT, M-AVWAP	`_Ex16SwingPullback.Strategy.CS`
6	3	AV-08	week-open / month-open VWAP regime layer	easy–medium	F-ROLL, F-CAL, F-VWAP, F-JRNL, F-EVT	`_Ex03PeriodAvwapLayer.Strategy.CS`
7	14	VW-05	late-day continuation	easy–medium	F-ROLL, F-CAL, F-VWAP, F-JRNL, F-EVT, M-MTF	`_Ex14LateDayVwap.Strategy.CS`
8	18	A2-56	first-hour breakout	easy–medium	F-ROLL, F-CAL, F-TM, F-JRNL	`_Ex18FirstHourBreak.Strategy.CS`
9	20	DV-15	EMA20 pullback with a session-VWAP slope filter.	easy–medium	F-SESS, F-ROLL, F-CAL, F-VWAP, F-TM, F-JRNL, F-EVT, M-DAILY	`_Ex20EmaVwapPullback.Strategy.CS`
10	17	A2-64	intraday mean reversion to the RTH open, gated by A2-70	easy–medium	F-ROLL, F-CAL, F-TM, F-JRNL, M-DAILY	`_Ex17OpenFadeHvGate.Strategy.CS`
11	4	A2-50	same-slot relative-volume opening gate	medium	F-ROLL, F-CAL, M-TOD, M-EVENT	`_Ex04OpeningRvGate.Strategy.CS`
12	5	A2-57	60m EMA	medium	F-ROLL, F-CAL, F-JRNL, F-EVT, M-MTF	`_Ex05TwoTimeframeTurn.Strategy.CS`
13	9	A2-63	long-only open + k x ATR20d breakout, exit 15:55 ET, with the _RegimeGate arm switch	medium	F-JRNL, M-DAILY, _RegimeEngine	`_Ex09OpenAtrBreakout.Strategy.CS`
14	11	PS-24	overnight high/low break-and-go at RTH	medium	F-PROF, F-ROLL, F-CAL, F-JRNL, F-EVT, M-MTF	`_Ex11OvernightBreak.Strategy.CS`
15	15	PS-08	narrow initial balance breakout	medium	F-ROLL, F-CAL, F-JRNL, F-EVT	`_Ex15NarrowIbBreak.Strategy.CS`
16	19	PS-07	open drive, continuation	medium	F-SESS, F-ROLL, F-CAL, F-	`_Ex19OpenDrive.Strategy.CS`

Order	Example	ID	Idea	Difficulty	Features (direct; + inherited)	File
			on the first higher low		JRNL, M-SWING, M-TOD	
17	21	MT-03	composite-value gate on the example 15 host	medium	F-DEV, F-PROF, F-COMP; + F-ROLL, F-CAL, F-JRNL, F-EVT	`_Ex21CompositeValueGate.Strategy.CS`
18	23	A2-58	fade the fast trend inside the slow trend	medium	F-ROLL, F-CAL, F-TM, F-JRNL	`_Ex23FastInSlowFade.Strategy.CS`
19	24	RG-07	event, roll and thin-session exclusions, applied as a gate on the example 18 host	medium	F-CAL, F-EVT, M-EVENT, VP helpers; + F-ROLL, F-TM, F-JRNL	`_Ex24ExclusionGate.Strategy.CS`
20	25	A2-51	intraday shock-bar fade with the delayed stop	medium	F-SESS, F-ROLL, F-CAL, F-TM, F-JRNL, M-TOD	`_Ex25ShockBarFade.Strategy.CS`
21	37	OF-07	bar-level burst at a fresh extreme, standalone form	medium	F-SESS, F-ROLL, F-CAL, F-TM, F-JRNL, M-TICK, M-PORT	`_Ex37BurstExtremeBreak.Strategy.CS`
22	40	A2-78	daily kill switch as a strategy-level layer on the OF-07 host	medium	F-TM, M-PORT; + F-SESS, F-ROLL, F-CAL, F-JRNL, M-TICK	`_Ex40DailyKillSwitch.Strategy.CS`
23	6	PS-21	first touch of the nearest naked POC / VAH / VAL	medium–hard	F-PROF, F-NPOC, F-ROLL, F-CAL, F-JRNL, F-EVT, M-MTF	`_Ex06NakedLevelReaction.Strategy.CS`
24	8	A2-29	developing-profile entropy day type at 11:00 ET	medium–hard	F-ROLL, F-CAL, M-GMM, VP helpers	`_Ex08EntropyDayType.Strategy.CS`
25	22	PS-19	prior-session poor high / poor low run-through.	medium–hard	F-DEV, F-PROF, F-SESS, F-ROLL, F-CAL, F-TM, F-JRNL	`_Ex22PoorHighRun.Strategy.CS`
26	26	PS-01	balance break, acceptance, then the first higher-low pullback	medium–hard	F-DEV, F-PROF, F-COMP, F-SESS, F-ROLL, F-CAL, F-TM, F-JRNL, M-SWING	`_Ex26CompositeBreakPullback.Strategy.CS`
27	27	PS-02				`_Ex27CompositeBreakMarket.Strategy.CS`

Order	Example	ID	Idea	Difficulty	Features (direct; + inherited)	File
			balance break, market entry after acceptance	medium–hard	F-DEV, F-PROF, F-COMP, F-SESS, F-ROLL, F-CAL, F-TM, F-JRNL, M-DAILY, M-GMM	
28	29	PS-22	composite LVN-edge rejection, with the HVN-ceiling veto as a separate	medium–hard	F-DEV, F-COMP, F-HVN, F-SESS, F-ROLL, F-CAL, F-TM, F-JRNL, F-EVT	`_Ex29LvnEdgeRejection.Strategy.CS`
29	30	PS-05	rejected repricing open	medium–hard	F-DEV, F-PROF, F-SESS, F-ROLL, F-CAL, F-TM, F-JRNL, M-DAILY	`_Ex30RejectedRepricingOpen.Strategy.CS`
30	38	OF-01	stacked-imbalance origin, first retest continuation	medium–hard	F-SESS, F-ROLL, F-CAL, F-OF, F-TM, F-JRNL	`_Ex38StackedImbalanceRetest.Strategy.CS`
31	31	MT-01	multi-timeframe stack	medium–hard	F-DEV, F-COMP, F-SESS, F-ROLL, F-CAL, F-TM, F-JRNL, M-SWING, M-DAILY	`_Ex31MultiTfStack.Strategy.CS`
32	35	A2-25	POC bootstrap-stability gate + A2-26 KDE bandwidth-sweep veto, as a module on the DV-02 host	medium–hard	M-GMM, VP helpers; + F-DEV, F-PROF, F-SESS, F-ROLL, F-CAL, F-TM, F-JRNL, F-EVT, M-DAILY	`_Ex35PocStabilityGate.Strategy.CS`
33	7	PS-13 / DV-03	rotation -> breakout -> first touch of the rotation's highest-volume price	hard	F-ROLL, F-CAL, F-JRNL, F-EVT, M-MTF, VP helpers	`_Ex07RotationVolumeLevel.Strategy.CS`
34	10	MT-02	weekly accepted value + 60m trigger	hard	F-ROLL, F-CAL, F-JRNL, F-EVT, M-MTF, M-DAILY, VP helpers	`_Ex10WeeklyValueAcceptance.Strategy.CS`
35	28	AV-11	PS-01 break re-entry at the break-anchored AVWAP	hard	F-DEV, F-PROF, F-COMP, F-SESS, F-ROLL, F-CAL, F-TM, F-JRNL, M-SWING	`_Ex28BreakAvwapPullback.Strategy.CS`
36	32	DV-02		hard		`_Ex32FailedLevelFlip.Strategy.CS`

Order	Example	ID	Idea	Difficulty	Features (direct; + inherited)	File
			failed level -> reversal		F-DEV, F-PROF, F-SESS, F-ROLL, F-CAL, F-TM, F-JRNL, F-EVT, M-DAILY	
37	33	DV-08	Fair Value Gap + heavy-volume alignment as a level layer	hard	F-HVN, F-ROLL, F-JRNL, M-DAILY, VP helpers	`_Ex33FvgHvnAlignment.Strategy.CS`
38	34	PS-12	trend-cluster pullback, Dale limit variant	hard	F-DEV, F-COMP, F-HVN, F-ROLL, F-JRNL, M-SWING, VP helpers	`_Ex34TrendClusterPullback.Strategy.CS`
39	36	A2-27	bimodal-profile void traverse	hard	F-SESS, F-ROLL, F-CAL, F-TM, F-JRNL, M-MTF, M-GMM, VP helpers	`_Ex36BimodalVoidTraverse.Strategy.CS`
40	39	PF-03	cross-market hidden-beta monitor, RESEARCH PROXY	research	F-SESS, F-ROLL, F-CAL, M-XMKT	`_Ex39XmktBetaMonitor.Strategy.CS`
41	41	A2-33	level-persistence vs matched-control study	research	F-PROF, F-NPOC, F-ROLL, F-CAL, A2-33	`_Ex41LevelPersistenceStudy.Strategy.CS`
42	42	KD-08	forward-cone diagnostic, RESEARCH SNIPPET	research	F-ROLL, M-DVP	`_Ex42ForwardConeDiagnostic.Strategy.CS`

Common pitfalls for all examples:

- **Evening wrap.** On a Globex chart, bars stamped 18:00–23:59 ET belong to the **next** Globex day. A check like `tod >= 15:55` is also true at 19:00, so the examples write `tod >= X && tod < 17:00`.
- **Closed bars.** Every example returns early unless `Bars.Status == EBarState.Close`, and calls `base.CalcBar()` first.
- **Rolls.** Examples that set `RollDetection = ERollDetection.Series` clear multi-day state in `rollOn(1)`. With RollDetection on, M-DAILY's ATR is reseeded at each roll and stays NaN for `atrLength` sessions (§4.3).
- **Configs** may be set in `Create()` or in `StartCalc()` before `base.StartCalc()`; engines are always constructed by the base inside `StartCalc` (lazily, only if enabled). Per-run helper objects may be (re)built in `StartCalc` so a recalculation starts clean.
- **Fill bar.** Stops are sent from the bar after the fill (MC rule), so the fill bar itself is unprotected unless the example sends the stop with the entry (example 20 notes this).

3.1 Example 1 (`_Ex01KaufmanThreeDay`) — A2-69 Kaufman 3-day trade (easy)

Features (direct). F-ROLL (§5.7), F-CAL (§5.8), F-JRNL (§5.13), F-EVT (§5.14).

Catalogue rules (A2-69, core arm). Setup: C(t−1) < C(t−2) < C(t−3), two lower RTH closes. Entry (long only): if the open O(t) < C(t−1), buy at 09:31 (the first 1m bar's open); otherwise buy at 15:59 (MOC proxy). Exit at 15:59 on the following session. No stop in the core. The SMA, profit-take, spike and Friday-short arms are not coded.

Chart. @ES 1m, dxFeed continuous unadjusted, CME Globex template. Inputs: only `TraceLevel`. Settings are in the protected `A269Config`.

Walk-through.

- The RTH close of each session is the close of its last RTH bar (`isRthBar`, through 16:00 ET or the early close), collected per Globex day. NoRth days never set up.
- At the start of each Globex day the setup is tested on the last three completed RTH closes. At the close of the first RTH bar (close-stamped 09:31), if that bar's open is below C1, a market order goes in for the next bar's open.
- Otherwise it waits for the bar close-stamped 15:59 (one minute before an early close) and sends a market order for the next bar.
- The exit is sent the same way at 15:59 on any later session (normally the next one).
- With TraceLevel Journal, `journalArm` records the planned entry and `logSignal` writes an `MCEvents` row for each entry.

Pitfalls.

- "Lower open" compares the RTH open with the prior **RTH** close, not the Globex close.
- On an early-close day the "15:59" bar is one minute before `RthCloseEt`.
- **Coding choice (labelled):** RollDetection is Series and `rollOn(1)` clears the close history, so the setup never compares closes from two contracts. The catalogue does not say this.
- A 1m entry "at 09:31" is the next bar's open after the 09:31 bar closes. That is the closest MC can do with market-next-bar orders.

Full strategy — `_Ex01KaufmanThreeDay.Strategy.CS`:

```
// Manual v4 example 1 - A2-69 Kaufman 3-day trade (core arm: open-else-close entry, exit next close, long
only).
// Chart: @ES 1m, dxFeed continuous UNADJUSTED, CME Globex session template, Bar Magnifier off (1m is the
fill bar).
using System;
using System.Collections.Generic;
using PowerLanguage.Function;
using System.Globalization;

namespace PowerLanguage.Strategy
{
    public class _Ex01KaufmanThreeDay : _CustomSignal
    {
        // ---- protected config (no [Input]s besides TraceLevel) ----
        protected sealed class A269Config
        {
            public int contracts = 1;
            public TimeSpan rthOpen = new TimeSpan(9, 30, 0);
            public TimeSpan closeBar = new TimeSpan(15, 59, 0);   // ET close-stamp of the bar whose close
sends the "15:59" order
        }
        protected A269Config Cfg { get; } = new A269Config();

        private IOrderMarket m_le, m_lx;
        private _CmeSessionClock m_clock;
        private readonly List<double> m_closes = new
List<double>();   // 16:00 ET RTH closes C(t-1), C(t-2), C(t-3) ... newest first
        private DateTime m_day = DateTime.MinValue, m_entryDay = DateTime.MinValue;
        private double m_rthClose = double.NaN;   // last RTH value-window close of the current trading day
        private bool m_setup, m_openChecked, m_waitClose;

        public _Ex01KaufmanThreeDay(object _ctx) : base(_ctx) { }

        private _CmeSessionClock Clock { get { if (m_clock == null) m_clock = new _CmeSessionClock(resolveBar
TimeZone(Bars.Info)); return m_clock; } }
```

```
        // CME session adapter (section 1.6): Globex trading day + ET clock + RTH value window (09:30-16:00
ET, early close aware).
        protected override DateTime tradingDayKey(DateTime t) { return Clock.GlobexDate(t); }
        protected override TimeSpan sessionClock(DateTime t) { return Clock.ToEastern(t).TimeOfDay; }
        protected override bool isRthBar(DateTime t) { return Clock.IsRthValueBarEt(Clock.ToEastern(t)); }

        protected override void Create()
        {
            base.Create();
            m_le = OrderCreator.MarketNextBar(new SOrderParameters(Contracts.UserSpecified, "K3LE", EOrderAct
ion.Buy));
            m_lx = OrderCreator.MarketNextBar(new SOrderParameters(Contracts.Default, "K3LX", EOrderAction.Se
ll));
        }

        protected override void StartCalc()
        {
            m_clock = null;
            RollDetection = ERollDetection.Series;   // unadjusted series: detect splices
            base.StartCalc();
            m_closes.Clear(); m_rthClose = double.NaN; m_day = m_entryDay = DateTime.MinValue;
            m_setup = m_openChecked = m_waitClose = false;
        }

        protected override void CalcBar()
        {
            base.CalcBar();
            if (Bars.Status != EBarState.Close) return;
            DateTime et = Clock.ToEastern(Bars.Time[0]);
            DateTime day = tradingDayKey(Bars.Time[0]);
            _CmeHolidays.Rules.Day cme = _CmeHolidays.Rules.Get(day);
            TimeSpan closeBar = cme.RthCloseEt.HasValue ? cme.RthCloseEt.Value - TimeSpan.FromMinutes(1) : Cf
g.closeBar;   // half-day

            if (rollOn(1))
            {
                m_closes.Clear(); m_rthClose =
double.NaN;   // the series jumps by the roll gap: restart the 3-close count (journaled below)
                if (journalOn()) trace(ETraceLevel.Journal, $"A2-69 roll at {Bars.Time[0]:yyyy-MM-dd HH:mm}:
close history cleared");
            }

            // 1. new trading day: push the completed session's RTH close as C(t-1), then evaluate the setup
once
            if (day != m_day)
            {
                m_day = day; m_openChecked = false; m_waitClose = false;
                if (!double.IsNaN(m_rthClose)) { m_closes.Insert(0, m_rthClose); if (m_closes.Count > 3) m_cl
oses.RemoveAt(3); }
                m_rthClose = double.NaN;
                m_setup = !cme.NoRth && m_closes.Count == 3 && m_closes[0] < m_closes[1] && m_closes[1] < m_c
loses[2];
                if (journalOn())
                    trace(ETraceLevel.Journal, $"A2-69 {day:yyyy-MM-dd} closes({string.Join(",", m_closes)})
setup({m_setup}) noRth({cme.NoRth}) halfDay({cme.RthCloseEt.HasValue})");
            }

            bool rth = isRthBar(Bars.Time[0]);
            if (rth) m_rthClose = Bars.Close[0];   // ends as the 16:00 (or early-close) bar close

            // ET 18:00-23:59 bars belong to the NEXT trading day, so "after 15:59" must also be "before
17:00"
            bool atClose = et.TimeOfDay >= closeBar && et.TimeOfDay < new TimeSpan(17, 0, 0);

            int pos = StrategyInfo.MarketPosition;

            // 2. exit: on the session after the entry day, at the 15:59 bar
            if (pos > 0 && day > m_entryDay && atClose) { m_lx.Send(); return; }
            if (pos != 0 || !m_setup) return;

            // 3. first RTH bar (close-stamped 09:31): O(t) is that bar's open; lower open -> buy the next
bar's open
            if (!m_openChecked && rth)
            {
                m_openChecked = true;
                double o = Bars.Open[0];
                if (o < m_closes[0]) { arm(o, "open"); return; }
```

```
                m_waitClose = true;
            }
            // 4. otherwise buy at the 15:59 bar (MOC proxy)
            if (m_waitClose && atClose) { m_waitClose = false; arm(Bars.Close[0], "close"); }
        }

        private void arm(double planned, string how)
        {
            m_setup = false; m_entryDay = m_day;
            if (journalOn())
            {
                var ctx = new Dictionary<string, string> { { "c1", m_closes[0].ToString() }, { "how", how } }
;
                journalArm("K3LE", planned, double.NaN, double.NaN, ctx);
                logSignal("A2-69", +1, planned, double.NaN, ctx, true);
            }
            m_le.Send(Cfg.contracts);
        }
    }
}
```

Compile status: compiled — type-checked with Roslyn against the repo sources (`origin/main` a07257a) plus MC API stubs (stubs not authoritative). Not run in MultiCharts; no backtest results.

3.2 Example 2 (`_Ex02EuropeanOpenDrift`) — TS-01 European-open overnight drift (easy)

Features (direct). F-CAL (§5.8), F-JRNL (§5.13), F-EVT (§5.14), M-MTF (§5.17), M-EVENT (§5.20).

Catalogue rules (TS-01). Buy at 01:30 ET, sell at 04:30 ET every trading day. Skip CME holidays and, in the event variant, the nights before tier-1 08:30 releases. No stop, but a disaster stop at 2 × ATR60m. Sweep: window {01:30–04:30, 02:00–03:00} × event filter {off, on}.

Chart. Data1 = @ES 1m, Data2 = @ES 60m (same symbol and session), dxFeed unadjusted, 24h Globex template. `[Input] Variant` 1–4: 1 = 01:30–04:30, 2 = 02:00–03:00, 3 and 4 = the same windows with the event filter.

Walk-through.

- `HtfStreams = {2}` makes Data2 a same-instrument HTF stream, read with `HtfClosedBar`.
- The entry is a market order sent at the close of the **first closed bar** stamped at or after one minute before the window start (and before the window end). Normally that is the bar one minute before the start, so it fills at the window start. If bars are missing, the entry is late, and the journal's `sendBar` field shows when. If the window passes with no closed bar in it, the night is journaled as `skip: no bar` and not traded. The exit is sent at the close of the first bar stamped at or after the window end (and before 17:00 ET).
- Days that CME has Closed, NoRth days and halt days are skipped (`_CmeHolidays.Rules.Get`).
- Event variant: the engine is enabled with the Macro class. The night is skipped if a T1 event at `EventRules.T0830` is on the coming trading day (`EventsOn`), or if `CoverageBlocks(day, Macro, T1, out hole)` reports an Unknown coverage day (EV-4.6b). If the event engine is missing or `Disabled`, the filter **fails closed** (no trade), because a disabled engine answers neutrally and would let every night through.
- Disaster stop: planned price − 2 × ATR60m from closed 60m bars, widened to the 14-tick floor if narrower; the night is skipped (journaled) only if 1 contract × that stop × $50 exceeds 1% of equity. Sent with the entry and re-sent every bar.

Pitfalls.

- **Assumption (labelled):** ATR60m length 14, as a simple average of true range. The catalogue does not give the length.
- The window sits in the evening-wrapped part of the Globex day: 01:30 ET belongs to the same Globex date as the following RTH session, so "the night before an 08:30 release" means the same Globex date.

- `EventsOn` is as-of: an event added to the table after the bar is not visible.
- GDP/PCE coverage holes 2016–24 (EV-4.6b) are the case `CoverageBlocks` handles.
- RollDetection stays Off here, so an ATR60m window that spans a roll includes the roll gap.
- The catalogue's TS-01 is written for **MES** (kill line $2.47) on 30m/60m bars. This example uses @ES on 1m with a 60m stream, so its costs are ES costs (§2.5).

Full strategy — `_Ex02EuropeanOpenDrift.Strategy.CS`:

```
// Manual v4 example 2 - TS-01 European-open overnight drift (long 01:30-04:30 ET, disaster stop 2 x ATR60m).
// Chart: Data1 = @ES 1m, Data2 = @ES 60m (same symbol, same session), dxFeed continuous UNADJUSTED, 24h CME
Globex template.
using System;
using System.Collections.Generic;
using PowerLanguage.Function;

namespace PowerLanguage.Strategy
{
    public class _Ex02EuropeanOpenDrift : _CustomSignal
    {
        /// <summary>Sweep (catalogue): 1 = 01:30-04:30, 2 = 02:00-03:00, 3 = 1 + event filter, 4 = 2 + event
filter.</summary>
        [Input] public int Variant { get; set; }

        protected sealed class Ts01Config
        {
            public int contracts = 1;
            public double disasterAtr = 2.0;
            public int atrLength = 14;       // ATR60m length is not in the catalogue: an assumption of this
example
            public int costFloorTicks =
14;  // catalogue 2.1: ES cost floor; stop distance = max(structural, floor), size never shrunk
            public double riskBudgetFraction = 0.01;  // floor-rule risk budget: Ettienne, 2026-10-23: 1% of
equity
        }
        protected Ts01Config Cfg { get; } = new Ts01Config();

        private IOrderMarket m_le, m_lx;
        private IOrderPriced m_lxStp;
        private _CmeSessionClock m_clock;
        private TimeSpan m_start, m_end;
        private bool m_filter;
        private DateTime m_doneDay = DateTime.MinValue;
        private bool overRiskBudget(double stopDist) { return Cfg.contracts * stopDist * Bars.Info.BigPointVa
lue > equityAvailable(Cfg.riskBudgetFraction); }
        private double m_stop = double.NaN;

        public _Ex02EuropeanOpenDrift(object _ctx) : base(_ctx) { Variant = 1; }

        private _CmeSessionClock Clock { get { if (m_clock == null) m_clock = new _CmeSessionClock(resolveBar
TimeZone(Bars.Info)); return m_clock; } }
        protected override DateTime tradingDayKey(DateTime t) { return Clock.GlobexDate(t); }
        protected override TimeSpan sessionClock(DateTime t) { return Clock.ToEastern(t).TimeOfDay; }
        protected override bool isRthBar(DateTime t) { return Clock.IsRthValueBarEt(Clock.ToEastern(t)); }

        protected override void Create()
        {
            base.Create();
            m_le = OrderCreator.MarketNextBar(new SOrderParameters(Contracts.UserSpecified, "TS01LE", EOrderA
ction.Buy));
            m_lx = OrderCreator.MarketNextBar(new SOrderParameters(Contracts.Default, "TS01LX",
EOrderAction.Sell));
            m_lxStp = OrderCreator.Stop(new SOrderParameters(Contracts.Default, "TS01DS", EOrderAction.Sell,
OrderExit.FromAll));
            // M-EVENT: created only when enabled (StartCalc). T1 only; coverage holes of lower tiers never
block (EV-4.6b).
            EventConfig.minTier = EEventTier.T1;
            EventConfig.classes = EEventClassMask.Macro;
        }

        protected override void StartCalc()
        {
            if (Variant < 1 || Variant > 4) throw new
ArgumentException("_Ex02EuropeanOpenDrift: Variant must be 1-4");
            bool narrow = Variant == 2 || Variant == 4;
```

```
            m_start = narrow ? new TimeSpan(2, 0, 0) : new TimeSpan(1, 30, 0);
            m_end = narrow ? new TimeSpan(3, 0, 0) : new TimeSpan(4, 30, 0);
            m_filter = Variant >= 3;
            EventConfig.enabled = m_filter;   // set before base.StartCalc: the engine is built there
            HtfStreams = new[] {
2 };          // M-MTF: StartCalc fails loudly if Data2 is missing or another root
            m_clock = null;
            base.StartCalc();
            m_doneDay = DateTime.MinValue; m_stop = double.NaN;
        }

        protected override void CalcBar()
        {
            base.CalcBar();
            if (Bars.Status != EBarState.Close) return;
            DateTime et = Clock.ToEastern(Bars.Time[0]);
            DateTime day = tradingDayKey(Bars.Time[0]);   // the 01:30 ET bar belongs to the Globex day that
opened 18:00 ET the evening before
            TimeSpan tod = et.TimeOfDay;
            int pos = StrategyInfo.MarketPosition;

            if (pos > 0)
            {
                if (tod >= m_end && tod < new TimeSpan(17, 0, 0)) { m_lx.Send(); return; }
                if (!double.IsNaN(m_stop)) m_lxStp.Send(m_stop);   // re-send every bar (MC cancels unsent
orders)
                return;
            }
            m_stop = double.NaN;
            if (day == m_doneDay) return;
            // the entry order is sent on the close of the FIRST closed bar stamped at or after m_start - 1
min (normally that
            // bar, so it fills at the m_start bar's open); a missing bar delays the entry, it does not skip
the night
            TimeSpan sendFrom = m_start - TimeSpan.FromMinutes(1);
            if (tod >= m_end && tod < new TimeSpan(17, 0, 0))
            {
                m_doneDay = day;   // the window passed with no closed bar in it
                if (journalOn()) trace(ETraceLevel.Journal, $"TS-01 {day:yyyy-MM-dd} skip: no bar in
{sendFrom:hh\\:mm}-{m_end:hh\\:mm} ET");
                return;
            }
            if (tod < sendFrom || tod >= m_end) return;
            m_doneDay = day;

            string skip = skipReason(day);
            double atr = atrClosed(2, Cfg.atrLength);
            if (skip == null && !(atr > 0)) skip = "ATR60m warm-up";
            if (skip != null) { if (journalOn()) trace(ETraceLevel.Journal, $"TS-01 {day:yyyy-MM-dd} skip:
{skip}"); return; }

            double planned = Bars.Close[0];
            m_stop = planned - Cfg.disasterAtr * atr;   // planned from the signal close; the fill is the
next bar's open
            double tick = tickSize();
            if (planned - m_stop < Cfg.costFloorTicks * tick) m_stop = planned - Cfg.costFloorTicks * tick;
 // max(structural, 14 ticks)
            if (overRiskBudget(planned - m_stop))   // widened stop breaches the 1% budget: skip and log,
never resize
            {
                if (journalOn()) trace(ETraceLevel.Journal, $"TS-01 {day:yyyy-MM-dd} skip: stop {(planned -
m_stop) / tick:F0} ticks breaches risk budget {Cfg.riskBudgetFraction:P2}");
                m_stop = double.NaN; return;
            }
            if (journalOn())
            {
                var ctx = new Dictionary<string, string> { { "variant", Variant.ToString() }, { "atr60",
atr.ToString("F2") }, { "sendBar", tod.ToString(@"hh\:mm") } };
                journalArm("TS01LE", planned, m_stop, double.NaN, ctx);
                logSignal("TS-01", +1, planned, m_stop, ctx, true);
            }
            m_le.Send(Cfg.contracts);
            m_lxStp.Send(m_stop);   // also protects the entry bar
        }

        private string skipReason(DateTime day)
        {
```

```
            _CmeHolidays.Rules.Day cme = _CmeHolidays.Rules.Get(day);
            if (cme.Closed || cme.NoRth || cme.HaltEt.HasValue) return "CME holiday/halt " + cme.Name;
            if (!m_filter) return null;
            _EventCalendar ev = EventCalendar;
            if (ev == null || ev.Disabled) return "event filter on but M-EVENT disabled";   // fail closed
            EEventKind hole;
            if (ev.CoverageBlocks(day, EEventClassMask.Macro, EEventTier.T1, out hole)) return "coverage
Unknown " + hole;
            foreach (EventRec r in ev.EventsOn(day, EEventClassMask.Macro))
                if (r.tier == EEventTier.T1 && r.etTime.TimeOfDay == EventRules.T0830) return "T1 08:30 " +
r.KindName;
            return null;
        }

        /// <summary>Simple-average true range over the last n CLOSED bars of stream ds (needs n + 1 closed
bars).</summary>
        private double atrClosed(int ds, int n)
        {
            if (BarsOfData(ds).CurrentBar <= n + 1) return double.NaN;
            double sum = 0;
            for (int k = 0; k < n; k++)
            {
                DateTime t; double o, h, l, c, v, pc;
                HtfClosedBar(ds, k + 1, out t, out o, out h, out l, out pc, out v);
                HtfClosedBar(ds, k, out t, out o, out h, out l, out c, out v);
                sum += Math.Max(h - l, Math.Max(Math.Abs(h - pc), Math.Abs(l - pc)));
            }
            return sum / n;
        }
    }
}
```

Compile status: compiled — type-checked with Roslyn against the repo sources (`origin/main` a07257a) plus MC API stubs (stubs not authoritative). Not run in MultiCharts; no backtest results.

3.3 Example 13 (`_Ex13CloseDrift`) — TS-03 close regime, variant A: at 15:50 ET trade the 15:30-15:50 drift, exit at 16:00. Stop 1 x ATR5, floored (easy)

Features (direct). F-ROLL (§5.7), F-CAL (§5.8), F-TM (§5.12), F-JRNL (§5.13), F-EVT (§5.14), M-MTF (§5.17).

What it does (from the file header):

> TS-03 close regime, variant A: at 15:50 ET trade the 15:30-15:50 drift, exit at 16:00. Stop 1 x ATR5, floored. Chart: Data1 = @ES 1m, Data2 = @ES 5m (24h Globex template), dxFeed continuous UNADJUSTED. MOC imbalance data is NOT available: the drift is a proxy only (catalogue). Early-close days (F-CAL) are skipped: there is no 15:30-16:00 window. Features: F-CAL (_CmeHolidays.Rules), F-TM (MaxTradesPerDay, entryAllowed), M-MTF (HtfClosedBar), CME clock adapter, F-JRNL.

Code.

```
// Manual v4 example 13 - TS-03 close regime, variant A: at 15:50 ET trade the 15:30-15:50 drift, exit at
16:00. Stop 1 x ATR5, floored.
// Chart: Data1 = @ES 1m, Data2 = @ES 5m (24h Globex template), dxFeed continuous UNADJUSTED. MOC imbalance
data is NOT available:
// the drift is a proxy only (catalogue). Early-close days (F-CAL) are skipped: there is no 15:30-16:00
window.
// Features: F-CAL (_CmeHolidays.Rules), F-TM (MaxTradesPerDay, entryAllowed), M-MTF (HtfClosedBar), CME
clock adapter, F-JRNL.
using System;
using System.Collections.Generic;
using PowerLanguage.Function;

namespace PowerLanguage.Strategy
{
    public class _Ex13CloseDrift : _CustomSignal
    {
        protected sealed class Ts03Config
```

```
        {
            public int contracts = 1;
            public TimeSpan driftFrom = new TimeSpan(15, 30, 0), entryAt = new TimeSpan(15, 50, 0), exitAt =
new TimeSpan(16, 0, 0);
            public int minDriftTicks = 1;
            public int atrLength = 14;
            public int costFloorTicks = 14;
            public double riskBudgetFraction = 0.01;
        }
        protected Ts03Config Cfg { get; } = new Ts03Config();

        private IOrderMarket m_le, m_se, m_lx, m_sx;
        private IOrderPriced m_lxStp, m_sxStp;
        private _CmeSessionClock m_clock;
        private double m_ref = double.NaN, m_stop = double.NaN;
        private DateTime m_day = DateTime.MinValue;

        public _Ex13CloseDrift(object _ctx) : base(_ctx) { }

        private _CmeSessionClock Clock { get { if (m_clock == null) m_clock = new _CmeSessionClock(resolveBar
TimeZone(Bars.Info)); return m_clock; } }
        protected override DateTime tradingDayKey(DateTime t) { return Clock.GlobexDate(t); }
        protected override TimeSpan sessionClock(DateTime t) { return Clock.ToEastern(t).TimeOfDay; }
        protected override bool isRthBar(DateTime t) { return Clock.IsRthValueBarEt(Clock.ToEastern(t)); }

        protected override void Create()
        {
            base.Create();
            RollDetection = ERollDetection.Series;
            MaxTradesPerDay = 1;
            m_le = OrderCreator.MarketNextBar(new SOrderParameters(Contracts.UserSpecified, "TS03LE", EOrderA
ction.Buy));
            m_se = OrderCreator.MarketNextBar(new SOrderParameters(Contracts.UserSpecified, "TS03SE", EOrderA
ction.SellShort));
            m_lx = OrderCreator.MarketNextBar(new SOrderParameters(Contracts.Default, "TS03LX",
EOrderAction.Sell));
            m_sx = OrderCreator.MarketNextBar(new SOrderParameters(Contracts.Default, "TS03SX",
EOrderAction.BuyToCover));
            m_lxStp = OrderCreator.Stop(new SOrderParameters(Contracts.Default, "TS03LS", EOrderAction.Sell,
OrderExit.FromAll));
            m_sxStp = OrderCreator.Stop(new SOrderParameters(Contracts.Default, "TS03SS", EOrderAction.BuyToC
over, OrderExit.FromAll));
        }

        protected override void StartCalc()
        {
            HtfStreams = new[] { 2 };
            m_clock = null;
            base.StartCalc();
            m_ref = m_stop = double.NaN; m_day = DateTime.MinValue;
        }

        protected override void CalcBar()
        {
            base.CalcBar();
            if (Bars.Status != EBarState.Close) return;
            DateTime day = tradingDayKey(Bars.Time[0]);
            if (day != m_day) { m_day = day; m_ref = double.NaN; }
            TimeSpan tod = sessionClock(Bars.Time[0]);
            int pos = StrategyInfo.MarketPosition;
            if (pos != 0)
            {
                if (tod >= Cfg.exitAt && tod < new TimeSpan(17, 0, 0)) { if (pos > 0) m_lx.Send(); else
m_sx.Send(); return; }   // next-bar market: fills at the open of the 16:01 bar
                if (pos > 0) m_lxStp.Send(m_stop); else m_sxStp.Send(m_stop);
                return;
            }
            if (rollOn(1)) { m_ref = double.NaN; return; }
            if (_CmeHolidays.Rules.Get(day).RthCloseEt.HasValue) return;     // F-CAL: early close, no MOC
window
            if (tod == Cfg.driftFrom) m_ref = Bars.Close[0];                  // 1m bar closing at 15:30
            if (tod != Cfg.entryAt || double.IsNaN(m_ref)) return;
            double tick = tickSize(), drift = Bars.Close[0] - m_ref;
            if (Math.Abs(drift) < Cfg.minDriftTicks * tick) return;
            int side = Math.Sign(drift);
            string why;
            if (!entryAllowed(out why)) return;
```

```
            double atr = atr5();
            if (double.IsNaN(atr)) return;
            double r = Math.Max(atr, Cfg.costFloorTicks * tick);               // floor rule
            if (overRiskBudget(r)) { if (journalOn()) trace(ETraceLevel.Journal, $"TS-03 {Bars.Time[0]:yyyy-
MM-dd} skip: stop {r / tick:F0} ticks breaches risk budget {Cfg.riskBudgetFraction:P2}"); return; }
            double entry = Bars.Close[0];
            m_stop = entry - side * r;
            if (side > 0) m_le.Send(Cfg.contracts); else m_se.Send(Cfg.contracts);
            if (journalOn())
            {
                var ctx = new Dictionary<string, string> { { "drift", drift.ToString("F2") }, { "atr5", atr.T
oString("F2") } };
                journalArm(side > 0 ? "TS03LE" : "TS03SE", entry, m_stop, double.NaN, ctx);
                logSignal("TS-03", side, entry, m_stop, ctx, true);
            }
        }

        private double atr5()
        {
            int n = Cfg.atrLength;
            if (BarsOfData(2).CurrentBar < n + 2) return double.NaN;
            double sum = 0;
            for (int k = 0; k < n; k++)
            {
                DateTime t; double o, h, l, c, v, o1, h1, l1, pc, v1;
                HtfClosedBar(2, k, out t, out o, out h, out l, out c, out v);
                HtfClosedBar(2, k + 1, out t, out o1, out h1, out l1, out pc, out v1);
                sum += Math.Max(h, pc) - Math.Min(l, pc);
            }
            return sum / n;
        }

        private bool overRiskBudget(double stopDist) { return Cfg.contracts * stopDist * Bars.Info.BigPointVa
lue > equityAvailable(Cfg.riskBudgetFraction); }
    }
}
```

3.4 Example 12 (`_Ex12MiddayBreakout`) — A2-49 Kaufman afternoon breakout of the morning range (breakout arm, day-only variant, SAR inside 12:00-15:30) (easy)

Features (direct). F-ROLL (§5.7), F-CAL (§5.8), F-TM (§5.12), F-JRNL (§5.13), F-EVT (§5.14).

What it does (from the file header):

> A2-49 Kaufman afternoon breakout of the morning range (breakout arm, day-only variant, SAR inside 12:00-15:30). Chart: Data1 = @ES 30m (24h Globex template), dxFeed continuous UNADJUSTED. IBKR = broker only. Features: F-TM (NoEntryBefore/NoEntryAfter/FlatBy, entryAllowed, mustFlatten, MaxTradesPerDay), CME clock adapter on a plain _CustomSignal, F-JRNL, roll guard, floor rule. R (for the 1.5R disaster stop) = the morning range width: our reading of A2's "R".

Code.

```
// Manual v4 example 12 - A2-49 Kaufman afternoon breakout of the morning range (breakout arm, day-only
variant, SAR inside 12:00-15:30).
// Chart: Data1 = @ES 30m (24h Globex template), dxFeed continuous UNADJUSTED. IBKR = broker only.
// Features: F-TM (NoEntryBefore/NoEntryAfter/FlatBy, entryAllowed, mustFlatten, MaxTradesPerDay), CME clock
adapter on a plain
// _CustomSignal, F-JRNL, roll guard, floor rule. R (for the 1.5R disaster stop) = the morning range width:
our reading of A2's "R".
using System;
using System.Collections.Generic;
using PowerLanguage.Function;

namespace PowerLanguage.Strategy
{
    public class _Ex12MiddayBreakout : _CustomSignal
    {
        protected sealed class A249Config
```

```
        {
            public int contracts = 1;
            public TimeSpan rthOpen = new TimeSpan(9, 30, 0), midday = new TimeSpan(12, 0, 0);
            public TimeSpan lastEntry = new TimeSpan(15, 30, 0), flat = new TimeSpan(15, 55, 0);
            public TimeSpan halfDayExitBefore = new TimeSpan(0, 5, 0);
            public double disasterR = 1.5;
            public int maxEntries = 3;              // the first break plus at most two reversals per
afternoon
            public int costFloorTicks = 14;
            public double riskBudgetFraction = 0.01;
        }
        protected A249Config Cfg { get; } = new A249Config();

        private IOrderMarket m_le, m_se, m_lx, m_sx;
        private IOrderPriced m_lxStp, m_sxStp;
        private _CmeSessionClock m_clock;
        private DateTime m_day = DateTime.MinValue;
        private double m_hi = double.NaN, m_lo = double.NaN, m_stop = double.NaN;
        private bool m_skipDay;

        public _Ex12MiddayBreakout(object _ctx) : base(_ctx) { }

        private _CmeSessionClock Clock { get { if (m_clock == null) m_clock = new _CmeSessionClock(resolveBar
TimeZone(Bars.Info)); return m_clock; } }
        protected override DateTime tradingDayKey(DateTime t) { return Clock.GlobexDate(t); }
        protected override TimeSpan sessionClock(DateTime t) { return Clock.ToEastern(t).TimeOfDay; }
        protected override bool isRthBar(DateTime t) { return Clock.IsRthValueBarEt(Clock.ToEastern(t)); }

        protected override void Create()
        {
            base.Create();
            RollDetection = ERollDetection.Series;
            // CS-7 primitives: protected properties, set in code (never [Input])
            NoEntryBefore = Cfg.midday; NoEntryAfter = Cfg.lastEntry; FlatBy = Cfg.flat; MaxTradesPerDay = Cf
g.maxEntries;
            m_le = OrderCreator.MarketNextBar(new SOrderParameters(Contracts.UserSpecified, "A249LE", EOrderA
ction.Buy));
            m_se = OrderCreator.MarketNextBar(new SOrderParameters(Contracts.UserSpecified, "A249SE", EOrderA
ction.SellShort));
            m_lx = OrderCreator.MarketNextBar(new SOrderParameters(Contracts.Default, "A249LX",
EOrderAction.Sell));
            m_sx = OrderCreator.MarketNextBar(new SOrderParameters(Contracts.Default, "A249SX",
EOrderAction.BuyToCover));
            m_lxStp = OrderCreator.Stop(new SOrderParameters(Contracts.Default, "A249LS", EOrderAction.Sell,
OrderExit.FromAll));
            m_sxStp = OrderCreator.Stop(new SOrderParameters(Contracts.Default, "A249SS", EOrderAction.BuyToC
over, OrderExit.FromAll));
        }

        protected override void StartCalc()
        {
            m_clock = null;
            base.StartCalc();
            m_day = DateTime.MinValue; m_hi = m_lo = m_stop = double.NaN; m_skipDay = false;
        }

        protected override void CalcBar()
        {
            base.CalcBar();
            if (Bars.Status != EBarState.Close) return;
            DateTime day = tradingDayKey(Bars.Time[0]);
            if (day != m_day) { m_day = day; m_hi = m_lo = double.NaN; m_skipDay = false; FlatBy =
flatAt(day); }
            if (rollOn(1)) m_skipDay = true;                        // the morning range would straddle the
splice
            TimeSpan tod = sessionClock(Bars.Time[0]);
            int pos = StrategyInfo.MarketPosition;
            if (mustFlatten()) { if (pos > 0) m_lx.Send(); else m_sx.Send(); return; }

            // morning range: RTH bars that close in (09:30, 12:00]
            if (tod > Cfg.rthOpen && tod <= Cfg.midday && isRthBar(Bars.Time[0]))
            {
                m_hi = double.IsNaN(m_hi) ? Bars.High[0] : Math.Max(m_hi, Bars.High[0]);
                m_lo = double.IsNaN(m_lo) ? Bars.Low[0] : Math.Min(m_lo, Bars.Low[0]);
            }
            if (pos > 0) m_lxStp.Send(m_stop); else if (pos < 0) m_sxStp.Send(m_stop);
            if (m_skipDay || double.IsNaN(m_hi) || tod <= Cfg.midday) return;
```

```
            int side = Bars.Close[0] > m_hi ? +1 : Bars.Close[0] < m_lo ? -1 : 0;
            if (side == 0 || side == Math.Sign(pos)) return;        // SAR: only a break against the current
position acts
            string why;
            if (!entryAllowed(out why)) { if (journalOn()) logSignal("A2-49", side, Bars.Close[0],
double.NaN, new Dictionary<string, string> { { "blocked", why } }, false); return; }
            double tick = tickSize(), entry = Bars.Close[0];
            double r = Math.Max(Cfg.disasterR * (m_hi - m_lo), Cfg.costFloorTicks * tick);   // floor rule
            if (overRiskBudget(r)) { if (journalOn()) trace(ETraceLevel.Journal, $"A2-49 {Bars.Time[0]:yyyy-
MM-dd HH:mm} skip: stop {r / tick:F0} ticks breaches risk budget {Cfg.riskBudgetFraction:P2}"); return; }
            m_stop = entry - side * r;
            if (side > 0) m_le.Send(Cfg.contracts); else m_se.Send(Cfg.contracts);   // reverses an open
opposite position
            if (journalOn())
            {
                var ctx = new Dictionary<string, string> { { "mHigh", m_hi.ToString("F2") }, { "mLow", m_lo.T
oString("F2") }, { "tradesToday", TradesToday.ToString() } };
                journalArm(side > 0 ? "A249LE" : "A249SE", entry, m_stop, double.NaN, ctx);
                logSignal("A2-49", side, entry, m_stop, ctx, true);
            }
        }

        private bool overRiskBudget(double stopDist) { return Cfg.contracts * stopDist * Bars.Info.BigPointVa
lue > equityAvailable(Cfg.riskBudgetFraction); }

        private TimeSpan flatAt(DateTime day)
        {
            _CmeHolidays.Rules.Day cme = _CmeHolidays.Rules.Get(day);
            TimeSpan f = Cfg.flat;
            if (cme.RthCloseEt.HasValue && cme.RthCloseEt.Value - Cfg.halfDayExitBefore < f) f = cme.RthClose
Et.Value - Cfg.halfDayExitBefore;
            return f;
        }
    }
}
```

3.5 Example 16 (`_Ex16SwingPullback`) — SW-02 Connors RSI(2) pullback with the AVWAP filter variant (long only, daily) (easy)

Features (direct). F-ROLL (§5.7), F-JRNL (§5.13), F-EVT (§5.14), M-AVWAP (§5.15).

What it does (from the file header):

> SW-02 Connors RSI(2) pullback with the AVWAP filter variant (long only, daily). Entry: close > SMA200 and RSI(2) < 10 (and close above the latest valid swing-low AVWAP) -> buy next open. Exit: close > SMA5, or after 5 daily bars. Disaster stop 3 x ATR(20) daily, floored. Chart: Data1 = @ES daily, RTH session template, dxFeed continuous UNADJUSTED (roll gaps distort SMA200: see pitfalls). Features: M-AVWAP (AvwapEvents = SwingLow -> AvwapManager.Latest), M-PA (forced on by the swing anchor), averageTrueRange, F-JRNL.

Code.

```
// Manual v4 example 16 - SW-02 Connors RSI(2) pullback with the AVWAP filter variant (long only, daily).
// Entry: close > SMA200 and RSI(2) < 10 (and close above the latest valid swing-low AVWAP) -> buy next open.
// Exit: close > SMA5, or after 5 daily bars. Disaster stop 3 x ATR(20) daily, floored.
// Chart: Data1 = @ES daily, RTH session template, dxFeed continuous UNADJUSTED (roll gaps distort SMA200:
see pitfalls).
// Features: M-AVWAP (AvwapEvents = SwingLow -> AvwapManager.Latest), M-PA (forced on by the swing anchor),
averageTrueRange, F-JRNL.
using System;
using System.Collections.Generic;
using PowerLanguage.Function;

namespace PowerLanguage.Strategy
{
    public class _Ex16SwingPullback : _CustomSignal
    {
        protected sealed class Sw02Config
        {
```

```
            public int contracts = 1;
            public int smaLong = 200, smaExit = 5, rsiLength = 2, maxHoldBars = 5, atrLength = 20;
            public double rsiEntry = 10, disasterAtr = 3.0;
            public bool avwapFilter = true;
            public int costFloorTicks = 14;
            public double riskBudgetFraction = 0.01;
        }
        protected Sw02Config Cfg { get; } = new Sw02Config();

        private IOrderMarket m_le, m_lx;
        private IOrderPriced m_lxStp;
        private double m_stop = double.NaN;
        private int m_entryBar = -1;

        public _Ex16SwingPullback(object _ctx) : base(_ctx) { }

        protected override void Create()
        {
            base.Create();
            RollDetection = ERollDetection.Series;
            AvwapEvents =
EAvwapAnchor.SwingLow;      // lazy: creates the AVWAP manager; swing anchors force PRICEACTION on (Data1)
            AvwapLimits.maxAgeSessions = 60;          // daily bars: one session per bar
            m_le = OrderCreator.MarketNextBar(new SOrderParameters(Contracts.UserSpecified, "SW02LE", EOrderA
ction.Buy));
            m_lx = OrderCreator.MarketNextBar(new SOrderParameters(Contracts.Default, "SW02LX",
EOrderAction.Sell));
            m_lxStp = OrderCreator.Stop(new SOrderParameters(Contracts.Default, "SW02LS", EOrderAction.Sell,
OrderExit.FromAll));
        }

        protected override void StartCalc()
        {
            base.StartCalc();
            m_stop = double.NaN; m_entryBar = -1;
        }

        protected override void CalcBar()
        {
            base.CalcBar();
            if (Bars.Status != EBarState.Close || Bars.CurrentBar <= Cfg.smaLong + 1) return;
            double c = Bars.Close[0];
            int pos = StrategyInfo.MarketPosition;
            if (pos > 0)
            {
                if (m_entryBar < 0) m_entryBar = Bars.CurrentBar;
                if (c > sma(Cfg.smaExit) || Bars.CurrentBar - m_entryBar >= Cfg.maxHoldBars) { m_lx.Send(); r
eturn; }
                m_lxStp.Send(m_stop);
                return;
            }
            m_entryBar = -1;
            if (rollOn(1)) return;
            if (!(c > sma(Cfg.smaLong)) || !(rsi(Cfg.rsiLength) < Cfg.rsiEntry)) return;
            double av = double.NaN;
            if (Cfg.avwapFilter)
            {
                _AvwapManager mgr = AvwapManager;
                _Avwap a = mgr != null ? mgr.Latest(EAvwapAnchor.SwingLow) : null;
                av = a != null && a.Valid ? a.Value : double.NaN;
                if (double.IsNaN(av) || c <= av) { if (journalOn()) logSignal("SW-02", +1, c, double.NaN,
new Dictionary<string, string> { { "avwap", double.IsNaN(av) ? "na" : av.ToString("F2") } }, false);
return; }
            }
            double tick = tickSize(), atr = averageTrueRange(Bars, Cfg.atrLength);
            if (!(atr > 0)) return;
            double r = Math.Max(Cfg.disasterAtr * atr, Cfg.costFloorTicks * tick);
            if (overRiskBudget(r)) { if (journalOn()) trace(ETraceLevel.Journal, $"SW-02 {Bars.Time[0]:yyyy-
MM-dd} skip: stop {r / tick:F0} ticks breaches risk budget {Cfg.riskBudgetFraction:P2}"); return; }
            m_stop = c - r;
            m_le.Send(Cfg.contracts);
            if (journalOn())
            {
                var ctx = new Dictionary<string, string> { { "rsi2", rsi(Cfg.rsiLength).ToString("F1") }, { "
atr20", atr.ToString("F2") }, { "avwap", double.IsNaN(av) ? "off" : av.ToString("F2") } };
                journalArm("SW02LE", c, m_stop, double.NaN, ctx);
                logSignal("SW-02", +1, c, m_stop, ctx, true);
```

```
            }
        }

        private double sma(int n) { double s = 0; for (int i = 0; i < n; i++) s += Bars.Close[i]; return s /
n; }

        /// <summary>Cutler RSI (simple averages) over n closes; 50 when flat.</summary>
        private double rsi(int n)
        {
            double up = 0, dn = 0;
            for (int i = 0; i < n; i++) { double d = Bars.Close[i] - Bars.Close[i + 1]; if (d > 0) up += d; e
lse dn -= d; }
            return up + dn == 0 ? 50.0 : 100.0 * up / (up + dn);
        }

        private bool overRiskBudget(double stopDist) { return Cfg.contracts * stopDist * Bars.Info.BigPointVa
lue > equityAvailable(Cfg.riskBudgetFraction); }
    }
}
```

3.6 Example 3 (`_Ex03PeriodAvwapLayer`) — AV-08 week/month VWAP regime layer (easy–medium)

Features (direct). F-ROLL (§5.7), F-CAL (§5.8), F-VWAP (§5.10), F-JRNL (§5.13), F-EVT (§5.14).

Catalogue rules (AV-08). A layer, not a strategy: `aboveWTD && aboveMTD` allows longs only, `belowWTD && belowMTD` allows shorts only. It is applied to host events (PS-01, PS-07, MT-01, AV-02). Sweep: {WTD only, WTD+MTD, MTD+QTD}. Multi-day anchors reset at rolls.

Chart. @ES 1m, dxFeed unadjusted, 24h Globex template. The class derives from `_VolumeProfileFuturesBase` and sets the two VWAP slots on Data1 in `Create`: `Vwap1Type = EVwapType.Week`, `Vwap2Type = EVwapType.Month`. `StartCalc` throws if they are changed.

Walk-through.

- Only the **long** side of the layer is coded: `layerLong` = both slots active, close > WTD VWAP and close > MTD VWAP, and both periods "clean".
- A period is clean only if no roll happened since the week start (Monday Globex date) or the month start, because the Week/Month VWAPs do **not** reset at a roll. RollDetection is Series.
- The host is a close above the prior 20-bar high between 09:45 and 15:00 ET, with a 16-tick stop (above the 14-tick cost floor), flat at 15:55, or at `RthCloseEt` − 5 min on an early-close day (`flatAt`). **It is illustrative only, not a catalogue rule**: it exists so the layer has events to filter. The 16-tick stop is above the floor; 1 contract at $200 risk passes the 1% check only with equity ≥ $20,000, otherwise the signal is skipped and journaled.
- Host signals the layer blocks are logged with `logSignal(..., false)`, so the ablation (gated vs ungated) can be run from `MCEvents` without a second backtest.

Pitfalls.

- AV-08's test is a layer ablation on real hosts. Don't read anything into the illustrative host.
- The Week VWAP slot starts at the week's first bar; the catalogue's A2-07 merge anchors at Sunday 18:00. On a Globex chart those are the same bar.
- Not coded: the short side (`belowWTD && belowMTD`), the WTD-only arm and the MTD+QTD arm (`EVwapType.Quarter`).
- **Coding choice (labelled):** `wtdClean`/`mtdClean` switch the layer off for the rest of a week or month that holds a roll. The catalogue says multi-day anchors reset at rolls. The other reading is to re-anchor both VWAPs at `rollOn(1)`; it is not coded (Appendix F).

Full strategy — `_Ex03PeriodAvwapLayer.Strategy.CS`:

```
// Manual v4 example 3 - AV-08 week-open / month-open VWAP regime layer (aboveWTD && aboveMTD for longs
only).
// The host trigger below is ILLUSTRATIVE ONLY (not a catalogue rule): AV-08 is a layer for PS-01 / PS-07 /
```

```
MT-01 / AV-02.
// Chart: @ES 1m, dxFeed continuous UNADJUSTED, 24h CME Globex template.
using System;
using System.Collections.Generic;
using PowerLanguage.Function;

namespace PowerLanguage.Strategy
{
    public class _Ex03PeriodAvwapLayer : _VolumeProfileFuturesBase
    {
        protected sealed class Av08Config
        {
            public int contracts = 1;
            public int hostLookback = 20;                       // illustrative host: close above the prior
20-bar high
            public TimeSpan from = new TimeSpan(9, 45, 0), to = new TimeSpan(15, 0, 0), flat = new TimeSpan(1
5, 55, 0);
            public TimeSpan halfDayExitBefore = new TimeSpan(0, 5, 0);   // early-close days: flat at
RthCloseEt - 5 min
            public double stopTicks = 16;                       // illustrative host stop (>= the ES cost
floor of 14 ticks)
            public double riskBudgetFraction = 0.01;            // Ettienne, 2026-10-23: skip if q x stop x
$50 > 1% of equity (16 ticks, 1 ct = $200: equity >= $20,000)
        }
        protected Av08Config Cfg { get; } = new Av08Config();

        private IOrderMarket m_le, m_lx;
        private IOrderPriced m_lxStp;
        private DateTime m_lastRollDay = DateTime.MinValue;
        private double m_stop = double.NaN;

        public _Ex03PeriodAvwapLayer(object _ctx) : base(_ctx) { }

        protected override void Create()
        {
            base.Create();
            RollDetection = ERollDetection.Series;
            m_le = OrderCreator.MarketNextBar(new SOrderParameters(Contracts.UserSpecified, "AV08LE", EOrderA
ction.Buy));
            m_lx = OrderCreator.MarketNextBar(new SOrderParameters(Contracts.Default, "AV08LX",
EOrderAction.Sell));
            m_lxStp = OrderCreator.Stop(new SOrderParameters(Contracts.Default, "AV08LS", EOrderAction.Sell,
OrderExit.FromAll));
            // The VWAP slots are [Input]s of the VP base: these are defaults, the Format Signal dialog can
override them.
            Vwap1Type = EVwapType.Week;  Vwap1DataStream = 1;   // WTD: resets on the first bar of a new
Monday-start Globex week
            Vwap2Type = EVwapType.Month; Vwap2DataStream = 1;   // MTD
            Vwap3Type = EVwapType.Disabled;
        }

        protected override void StartCalc()
        {
            base.StartCalc();
            if (Vwap1Type != EVwapType.Week || Vwap2Type != EVwapType.Month || Vwap1DataStream != 1 || Vwap2D
ataStream != 1)
                throw new ArgumentException("_Ex03PeriodAvwapLayer: needs Vwap1Type = Week and Vwap2Type =
Month on Data1");
            m_lastRollDay = DateTime.MinValue; m_stop = double.NaN;
        }

        protected override void CalcBar()
        {
            base.CalcBar();
            if (Bars.Status != EBarState.Close) return;
            DateTime t = Bars.Time[0], day = GlobexDate(t);
            TimeSpan tod = ToEastern(t).TimeOfDay;
            if (rollOn(1)) m_lastRollDay = day;
            int pos = StrategyInfo.MarketPosition;
            TimeSpan flat = flatAt(day);

            if (pos > 0)
            {
                if (tod >= flat && tod < new TimeSpan(17, 0, 0)) m_lx.Send();
                else if (!double.IsNaN(m_stop)) m_lxStp.Send(m_stop);
                return;
            }
```

```
            if (tod < Cfg.from || tod >= Cfg.to || tod >= flat || Bars.CurrentBar <= Cfg.hostLookback + 1) re
turn;

            // --- the AV-08 layer (closed bar: the VWAP slots include this bar) ---
            double wtd = CurrentVwap1, mtd = CurrentVwap2, c = Bars.Close[0];
            DateTime weekStart = day.AddDays(-(((int)day.DayOfWeek + 6) % 7));   // Monday of the Globex week
            DateTime monthStart = new DateTime(day.Year, day.Month, 1);
            // Week/Month VWAPs do NOT reset at a roll. CODING CHOICE: the layer is off for the rest of a
week/month that holds a roll
            // (the catalogue says multi-day anchors reset at rolls; the alternative is to re-anchor the
VWAPs at rollOn(1))
            bool wtdClean = m_lastRollDay < weekStart, mtdClean = m_lastRollDay < monthStart;
            bool layerLong = Vwap1Active && Vwap2Active && wtdClean && mtdClean && c > wtd && c > mtd;

            // --- illustrative host event ---
            double hh = double.MinValue;
            for (int i = 1; i <= Cfg.hostLookback; i++) hh = Math.Max(hh, Bars.High[i]);
            if (!(c > hh)) return;

            var ctx = new Dictionary<string, string> { { "wtd", wtd.ToString("F2") }, { "mtd", mtd.ToString("
F2") },
                { "wtdClean", wtdClean.ToString() }, { "mtdClean", mtdClean.ToString() } };
            if (!layerLong) { if (journalOn()) logSignal("AV-08", +1, c, double.NaN, ctx, false); return; }
            m_stop = c - Cfg.stopTicks * tickSize();
            if (Cfg.contracts * (c - m_stop) * Bars.Info.BigPointValue > equityAvailable(Cfg.riskBudgetFracti
on))
            {
                if (journalOn()) { trace(ETraceLevel.Journal, $"AV-08 {t:yyyy-MM-dd HH:mm} skip: risk budget
{Cfg.riskBudgetFraction:P2}"); logSignal("AV-08", +1, c, m_stop, ctx, false); }
                m_stop = double.NaN; return;
            }
            if (journalOn()) { journalArm("AV08LE", c, m_stop, double.NaN, ctx); logSignal("AV-08", +1, c, m_
stop, ctx, true); }
            m_le.Send(Cfg.contracts);
            m_lxStp.Send(m_stop);
        }

        /// <summary>Flat time for a Globex day: Cfg.flat, or on a CME early-close day RthCloseEt -
halfDayExitBefore if earlier
        /// (_CmeHolidays.Rules.Get(day).RthCloseEt; same rule as A263Config.halfDayExitBefore).</summary>
        private TimeSpan flatAt(DateTime day)
        {
            _CmeHolidays.Rules.Day cme = _CmeHolidays.Rules.Get(day);
            TimeSpan f = Cfg.flat;
            if (cme.RthCloseEt.HasValue && cme.RthCloseEt.Value - Cfg.halfDayExitBefore < f) f = cme.RthClose
Et.Value - Cfg.halfDayExitBefore;
            return f;
        }
    }
}
```

Compile status: compiled — type-checked with Roslyn against the repo sources (`origin/main` a07257a) plus MC API stubs (stubs not authoritative). Not run in MultiCharts; no backtest results.

3.7 Example 14 (`_Ex14LateDayVwap`) — VW-05 late-day continuation (v1 15:00 arm): sign(15:00 price - prior RTH close) == sign(price - session VWAP) (easy–medium)

Features (direct). F-ROLL (§5.7), F-CAL (§5.8), F-VWAP (§5.10), F-JRNL (§5.13), F-EVT (§5.14), M-MTF (§5.17).

What it does (from the file header):

> VW-05 late-day continuation (v1 15:00 arm): sign(15:00 price - prior RTH close) == sign(price - session VWAP) -> 15:00 market in that direction, exit 15:55, stop 1 x ATR30m floored. (A2-06's 15:29 primary rule is a variant; not coded here.) Chart: Data1 = @ES 1m, Data2 = @ES 30m (24h Globex template), dxFeed continuous UNADJUSTED. Features: F-VWAP (Vwap1Type = Session -> CurrentVwap1, Vwap1Active), F-SESS (Session.priorRthClose), F-TM, F-CAL half-days.

Code.

```
// Manual v4 example 14 - VW-05 late-day continuation (v1 15:00 arm): sign(15:00 price - prior RTH close) ==
sign(price - session VWAP)
// -> 15:00 market in that direction, exit 15:55, stop 1 x ATR30m floored. (A2-06's 15:29 primary rule is a
variant; not coded here.)
// Chart: Data1 = @ES 1m, Data2 = @ES 30m (24h Globex template), dxFeed continuous UNADJUSTED.
// Features: F-VWAP (Vwap1Type = Session -> CurrentVwap1, Vwap1Active), F-SESS (Session.priorRthClose), F-TM,
F-CAL half-days.
using System;
using System.Collections.Generic;
using PowerLanguage.Function;

namespace PowerLanguage.Strategy
{
    public class _Ex14LateDayVwap : _VolumeProfileFuturesBase
    {
        protected sealed class Vw05Config
        {
            public int contracts = 1;
            public TimeSpan entryAt = new TimeSpan(15, 0, 0), flat = new TimeSpan(15, 55, 0);
            public TimeSpan halfDayExitBefore = new TimeSpan(0, 5, 0);
            public int atrLength = 14;
            public int costFloorTicks = 14;
            public double riskBudgetFraction = 0.01;
        }
        protected Vw05Config Cfg { get; } = new Vw05Config();

        private IOrderMarket m_le, m_se, m_lx, m_sx;
        private IOrderPriced m_lxStp, m_sxStp;
        private double m_stop = double.NaN;
        private DateTime m_tradedDay = DateTime.MinValue;

        public _Ex14LateDayVwap(object _ctx) : base(_ctx) { }

        protected override void Create()
        {
            base.Create();
            RollDetection = ERollDetection.Series;
            // VWAP slot 1 = session VWAP on Data1. These VP-base settings are [Input]s on main: set them in
code AND do not edit them in
            // the Format Signal dialog (see the F-VWAP pitfalls).
            Vwap1Type = EVwapType.Session; Vwap1DataStream = 1;
            m_le = OrderCreator.MarketNextBar(new SOrderParameters(Contracts.UserSpecified, "VW05LE", EOrderA
ction.Buy));
            m_se = OrderCreator.MarketNextBar(new SOrderParameters(Contracts.UserSpecified, "VW05SE", EOrderA
ction.SellShort));
            m_lx = OrderCreator.MarketNextBar(new SOrderParameters(Contracts.Default, "VW05LX",
EOrderAction.Sell));
            m_sx = OrderCreator.MarketNextBar(new SOrderParameters(Contracts.Default, "VW05SX",
EOrderAction.BuyToCover));
            m_lxStp = OrderCreator.Stop(new SOrderParameters(Contracts.Default, "VW05LS", EOrderAction.Sell,
OrderExit.FromAll));
            m_sxStp = OrderCreator.Stop(new SOrderParameters(Contracts.Default, "VW05SS", EOrderAction.BuyToC
over, OrderExit.FromAll));
        }

        protected override void StartCalc()
        {
            HtfStreams = new[] { 2 };
            base.StartCalc();
            if (Vwap1Type != EVwapType.Session) throw new
ArgumentException("_Ex14LateDayVwap: Vwap1Type must be Session");
            m_stop = double.NaN; m_tradedDay = DateTime.MinValue;
        }

        protected override void CalcBar()
        {
            base.CalcBar();
            if (Bars.Status != EBarState.Close) return;
            DateTime day = GlobexDate(Bars.Time[0]);
            TimeSpan tod = ToEastern(Bars.Time[0]).TimeOfDay, flat = flatAt(day);
            int pos = StrategyInfo.MarketPosition;
            if (pos != 0)
            {
                if (tod >= flat && tod < new TimeSpan(17, 0, 0)) { if (pos > 0) m_lx.Send(); else m_sx.Send()
```

```
; return; }
                if (pos > 0) m_lxStp.Send(m_stop); else m_sxStp.Send(m_stop);
                return;
            }
            if (rollOn(1)) { m_tradedDay = day; return; }                        // prior close is the old
contract on the splice day
            if (tod != Cfg.entryAt || m_tradedDay == day || tod >= flat) return;
            SessionStats s = Session;
            if (s == null || s.priorRthClose <= 0 || !Vwap1Active) return;
            double px = Bars.Close[0], vwap = CurrentVwap1;
            if (vwap <= 0) return;
            int a = Math.Sign(px - s.priorRthClose), b = Math.Sign(px - vwap);
            m_tradedDay = day;
            if (a == 0 || a != b) { if (journalOn()) logSignal("VW-05", a, px, double.NaN, new Dictionary<str
ing, string> { { "vwapSide", b.ToString() } }, false); return; }
            double tick = tickSize(), atr = atr30();
            if (double.IsNaN(atr)) return;
            double r = Math.Max(atr, Cfg.costFloorTicks * tick);
            if (overRiskBudget(r)) { if (journalOn()) trace(ETraceLevel.Journal, $"VW-05 {Bars.Time[0]:yyyy-
MM-dd} skip: stop {r / tick:F0} ticks breaches risk budget {Cfg.riskBudgetFraction:P2}"); return; }
            m_stop = px - a * r;
            if (a > 0) m_le.Send(Cfg.contracts); else m_se.Send(Cfg.contracts);
            if (journalOn())
            {
                var ctx = new Dictionary<string, string> { { "vwap", vwap.ToString("F2") }, { "priorClose",
s.priorRthClose.ToString("F2") }, { "atr30", atr.ToString("F2") } };
                journalArm(a > 0 ? "VW05LE" : "VW05SE", px, m_stop, double.NaN, ctx);
                logSignal("VW-05", a, px, m_stop, ctx, true);
            }
        }

        private double atr30()
        {
            int n = Cfg.atrLength;
            if (BarsOfData(2).CurrentBar < n + 2) return double.NaN;
            double sum = 0;
            for (int k = 0; k < n; k++)
            {
                DateTime t; double o, h, l, c, v, o1, h1, l1, pc, v1;
                HtfClosedBar(2, k, out t, out o, out h, out l, out c, out v);
                HtfClosedBar(2, k + 1, out t, out o1, out h1, out l1, out pc, out v1);
                sum += Math.Max(h, pc) - Math.Min(l, pc);
            }
            return sum / n;
        }

        private bool overRiskBudget(double stopDist) { return Cfg.contracts * stopDist * Bars.Info.BigPointVa
lue > equityAvailable(Cfg.riskBudgetFraction); }

        private TimeSpan flatAt(DateTime day)
        {
            _CmeHolidays.Rules.Day cme = _CmeHolidays.Rules.Get(day);
            TimeSpan f = Cfg.flat;
            if (cme.RthCloseEt.HasValue && cme.RthCloseEt.Value - Cfg.halfDayExitBefore < f) f = cme.RthClose
Et.Value - Cfg.halfDayExitBefore;
            return f;
        }
    }
}
```

3.8 Example 18 (`_Ex18FirstHourBreak`) — A2-56 first-hour breakout (core arm, no LBR setup filter): after 10:30 ET a buy stop at FH high + 1 tick and a (easy–medium)

Features (direct). F-ROLL (§5.7), F-CAL (§5.8), F-TM (§5.12), F-JRNL (§5.13).

What it does (from the file header):

> A2-56 first-hour breakout (core arm, no LBR setup filter): after 10:30 ET a buy stop at FH high + 1 tick and a sell stop at FH low - 1 tick until 15:00; stop at the other side of the first hour (+/- 1 tick), floored; flat 15:55. Chart: Data1 = @ES 1m (24h Globex template), dxFeed continuous

> UNADJUSTED. Features: F-TM (NoEntryBefore/After, FlatBy, MaxTradesPerDay, entryAllowed, mustFlatten), CME clock adapter, F-JRNL. Extension point: hostGate() is virtual; example 24 (RG-07) derives from this class and adds the exclusions.

Code.

```
// Manual v4 example 18 - A2-56 first-hour breakout (core arm, no LBR setup filter): after 10:30 ET a buy stop at FH high + 1 tick and a
// sell stop at FH low - 1 tick until 15:00; stop at the other side of the first hour (+/- 1 tick), floored; flat 15:55.
// Chart: Data1 = @ES 1m (24h Globex template), dxFeed continuous UNADJUSTED.
// Features: F-TM (NoEntryBefore/After, FlatBy, MaxTradesPerDay, entryAllowed, mustFlatten), CME clock adapter, F-JRNL.
// Extension point: hostGate() is virtual; example 24 (RG-07) derives from this class and adds the exclusions.
using System;
using System.Collections.Generic;
using PowerLanguage.Function;

namespace PowerLanguage.Strategy
{
    public class _Ex18FirstHourBreak : _CustomSignal
    {
        protected sealed class A256Config
        {
            public int contracts = 1;
            public TimeSpan open = new TimeSpan(9, 30, 0), fhEnd = new TimeSpan(10, 30, 0);
            public TimeSpan lastEntry = new TimeSpan(15, 0, 0), flat = new TimeSpan(15, 55, 0);
            public TimeSpan halfDayExitBefore = new TimeSpan(0, 5, 0);
            public int costFloorTicks = 14;
            public double riskBudgetFraction = 0.01;
        }
        protected A256Config Cfg { get; } = new A256Config();

        private IOrderPriced m_leStp, m_seStp, m_lxStp, m_sxStp;
        private IOrderMarket m_lx, m_sx;
        private _CmeSessionClock m_clock;
        private DateTime m_day = DateTime.MinValue;
        private double m_fhH = double.NaN, m_fhL = double.NaN, m_stop = double.NaN;
        private bool m_skip;

        public _Ex18FirstHourBreak(object _ctx) : base(_ctx) { }

        private _CmeSessionClock Clock { get { if (m_clock == null) m_clock = new _CmeSessionClock(resolveBar
TimeZone(Bars.Info)); return m_clock; } }
        protected override DateTime tradingDayKey(DateTime t) { return Clock.GlobexDate(t); }
        protected override TimeSpan sessionClock(DateTime t) { return Clock.ToEastern(t).TimeOfDay; }
        protected override bool isRthBar(DateTime t) { return Clock.IsRthValueBarEt(Clock.ToEastern(t)); }

        /// <summary>Extra entry gate for derived strategies (true = allowed). Called once per bar while the
stops would be sent.</summary>
        protected virtual bool hostGate(DateTime day, out string why) { why = null; return true; }

        protected override void Create()
        {
            base.Create();
            RollDetection = ERollDetection.Series;
            NoEntryBefore = Cfg.fhEnd; NoEntryAfter = Cfg.lastEntry; FlatBy = Cfg.flat; MaxTradesPerDay = 1;
            m_leStp = OrderCreator.Stop(new SOrderParameters(Contracts.UserSpecified, "A256LE",
EOrderAction.Buy));
            m_seStp = OrderCreator.Stop(new SOrderParameters(Contracts.UserSpecified, "A256SE",
EOrderAction.SellShort));
            m_lx = OrderCreator.MarketNextBar(new SOrderParameters(Contracts.Default, "A256LX",
EOrderAction.Sell));
            m_sx = OrderCreator.MarketNextBar(new SOrderParameters(Contracts.Default, "A256SX",
EOrderAction.BuyToCover));
            m_lxStp = OrderCreator.Stop(new SOrderParameters(Contracts.Default, "A256LS", EOrderAction.Sell,
OrderExit.FromAll));
            m_sxStp = OrderCreator.Stop(new SOrderParameters(Contracts.Default, "A256SS", EOrderAction.BuyToC
over, OrderExit.FromAll));
        }

        protected override void StartCalc()
        {
            m_clock = null;
```

```
            base.StartCalc();
            m_day = DateTime.MinValue; m_fhH = m_fhL = m_stop = double.NaN; m_skip = false;
        }

        protected override void CalcBar()
        {
            base.CalcBar();
            if (Bars.Status != EBarState.Close) return;
            DateTime day = tradingDayKey(Bars.Time[0]);
            if (day != m_day) { m_day = day; m_fhH = m_fhL = double.NaN; m_skip = false; FlatBy = flatAt(day)
; }
            if (rollOn(1)) m_skip = true;
            TimeSpan tod = sessionClock(Bars.Time[0]);
            int pos = StrategyInfo.MarketPosition;
            if (mustFlatten()) { if (pos > 0) m_lx.Send(); else if (pos < 0) m_sx.Send(); return; }
            if (tod > Cfg.open && tod <= Cfg.fhEnd && isRthBar(Bars.Time[0]))
            {
                m_fhH = double.IsNaN(m_fhH) ? Bars.High[0] : Math.Max(m_fhH, Bars.High[0]);
                m_fhL = double.IsNaN(m_fhL) ? Bars.Low[0] : Math.Min(m_fhL, Bars.Low[0]);
            }
            double tick = tickSize();
            if (pos != 0)
            {
                if (double.IsNaN(m_stop)) m_stop = pos > 0 ? m_fhL - tick : m_fhH + tick;
                double e = StrategyInfo.AvgEntryPrice;                                    // floor from the
actual fill
                if (Math.Abs(e - m_stop) < Cfg.costFloorTicks * tick) m_stop = e - Math.Sign(pos) * Cfg.costF
loorTicks * tick;
                if (pos > 0) m_lxStp.Send(m_stop); else m_sxStp.Send(m_stop);
                return;
            }
            m_stop = double.NaN;
            if (m_skip || double.IsNaN(m_fhH) || tod <= Cfg.fhEnd) return;
            string why;
            if (!entryAllowed(out why) || !hostGate(day, out why)) return;
            double buy = m_fhH + tick, sell = m_fhL - tick;
            double r = Math.Max(buy - sell, Cfg.costFloorTicks *
tick);   // either side risks the first-hour range (+2 ticks), floored
            if (overRiskBudget(r)) { if (!m_skip && journalOn()) trace(ETraceLevel.Journal, $"A2-56
{day:yyyy-MM-dd} skip: stop {r / tick:F0} ticks breaches risk budget {Cfg.riskBudgetFraction:P2}"); m_skip =
true; return; }
            m_leStp.Send(buy, Cfg.contracts); m_seStp.Send(sell, Cfg.contracts);
            if (journalOn() && Bars.High[0] < buy && Bars.Low[0] > sell && tod == Cfg.fhEnd + TimeSpan.FromMi
nutes(1))
            {
                var ctx = new Dictionary<string, string> { { "fhHigh", m_fhH.ToString("F2") }, { "fhLow", m_f
hL.ToString("F2") } };
                journalArm("A256LE", buy, sell, double.NaN, ctx);
                journalArm("A256SE", sell, buy, double.NaN, ctx);
            }
        }

        private bool overRiskBudget(double stopDist) { return Cfg.contracts * stopDist * Bars.Info.BigPointVa
lue > equityAvailable(Cfg.riskBudgetFraction); }

        private TimeSpan flatAt(DateTime day)
        {
            _CmeHolidays.Rules.Day cme = _CmeHolidays.Rules.Get(day);
            TimeSpan f = Cfg.flat;
            if (cme.RthCloseEt.HasValue && cme.RthCloseEt.Value - Cfg.halfDayExitBefore < f) f = cme.RthClose
Et.Value - Cfg.halfDayExitBefore;
            return f;
        }
    }
}
```

3.9 Example 20 (`_Ex20EmaVwapPullback`) — DV-15 EMA20 pullback with a session-VWAP slope filter (easy–medium)

Features (direct). F-SESS (§5.6), F-ROLL (§5.7), F-CAL (§5.8), F-VWAP (§5.10), F-TM (§5.12), F-JRNL (§5.13), F-EVT (§5.14), M-DAILY (§5.19).

What it does (from the file header):

> DV-15 EMA20 pullback with a session-VWAP slope filter. Side = sign(close - EMA20) held for >= 3 closed 30m bars. On a touch of the EMA (bar range contains it) while the session-VWAP slope over 8 bars has the same sign with |slope| >= s x ATR(14, 30m) and price is on the same side of VWAP: a limit at the EMA for the next bar. Pitfall: stop/target are sent from the bar after the fill, so the fill bar itself is unprotected. Bracket 1:1 at f x ATR20d (DailyContext), floored at 14 ticks. RTH 10:00-15:00 entries, flat 15:55. s = 0.05, f = 0.15 (grid values from the catalogue, not tuned results). Chart: Data1 = @ES 30m (24h Globex template), dxFeed continuous UNADJUSTED. Features: F-VWAP (Vwap1Type/Vwap1DataStream set in code, CurrentVwap1, Vwap1Active), M-DAILY (NeedsDaily, Atr20d), F-TM (FlatBy).

Pitfalls.

- The fill bar is unprotected: the stop is first sent on the bar after the fill.

Code.

```
// Manual v4 example 20 - DV-15 EMA20 pullback with a session-VWAP slope filter.
// Side = sign(close - EMA20) held for >= 3 closed 30m bars. On a touch of the EMA (bar range contains it)
while the session-VWAP slope
// over 8 bars has the same sign with |slope| >= s x ATR(14, 30m) and price is on the same side of VWAP: a
limit at the EMA for the next bar.
// Pitfall: stop/target are sent from the bar after the fill, so the fill bar itself is unprotected.
// Bracket 1:1 at f x ATR20d (DailyContext), floored at 14 ticks. RTH 10:00-15:00 entries, flat 15:55. s =
0.05, f = 0.15 (grid values from
// the catalogue, not tuned results).
// Chart: Data1 = @ES 30m (24h Globex template), dxFeed continuous UNADJUSTED.
// Features: F-VWAP (Vwap1Type/Vwap1DataStream set in code, CurrentVwap1, Vwap1Active), M-DAILY (NeedsDaily,
Atr20d), F-TM (FlatBy).
using System;
using System.Collections.Generic;
using PowerLanguage.Function;

namespace PowerLanguage.Strategy
{
    public class _Ex20EmaVwapPullback : _VolumeProfileFuturesBase
    {
        protected sealed class Dv15Config
        {
            public int contracts = 1, emaLength = 20, sideBars = 3, slopeBars = 8, atrLength = 14;
            public double slopeAtr = 0.05, bracketAtr20d = 0.15;
            public TimeSpan firstEntry = new TimeSpan(10, 0, 0), lastEntry = new TimeSpan(15, 0, 0), flat = n
ew TimeSpan(15, 55, 0);
            public int costFloorTicks = 14;
            public double riskBudgetFraction = 0.01;
        }
        protected Dv15Config Cfg { get; } = new Dv15Config();

        private IOrderPriced m_leLmt, m_seLmt, m_lxStp, m_sxStp, m_lxLmt, m_sxLmt;
        private IOrderMarket m_lx, m_sx;
        private double m_ema = double.NaN, m_stop, m_target;
        private int m_sideRun, m_side, m_emaBars;
        private readonly List<double> m_vwap = new List<double>();   // closed-bar VWAP values, newest last
(session-reset aware)
        private DateTime m_vwapDay = DateTime.MinValue;

        public _Ex20EmaVwapPullback(object _ctx) : base(_ctx) { }

        protected override void Create()
        {
            base.Create();
            RollDetection = ERollDetection.Series;
            Vwap1Type = EVwapType.Session; Vwap1DataStream = 1;     // [Input]s on the base: set in code
            FlatBy = Cfg.flat;
            m_leLmt = OrderCreator.Limit(new SOrderParameters(Contracts.UserSpecified, "DV15LE",
EOrderAction.Buy));
            m_seLmt = OrderCreator.Limit(new SOrderParameters(Contracts.UserSpecified, "DV15SE",
EOrderAction.SellShort));
            m_lx = OrderCreator.MarketNextBar(new SOrderParameters(Contracts.Default, "DV15LX",
EOrderAction.Sell));
            m_sx = OrderCreator.MarketNextBar(new SOrderParameters(Contracts.Default, "DV15SX",
EOrderAction.BuyToCover));
            m_lxStp = OrderCreator.Stop(new SOrderParameters(Contracts.Default, "DV15LS", EOrderAction.Sell,
```

```
OrderExit.FromAll));
            m_sxStp = OrderCreator.Stop(new SOrderParameters(Contracts.Default, "DV15SS", EOrderAction.BuyToC
over, OrderExit.FromAll));
            m_lxLmt = OrderCreator.Limit(new SOrderParameters(Contracts.Default, "DV15LT",
EOrderAction.Sell, OrderExit.FromAll));
            m_sxLmt = OrderCreator.Limit(new SOrderParameters(Contracts.Default, "DV15ST", EOrderAction.BuyTo
Cover, OrderExit.FromAll));
        }

        protected override void StartCalc()
        {
            NeedsDaily = true;                                          // before base.StartCalc (M-DAILY is
built there)
            base.StartCalc();
            m_ema = double.NaN; m_emaBars = 0; m_sideRun = 0; m_side = 0; m_vwap.Clear(); m_vwapDay = DateTim
e.MinValue;
        }

        protected override void CalcBar()
        {
            base.CalcBar();
            if (Bars.Status != EBarState.Close) return;
            double c = Bars.Close[0], tick = tickSize();
            double k = 2.0 / (Cfg.emaLength + 1);
            m_ema = double.IsNaN(m_ema) ? c : m_ema + k * (c - m_ema); m_emaBars++;
            int prevSide = m_side, prevRun = m_sideRun;              // side state up to the previous bar
            int s = Math.Sign(c - m_ema);
            m_sideRun = s != 0 && s == m_side ? m_sideRun + 1 : 1; m_side = s;
            DateTime day = GlobexDate(Bars.Time[0]);
            if (day != m_vwapDay) { m_vwapDay = day; m_vwap.Clear(); }
            if (Vwap1Active) m_vwap.Add(CurrentVwap1);

            int pos = StrategyInfo.MarketPosition;
            if (pos != 0)
            {
                if (mustFlatten()) { if (pos > 0) m_lx.Send(); else m_sx.Send(); return; }
                if (pos > 0) { m_lxStp.Send(m_stop); m_lxLmt.Send(m_target); } else { m_sxStp.Send(m_stop); m
_sxLmt.Send(m_target); }
                return;
            }
            TimeSpan tod = ToEastern(Bars.Time[0]).TimeOfDay;
            if (!IsRth() || tod < Cfg.firstEntry || tod > Cfg.lastEntry || m_emaBars < Cfg.emaLength || rollO
n(1)) return;
            if (m_vwap.Count <= Cfg.slopeBars || DailyContext == null || !DailyContext.Valid) return;
            int side = prevRun >= Cfg.sideBars ? prevSide : 0;                                  // side
established BEFORE this bar
            if (side == 0 || !(Bars.Low[0] <= m_ema && Bars.High[0] >= m_ema)) return;                //
touch
            double vw = m_vwap[m_vwap.Count - 1], slope = vw - m_vwap[m_vwap.Count - 1 - Cfg.slopeBars];
            double atr = averageTrueRange(Bars, Cfg.atrLength);
            if (!(atr > 0) || Math.Sign(slope) != side || Math.Abs(slope) < Cfg.slopeAtr * atr ||
Math.Sign(c - vw) != side) return;
            double d = Math.Max(Cfg.bracketAtr20d * DailyContext.Atr20d, Cfg.costFloorTicks * tick);
            if (!(d > 0) || overRiskBudget(d)) { if (journalOn()) trace(ETraceLevel.Journal, $"DV-15
{Bars.Time[0]:yyyy-MM-dd HH:mm} skip: bracket {d / tick:F0} ticks breaches risk budget"); return; }
            double entry = Math.Round(m_ema / tick) * tick;
            m_stop = entry - side * d; m_target = entry + side * d;      // 1:1 bracket
            if (side > 0) m_leLmt.Send(entry, Cfg.contracts); else m_seLmt.Send(entry, Cfg.contracts);
            if (journalOn()) logSignal("DV-15", side, entry, m_stop, new Dictionary<string, string> { { "vwap
", vw.ToString("F2") }, { "slope", slope.ToString("F2") } }, true);
        }

        private bool overRiskBudget(double stopDist) { return Cfg.contracts * stopDist * Bars.Info.BigPointVa
lue > equityAvailable(Cfg.riskBudgetFraction); }
    }
}
```

3.10 Example 17 (`_Ex17OpenFadeHvGate`) — A2-64 intraday mean reversion to the RTH open, gated by A2-70 (HV20 off-switch with hysteresis) (easy–medium)

Features (direct). F-ROLL (§5.7), F-CAL (§5.8), F-TM (§5.12), F-JRNL (§5.13), M-DAILY (§5.19).

What it does (from the file header):

> A2-64 intraday mean reversion to the RTH open, gated by A2-70 (HV20 off-switch with hysteresis). Entry: limit fades at R +/- m x ATR20d (R = 09:30 RTH open), 09:45-15:00 ET. Target R. Flat 15:55. Disaster stop 2m x ATR20d, floored. Gate (A2-70): HV20 (annualised close-to-close, daily) > thetaOff blocks entries; re-enabled when HV20 < 0.8 x thetaOff. thetaOff below is a PLACEHOLDER to be set by the A2-70 study; it is not a result. Chart: Data1 = @ES 1m (24h Globex template), dxFeed continuous UNADJUSTED. Daily context aggregated from Data1 (no daily stream). Features: F-SESS (Session.rthOpen), M-DAILY (NeedsDaily, DailyConfig.hvLengths, Atr20d, Hv(n)), F-TM (entryAllowed), F-JRNL.

Pitfalls.

- Contains PLACEHOLDER values (marked in the code); calibrate them before reading any result.

Code.

```
// Manual v4 example 17 - A2-64 intraday mean reversion to the RTH open, gated by A2-70 (HV20 off-switch with
hysteresis).
// Entry: limit fades at R +/- m x ATR20d (R = 09:30 RTH open), 09:45-15:00 ET. Target R. Flat 15:55.
Disaster stop 2m x ATR20d, floored.
// Gate (A2-70): HV20 (annualised close-to-close, daily) > thetaOff blocks entries; re-enabled when HV20 <
0.8 x thetaOff.
// thetaOff below is a PLACEHOLDER to be set by the A2-70 study; it is not a result.
// Chart: Data1 = @ES 1m (24h Globex template), dxFeed continuous UNADJUSTED. Daily context aggregated from
Data1 (no daily stream).
// Features: F-SESS (Session.rthOpen), M-DAILY (NeedsDaily, DailyConfig.hvLengths, Atr20d, Hv(n)), F-TM
(entryAllowed), F-JRNL.
using System;
using System.Collections.Generic;
using PowerLanguage.Function;

namespace PowerLanguage.Strategy
{
    public class _Ex17OpenFadeHvGate : _VolumeProfileFuturesBase
    {
        protected sealed class A264Config
        {
            public int contracts = 1;
            public TimeSpan from = new TimeSpan(9, 45, 0), to = new TimeSpan(15, 0, 0), flat = new TimeSpan(1
5, 55, 0);
            public TimeSpan halfDayExitBefore = new TimeSpan(0, 5, 0);
            public double m = 0.5;                  // band multiple of ATR20d (A2-64 sweep parameter)
            public int atrLength = 20, hvLength = 20;
            public double thetaOff = 0.30;          // PLACEHOLDER (annualised HV); A2-70 sets it
            public double reEnable = 0.8;
            public int costFloorTicks = 14;
            public double riskBudgetFraction = 0.01;
        }
        protected A264Config Cfg { get; } = new A264Config();

        private IOrderPriced m_leLmt, m_seLmt, m_lxStp, m_sxStp, m_lxLmt, m_sxLmt;
        private IOrderMarket m_lx, m_sx;
        private DateTime m_lastDaily = DateTime.MinValue, m_day = DateTime.MinValue;
        private bool m_gateOff;
        private double m_atr = double.NaN, m_hv = double.NaN, m_stop = double.NaN;
        private bool m_traded;
        private double m_r = double.NaN;

        public _Ex17OpenFadeHvGate(object _ctx) : base(_ctx) { }

        protected override void Create()
        {
            base.Create();
            RollDetection = ERollDetection.Series;
            MaxTradesPerDay = 1;
            m_leLmt = OrderCreator.Limit(new SOrderParameters(Contracts.UserSpecified, "A264LE",
EOrderAction.Buy));
            m_seLmt = OrderCreator.Limit(new SOrderParameters(Contracts.UserSpecified, "A264SE",
EOrderAction.SellShort));
            m_lx = OrderCreator.MarketNextBar(new SOrderParameters(Contracts.Default, "A264LX",
EOrderAction.Sell));
```

```
            m_sx = OrderCreator.MarketNextBar(new SOrderParameters(Contracts.Default, "A264SX",
EOrderAction.BuyToCover));
            m_lxStp = OrderCreator.Stop(new SOrderParameters(Contracts.Default, "A264LS", EOrderAction.Sell,
OrderExit.FromAll));
            m_sxStp = OrderCreator.Stop(new SOrderParameters(Contracts.Default, "A264SS", EOrderAction.BuyToC
over, OrderExit.FromAll));
            m_lxLmt = OrderCreator.Limit(new SOrderParameters(Contracts.Default, "A264LT",
EOrderAction.Sell, OrderExit.FromAll));
            m_sxLmt = OrderCreator.Limit(new SOrderParameters(Contracts.Default, "A264ST", EOrderAction.BuyTo
Cover, OrderExit.FromAll));
        }

        protected override void StartCalc()
        {
            NeedsDaily =
true;                       // M-DAILY, dataStream 0 = aggregate Data1 per Globex day (roll-adjusted)
            DailyConfig.atrLength = Cfg.atrLength; DailyConfig.hvLengths = new[] { Cfg.hvLength };
            base.StartCalc();
            m_lastDaily = m_day = DateTime.MinValue; m_gateOff = false; m_atr = m_hv = m_stop = double.NaN; m
_traded = false;
        }

        protected override void CalcBar()
        {
            base.CalcBar();
            if (Bars.Status != EBarState.Close) return;
            stepDaily();
            DateTime day = GlobexDate(Bars.Time[0]);
            if (day != m_day) { m_day = day; m_traded = false; }
            TimeSpan tod = ToEastern(Bars.Time[0]).TimeOfDay, flat = flatAt(day);
            SessionStats s = Session;
            int pos = StrategyInfo.MarketPosition;
            if (pos != 0)
            {
                m_traded = true;
                if (double.IsNaN(m_stop)) m_stop = StrategyInfo.AvgEntryPrice - Math.Sign(pos) * m_r;   //
first bar in the trade
                if (tod >= flat && tod < new TimeSpan(17, 0, 0)) { if (pos > 0) m_lx.Send(); else m_sx.Send()
; return; }
                if (pos > 0) { m_lxStp.Send(m_stop); m_lxLmt.Send(s.rthOpen); } else { m_sxStp.Send(m_stop);
m_sxLmt.Send(s.rthOpen); }
                return;
            }
            if (rollOn(1)) m_traded = true;
            if (m_traded || s == null || s.rthOpen <= 0 || !(m_atr > 0) || m_gateOff) return;
            if (tod < Cfg.from || tod >= Cfg.to || tod >= flat) return;
            string why;
            if (!entryAllowed(out why)) return;
            double tick = tickSize(), band = Cfg.m * m_atr;
            double r = Math.Max(2 * Cfg.m * m_atr, Cfg.costFloorTicks * tick);   // disaster stop 2m x ATR,
floored
            if (overRiskBudget(r)) { if (journalOn()) trace(ETraceLevel.Journal, $"A2-64 {day:yyyy-MM-dd}
skip: stop {r / tick:F0} ticks breaches risk budget {Cfg.riskBudgetFraction:P2}"); m_traded = true; return; }
            double buyAt = Math.Round((s.rthOpen - band) / tick) * tick, sellAt = Math.Round((s.rthOpen + ban
d) / tick) * tick;
            // both limits stand; whichever fills first sets the side (MaxTradesPerDay = 1 blocks a second
entry). The protective stop
            // is placed from the fill price on the first closed bar in the trade: a fill-bar spike is
unprotected (pitfall).
            m_leLmt.Send(buyAt, Cfg.contracts); m_seLmt.Send(sellAt, Cfg.contracts);
            m_r = r; m_stop = double.NaN;
        }

        /// <summary>New completed day in M-DAILY: ATR20d and HV20 from the (roll-adjusted) daily context,
and the A2-70 hysteresis gate.</summary>
        private void stepDaily()
        {
            _DailyContext d = DailyContext;
            if (d == null || !d.Valid || d.PrevDate == m_lastDaily) return;
            m_lastDaily = d.PrevDate;
            m_atr = d.Atr20d; m_hv = d.Hv(Cfg.hvLength);
            if (double.IsNaN(m_hv)) return;
            bool was = m_gateOff;
            if (!m_gateOff && m_hv > Cfg.thetaOff) m_gateOff = true;
            else if (m_gateOff && m_hv < Cfg.reEnable * Cfg.thetaOff) m_gateOff = false;
            if (was != m_gateOff && journalOn()) trace(ETraceLevel.Journal, $"A2-70 {d.PrevDate:yyyy-MM-dd}
HV20 {m_hv:P1} gate {(m_gateOff ? "OFF" : "ON")}");
```

```
        }

        private bool overRiskBudget(double stopDist) { return Cfg.contracts * stopDist * Bars.Info.BigPointVa
lue > equityAvailable(Cfg.riskBudgetFraction); }

        private TimeSpan flatAt(DateTime day)
        {
            _CmeHolidays.Rules.Day cme = _CmeHolidays.Rules.Get(day);
            TimeSpan f = Cfg.flat;
            if (cme.RthCloseEt.HasValue && cme.RthCloseEt.Value - Cfg.halfDayExitBefore < f) f = cme.RthClose
Et.Value - Cfg.halfDayExitBefore;
            return f;
        }
    }
}
```

3.11 Example 4 (`_Ex04OpeningRvGate`) — A2-50 same-slot relative-volume gate (medium)

Features (direct). F-ROLL (§5.7), F-CAL (§5.8), M-TOD (§5.18), M-EVENT (§5.20).

Catalogue rules (A2-50). RV30 = Σ volume(09:30–10:00) / SSB(N = 20) of the same 30-minute window. HIGH if RV30 ≥ q_hi, LOW if ≤ q_lo, else MID. q_hi/q_lo are the trailing 250-session 70th/30th percentiles, rolled daily and causal. Computed once at 10:00; 08:30-report days are a separate stratum. It is a module that routes hosts, so it places no orders.

Chart. Data1 = @ES **30m** (bars end 10:00, 10:30, …); the strategy throws on another bar size. dxFeed unadjusted, 24h Globex template.

Walk-through.

- M-TOD is enabled for Volume with `TodSessions = 20`, `TodMinSessions = 20` and bucket 0 (the host then uses the Data1 bar size, 30 minutes).
- `todHalfDay` is overridden so early-close days stay out of the baseline.
- At the 10:00 bar close, RV30 = that bar's volume / the mean of the last 20 values in `TodNormaliser.VolumePool(todMinuteOfSession(t))` (prior sessions only; `base.CalcBar()` runs first). Half-days and NoRth days get no state.
- RV30 is compared with the trailing 250 completed sessions' values (strictly earlier), then pushed. The state is HIGH, MID or LOW, exposed as `Rv30State` and journaled.
- The event engine is used only to **tag** the day as an 08:30-report day (or as unknown coverage), so the stratum can be split later.

Pitfalls.

- **Assumption (labelled):** nearest-rank percentiles.
- The first 250 sessions after the TOD warm-up have no state (warm-up).
- On a 1m chart the 09:30–10:00 sum would need 30 buckets; the 30m chart makes it one bucket. The 15-minute window arm needs a 15m chart.

Full strategy — `_Ex04OpeningRvGate.Strategy.CS`:

```
// Manual v4 example 4 - A2-50 same-slot relative-volume opening gate (module). Computes RV30 once per
session at the 10:00 ET
// bar close and journals HIGH / MID / LOW. It places NO orders: hosts read Rv30State (A2-50 routes hosts, it
is not a strategy).
// Chart: Data1 = @ES 30m (bars end 10:00, 10:30, ...), dxFeed continuous UNADJUSTED, 24h CME Globex
template. A 30m chart is used
// because a 30m TOD bucket on a 1m chart groups bar ENDS 09:31-10:00 into two buckets (09:31-09:59 and
10:00-10:29).
using System;
using System.Collections.Generic;
using System.Linq;
using PowerLanguage.Function;

namespace PowerLanguage.Strategy
```

```
{
    public class _Ex04OpeningRvGate : _CustomSignal
    {
        public enum ERvState { Unknown = 0, Low = 1, Mid = 2, High = 3 }

        protected sealed class A250Config
        {
            public TimeSpan slotEnd = new TimeSpan(10, 0, 0);   // ET close-stamp of the 09:30-10:00 bar
            public int ssbSessions = 20;                         // SSB(N = 20)
            public int pctSessions = 250;                        // trailing percentile window
            public double qHi = 0.70, qLo = 0.30;
        }
        protected A250Config Cfg { get; } = new A250Config();

        /// <summary>Today's gate state (Unknown before 10:00 ET, on warm-up, half-days and holidays).</
summary>
        public ERvState Rv30State { get; private set; }
        public double Rv30 { get; private set; }

        private _CmeSessionClock m_clock;
        private readonly List<double> m_history = new List<double>();   // prior sessions' RV30, oldest first
        private DateTime m_day = DateTime.MinValue;

        public _Ex04OpeningRvGate(object _ctx) : base(_ctx) { }

        private _CmeSessionClock Clock { get { if (m_clock == null) m_clock = new _CmeSessionClock(resolveBar
TimeZone(Bars.Info)); return m_clock; } }
        protected override DateTime tradingDayKey(DateTime t) { return Clock.GlobexDate(t); }
        protected override TimeSpan sessionClock(DateTime t) { return Clock.ToEastern(t).TimeOfDay; }
        protected override bool isRthBar(DateTime t) { return Clock.IsRthValueBarEt(Clock.ToEastern(t)); }
        /// <summary>Half-days are kept out of the same-slot pools (M-TOD) and out of the gate.</summary>
        protected override bool todHalfDay(DateTime sessionDate) { return _CmeHolidays.Rules.Get(sessionDate)
.RthCloseEt.HasValue; }

        protected override void Create()
        {
            base.Create();
            RollDetection = ERollDetection.Series;
            EventConfig.enabled =
true;               // only to tag 08:30-report days (reported as a separate stratum)
            EventConfig.classes = EEventClassMask.Macro;
        }

        protected override void StartCalc()
        {
            TodMetrics = ETodMetric.Volume;
            TodSessions = Cfg.ssbSessions;
            TodMinSessions = Cfg.ssbSessions;        // no SSB until 20 completed sessions
            TodBucketMinutes = 0;                    // 0 = Data1 bar size (30)
            m_clock = null;
            base.StartCalc();
            m_history.Clear(); m_day = DateTime.MinValue; Rv30State = ERvState.Unknown; Rv30 = double.NaN;
        }

        protected override void CalcBar()
        {
            base.CalcBar();                          // steps M-TOD first: the pool below never holds today
            if (Bars.Status != EBarState.Close) return;
            DateTime t = Bars.Time[0], day = tradingDayKey(t);
            if (day != m_day) { m_day = day; Rv30State = ERvState.Unknown; Rv30 = double.NaN; }
            if (Clock.ToEastern(t).TimeOfDay != Cfg.slotEnd) return;
            if (Bars.Info.Resolution.Size != 30) throw new ArgumentException("_Ex04OpeningRvGate: Data1 must
be 30-minute bars");

            _CmeHolidays.Rules.Day cme = _CmeHolidays.Rules.Get(day);
            if (cme.NoRth || cme.RthCloseEt.HasValue) { journal(day, "skip half-day/no-RTH", double.NaN, doub
le.NaN); return; }

            IList<double> pool = TodNormaliser.VolumePool(todMinuteOfSession(t));
            if (pool == null || pool.Count < Cfg.ssbSessions) { journal(day, "SSB warm-up", double.NaN, doubl
e.NaN); return; }
            double ssb = pool.Skip(pool.Count - Cfg.ssbSessions).Average();   // mean of the last 20
completed sessions' 09:30-10:00 volume
            if (!(ssb > 0)) return;
            double rv = Bars.Volume[0] / ssb;
            Rv30 = rv;
```

```
            if (m_history.Count >= Cfg.pctSessions)
            {
                var w = m_history.Skip(m_history.Count - Cfg.pctSessions).OrderBy(x => x).ToList();   //
prior 250 only (causal)
                double hi = nearestRank(w, Cfg.qHi), lo = nearestRank(w, Cfg.qLo);
                Rv30State = rv >= hi ? ERvState.High : rv <= lo ? ERvState.Low : ERvState.Mid;
                journal(day, Rv30State.ToString(), hi, lo);
            }
            else journal(day, "percentile warm-up " + m_history.Count + "/" + Cfg.pctSessions, double.NaN, do
uble.NaN);
            m_history.Add(rv);
            if (m_history.Count > Cfg.pctSessions + 5) m_history.RemoveAt(0);
        }

        /// <summary>Nearest-rank quantile of a sorted list (the catalogue does not fix the quantile method:
an assumption).</summary>
        private static double nearestRank(List<double> sorted, double q)
        {
            int k = (int)Math.Ceiling(q * sorted.Count) - 1;
            return sorted[Math.Max(0, Math.Min(sorted.Count - 1, k))];
        }

        private void journal(DateTime day, string state, double hi, double lo)
        {
            if (!journalOn()) return;
            string rep = "n/a";
            _EventCalendar ev = EventCalendar;
            if (ev != null && !ev.Disabled)
            {
                EEventKind hole;
                if (ev.CoverageBlocks(day, EEventClassMask.Macro, EEventTier.None, out hole)) rep = "unknown(
" + hole + ")";
                else rep = ev.EventsOn(day, EEventClassMask.Macro).Any(r => r.etTime.TimeOfDay ==
EventRules.T0830) ? "yes" : "no";
            }
            trace(ETraceLevel.Journal, $"A2-50 {day:yyyy-MM-dd} RV30({Rv30:F3}) qHi({hi:F3}) qLo({lo:F3})
state({state}) report0830({rep})");
        }
    }
}
```

Compile status: compiled — type-checked with Roslyn against the repo sources (`origin/main` a07257a) plus MC API stubs (stubs not authoritative). Not run in MultiCharts; no backtest results.

3.12 Example 5 (`_Ex05TwoTimeframeTurn`) — A2-57 60m turn + first 12m oversold (medium)

Features (direct). F-ROLL (§5.7), F-CAL (§5.8), F-JRNL (§5.13), F-EVT (§5.14), M-MTF (§5.17).

Catalogue rules (A2-57, base configuration). Trend: EMA(13) of 60m closes (closed bars, 24h). Up-turn = the EMA slope changes from ≤ 0 to > 0. Oscillator: raw %K(3) on 12m. Entry: the **first** 12m bar after the up-turn with %K(3) ≤ 20, then a buy stop at that bar's high + 1 tick, valid for the next two 12m bars; RTH 09:45–15:00 only; one trade per up-turn. Stop: the lower of the signal-bar low and the prior 12m low − 1 tick; break-even at +1R; then trail to protect 50% of the peak open profit. Exit: the 60m slope turns ≤ 0, the trail, or 15:55.

Chart. Data1 = @ES 1m, Data2 = @ES 12m, Data3 = @ES 60m (`HtfStreams = {2, 3}`), dxFeed unadjusted, 24h Globex template.

Walk-through.

- Each new closed 60m bar updates the EMA and its slope. An up-turn arms the "first oversold" search.
- Each new closed 12m bar computes %K(3). The first one at ≤ 20 inside 09:45–15:00 becomes the signal bar. An oversold bar outside that window **forfeits** the up-turn.
- The buy stop is sent on every 1m bar until two 12m bars have closed after the signal bar.

- The structural stop is min(signal-bar low, prior 12m low) − 1 tick. If trigger − stop is under the 14-tick cost floor (`costFloorTicks`), the stop is widened to trigger − 14 ticks. If the widened stop breaches the per-trade risk budget (`riskBudgetFraction` = 0.01, 1% of equity, decided by Ettienne 2026-10-23; PF-04's 0.5% cap is not the default for the floor rule), the signal is skipped and journaled, and that up-turn is used up. Size is never shrunk.
- In the trade, the stop moves to break-even at +1R, then to entry + 50% of the peak open profit. The position exits on a 60m slope ≤ 0 or at 15:55 (`RthCloseEt` − 5 min on an early-close day, `flatAt`).
- RollDetection is Series. `rollOn(1)` clears a pending trigger, so a buy stop never spans the splice. It also **re-seeds the 60m EMA**: `m_ema`, `m_slope`, `m_ema60Count` and `m_up` are reset and a journal line is written. No slope turn counts until 3 × 13 new closed 60m bars, so the roll gap can't make a false turn.

Pitfalls.

- **Assumptions (labelled):** the EMA warm-up (3 × 13 closed 60m bars before the first turn counts), and the reading that an oversold bar outside 09:45–15:00 uses up "the first OS".
- The protective stop is sent from the bar after the fill. On a 1m chart the fill bar itself is unprotected for that minute.
- **Rolls.** Without the re-seed, the first EMA updates on the new contract include the roll gap, and an up-turn in the slope can come from the splice alone. **Coding choice (labelled):** re-seed rather than ignore turns for 3 × 13 bars. The two differ only in whether the warm-up uses old-contract closes. During the warm-up the slope exit is off too, which doesn't matter while the position is flat by 15:55 before a session-start splice.
- Run MC Probe P2 (HTF bar-end stamping) first, as the catalogue says (§4.12 VERIFY).

Full strategy — `_Ex05TwoTimeframeTurn.Strategy.CS`:

```
// Manual v4 example 5 - A2-57 60m EMA(13) slope up-turn + FIRST 12m %K(3) <= 20, buy stop above the signal
bar (long arm, base config).
// Chart: Data1 = @ES 1m, Data2 = @ES 12m, Data3 = @ES 60m (same symbol and 24h Globex template), dxFeed
continuous UNADJUSTED.
using System;
using System.Collections.Generic;
using PowerLanguage.Function;

namespace PowerLanguage.Strategy
{
    public class _Ex05TwoTimeframeTurn : _CustomSignal
    {
        protected sealed class A257Config
        {
            public int contracts = 1;
            public int emaLength = 13, stochLength = 3, validBars = 2;
            public double osLevel = 20;
            public TimeSpan from = new TimeSpan(9, 45, 0), to = new TimeSpan(15, 0, 0), flat = new TimeSpan(1
5, 55, 0);
            public TimeSpan halfDayExitBefore = new TimeSpan(0, 5, 0);   // early-close days: flat at
RthCloseEt - 5 min
            public int costFloorTicks =
14;  // catalogue 2.1: ES cost floor; stop distance = max(structural, floor), size never shrunk
            public double riskBudgetFraction = 0.01;  // floor-rule risk budget: Ettienne, 2026-10-23: 1% of
equity (PF-04's 0.5% is not the default here)
            public double trailKeep = 0.5;   // after +1R: protect 50% of the peak open profit
        }
        protected A257Config Cfg { get; } = new A257Config();

        private IOrderPriced m_leStp, m_lxStp;
        private IOrderMarket m_lx;
        private _CmeSessionClock m_clock;
        private DateTime m_last12 = DateTime.MinValue, m_last60 = DateTime.MinValue;
        private double m_ema = double.NaN, m_slope = double.NaN;
        private int m_ema60Count;
        private bool m_up, m_osTaken;          // 60m state; the first OS after this up-turn was already seen
        private double m_trigger = double.NaN, m_planStop = double.NaN; private int m_validLeft;
        private double m_stop = double.NaN, m_entry = double.NaN, m_risk = double.NaN, m_peak = double.NaN;
        private bool m_slopeDown;
```

```
        /// <summary>Catalogue 2.1 / App F #9 (resolved): true when Cfg.contracts at this stop distance risk
more than the per-trade budget.</summary>
        private bool overRiskBudget(double stopDist) { return Cfg.contracts * stopDist * Bars.Info.BigPointVa
lue > equityAvailable(Cfg.riskBudgetFraction); }
        public _Ex05TwoTimeframeTurn(object _ctx) : base(_ctx) { }

        private _CmeSessionClock Clock { get { if (m_clock == null) m_clock = new _CmeSessionClock(resolveBar
TimeZone(Bars.Info)); return m_clock; } }
        protected override DateTime tradingDayKey(DateTime t) { return Clock.GlobexDate(t); }
        protected override TimeSpan sessionClock(DateTime t) { return Clock.ToEastern(t).TimeOfDay; }
        protected override bool isRthBar(DateTime t) { return Clock.IsRthValueBarEt(Clock.ToEastern(t)); }

        protected override void Create()
        {
            base.Create();
            RollDetection =
ERollDetection.Series;   // unadjusted series: rollOn(1) below needs detection on (default is Off)
            m_leStp = OrderCreator.Stop(new SOrderParameters(Contracts.UserSpecified, "A257LE",
EOrderAction.Buy));
            m_lxStp = OrderCreator.Stop(new SOrderParameters(Contracts.Default, "A257LS", EOrderAction.Sell,
OrderExit.FromAll));
            m_lx = OrderCreator.MarketNextBar(new SOrderParameters(Contracts.Default, "A257LX",
EOrderAction.Sell));
        }

        protected override void StartCalc()
        {
            HtfStreams = new[] { 2, 3 };
            m_clock = null;
            base.StartCalc();
            m_last12 = m_last60 = DateTime.MinValue; m_ema = m_slope = double.NaN; m_ema60Count = 0;
            m_up = m_osTaken = m_slopeDown = false; m_trigger = m_planStop = double.NaN; m_validLeft = 0;
            m_stop = m_entry = m_risk = m_peak = double.NaN;
        }

        protected override void CalcBar()
        {
            base.CalcBar();
            if (Bars.Status != EBarState.Close) return;
            TimeSpan tod = Clock.ToEastern(Bars.Time[0]).TimeOfDay;
            TimeSpan flat = flatAt(tradingDayKey(Bars.Time[0]));
            bool rthWindow = tod >= Cfg.from && tod < Cfg.to && tod < flat;

            if (rollOn(1))
            {
                m_trigger = double.NaN; m_validLeft = 0;   // never carry a level across the splice
                // re-seed the 60m EMA from the new contract: no slope turn counts until 3 x emaLength new
closed 60m bars
                m_ema = m_slope = double.NaN; m_ema60Count = 0; m_up = false;
                if (journalOn()) trace(ETraceLevel.Journal,
$"A2-57 roll {Bars.Time[0]:yyyy-MM-dd HH:mm}: 60m EMA re-seeded");
            }
            step60();
            bool new12 = step12(rthWindow);

            int pos = StrategyInfo.MarketPosition;
            if (pos > 0) { manage(tod, flat); return; }
            m_stop = m_entry = m_risk = m_peak = double.NaN;
            if (new12 && m_validLeft > 0) m_validLeft--;                    // "valid for the next 2 12m
bars"
            if (m_validLeft > 0 && rthWindow && m_up && !double.IsNaN(m_trigger)) m_leStp.Send(m_trigger,
Cfg.contracts);
        }

        /// <summary>New CLOSED 60m bar: update EMA(13) and the slope state.</summary>
        private void step60()
        {
            DateTime t; double o, h, l, c, v;
            HtfClosedBar(3, 0, out t, out o, out h, out l, out c, out v);
            if (t == m_last60) return;
            m_last60 = t;
            double prevEma = m_ema, prevSlope = m_slope, a = 2.0 / (Cfg.emaLength + 1);
            m_ema = double.IsNaN(m_ema) ? c : m_ema + a * (c - m_ema);
            m_ema60Count++;
            if (double.IsNaN(prevEma) || m_ema60Count < 3 * Cfg.emaLength) return;   // EMA warm-up (3 x
length: an assumption)
            m_slope = m_ema - prevEma;
```

```
            bool turnUp = !double.IsNaN(prevSlope) && prevSlope <= 0 && m_slope > 0;
            if (turnUp) { m_up = true; m_osTaken = false; if (journalOn()) trace(ETraceLevel.Journal,
$"A2-57 60m up-turn {t:yyyy-MM-dd HH:mm} ema({m_ema:F2})"); }
            if (m_slope <= 0) { m_up = false; m_slopeDown = true; m_trigger = double.NaN; m_validLeft = 0; }
else m_slopeDown = false;
        }

        /// <summary>New CLOSED 12m bar: the first %K(3) &lt;= 20 after the up-turn arms the buy stop.
Returns true on a new 12m bar.</summary>
        private bool step12(bool rthWindow)
        {
            DateTime t; double o, h, l, c, v;
            HtfClosedBar(2, 0, out t, out o, out h, out l, out c, out v);
            if (t == m_last12) return false;
            m_last12 = t;
            if (!m_up || m_osTaken || BarsOfData(2).CurrentBar <= Cfg.stochLength + 1) return true;
            double hh = double.MinValue, ll = double.MaxValue, prevLow = double.NaN;
            for (int k = 0; k < Cfg.stochLength; k++)
            {
                DateTime tk; double ok, hk, lk, ck, vk;
                HtfClosedBar(2, k, out tk, out ok, out hk, out lk, out ck, out vk);
                hh = Math.Max(hh, hk); ll = Math.Min(ll, lk);
                if (k == 1) prevLow = lk;
            }
            double pctK = hh > ll ? 100.0 * (c - ll) / (hh - ll) : 50.0;
            if (pctK > Cfg.osLevel) return true;
            m_osTaken = true;                       // FIRST oversold only: later ones are ignored (the key
rule)
            if (!rthWindow) { if (journalOn()) trace(ETraceLevel.Journal, $"A2-57 first OS {t:HH:mm} outside
09:45-15:00: turn forfeited"); return true; }
            double tick = tickSize();
            double trigger = h + tick, planStop = Math.Min(l, double.IsNaN(prevLow) ? l : prevLow) - tick;
            if (trigger - planStop < Cfg.costFloorTicks * tick) planStop = trigger - Cfg.costFloorTicks * tic
k;   // catalogue 2.1: widen to the cost floor
            if (overRiskBudget(trigger - planStop))   // widened stop breaches the per-trade risk budget:
skipped and logged, not resized (this up-turn is used up)
            {
                if (journalOn()) trace(ETraceLevel.Journal,
$"A2-57 {t:yyyy-MM-dd HH:mm} skip: stop {(trigger - planStop) / tick:F0} ticks breaches risk budget
{Cfg.riskBudgetFraction:P2}");
                return true;
            }
            m_trigger = trigger; m_planStop = planStop;
            m_validLeft = Cfg.validBars + 1;        // decremented on this bar's new12 pass, leaving 2 full
12m bars
            if (journalOn())
            {
                var ctx = new Dictionary<string, string> { { "pctK", pctK.ToString("F1") }, { "ema60",
m_ema.ToString("F2") } };
                journalArm("A257LE", m_trigger, m_planStop, double.NaN, ctx);
                logSignal("A2-57", +1, m_trigger, m_planStop, ctx, true);
            }
            return true;
        }

        private void manage(TimeSpan tod, TimeSpan flat)
        {
            if (double.IsNaN(m_entry))
            {
                m_entry = StrategyInfo.AvgEntryPrice; m_stop = m_planStop; m_risk = m_entry - m_stop; m_peak
= m_entry;
                m_trigger = double.NaN; m_validLeft = 0;   // one trade per up-turn
            }
            if ((tod >= flat && tod < new TimeSpan(17, 0, 0)) || m_slopeDown) { m_lx.Send(); return; }
            m_peak = Math.Max(m_peak, Bars.High[0]);
            if (m_risk > 0 && m_peak - m_entry >= m_risk)
            {
                double trail = m_entry + Cfg.trailKeep * (m_peak - m_entry);   // >= break-even once +1R is
reached
                m_stop = Math.Max(m_stop, Math.Max(m_entry, trail));
            }
            m_lxStp.Send(m_stop);
        }

        /// <summary>Flat time for a Globex day: Cfg.flat, or on a CME early-close day RthCloseEt -
halfDayExitBefore if earlier
        /// (_CmeHolidays.Rules.Get(day).RthCloseEt; same rule as A263Config.halfDayExitBefore).</summary>
```

```
        private TimeSpan flatAt(DateTime day)
        {
            _CmeHolidays.Rules.Day cme = _CmeHolidays.Rules.Get(day);
            TimeSpan f = Cfg.flat;
            if (cme.RthCloseEt.HasValue && cme.RthCloseEt.Value - Cfg.halfDayExitBefore < f) f = cme.RthClose
Et.Value - Cfg.halfDayExitBefore;
            return f;
        }
    }
}
```

Compile status: compiled — type-checked with Roslyn against the repo sources (`origin/main` a07257a) plus MC API stubs (stubs not authoritative). Not run in MultiCharts; no backtest results.

3.13 Example 9 (`_Ex09OpenAtrBreakout`) — A2-63 open + k × ATR20d breakout with RegimeGate (medium)

Features (direct). F-JRNL (§5.13), M-DAILY (§5.19), _RegimeEngine (§5.30).

Catalogue rules (A2-63, core). Buy stop at O + k × ATR20d, O = the 09:30 ET RTH open; active 09:30–15:30; one entry per day; exit 15:55. Core: k 0.5, 30m bars, ATR 20. The G-R consumer `_RegimeGate` (C1) adds arms U (ungated), G and A, pre-registered in `specs/regime-consumer-prereg.md`; its test has **not** been run.

Chart. Data1 = @ES 30m (the book's size) or 1m, dxFeed unadjusted, 24h Globex template. `[Input] GateArm` 0 = U, 1 = G, 2 = A.

Walk-through. This is `_Test6` cut down to show the API; `_Test6` stays the pre-registered harness.

- `StartCalc` validates the arm, then calls `RegimeGate.ApplyArm(GateCfg, RegimeConfig)` **before** `base.StartCalc()`, because the engines are built there. It sets `NeedsDaily` and `DailyConfig.atrLength`.
- `AtrTercileTracker` gets one value per completed day (`daily.PrevDate`, `daily.Atr20d`).
- On the latch bar (`A263Core.IsLatchBar`, the first bar closing after 09:30 ET) it reads ATR and as-of date, asks `RegimeGate.Decide(arm, Regime, day, tercile)` and latches the open, threshold and gate decision in `A263Core.Latch`.
- With TraceLevel Journal it writes `RegimeGate.Line(...)` plus the core's suffix, and `journalArm` with `RegimeGate.TradeFields` when an allowed signal exists.
- `A263Core.Step` returns the stop price to send or the exit.

Pitfalls.

- Like `_Test6`, this example is a plain `_CustomSignal` that converts time with `ToZone(..., "Eastern Standard Time")`. Unlike `_Test6` (Globex key since PR #46) it does not override `tradingDayKey`, so M-DAILY days are calendar days on a Globex chart (**VERIFY**, §1.6 and Appendix F).
- RollDetection is Off here: the roll gap enters the daily ATR. With it on and the default `atrRestartOnRoll = true`, `Atr20d` is NaN for 20 sessions after each roll (§4.3). `_Test6` itself switched in PR #46 to `Table` detection with `atrRestartOnRoll = false` (ATR carried on, splice-day TR skipped), which is the closest match to the catalogue's "Atr20d, roll-gap removed"; copy those two `StartCalc` lines (§4.3) to match it. See Appendix F.
- Change the gate config only before `base.StartCalc()`; the hash (`RegimeGate.Hash`) records it.
- **Not coded:** the host H2 "disaster-stop-at-the-open" arm named in A2-63's gate line (a disaster stop at O). `A263Config` has no disaster stop, so the core has no stop and the cost floor doesn't apply.

Full strategy — `_Ex09OpenAtrBreakout.Strategy.CS`:

```
// Manual v4 example 9 - A2-63 long-only open + k x ATR20d breakout, exit 15:55 ET, with the _RegimeGate arm
switch (G-R consumer C1).
// Chart: Data1 = @ES 30m (book) or 1m, dxFeed continuous UNADJUSTED, 24h CME Globex template.
// The pre-registered harness is _Test6; this example is the same wiring cut down to show the API (A263Core
```

```
does the rules).
using System;
using System.Collections.Generic;
using PowerLanguage.Function;

namespace PowerLanguage.Strategy
{
    public class _Ex09OpenAtrBreakout : _CustomSignal
    {
        [Input] public int GateArm { get; set; }      // 0 = U (ungated), 1 = G, 2 = A (see RegimeGate pre-
registration)

        protected RegimeGateConfig GateCfg { get; } = new RegimeGateConfig();
        protected A263Config HostCfg { get; } = new A263Config();   // k 0.5, ATR 20, 09:30-15:30 active,
exit 15:55

        private IOrderPriced m_le; private IOrderMarket m_lx;
        private A263Core m_core; private AtrTercileTracker m_terc;
        private string m_hash;

        public _Ex09OpenAtrBreakout(object _ctx) : base(_ctx) { GateArm = 0; }

        protected override void Create()
        {
            base.Create();
            m_le = OrderCreator.Stop(new SOrderParameters(Contracts.UserSpecified, "A263LE",
EOrderAction.Buy));
            m_lx = OrderCreator.MarketNextBar(new SOrderParameters(Contracts.Default, "A263LX",
EOrderAction.Sell));
        }

        protected override void StartCalc()
        {
            if (GateArm < 0 || GateArm > 2) throw new ArgumentException("GateArm must be 0, 1 or 2");
            GateCfg.arm = (ERegimeGateArm)GateArm;
            string err = GateCfg.Validate(); if (err != null) throw new ArgumentException(err);
            RegimeGate.ApplyArm(GateCfg, RegimeConfig);   // BEFORE base.StartCalc: the engines are built
there
            NeedsDaily = true;                            // M-DAILY, lazily created by the base
            DailyConfig.atrLength = HostCfg.atrLength;
            base.StartCalc();
            m_core = new A263Core(HostCfg);
            m_terc = new AtrTercileTracker(RegimeConfig.atrPctLookback, RegimeConfig.K);
            m_hash = RegimeGate.Hash(RegimeConfig, GateCfg, HostCfg);
        }

        protected override void CalcBar()
        {
            base.CalcBar();
            if (Bars.Status != EBarState.Close) return;
            DateTime et = ToZone(Bars.Time[0], "Eastern Standard Time");
            _DailyContext daily = DailyContext;
            if (daily != null) m_terc.AddDay(daily.PrevDate, daily.Atr20d);
            bool isLong = StrategyInfo.MarketPosition > 0;

            if (m_core.IsLatchBar(et))   // first bar closing after 09:30 ET: O = its open
            {
                double atr = daily != null ? daily.Atr20d : double.NaN;      // RollDetection is Off here
(_Test6 uses Table + atrRestartOnRoll = false since PR #46): roll gaps enter the ATR
                DateTime asOf = daily != null ? daily.PrevDate : DateTime.MinValue;
                _CmeHolidays.Rules.Day cme = _CmeHolidays.Rules.Get(et.Date);
                RegimeGateDecision d = RegimeGate.Decide(GateCfg.arm, Regime, et.Date, m_terc.LastDate == asO
f ? m_terc.Tercile : -1);
                m_core.Latch(et, Bars.Open[0], atr, asOf, d.allow, cme.RthCloseEt, cme.NoRth);
                if (journalOn())
                {
                    trace(ETraceLevel.Journal, RegimeGate.Line(d, m_core.HasSignal, m_hash, GateCfg.placeboSe
ed) + m_core.Suffix());
                    if (d.allow && m_core.HasSignal) journalArm("A263LE", m_core.Threshold, double.NaN, doubl
e.NaN, RegimeGate.TradeFields(d, m_hash, GateCfg.placeboSeed));
                }
            }

            A263Action a = m_core.Step(et, isLong);
            if (a.exit) m_lx.Send();
            else if (a.sendStop) m_le.Send(a.stop, HostCfg.contracts);
        }
```

```
    }
}
```

Compile status: compiled — type-checked with Roslyn against the repo sources (`origin/main` a07257a) plus MC API stubs (stubs not authoritative). Not run in MultiCharts; no backtest results.

- **v4 note.** This v3 example still exposes `[Input]`s (house rule: protected config); kept unchanged for continuity.

3.14 Example 11 (`_Ex11OvernightBreak`) — PS-24 overnight high/low break-and-go at RTH (core arm: 2 closed 5m closes beyond the ON extreme, market entry) (medium)

Features (direct). F-PROF (§5.2), F-ROLL (§5.7), F-CAL (§5.8), F-JRNL (§5.13), F-EVT (§5.14), M-MTF (§5.17).

What it does (from the file header):

> PS-24 overnight high/low break-and-go at RTH (core arm: 2 closed 5m closes beyond the ON extreme, market entry). Chart: Data1 = @ES 1m, Data2 = @ES 5m (same symbol, 24h Globex template), dxFeed continuous UNADJUSTED. IBKR = broker only. Features: F-SESS (Session.ethHigh/ethLow, rthOpen), F-PROF V-score (ValueMigrationScore), F-TM, F-JRNL, roll guard, floor rule.

Code.

```
// Manual v4 example 11 - PS-24 overnight high/low break-and-go at RTH (core arm: 2 closed 5m closes beyond
the ON extreme, market entry).
// Chart: Data1 = @ES 1m, Data2 = @ES 5m (same symbol, 24h Globex template), dxFeed continuous UNADJUSTED.
IBKR = broker only.
// Features: F-SESS (Session.ethHigh/ethLow, rthOpen), F-PROF V-score (ValueMigrationScore), F-TM, F-JRNL,
roll guard, floor rule.
using System;
using System.Collections.Generic;
using PowerLanguage.Function;

namespace PowerLanguage.Strategy
{
    public class _Ex11OvernightBreak : _VolumeProfileFuturesBase
    {
        protected sealed class Ps24Config
        {
            public int contracts = 1;
            public TimeSpan from = new TimeSpan(9, 45, 0), to = new TimeSpan(11, 30, 0), flat = new
TimeSpan(15, 55, 0);
            public TimeSpan halfDayExitBefore = new TimeSpan(0, 5, 0);
            public double minOutsideAtr = 0.5;     // the ON extreme must be >= 0.5 x ATR5 outside the RTH
open
            public int deltaTicks = 1;             // delta: closes must be beyond the extreme by this many
ticks
            public int closesNeeded = 2;
            public double stopAtr = 0.5;           // stop 0.5 x ATR5 back inside the ON extreme
            public double targetR = 1.0;           // b (catalogue sweep {0.75, 1.5})
            public int atrLength = 14;
            public int costFloorTicks = 14;        // catalogue 2.1 cost floor; stop = max(structural,
floor), size never shrunk
            public double riskBudgetFraction = 0.01;
        }
        protected Ps24Config Cfg { get; } = new Ps24Config();

        private IOrderMarket m_le, m_se, m_lx, m_sx;
        private IOrderPriced m_lxStp, m_sxStp, m_lxLmt, m_sxLmt;
        private DateTime m_last5 = DateTime.MinValue, m_day = DateTime.MinValue;
        private int m_runUp, m_runDn; private bool m_tradedToday;
        private double m_stop = double.NaN, m_target = double.NaN;

        public _Ex11OvernightBreak(object _ctx) : base(_ctx) { }
```

```
        protected override void Create()
        {
            base.Create();
            RollDetection = ERollDetection.Series;
            m_le = OrderCreator.MarketNextBar(new SOrderParameters(Contracts.UserSpecified, "PS24LE", EOrderA
ction.Buy));
            m_se = OrderCreator.MarketNextBar(new SOrderParameters(Contracts.UserSpecified, "PS24SE", EOrderA
ction.SellShort));
            m_lx = OrderCreator.MarketNextBar(new SOrderParameters(Contracts.Default, "PS24LX",
EOrderAction.Sell));
            m_sx = OrderCreator.MarketNextBar(new SOrderParameters(Contracts.Default, "PS24SX",
EOrderAction.BuyToCover));
            m_lxStp = OrderCreator.Stop(new SOrderParameters(Contracts.Default, "PS24LS", EOrderAction.Sell,
OrderExit.FromAll));
            m_sxStp = OrderCreator.Stop(new SOrderParameters(Contracts.Default, "PS24SS", EOrderAction.BuyToC
over, OrderExit.FromAll));
            m_lxLmt = OrderCreator.Limit(new SOrderParameters(Contracts.Default, "PS24LT",
EOrderAction.Sell, OrderExit.FromAll));
            m_sxLmt = OrderCreator.Limit(new SOrderParameters(Contracts.Default, "PS24ST", EOrderAction.BuyTo
Cover, OrderExit.FromAll));
        }

        protected override void StartCalc()
        {
            HtfStreams = new[] { 2 };
            base.StartCalc();
            m_last5 = m_day = DateTime.MinValue; m_runUp = m_runDn = 0; m_tradedToday = false;
            m_stop = m_target = double.NaN;
        }

        protected override void CalcBar()
        {
            base.CalcBar();
            if (Bars.Status != EBarState.Close) return;
            DateTime day = GlobexDate(Bars.Time[0]);
            if (day != m_day) { m_day = day; m_runUp = m_runDn = 0; m_tradedToday = false; }
            if (rollOn(1)) { m_runUp = m_runDn = 0; m_tradedToday = true; }   // no ON level on the splice
day
            TimeSpan tod = ToEastern(Bars.Time[0]).TimeOfDay, flat = flatAt(day);
            int pos = StrategyInfo.MarketPosition;
            if (pos != 0)
            {
                if (tod >= flat && tod < new TimeSpan(17, 0, 0)) { if (pos > 0) m_lx.Send(); else m_sx.Send()
; return; }
                if (pos > 0) { m_lxStp.Send(m_stop); m_lxLmt.Send(m_target); } else { m_sxStp.Send(m_stop); m
_sxLmt.Send(m_target); }
                return;
            }

            DateTime t5; double o5, h5, l5, c5, v5;
            HtfClosedBar(2, 0, out t5, out o5, out h5, out l5, out c5, out v5);
            if (t5 == m_last5) return;
            m_last5 = t5;
            SessionStats s = Session;
            if (s == null || s.ethHigh <= 0 || s.ethLow <= 0 || s.rthOpen <= 0 || m_tradedToday) return;
            if (tod < Cfg.from || tod >= Cfg.to || tod >= flat) { m_runUp = m_runDn = 0; return; }
            double tick = tickSize(), atr = atr5();
            if (double.IsNaN(atr)) return;
            m_runUp = c5 > s.ethHigh + Cfg.deltaTicks * tick ? m_runUp + 1 : 0;
            m_runDn = c5 < s.ethLow - Cfg.deltaTicks * tick ? m_runDn + 1 : 0;
            int side = m_runUp >= Cfg.closesNeeded ? +1 : m_runDn >= Cfg.closesNeeded ? -1 : 0;
            if (side == 0) return;
            double level = side > 0 ? s.ethHigh : s.ethLow;
            if (side * (level - s.rthOpen) < Cfg.minOutsideAtr * atr) { m_runUp = m_runDn = 0;
return; }   // extreme too close to the open
            int? vs = ValueMigrationScore(1);
            if (!vs.HasValue || side * vs.Value < 0)
 // V-score opposed or unknown
            {
                if (journalOn()) logSignal("PS-24", side, level, double.NaN, new Dictionary<string, string>
{ { "vscore", vs.HasValue ? vs.Value.ToString() : "na" } }, false);
                m_runUp = m_runDn = 0; return;
            }
            double entry = Bars.Close[0], stop = level - side * Cfg.stopAtr * atr;
            if (side * (entry - stop) < Cfg.costFloorTicks * tick) stop = entry - side * Cfg.costFloorTicks
* tick;   // catalogue 2.1 floor
            double r = Math.Abs(entry - stop);
```

```
            if (overRiskBudget(r)) { if (journalOn()) trace(ETraceLevel.Journal, $"PS-24 {Bars.Time[0]:yyyy-
MM-dd HH:mm} skip: stop {r / tick:F0} ticks breaches risk budget {Cfg.riskBudgetFraction:P2}");
m_tradedToday = true; return; }
            m_stop = stop; m_target = entry + side * Cfg.targetR * r; m_tradedToday = true;
            if (side > 0) m_le.Send(Cfg.contracts); else m_se.Send(Cfg.contracts);
            if (journalOn())
            {
                var ctx = new Dictionary<string, string> { { "on", level.ToString("F2") }, { "atr5", atr.ToSt
ring("F2") }, { "vscore", vs.Value.ToString() } };
                journalArm(side > 0 ? "PS24LE" : "PS24SE", entry, m_stop, m_target, ctx);
                logSignal("PS-24", side, entry, m_stop, ctx, true);
            }
        }

        /// <summary>Mean true range of the last atrLength CLOSED 5m bars (simple mean, not Wilder), NaN in
warm-up.</summary>
        private double atr5()
        {
            int n = Cfg.atrLength;
            if (BarsOfData(2).CurrentBar < n + 2) return double.NaN;
            double sum = 0;
            for (int k = 0; k < n; k++)
            {
                DateTime t; double o, h, l, c, v, o1, h1, l1, pc, v1;
                HtfClosedBar(2, k, out t, out o, out h, out l, out c, out v);
                HtfClosedBar(2, k + 1, out t, out o1, out h1, out l1, out pc, out v1);
                sum += Math.Max(h, pc) - Math.Min(l, pc);
            }
            return sum / n;
        }

        private bool overRiskBudget(double stopDist) { return Cfg.contracts * stopDist * Bars.Info.BigPointVa
lue > equityAvailable(Cfg.riskBudgetFraction); }

        private TimeSpan flatAt(DateTime day)
        {
            _CmeHolidays.Rules.Day cme = _CmeHolidays.Rules.Get(day);
            TimeSpan f = Cfg.flat;
            if (cme.RthCloseEt.HasValue && cme.RthCloseEt.Value - Cfg.halfDayExitBefore < f) f = cme.RthClose
Et.Value - Cfg.halfDayExitBefore;
            return f;
        }
    }
}
```

3.15 Example 15 (`_Ex15NarrowIbBreak`) — PS-08 narrow initial balance breakout (core arm). IB width <= 33rd percentile of the last 20 sessions; (medium)

Features (direct). F-ROLL (§5.7), F-CAL (§5.8), F-JRNL (§5.13), F-EVT (§5.14).

What it does (from the file header):

> PS-08 narrow initial balance breakout (core arm). IB width <= 33rd percentile of the last 20 sessions; after 10:30 ET, 2 consecutive closed 5m bars beyond IB +/- delta -> market; stop back inside by 0.25 x IB (floored); target 1 x IB beyond. Chart: Data1 = @ES 5m (24h Globex template), dxFeed continuous UNADJUSTED. IbMinutes = 60 (default). Features: F-SESS (Session.ibHigh/ibLow/ibComplete, IbRange), F-CAL half-days, F-JRNL, roll guard, floor rule.

Code.

```
// Manual v4 example 15 - PS-08 narrow initial balance breakout (core arm). IB width <= 33rd percentile of
the last 20 sessions;
// after 10:30 ET, 2 consecutive closed 5m bars beyond IB +/- delta -> market; stop back inside by 0.25 x IB
(floored); target 1 x IB beyond.
// Chart: Data1 = @ES 5m (24h Globex template), dxFeed continuous UNADJUSTED. IbMinutes = 60 (default).
// Features: F-SESS (Session.ibHigh/ibLow/ibComplete, IbRange), F-CAL half-days, F-JRNL, roll guard, floor
rule.
using System;
using System.Collections.Generic;
using PowerLanguage.Function;
```

```
namespace PowerLanguage.Strategy
{
    public class _Ex15NarrowIbBreak : _VolumeProfileFuturesBase
    {
        protected sealed class Ps08Config
        {
            public int contracts = 1;
            public TimeSpan from = new TimeSpan(10, 30, 0), to = new TimeSpan(15, 0, 0), flat = new
TimeSpan(15, 55, 0);
            public TimeSpan halfDayExitBefore = new TimeSpan(0, 5, 0);
            public int lookback = 20;
            public double maxPercentile = 0.33;
            public int deltaTicks = 1, closesNeeded = 2;
            public double stopInsideIb = 0.25, targetIb = 1.0;
            public int costFloorTicks = 14;
            public double riskBudgetFraction = 0.01;
        }
        protected Ps08Config Cfg { get; } = new Ps08Config();

        private IOrderMarket m_le, m_se, m_lx, m_sx;
        private IOrderPriced m_lxStp, m_sxStp, m_lxLmt, m_sxLmt;
        private readonly List<double> m_ibHist = new List<double>();   // completed IB widths, oldest first
        private DateTime m_day = DateTime.MinValue;
        private bool m_ibLogged, m_narrow, m_done;
        private int m_up, m_dn;
        private double m_stop = double.NaN, m_target = double.NaN;

        public _Ex15NarrowIbBreak(object _ctx) : base(_ctx) { }

        protected override void Create()
        {
            base.Create();
            RollDetection = ERollDetection.Series;
            m_le = OrderCreator.MarketNextBar(new SOrderParameters(Contracts.UserSpecified, "PS08LE", EOrderA
ction.Buy));
            m_se = OrderCreator.MarketNextBar(new SOrderParameters(Contracts.UserSpecified, "PS08SE", EOrderA
ction.SellShort));
            m_lx = OrderCreator.MarketNextBar(new SOrderParameters(Contracts.Default, "PS08LX",
EOrderAction.Sell));
            m_sx = OrderCreator.MarketNextBar(new SOrderParameters(Contracts.Default, "PS08SX",
EOrderAction.BuyToCover));
            m_lxStp = OrderCreator.Stop(new SOrderParameters(Contracts.Default, "PS08LS", EOrderAction.Sell,
OrderExit.FromAll));
            m_sxStp = OrderCreator.Stop(new SOrderParameters(Contracts.Default, "PS08SS", EOrderAction.BuyToC
over, OrderExit.FromAll));
            m_lxLmt = OrderCreator.Limit(new SOrderParameters(Contracts.Default, "PS08LT",
EOrderAction.Sell, OrderExit.FromAll));
            m_sxLmt = OrderCreator.Limit(new SOrderParameters(Contracts.Default, "PS08ST", EOrderAction.BuyTo
Cover, OrderExit.FromAll));
        }

        protected override void StartCalc()
        {
            base.StartCalc();
            m_ibHist.Clear(); m_day = DateTime.MinValue; m_ibLogged = m_narrow = m_done = false; m_up = m_dn
= 0;
            m_stop = m_target = double.NaN;
        }

        protected override void CalcBar()
        {
            base.CalcBar();
            if (Bars.Status != EBarState.Close) return;
            DateTime day = GlobexDate(Bars.Time[0]);
            if (day != m_day) { m_day = day; m_ibLogged = m_narrow = m_done = false; m_up = m_dn = 0; }
            if (rollOn(1)) m_done = true;
            TimeSpan tod = ToEastern(Bars.Time[0]).TimeOfDay, flat = flatAt(day);
            int pos = StrategyInfo.MarketPosition;
            if (pos != 0)
            {
                if (tod >= flat && tod < new TimeSpan(17, 0, 0)) { if (pos > 0) m_lx.Send(); else m_sx.Send()
; return; }
                if (pos > 0) { m_lxStp.Send(m_stop); m_lxLmt.Send(m_target); } else { m_sxStp.Send(m_stop); m
_sxLmt.Send(m_target); }
                return;
            }
            SessionStats s = Session;
```

```
            if (s == null || !s.ibComplete) return;
            double ib = s.IbRange;
            if (!m_ibLogged)                                        // once per session, the first bar after
the IB completes
            {
                m_ibLogged = true;
                m_narrow = m_ibHist.Count >= Cfg.lookback && percentileRank(ib) <= Cfg.maxPercentile;
                m_ibHist.Add(ib); if (m_ibHist.Count > Cfg.lookback) m_ibHist.RemoveAt(0);
                if (journalOn()) trace(ETraceLevel.Journal, $"PS-08 {day:yyyy-MM-dd} IB {s.ibLow:F2}-
{s.ibHigh:F2} width {ib:F2} narrow({m_narrow})");
            }
            if (!m_narrow || m_done || tod < Cfg.from || tod >= Cfg.to || tod >= flat) return;
            double tick = tickSize();
            m_up = Bars.Close[0] > s.ibHigh + Cfg.deltaTicks * tick ? m_up + 1 : 0;
            m_dn = Bars.Close[0] < s.ibLow - Cfg.deltaTicks * tick ? m_dn + 1 : 0;
            int side = m_up >= Cfg.closesNeeded ? +1 : m_dn >= Cfg.closesNeeded ? -1 : 0;
            if (side == 0) return;
            m_done = true;                                          // one attempt per session
            string why;
            if (!hostGate(side, out why)) { if (journalOn()) trace(ETraceLevel.Journal, $"PS-08
{Bars.Time[0]:yyyy-MM-dd HH:mm} gate blocked side {side}: {why}"); return; }
            double edge = side > 0 ? s.ibHigh : s.ibLow, entry = Bars.Close[0];
            double stop = edge - side * Cfg.stopInsideIb * ib;
            if (side * (entry - stop) < Cfg.costFloorTicks * tick) stop = entry - side * Cfg.costFloorTicks
* tick;
            double r = Math.Abs(entry - stop);
            if (overRiskBudget(r)) { if (journalOn()) trace(ETraceLevel.Journal, $"PS-08 {Bars.Time[0]:yyyy-
MM-dd HH:mm} skip: stop {r / tick:F0} ticks breaches risk budget {Cfg.riskBudgetFraction:P2}"); return; }
            m_stop = stop; m_target = edge + side * Cfg.targetIb * ib;
            if (side > 0) m_le.Send(Cfg.contracts); else m_se.Send(Cfg.contracts);
            if (journalOn())
            {
                var ctx = new Dictionary<string, string> { { "ib", ib.ToString("F2") }, { "edge", edge.ToStri
ng("F2") } };
                journalArm(side > 0 ? "PS08LE" : "PS08SE", entry, m_stop, m_target, ctx);
                logSignal("PS-08", side, entry, m_stop, ctx, true);
            }
        }

        /// <summary>Fraction of the stored IB widths that are &lt;= w.</summary>
        /// <summary>Extra entry gate for derived strategies (example 21 / MT-03). true = allowed.</summary>
        protected virtual bool hostGate(int side, out string why) { why = null; return true; }

        private double percentileRank(double w)
        {
            int le = 0; foreach (double x in m_ibHist) if (x <= w) le++;
            return m_ibHist.Count == 0 ? 1.0 : (double)le / m_ibHist.Count;
        }

        private bool overRiskBudget(double stopDist) { return Cfg.contracts * stopDist * Bars.Info.BigPointVa
lue > equityAvailable(Cfg.riskBudgetFraction); }

        private TimeSpan flatAt(DateTime day)
        {
            _CmeHolidays.Rules.Day cme = _CmeHolidays.Rules.Get(day);
            TimeSpan f = Cfg.flat;
            if (cme.RthCloseEt.HasValue && cme.RthCloseEt.Value - Cfg.halfDayExitBefore < f) f = cme.RthClose
Et.Value - Cfg.halfDayExitBefore;
            return f;
        }
    }
}
```

3.16 Example 19 (`_Ex19OpenDrive`) — PS-07 open drive, continuation on the first higher low (core arm) (medium)

Features (direct). F-SESS (§5.6), F-ROLL (§5.7), F-CAL (§5.8), F-JRNL (§5.13), M-SWING (§5.16), M-TOD (§5.18).

What it does (from the file header):

> PS-07 open drive, continuation on the first higher low (core arm). Drive test (operationalised): the 09:30-09:45 range >= driveMult x the sum of the three same-slot TOD Range medians, and the 09:45 close in the outer third of that range in the drive direction. Entry: after the first PA pivot low labelled HL (long) / pivot high labelled LH (short) that confirms before 11:30, a stop order 1 tick beyond the confirming bar. Stop: the pivot -/+ 1 tick, floored. Exit: 11:30 time stop. One trade per day. Chart: Data1 = @ES 5m (24h Globex template), dxFeed continuous UNADJUSTED. Features: M-TOD (TodMetrics = Range, Median, Valid), M-PA (PriceActionConfig, NewPivotLow/High, LastLow/LastHigh, EPivotLabel), F-SESS.

Pitfalls.

- Contains PLACEHOLDER values (marked in the code); calibrate them before reading any result.

Code.

```
// Manual v4 example 19 - PS-07 open drive, continuation on the first higher low (core arm).
// Drive test (operationalised): the 09:30-09:45 range >= driveMult x the sum of the three same-slot TOD
Range medians, and the
// 09:45 close in the outer third of that range in the drive direction. Entry: after the first PA pivot low
labelled HL (long) / pivot
// high labelled LH (short) that confirms before 11:30, a stop order 1 tick beyond the confirming bar. Stop:
the pivot -/+ 1 tick, floored.
// Exit: 11:30 time stop. One trade per day.
// Chart: Data1 = @ES 5m (24h Globex template), dxFeed continuous UNADJUSTED.
// Features: M-TOD (TodMetrics = Range, Median, Valid), M-PA (PriceActionConfig, NewPivotLow/High, LastLow/
LastHigh, EPivotLabel), F-SESS.
using System;
using PowerLanguage.Function;

namespace PowerLanguage.Strategy
{
    public class _Ex19OpenDrive : _VolumeProfileFuturesBase
    {
        protected sealed class Ps07Config
        {
            public int contracts = 1;
            public double driveMult = 0.6;                  // PLACEHOLDER threshold, not a tested value
            public TimeSpan driveEnd = new TimeSpan(9, 45, 0), timeStop = new TimeSpan(11, 30, 0);
            public int costFloorTicks = 14;
            public double riskBudgetFraction = 0.01;
        }
        protected Ps07Config Cfg { get; } = new Ps07Config();

        private IOrderPriced m_leStp, m_seStp, m_lxStp, m_sxStp;
        private IOrderMarket m_lx, m_sx;
        private DateTime m_day = DateTime.MinValue;
        private double m_hi, m_lo, m_medSum, m_trigger = double.NaN, m_stop = double.NaN;
        private int m_slots, m_side;
        private bool m_done;

        public _Ex19OpenDrive(object _ctx) : base(_ctx) { }

        protected override void Create()
        {
            base.Create();
            RollDetection = ERollDetection.Series;
            TodMetrics = ETodMetric.Range; TodSessions = 20; TodMinSessions = 10; TodBucketMinutes = 5;
            PriceActionConfig.enabled = true;               // Futures preset applied by the base
            PriceActionConfig.dataStream = 1;
            m_leStp = OrderCreator.Stop(new SOrderParameters(Contracts.UserSpecified, "PS07LE",
EOrderAction.Buy));
            m_seStp = OrderCreator.Stop(new SOrderParameters(Contracts.UserSpecified, "PS07SE",
EOrderAction.SellShort));
            m_lx = OrderCreator.MarketNextBar(new SOrderParameters(Contracts.Default, "PS07LX",
EOrderAction.Sell));
            m_sx = OrderCreator.MarketNextBar(new SOrderParameters(Contracts.Default, "PS07SX",
EOrderAction.BuyToCover));
            m_lxStp = OrderCreator.Stop(new SOrderParameters(Contracts.Default, "PS07LS", EOrderAction.Sell,
OrderExit.FromAll));
            m_sxStp = OrderCreator.Stop(new SOrderParameters(Contracts.Default, "PS07SS", EOrderAction.BuyToC
over, OrderExit.FromAll));
        }
```

```
        protected override void StartCalc()
        {
            base.StartCalc();
            m_day = DateTime.MinValue; m_side = 0; m_done = false; m_trigger = m_stop = double.NaN;
        }

        protected override void CalcBar()
        {
            base.CalcBar();
            if (Bars.Status != EBarState.Close) return;
            DateTime day = GlobexDate(Bars.Time[0]);
            if (day != m_day) { m_day = day; m_hi = double.MinValue; m_lo = double.MaxValue; m_medSum = 0; m_
slots = 0; m_side = 0; m_done = false; m_trigger = double.NaN; }
            if (rollOn(1)) m_done = true;
            TimeSpan tod = ToEastern(Bars.Time[0]).TimeOfDay;
            int pos = StrategyInfo.MarketPosition;
            double tick = tickSize();
            if (pos != 0)
            {
                m_done = true;                                          // one trade per day: never re-arm
after this position
                if (tod >= Cfg.timeStop) { if (pos > 0) m_lx.Send(); else m_sx.Send(); return; }
                if (pos > 0) m_lxStp.Send(m_stop); else m_sxStp.Send(m_stop);
                return;
            }
            if (m_done || !IsRth()) return;
            if (tod > new TimeSpan(9, 30, 0) && tod <= Cfg.driveEnd)
            {
                m_hi = Math.Max(m_hi, Bars.High[0]); m_lo = Math.Min(m_lo, Bars.Low[0]);
                if (TodNormaliser != null && TodNormaliser.Valid(ETodMetric.Range)) { m_medSum += TodNormalis
er.Median(ETodMetric.Range); m_slots++; }
                if (tod == Cfg.driveEnd) classifyDrive();
                return;
            }
            if (m_side == 0 || tod >= Cfg.timeStop) return;
            _PriceActionEngine pa = PriceActionEngine;
            if (pa == null || !pa.IsWarm) return;
            if (double.IsNaN(m_trigger))
            {
                _Pivot p = m_side > 0 ? (pa.NewPivotLow ? pa.LastLow(0) : null) : (pa.NewPivotHigh ? pa.LastH
igh(0) : null);
                if (p == null || p.label != (m_side > 0 ? EPivotLabel.HL : EPivotLabel.LH)) return;
                m_trigger = m_side > 0 ? Bars.High[0] + tick : Bars.Low[0] - tick;
                m_stop = m_side > 0 ? p.price - tick : p.price + tick;
                if (m_side * (m_trigger - m_stop) < Cfg.costFloorTicks * tick) m_stop = m_trigger - m_side *
Cfg.costFloorTicks * tick;
                double r = Math.Abs(m_trigger - m_stop);
                if (overRiskBudget(r)) { if (journalOn()) trace(ETraceLevel.Journal,
$"PS-07 {day:yyyy-MM-dd} skip: stop {r / tick:F0} ticks breaches risk budget"); m_done = true; return; }
                if (journalOn()) journalArm(m_side > 0 ? "PS07LE" : "PS07SE", m_trigger, m_stop, double.NaN,
null);
            }
            if (m_side > 0) m_leStp.Send(m_trigger, Cfg.contracts); else m_seStp.Send(m_trigger, Cfg.contract
s);
        }


        private void classifyDrive()
        {
            if (m_slots < 3) return;                                    // TOD history not warm for all three
slots
            double range = m_hi - m_lo, c = Bars.Close[0];
            if (!(range > 0) || range < Cfg.driveMult * m_medSum) return;
            if (c >= m_hi - range / 3) m_side = +1; else if (c <= m_lo + range / 3) m_side = -1;
            if (m_side != 0 && journalOn()) trace(ETraceLevel.Journal, $"PS-07 {m_day:yyyy-MM-dd} drive
{(m_side > 0 ? "up" : "down")} range {range:F2} vs sum-median {m_medSum:F2}");
        }

        private bool overRiskBudget(double stopDist) { return Cfg.contracts * stopDist * Bars.Info.BigPointVa
lue > equityAvailable(Cfg.riskBudgetFraction); }
    }
}
```

3.17 Example 21 (`_Ex21CompositeValueGate`) — MT-03 composite-value gate on the example 15 host (PS-08 narrow-IB breakout) (medium)

Features (direct). F-DEV (§5.1), F-PROF (§5.2), F-COMP (§5.3). **Inherited from example 15:** F-ROLL, F-CAL, F-JRNL, F-EVT.

What it does (from the file header):

> MT-03 composite-value gate on the example 15 host (PS-08 narrow-IB breakout). Rule: long only if price >= the 20-session composite POC and the 20-session V-score >= 0; short mirrors (price <= POC, V-score <= 0). Chart: as example 15 (@ES 5m). The completed-session profiles come from the base's Own profiles (OwnProfileWindow = Rth), so the gate does not depend on the MC Volume Profile study settings. Features: F-VP (Composite, ValueMigrationScore, OwnProfileWindow, DevelopingSource), M-VSCORE.

Pitfalls.

- Module on example 15: compile both files; the host's rules, stops and settings apply unless overridden.

Code.

```
// Manual v4 example 21 - MT-03 composite-value gate on the example 15 host (PS-08 narrow-IB breakout).
// Rule: long only if price >= the 20-session composite POC and the 20-session V-score >= 0; short mirrors
(price <= POC, V-score <= 0).
// Chart: as example 15 (@ES 5m). The completed-session profiles come from the base's Own profiles
(OwnProfileWindow = Rth), so the gate
// does not depend on the MC Volume Profile study settings.
// Features: F-VP (Composite, ValueMigrationScore, OwnProfileWindow, DevelopingSource), M-VSCORE.
using System;
using PowerLanguage.Function;

namespace PowerLanguage.Strategy
{
    public class _Ex21CompositeValueGate : _Ex15NarrowIbBreak
    {
        private const int N = 20;
        public _Ex21CompositeValueGate(object _ctx) : base(_ctx) { }

        protected override void Create()
        {
            base.Create();
            DevelopingSource = EDevelopingSource.Own;   // [Input] on the base: set in code, not in the
Format dialog
            OwnProfileWindow = EOwnProfileWindow.Rth;
        }

        protected override bool hostGate(int side, out string why)
        {
            why = null;
            VpExtra comp = Composite(1, N);
            int? v = ValueMigrationScore(1, N);
            if (comp == null || !v.HasValue) { why = "composite/V-score not ready"; return false; }
            double px = Bars.Close[0];
            bool ok = side > 0 ? (px >= comp.poc && v.Value >= 0) : (px <= comp.poc && v.Value <= 0);
            if (!ok) why = $"px {px:F2} vs POC{N} {comp.poc:F2}, V{N} {v.Value}";
            return ok;
        }
    }
}
```

3.18 Example 23 (`_Ex23FastInSlowFade`) — A2-58 fade the fast trend inside the slow trend (Kaufman), intraday 5m (medium)

Features (direct). F-ROLL (§5.7), F-CAL (§5.8), F-TM (§5.12), F-JRNL (§5.13).

What it does (from the file header):

> A2-58 fade the fast trend inside the slow trend (Kaufman), intraday 5m. Slow direction = sign(SMA_S[t] - SMA_S[t-1]); fast direction likewise with SMA_F. Long: slow up and fast turns from up to down on this closed bar, 09:45-15:00 ET; market next bar. Mirror short. Exit: slow direction turns, or 15:55. Disaster stop 2.0 x ATR(14, 5m), floored. S = 30, F = 3 (one point of the catalogue's 4 x 3 grid, not a tuned value). Chart: Data1 = @ES 5m (24h Globex template), dxFeed continuous UNADJUSTED. Indicators run on all bars (ETH included). Features: CME clock adapter on a plain _CustomSignal, F-TM (NoEntryBefore/After, FlatBy, entryAllowed, mustFlatten), averageTrueRange.

Code.

```
// Manual v4 example 23 - A2-58 fade the fast trend inside the slow trend (Kaufman), intraday 5m.
// Slow direction = sign(SMA_S[t] - SMA_S[t-1]); fast direction likewise with SMA_F. Long: slow up and fast turns from up to down on this
// closed bar, 09:45-15:00 ET; market next bar. Mirror short. Exit: slow direction turns, or 15:55. Disaster stop 2.0 x ATR(14, 5m), floored.
// S = 30, F = 3 (one point of the catalogue's 4 x 3 grid, not a tuned value).
// Chart: Data1 = @ES 5m (24h Globex template), dxFeed continuous UNADJUSTED. Indicators run on all bars (ETH included).
// Features: CME clock adapter on a plain _CustomSignal, F-TM (NoEntryBefore/After, FlatBy, entryAllowed, mustFlatten), averageTrueRange.
using System;
using PowerLanguage.Function;

namespace PowerLanguage.Strategy
{
    public class _Ex23FastInSlowFade : _CustomSignal
    {
        protected sealed class A258Config
        {
            public int contracts = 1, slow = 30, fast = 3, atrLength = 14;
            public double stopAtr = 2.0;
            public int costFloorTicks = 14;
            public double riskBudgetFraction = 0.01;
        }
        protected A258Config Cfg { get; } = new A258Config();

        private IOrderMarket m_le, m_se, m_lx, m_sx;
        private IOrderPriced m_lxStp, m_sxStp;
        private _CmeSessionClock m_clock;
        private double m_slowPrev = double.NaN, m_fastPrev = double.NaN, m_stop;
        private int m_slowDir, m_fastDir;

        public _Ex23FastInSlowFade(object _ctx) : base(_ctx) { }

        private _CmeSessionClock Clock { get { if (m_clock == null) m_clock = new _CmeSessionClock(resolveBar
TimeZone(Bars.Info)); return m_clock; } }
        protected override DateTime tradingDayKey(DateTime t) { return Clock.GlobexDate(t); }
        protected override TimeSpan sessionClock(DateTime t) { return Clock.ToEastern(t).TimeOfDay; }
        protected override bool isRthBar(DateTime t) { return Clock.IsRthValueBarEt(Clock.ToEastern(t)); }

        protected override void Create()
        {
            base.Create();
            RollDetection = ERollDetection.Series;
            NoEntryBefore = new TimeSpan(9, 45, 0); NoEntryAfter = new TimeSpan(15, 0, 0); FlatBy = new TimeS
pan(15, 55, 0);
            m_le = OrderCreator.MarketNextBar(new SOrderParameters(Contracts.UserSpecified, "A258LE", EOrderA
ction.Buy));
            m_se = OrderCreator.MarketNextBar(new SOrderParameters(Contracts.UserSpecified, "A258SE", EOrderA
ction.SellShort));
            m_lx = OrderCreator.MarketNextBar(new SOrderParameters(Contracts.Default, "A258LX",
EOrderAction.Sell));
            m_sx = OrderCreator.MarketNextBar(new SOrderParameters(Contracts.Default, "A258SX",
EOrderAction.BuyToCover));
            m_lxStp = OrderCreator.Stop(new SOrderParameters(Contracts.Default, "A258LS", EOrderAction.Sell,
OrderExit.FromAll));
            m_sxStp = OrderCreator.Stop(new SOrderParameters(Contracts.Default, "A258SS", EOrderAction.BuyToC
over, OrderExit.FromAll));
        }

        protected override void StartCalc()
        {
```

```
            m_clock = null;
            base.StartCalc();
            m_slowPrev = m_fastPrev = double.NaN; m_slowDir = m_fastDir = 0;
        }

        private double sma(int n) { double s = 0; for (int i = 0; i < n; i++) s += Bars.Close[i]; return s /
n; }

        protected override void CalcBar()
        {
            base.CalcBar();
            if (Bars.Status != EBarState.Close || Bars.CurrentBar <= Cfg.slow) return;
            if (rollOn(1)) { m_slowPrev = m_fastPrev = double.NaN; m_slowDir = m_fastDir = 0; }   //
directions restart; the SMAs still span the roll gap for S bars (pitfall: use a back-adjusted series if that
matters)
            double sl = sma(Cfg.slow), fs = sma(Cfg.fast);
            int slowDir = double.IsNaN(m_slowPrev) ? 0 : Math.Sign(sl - m_slowPrev);
            int fastDir = double.IsNaN(m_fastPrev) ? 0 : Math.Sign(fs - m_fastPrev);
            int prevSlow = m_slowDir, prevFast = m_fastDir;
            m_slowPrev = sl; m_fastPrev = fs;
            if (slowDir != 0) m_slowDir = slowDir;
            if (fastDir != 0) m_fastDir = fastDir;

            int pos = StrategyInfo.MarketPosition;
            if (pos != 0)
            {
                if (mustFlatten() || m_slowDir != Math.Sign(pos)) { if (pos > 0) m_lx.Send(); else m_sx.Send(
); return; }
                if (pos > 0) m_lxStp.Send(m_stop); else m_sxStp.Send(m_stop);
                return;
            }
            string why;
            if (!entryAllowed(out why) || !isRthBar(Bars.Time[0])) return;
            int side = 0;
            if (m_slowDir > 0 && prevFast > 0 && m_fastDir < 0 && prevSlow > 0) side = +1;
            else if (m_slowDir < 0 && prevFast < 0 && m_fastDir > 0 && prevSlow < 0) side = -1;
            if (side == 0) return;
            double atr = averageTrueRange(Bars, Cfg.atrLength), tick = tickSize(), c = Bars.Close[0];
            if (!(atr > 0)) return;
            double r = Math.Max(Cfg.stopAtr * atr, Cfg.costFloorTicks * tick);
            if (overRiskBudget(r)) { if (journalOn()) trace(ETraceLevel.Journal, $"A2-58 {Bars.Time[0]:yyyy-
MM-dd HH:mm} skip: stop {r / tick:F0} ticks breaches risk budget"); return; }
            m_stop = c - side * r;                                      // approximate: from the signal close,
not the fill
            if (side > 0) m_le.Send(Cfg.contracts); else m_se.Send(Cfg.contracts);
            if (journalOn()) journalArm(side > 0 ? "A258LE" : "A258SE", c, m_stop, double.NaN, null);
        }

        private bool overRiskBudget(double stopDist) { return Cfg.contracts * stopDist * Bars.Info.BigPointVa
lue > equityAvailable(Cfg.riskBudgetFraction); }
    }
}
```

3.19 Example 24 (`_Ex24ExclusionGate`) — RG-07 event, roll and thin-session exclusions, applied as a gate on the example 18 host (A2-56) (medium)

Features (direct). F-CAL (§5.8), F-EVT (§5.14), M-EVENT (§5.20), VP helpers (§5.27). **Inherited from example 18:** F-ROLL, F-TM, F-JRNL.

What it does (from the file header):

> RG-07 event, roll and thin-session exclusions, applied as a gate on the example 18 host (A2-56). Rules: tier-1 news -2/+10 min blackout (and flatten); FOMC day after 14:00 ET -> no new entries; roll week -> no new entries; CME half-days and holiday-adjacent sessions -> off. Unscheduled news is NOT covered. GDP/PCE coverage holes 2016-24 (EV-4.6b): allowPartialCoverage = false makes an Unknown-coverage day block (conservative). Chart: as example 18. M-EVENT uses its embedded table unless EventConfig.csvPath is set. Features: M-EVENT (EventConfig, EventBlackout, EventFlattenNow, EventCalendar.Classify, IsRollWeek, CoverageBlocks), F-CAL, F-ROLL.

Pitfalls.

- Module on example 18: compile both files; the host's rules, stops and settings apply unless overridden.

Code.

```
// Manual v4 example 24 - RG-07 event, roll and thin-session exclusions, applied as a gate on the example 18
host (A2-56).
// Rules: tier-1 news -2/+10 min blackout (and flatten); FOMC day after 14:00 ET -> no new entries; roll week
-> no new entries;
// CME half-days and holiday-adjacent sessions -> off. Unscheduled news is NOT covered. GDP/PCE coverage
holes 2016-24 (EV-4.6b):
// allowPartialCoverage = false makes an Unknown-coverage day block (conservative).
// Chart: as example 18. M-EVENT uses its embedded table unless EventConfig.csvPath is set.
// Features: M-EVENT (EventConfig, EventBlackout, EventFlattenNow, EventCalendar.Classify, IsRollWeek,
CoverageBlocks), F-CAL, F-ROLL.
using System;
using System.Collections.Generic;
using PowerLanguage.Function;

namespace PowerLanguage.Strategy
{
    public class _Ex24ExclusionGate : _Ex18FirstHourBreak
    {
        private IOrderMarket m_flatL, m_flatS;
        public _Ex24ExclusionGate(object _ctx) : base(_ctx) { }

        protected override void Create()
        {
            base.Create();
            EventConfig.enabled = true;                          // lazy: the calendar is built only when
enabled
            EventConfig.blackoutTier = EEventTier.T1;
            EventConfig.blackoutPreMin = 2; EventConfig.blackoutPostMin = 10;
            EventConfig.blackoutFlatten =
true;                  // the base only RAISES EventFlattenNow; this strategy acts on it
            EventConfig.allowPartialCoverage = false;
            m_flatL = OrderCreator.MarketNextBar(new SOrderParameters(Contracts.Default, "RG07LX", EOrderActi
on.Sell));
            m_flatS = OrderCreator.MarketNextBar(new SOrderParameters(Contracts.Default, "RG07SX", EOrderActi
on.BuyToCover));
        }

        protected override void CalcBar()
        {
            base.CalcBar();                                      // runs the host (which calls hostGate) and
the event engine
            if (Bars.Status != EBarState.Close) return;
            if (EventFlattenNow)
            {
                int pos = StrategyInfo.MarketPosition;
                if (pos > 0) m_flatL.Send(); else if (pos < 0) m_flatS.Send();
                if (pos != 0 && journalOn()) trace(ETraceLevel.Journal, $"RG-07 {Bars.Time[0]:yyyy-MM-dd
HH:mm} tier-1 blackout: flatten");
            }
        }

        protected override bool hostGate(DateTime day, out string why)
        {
            why = null;
            if (EventBlackout) { why = "T1 blackout"; return false; }
            _EventCalendar ev = EventCalendar;
            if (ev == null) { why = "no calendar"; return false; }
            EventDay d = ev.Classify(day);
            TimeSpan tod = sessionClock(Bars.Time[0]);
            if (d.hasFomc && tod >= new TimeSpan(14, 0, 0)) why = "FOMC day after 14:00";
            else if (d.isRollWeek || ev.IsRollWeek()) why = "roll week";
            else if (d.isHalfDay || _CmeHolidays.Rules.Get(day).RthCloseEt.HasValue) why = "half-day";
            else if (holidayAdjacent(day)) why = "holiday-adjacent";
            else if (ev.CoverageBlocks(day, EEventClassMask.Macro)) why = "coverage hole";
            if (why != null && journalOn()) logSignal("RG-07", 0, Bars.Close[0], double.NaN, new Dictionary<s
tring, string> { { "blocked", why } }, false);
            return why == null;
        }
```

```
        /// <summary>Day before or after a weekday CME holiday (no RTH session) or early close.</summary>
        private static bool holidayAdjacent(DateTime day)
        {
            foreach (int k in new[] { -1, 1 })
            {
                DateTime d = day.AddDays(k);
                while (d.DayOfWeek == DayOfWeek.Saturday || d.DayOfWeek == DayOfWeek.Sunday) d = d.AddDays(k)
;
                _CmeHolidays.Rules.Day r = _CmeHolidays.Rules.Get(d);
                if (r.Closed || r.NoRth || r.RthCloseEt.HasValue) return true;
            }
            return false;
        }
    }
}
```

3.20 Example 25 (`_Ex25ShockBarFade`) — A2-51 intraday shock-bar fade with the delayed stop (Kaufman Ch 16), exit arm E1 (volume decay) (medium)

Features (direct). F-SESS (§5.6), F-ROLL (§5.7), F-CAL (§5.8), F-TM (§5.12), F-JRNL (§5.13), M-TOD (§5.18).

What it does (from the file header):

> A2-51 intraday shock-bar fade with the delayed stop (Kaufman Ch 16), exit arm E1 (volume decay). Signal (closed 5m bar b, 10:00-15:00 ET): TR_b >= k x same-slot TOD median TR (N = 100 sessions; the catalogue's SSB average is approximated by the normaliser's MEDIAN), close in the top 20% of the bar (-> short) or bottom 20% (-> long). Bars whose window contains 08:30 / 10:00 / 14:00 ET +/- 2 min are skipped. Entry: market next bar. Stop: disaster entry -/+ 1.5 x TR_b throughout; from the close of b+2 a stop beyond the b..b+2 extreme + 1 tick (both floored at 14 ticks). Exit E1: the first closed bar with volume <= the same-slot TOD median volume; E0 15:55 as backstop. k = 2.5. Chart: Data1 = @ES 5m (24h Globex template), dxFeed continuous UNADJUSTED, tick-built volume. Features: M-TOD (TodMetrics = TrueRange | Volume, TodSessions = 100, Valid, Median), F-SESS, F-TM (FlatBy, mustFlatten).

Code.

```
// Manual v4 example 25 - A2-51 intraday shock-bar fade with the delayed stop (Kaufman Ch 16), exit arm E1
(volume decay).
// Signal (closed 5m bar b, 10:00-15:00 ET): TR_b >= k x same-slot TOD median TR (N = 100 sessions; the
catalogue's SSB average is
// approximated by the normaliser's MEDIAN), close in the top 20% of the bar (-> short) or bottom 20% (->
long). Bars whose window contains
// 08:30 / 10:00 / 14:00 ET +/- 2 min are skipped. Entry: market next bar. Stop: disaster entry -/+ 1.5 x
TR_b throughout; from the close of
// b+2 a stop beyond the b..b+2 extreme + 1 tick (both floored at 14 ticks). Exit E1: the first closed bar
with volume <= the same-slot TOD
// median volume; E0 15:55 as backstop. k = 2.5.
// Chart: Data1 = @ES 5m (24h Globex template), dxFeed continuous UNADJUSTED, tick-built volume.
// Features: M-TOD (TodMetrics = TrueRange | Volume, TodSessions = 100, Valid, Median), F-SESS, F-TM (FlatBy,
mustFlatten).
using System;
using PowerLanguage.Function;

namespace PowerLanguage.Strategy
{
    public class _Ex25ShockBarFade : _VolumeProfileFuturesBase
    {
        protected sealed class A251Config
        {
            public int contracts = 1;
            public double k = 2.5, closeZone = 0.2, disasterTr = 1.5;
            public TimeSpan from = new TimeSpan(10, 0, 0), to = new TimeSpan(15, 0, 0), flat = new TimeSpan(1
5, 55, 0);
            public int costFloorTicks = 14;
            public double riskBudgetFraction = 0.01;
        }
        protected A251Config Cfg { get; } = new A251Config();
```

```
        private static readonly TimeSpan[] Releases = { new TimeSpan(8, 30, 0), new TimeSpan(10, 0, 0), new T
imeSpan(14, 0, 0) };

        private IOrderMarket m_le, m_se, m_lx, m_sx;
        private IOrderPriced m_lxStp, m_sxStp;
        private int m_shockBar = -1, m_side;
        private double m_shockTr, m_stop;

        public _Ex25ShockBarFade(object _ctx) : base(_ctx) { }

        protected override void Create()
        {
            base.Create();
            RollDetection = ERollDetection.Series;
            TodMetrics = ETodMetric.TrueRange | ETodMetric.Volume; TodSessions = 100; TodMinSessions = 20; To
dBucketMinutes = 5;
            FlatBy = Cfg.flat;
            m_le = OrderCreator.MarketNextBar(new SOrderParameters(Contracts.UserSpecified, "A251LE", EOrderA
ction.Buy));
            m_se = OrderCreator.MarketNextBar(new SOrderParameters(Contracts.UserSpecified, "A251SE", EOrderA
ction.SellShort));
            m_lx = OrderCreator.MarketNextBar(new SOrderParameters(Contracts.Default, "A251LX",
EOrderAction.Sell));
            m_sx = OrderCreator.MarketNextBar(new SOrderParameters(Contracts.Default, "A251SX",
EOrderAction.BuyToCover));
            m_lxStp = OrderCreator.Stop(new SOrderParameters(Contracts.Default, "A251LS", EOrderAction.Sell,
OrderExit.FromAll));
            m_sxStp = OrderCreator.Stop(new SOrderParameters(Contracts.Default, "A251SS", EOrderAction.BuyToC
over, OrderExit.FromAll));
        }

        protected override void StartCalc() { base.StartCalc(); m_shockBar = -1; m_side = 0; }

        protected override void CalcBar()
        {
            base.CalcBar();
            if (Bars.Status != EBarState.Close || Bars.CurrentBar < 2) return;
            double tick = tickSize(), floor = Cfg.costFloorTicks * tick;
            int pos = StrategyInfo.MarketPosition;
            _TodNormaliser tod = TodNormaliser;
            if (pos != 0)
            {
                if (mustFlatten()) { if (pos > 0) m_lx.Send(); else m_sx.Send(); return; }
                int since = Bars.CurrentBar - m_shockBar;                    // 1 = entry bar b+1, 2 = b+2
                if (since == 1)
                {
                    double e = StrategyInfo.AvgEntryPrice, d = Math.Max(Cfg.disasterTr * m_shockTr, floor);
                    m_stop = e - Math.Sign(pos) * d;
                }
                else if (since >= 2)
                {
                    double ext = pos > 0 ? Math.Min(Bars.Low[0], Math.Min(Bars.Low[1], Bars.Low[2])) - tick
                                         : Math.Max(Bars.High[0], Math.Max(Bars.High[1], Bars.High[2])) + tic
k;
                    if (since == 2) { double e = StrategyInfo.AvgEntryPrice; if (Math.Abs(e - ext) < floor) e
xt = e - Math.Sign(pos) * floor; m_stop = pos > 0 ? Math.Max(m_stop, ext) : Math.Min(m_stop, ext); }
                }
                if (since >= 2 && tod != null && tod.Valid(ETodMetric.Volume) && Bars.Volume[0] <=
tod.Median(ETodMetric.Volume))
                { if (pos > 0) m_lx.Send(); else m_sx.Send(); return; }      // E1 volume decay
                if (pos > 0) m_lxStp.Send(m_stop); else m_sxStp.Send(m_stop);
                return;
            }
            if (rollOn(1) || tod == null || !tod.Valid(ETodMetric.TrueRange)) return;
            DateTime et = ToEastern(Bars.Time[0]);
            TimeSpan t = et.TimeOfDay;
            if (!IsRth() || t < Cfg.from || t > Cfg.to || nearRelease(t)) return;
            double h = Bars.High[0], l = Bars.Low[0], c = Bars.Close[0];
            double tr = Math.Max(h, Bars.Close[1]) - Math.Min(l, Bars.Close[1]);
            double med = tod.Median(ETodMetric.TrueRange);
            if (!(med > 0) || tr < Cfg.k * med || !(h > l)) return;
            double loc = (c - l) / (h - l);
            int side = loc >= 1 - Cfg.closeZone ? -1 : loc <= Cfg.closeZone ? +1 : 0;
            if (side == 0) return;
            double r = Math.Max(Cfg.disasterTr * tr, floor);
            if (overRiskBudget(r)) { if (journalOn()) trace(ETraceLevel.Journal, $"A2-51 {et:yyyy-MM-dd
HH:mm} skip: disaster stop {r / tick:F0} ticks breaches risk budget"); return; }
```

```
            m_shockBar = Bars.CurrentBar; m_shockTr = tr; m_stop = c - side * r;
            if (side > 0) m_le.Send(Cfg.contracts); else m_se.Send(Cfg.contracts);
            if (journalOn()) trace(ETraceLevel.Journal, $"A2-51 {et:yyyy-MM-dd HH:mm} shock TR {tr:F2} =
{tr / med:F2} x median; fade {(side > 0 ? "long" : "short")}");
        }

        private bool nearRelease(TimeSpan barEndEt)
        {
            TimeSpan start = barEndEt - TimeSpan.FromMinutes(5);
            foreach (TimeSpan r in Releases) if (r >= start - TimeSpan.FromMinutes(2) && r <= barEndEt + Time
Span.FromMinutes(2)) return true;
            return false;
        }

        private bool overRiskBudget(double stopDist) { return Cfg.contracts * stopDist * Bars.Info.BigPointVa
lue > equityAvailable(Cfg.riskBudgetFraction); }
    }
}
```

3.21 Example 37 (`_Ex37BurstExtremeBreak`) — OF-07 bar-level burst at a fresh extreme, standalone form (both sides) (medium)

Features (direct). F-SESS (§5.6), F-ROLL (§5.7), F-CAL (§5.8), F-TM (§5.12), F-JRNL (§5.13), M-TICK (§5.21), M-PORT (§5.23).

What it does (from the file header):

> OF-07 bar-level burst at a fresh extreme, standalone form (both sides). Event: a closed 1m RTH bar takes out the session's RTH high (low) so far, M-TICK flags a print-rate burst on that bar (TickStats.valid && TickStats.burst, z >= TickConfig.burstZ), and its range >= 1.5 x the mean range of the prior 20 bars. Deviations (stated): the catalogue's volume z >= 2 and "clock-minute norm" are operationalised as the M-TICK print-rate burst and a rolling 20-bar range mean; ON extremes are not used. Entry: market next bar in the break direction, 09:35-15:30 ET, max 3 per day. Stop: the break bar's midpoint, floored at 14 ticks. Target b = 0.75R (catalogue grid {0.5, 0.75, 1}). Risk budget 0.01 x equity. Chart: Data1 @ES 1m, dxFeed continuous UNADJUSTED (M-TICK loads the same symbol's 1-tick Trade stream via DataLoader). Features: M-TICK (TickConfig, TickStats), CS-7 (MaxTradesPerDay, FlatBy, entryAllowed), M-PORT (portfolioArm, portfolioStop).

Code.

```
// Manual v4 example 37 - OF-07 bar-level burst at a fresh extreme, standalone form (both sides).
// Event: a closed 1m RTH bar takes out the session's RTH high (low) so far, M-TICK flags a print-rate burst
on that bar
// (TickStats.valid && TickStats.burst, z >= TickConfig.burstZ), and its range >= 1.5 x the mean range of the
prior 20 bars.
// Deviations (stated): the catalogue's volume z >= 2 and "clock-minute norm" are operationalised as the M-
TICK print-rate burst and a
// rolling 20-bar range mean; ON extremes are not used. Entry: market next bar in the break direction,
09:35-15:30 ET, max 3 per day.
// Stop: the break bar's midpoint, floored at 14 ticks. Target b = 0.75R (catalogue grid {0.5, 0.75, 1}).
Risk budget 0.01 x equity.
// Chart: Data1 @ES 1m, dxFeed continuous UNADJUSTED (M-TICK loads the same symbol's 1-tick Trade stream via
DataLoader).
// Features: M-TICK (TickConfig, TickStats), CS-7 (MaxTradesPerDay, FlatBy, entryAllowed), M-PORT
(portfolioArm, portfolioStop).
using System;
using PowerLanguage.Function;

namespace PowerLanguage.Strategy
{
    public class _Ex37BurstExtremeBreak : _VolumeProfileFuturesBase
    {
        protected sealed class Of07Config
        {
            public int contracts = 1, rangeLen = 20, costFloorTicks = 14;
            public double minRangeMult = 1.5, targetR = 0.75, riskBudgetFraction = 0.01;
            public TimeSpan from = new TimeSpan(9, 35, 0), to = new TimeSpan(15, 30, 0), flat = new
TimeSpan(15, 55, 0);
        }
```

```
        protected Of07Config Cfg { get; } = new Of07Config();

        private IOrderMarket m_le, m_se, m_lx, m_sx;
        private IOrderPriced m_lxStp, m_sxStp, m_lxLmt, m_sxLmt;
        private DateTime m_day = DateTime.MinValue;
        private double m_hi, m_lo, m_stop, m_target;

        public _Ex37BurstExtremeBreak(object _ctx) : base(_ctx) { }

        protected override void Create()
        {
            base.Create();
            RollDetection = ERollDetection.Series;
            TickConfig.enabled = true;                          // lazy: no DataLoader tick request unless
enabled
            TickConfig.burstZ = 3.0;
            MaxTradesPerDay = 3; FlatBy = Cfg.flat;
            m_le = OrderCreator.MarketNextBar(new SOrderParameters(Contracts.UserSpecified, "OF07LE", EOrderA
ction.Buy));
            m_se = OrderCreator.MarketNextBar(new SOrderParameters(Contracts.UserSpecified, "OF07SE", EOrderA
ction.SellShort));
            m_lx = OrderCreator.MarketNextBar(new SOrderParameters(Contracts.Default, "OF07LX",
EOrderAction.Sell));
            m_sx = OrderCreator.MarketNextBar(new SOrderParameters(Contracts.Default, "OF07SX",
EOrderAction.BuyToCover));
            m_lxStp = OrderCreator.Stop(new SOrderParameters(Contracts.Default, "OF07LS", EOrderAction.Sell,
OrderExit.FromAll));
            m_sxStp = OrderCreator.Stop(new SOrderParameters(Contracts.Default, "OF07SS", EOrderAction.BuyToC
over, OrderExit.FromAll));
            m_lxLmt = OrderCreator.Limit(new SOrderParameters(Contracts.Default, "OF07LT",
EOrderAction.Sell, OrderExit.FromAll));
            m_sxLmt = OrderCreator.Limit(new SOrderParameters(Contracts.Default, "OF07ST", EOrderAction.BuyTo
Cover, OrderExit.FromAll));
        }

        protected override void StartCalc() { base.StartCalc(); m_day = DateTime.MinValue; }

        protected override void CalcBar()
        {
            base.CalcBar();
            if (Bars.Status != EBarState.Close) return;
            DateTime day = GlobexDate(Bars.Time[0]);
            if (day != m_day) { m_day = day; m_hi = double.NaN; m_lo = double.NaN; }
            int pos = StrategyInfo.MarketPosition;
            if (pos != 0)
            {
                if (mustFlatten()) { if (pos > 0) m_lx.Send(); else m_sx.Send(); return; }
                if (pos > 0) { m_lxStp.Send(m_stop); m_lxLmt.Send(m_target); } else { m_sxStp.Send(m_stop); m
_sxLmt.Send(m_target); }
                portfolioStop(m_stop);                          // M-PORT: no-op outside Portfolio Trader
            }
            if (!IsRth()) return;
            double h = Bars.High[0], l = Bars.Low[0], c = Bars.Close[0], tick = tickSize();
            bool first = double.IsNaN(m_hi);
            int side = first ? 0 : h > m_hi ? +1 : l < m_lo ? -1 : 0;
            if (h > m_hi || first) m_hi = h;
            if (l < m_lo || first) m_lo = l;
            if (side == 0 || pos != 0 || rollOn(1)) return;
            TimeSpan tod = ToEastern(Bars.Time[0]).TimeOfDay;
            if (tod < Cfg.from || tod > Cfg.to) return;
            string reason;
            if (!entryAllowed(out reason)) return;
            TickBarStats ts = TickStats;
            if (ts == null || !ts.valid || !ts.burst) return;
            if (Bars.CurrentBar <= Cfg.rangeLen) return;
            double mean = 0;
            for (int i = 1; i <= Cfg.rangeLen; i++) mean += Bars.High[i] - Bars.Low[i];
            mean /= Cfg.rangeLen;
            if (h - l < Cfg.minRangeMult * mean) return;
            double stop = (h + l) / 2;
            if (side * (c - stop) < Cfg.costFloorTicks * tick) stop = c - side * Cfg.costFloorTicks * tick;
            double r = Math.Abs(c - stop);
            if (Cfg.contracts * r * Bars.Info.BigPointValue > equityAvailable(Cfg.riskBudgetFraction))
            { if (journalOn()) trace(ETraceLevel.Journal, $"OF-07 {Bars.Time[0]:yyyy-MM-dd HH:mm} skip: risk
budget"); return; }
            m_stop = stop; m_target = c + side * Cfg.targetR * r;
            if (journalOn()) journalArm(side > 0 ? "OF07LE" : "OF07SE", c, m_stop, m_target, null);
```

```
            portfolioArm(side, c, m_stop, Cfg.contracts);    // M-PORT: no-op outside Portfolio Trader
            if (side > 0) m_le.Send(Cfg.contracts); else m_se.Send(Cfg.contracts);
        }
    }
}
```

3.22 Example 40 (`_Ex40DailyKillSwitch`) — A2-78 daily kill switch as a strategy-level layer on the OF-07 host (Ex37) (medium)

Features (direct). F-TM (§5.12), M-PORT (§5.23). **Inherited from example 37:** F-SESS, F-ROLL, F-CAL, F-JRNL, M-TICK.

What it does (from the file header):

> A2-78 daily kill switch as a strategy-level layer on the OF-07 host (Ex37). A2-78 rules: when the day's P&L <= -D x R, cancel and flatten and take no new entries until the next session; two-strike rule: stop after K losers. Grid: D in {2, 3, 4}, K in {off, 3}; this file shows D = 2, K = 3 (grid points, not tuned values). What the framework provides per strategy (CS-7): DailyLossStopR (closed net loss in R of RiskUnitDollars) and DailyLossStopCount (losing closed trades in the trading day) block new entries through entryAllowed(). Deviations (stated): CS-7 counts ALL losers of the day, not CONSECUTIVE losers, and uses CLOSED P&L only; the open position keeps its bracket. The BOOK-level rule (open + closed, cancel-and-flatten) is _PortfolioRiskManager / PortMmConfig (dailyLossStop, dailyLossFlatten) in Portfolio Trader. RiskUnitDollars = 14 ticks x $12.50 = $175, i.e. the cost-floor stop of one ES contract (a definition of R, not a result). M-PORT: PortfolioHalted reports the strategy's own kill to the portfolio MM; PortfolioFamily = "OF".

Pitfalls.

- Module on example 37: compile both files; the host's rules, stops and settings apply unless overridden.

Code.

```
// Manual v4 example 40 - A2-78 daily kill switch as a strategy-level layer on the OF-07 host (Ex37).
// A2-78 rules: when the day's P&L <= -D x R, cancel and flatten and take no new entries until the next
session; two-strike rule:
// stop after K losers. Grid: D in {2, 3, 4}, K in {off, 3}; this file shows D = 2, K = 3 (grid points, not
tuned values).
// What the framework provides per strategy (CS-7): DailyLossStopR (closed net loss in R of RiskUnitDollars)
and DailyLossStopCount
// (losing closed trades in the trading day) block new entries through entryAllowed(). Deviations (stated):
CS-7 counts ALL losers of
// the day, not CONSECUTIVE losers, and uses CLOSED P&L only; the open position keeps its bracket. The BOOK-
level rule (open + closed,
// cancel-and-flatten) is _PortfolioRiskManager / PortMmConfig (dailyLossStop, dailyLossFlatten) in Portfolio
Trader.
// RiskUnitDollars = 14 ticks x $12.50 = $175, i.e. the cost-floor stop of one ES contract (a definition of
R, not a result).
// M-PORT: PortfolioHalted reports the strategy's own kill to the portfolio MM; PortfolioFamily = "OF".
using System;
using PowerLanguage.Function;

namespace PowerLanguage.Strategy
{
    public class _Ex40DailyKillSwitch : _Ex37BurstExtremeBreak
    {
        private bool m_halted;

        public _Ex40DailyKillSwitch(object _ctx) : base(_ctx) { }

        protected override string PortfolioFamily { get { return "OF"; } }
        protected override bool PortfolioHalted { get { return m_halted; } }

        protected override void Create()
        {
            base.Create();
            DailyLossStopR = 2;            // D
```

```
            DailyLossStopCount = 3;        // K (all losers of the day; see header)
            RiskUnitDollars = 14 * 12.50;  // 1 R = cost-floor stop of one full-size ES
        }

        protected override void StartCalc() { base.StartCalc(); m_halted = false; }

        protected override void CalcBar()
        {
            string reason;
            bool ok = entryAllowed(out reason);              // CS-7 resets per trading day
            bool halted = !ok && reason != null && reason.StartsWith("DailyLossStop");
            if (halted && !m_halted && journalOn()) trace(ETraceLevel.Journal, $"A2-78 {Bars.Time[0]:yyyy-MM-
dd HH:mm} kill: {reason}");
            m_halted = halted;
            base.CalcBar();
        }
    }
}
```

3.23 Example 6 (`_Ex06NakedLevelReaction`) — PS-21 naked POC/VAH/VAL first touch (medium–hard)

Features (direct). F-PROF (§5.2), F-NPOC (§5.4), F-ROLL (§5.7), F-CAL (§5.8), F-JRNL (§5.13), F-EVT (§5.14), M-MTF (§5.17).

Catalogue rules (PS-21). Market and timeframe: ES/NQ, GC, CL; 5m. Level: the nearest naked POC/VAH/VAL ≤ 20 sessions old. Entry: first touch ± δ, then a 5m rejection close. Stop: level + floor. Target: 1R. Sweep: age {≤ 5, ≤ 20} × level {POC, VAH/VAL}. Multi-day levels reset at rolls. The catalogue expects a kill and keeps it as a control for EX-02. **G0-d-gated** (families 1 and 2).

Chart. Data1 = @ES 1m with the MC Volume Profile **on** (session profiles), Data2 = @ES 5m, dxFeed unadjusted, 24h. Derives from `_VolumeProfileFuturesBase`, `TrackNakedPoc = true` (StartCalc throws otherwise).

Walk-through.

- The level search uses `RecentCompletedProfiles(1, maxAge, barNo - 1)`: profiles completed **as of the previous bar**. `barNo` is `Bars.FullSymbolData.Current`, the 1-based Status Bar number that `asOfBar` expects (not `Bars.CurrentBar`).
- Profiles that cross a roll or start at or before the last roll bar are skipped.
- Naked tests: `VpExtra.IsNakedPoc(asOfBar)` for POCs, `IsNakedVah`/`IsNakedVal` for value edges. A price already traded once is kept in a set and never used again (first touch only).
- Side: price above the level means support (long), below means resistance (short). The nearest candidate to the prior close wins.
- A touch is the 1m low (or high) within `touchTicks` of the level. The decision waits for the first **closed** 5m bar at or after the touch: a close back on the entry side is a rejection, and the trade is armed at market; anything else is discarded and logged as untaken.
- Stop = level − side × 4 ticks (`stopBufferTicks`); target = 1R from the 1m close of the decision bar. If that R is under the 14-tick cost floor (`costFloorTicks`), the stop is widened to 14 ticks from the 1m close (and the 1R target moves with it). If the widened stop breaches the per-trade risk budget (`riskBudgetFraction` = 0.01, 1% of equity, decided by Ettienne 2026-10-23; PF-04's 0.5% cap is not the default for the floor rule), the trade is skipped and journaled. Size is never shrunk. Short entries mirror longs. Entries until 15:30; flat at 15:55, or at `RthCloseEt` − 5 min on an early-close day (`flatAt`).

Pitfalls.

- **Assumption (labelled):** the 4-tick buffer beyond the level. When the 1m close is under 10 ticks from the level, the 4-tick buffer gives a stop under the 14-tick floor, so the stop is widened to 14 ticks from the close.
- **First touch uses up the level.** The level goes into `m_used` at the touch, so a touch that is discarded (no rejection) or skipped (risk-budget breach) uses up that level too.

- Stop and target are sent from the bar after the fill (§1.7), so the fill bar has no working exit.
- `asOfBar` is the Status Bar number. Passing `Bars.CurrentBar` silently shifts the as-of point once MaxBarsBack is not 0.
- If the touch bar also closes the 5m bar that holds it, the rejection is checked on the same bar.
- PS-21's K-FL control with distance matching is mandatory (catalogue); it is not coded here.

Full strategy — `_Ex06NakedLevelReaction.Strategy.CS`:

```
// Manual v4 example 6 - PS-21 first touch of the nearest naked POC / VAH / VAL (<= 20 sessions) + 5m
rejection close, target 1R.
// Chart: Data1 = @ES 1m with the MultiCharts Volume Profile ON (session profiles), Data2 = @ES 5m, dxFeed
continuous UNADJUSTED, 24h.
using System;
using System.Collections.Generic;
using PowerLanguage.Function;

namespace PowerLanguage.Strategy
{
    public class _Ex06NakedLevelReaction : _VolumeProfileFuturesBase
    {
        public enum ELevelSet { Poc = 1, ValueEdges = 2 }

        protected sealed class Ps21Config
        {
            public int contracts = 1;
            public int maxAgeSessions = 20;          // sweep {5, 20}
            public ELevelSet levels = ELevelSet.Poc; // sweep {POC, VAH/VAL}
            public int touchTicks = 1;               // delta of "first touch +/- delta"
            public int stopBufferTicks = 4;          // structural stop: this far beyond the level (an
assumption of this example)
            public int costFloorTicks = 14;          // catalogue 2.1: ES cost floor; stop distance =
max(structural, floor), size never shrunk
            public double riskBudgetFraction = 0.01;  // floor-rule risk budget: Ettienne, 2026-10-23: 1% of
equity (PF-04's 0.5% is not the default here)
            public TimeSpan from = new TimeSpan(9, 30, 0), to = new TimeSpan(15, 30, 0), flat = new
TimeSpan(15, 55, 0);
            public TimeSpan halfDayExitBefore = new TimeSpan(0, 5, 0);   // early-close days: flat at
RthCloseEt - 5 min
        }
        protected Ps21Config Cfg { get; } = new Ps21Config();

        private IOrderMarket m_le, m_se, m_lx, m_sx;
        private IOrderPriced m_lxStp, m_sxStp, m_lxLmt, m_sxLmt;
        private int m_lastRollBar;                       // 1-based Status Bar # of the last roll bar
        private double m_level = double.NaN; private int m_side; private DateTime m_touch5 = DateTime.MinValu
e, m_last5 = DateTime.MinValue;
        private readonly HashSet<double> m_used = new HashSet<double>();   // first touch only: a level is
never traded twice
        private double m_stop = double.NaN, m_target = double.NaN;

        /// <summary>Catalogue 2.1 / App F #9 (resolved): true when Cfg.contracts at this stop distance risk
more than the per-trade budget.</summary>
        private bool overRiskBudget(double stopDist) { return Cfg.contracts * stopDist * Bars.Info.BigPointVa
lue > equityAvailable(Cfg.riskBudgetFraction); }
        public _Ex06NakedLevelReaction(object _ctx) : base(_ctx) { }

        protected override void Create()
        {
            base.Create();
            RollDetection = ERollDetection.Series;
            TrackNakedPoc = true;
            m_le = OrderCreator.MarketNextBar(new SOrderParameters(Contracts.UserSpecified, "PS21LE", EOrderA
ction.Buy));
            m_se = OrderCreator.MarketNextBar(new SOrderParameters(Contracts.UserSpecified, "PS21SE", EOrderA
ction.SellShort));
            m_lx = OrderCreator.MarketNextBar(new SOrderParameters(Contracts.Default, "PS21LX",
EOrderAction.Sell));
            m_sx = OrderCreator.MarketNextBar(new SOrderParameters(Contracts.Default, "PS21SX",
EOrderAction.BuyToCover));
            m_lxStp = OrderCreator.Stop(new SOrderParameters(Contracts.Default, "PS21LS", EOrderAction.Sell,
OrderExit.FromAll));
            m_sxStp = OrderCreator.Stop(new SOrderParameters(Contracts.Default, "PS21SS", EOrderAction.BuyToC
over, OrderExit.FromAll));
            m_lxLmt = OrderCreator.Limit(new SOrderParameters(Contracts.Default, "PS21LT",
```

```
EOrderAction.Sell, OrderExit.FromAll));
            m_sxLmt = OrderCreator.Limit(new SOrderParameters(Contracts.Default, "PS21ST", EOrderAction.BuyTo
Cover, OrderExit.FromAll));
        }

        protected override void StartCalc()
        {
            HtfStreams = new[] { 2 };
            base.StartCalc();
            if (!TrackNakedPoc) throw new ArgumentException("_Ex06NakedLevelReaction: TrackNakedPoc must be
true");
            m_lastRollBar = 0; m_level = double.NaN; m_side = 0; m_touch5 = m_last5 = DateTime.MinValue; m_us
ed.Clear();
            m_stop = m_target = double.NaN;
        }

        protected override void CalcBar()
        {
            base.CalcBar();
            if (Bars.Status != EBarState.Close) return;
            int barNo = Bars.FullSymbolData.Current;           // 1-based Status Bar #: the unit of asOfBar
(NOT Bars.CurrentBar)
            if (rollOn(1)) { m_lastRollBar = barNo; m_level = double.NaN; m_side = 0; }
            TimeSpan tod = ToEastern(Bars.Time[0]).TimeOfDay;
            TimeSpan flat = flatAt(GlobexDate(Bars.Time[0]));
            TimeSpan entryEnd = Cfg.to < flat ? Cfg.to : flat;
            double tick = tickSize();
            int pos = StrategyInfo.MarketPosition;

            if (pos != 0)
            {
                if (tod >= flat && tod < new TimeSpan(17, 0, 0)) { if (pos > 0) m_lx.Send(); else m_sx.Send()
; return; }
                if (pos > 0) { m_lxStp.Send(m_stop); m_lxLmt.Send(m_target); } else { m_sxStp.Send(m_stop); m
_sxLmt.Send(m_target); }
                return;
            }

            // a. pending touch: wait for the first 5m bar that CLOSES after the touch (closed 5m bars only)
            DateTime t5; double o5, h5, l5, c5, v5;
            HtfClosedBar(2, 0, out t5, out o5, out h5, out l5, out c5, out v5);
            bool new5 = t5 != m_last5; m_last5 = t5;
            if (m_side != 0 && new5 && t5 >= m_touch5) { checkReject(c5, tod, entryEnd, tick); return; }
            if (m_side != 0 || tod < Cfg.from || tod >= entryEnd) return;

            // b. levels naked as of the PREVIOUS bar, from the last maxAgeSessions completed profiles, none
older than the last roll
            double best = double.NaN; int side = 0; double c1 = Bars.Close[1];
            foreach (VpExtra e in RecentCompletedProfiles(1, Cfg.maxAgeSessions, barNo - 1))
            {
                if (e.crossesRoll || e.startBar <= m_lastRollBar) continue;   // multi-day levels reset at
rolls
                var cand = new List<double>();
                if (Cfg.levels == ELevelSet.Poc) { if (e.IsNakedPoc(barNo - 1)) cand.Add(e.poc); }
                else { if (IsNakedVah(e, barNo - 1)) cand.Add(e.vah); if (IsNakedVal(e, barNo - 1))
cand.Add(e.val); }
                foreach (double lv in cand)
                {
                    if (m_used.Contains(lv)) continue;
                    int s = c1 > lv ? +1 : c1 < lv ? -1 : 0;     // approached from above = support (long);
from below = resistance (short)
                    if (s == 0) continue;
                    if (double.IsNaN(best) || Math.Abs(c1 - lv) < Math.Abs(c1 - best)) { best = lv; side =
s; }
                }
            }
            if (double.IsNaN(best)) return;
            bool touched = side > 0 ? Bars.Low[0] <= best + Cfg.touchTicks * tick : Bars.High[0] >= best - Cf
g.touchTicks * tick;
            if (!touched) return;
            m_used.Add(best); m_level = best; m_side = side; m_touch5 = Bars.Time[0];
            if (journalOn()) trace(ETraceLevel.Journal, $"PS-21 touch {Bars.Time[0]:yyyy-MM-dd HH:mm}
level({best}) side({side})");
            if (new5 && t5 >= m_touch5) checkReject(c5, tod, entryEnd, tick);   // the touch bar also closed
the 5m bar that holds it
        }
```

```
        /// <summary>The first 5m close at or after the touch decides: back on the entry side of the level =
rejection, else discard.
        /// Either way the level stays in m_used: a discarded touch uses the level up for good.</summary>
        private void checkReject(double c5, TimeSpan tod, TimeSpan entryEnd, double tick)
        {
            bool reject = m_side > 0 ? c5 > m_level : c5 < m_level;
            if (reject && tod < entryEnd) arm(m_side, Bars.Close[0], tick);
            else if (journalOn()) logSignal("PS-21", m_side, m_level, double.NaN, null, false);
            m_side = 0; m_level = double.NaN;
        }

        private void arm(int side, double planned, double tick)
        {
            double stop = m_level - side * Cfg.stopBufferTicks * tick;
            double r = Math.Abs(planned - stop);
            if (r < Cfg.costFloorTicks * tick) { r = Cfg.costFloorTicks * tick; stop = planned - side *
r; }   // catalogue 2.1: widen to the cost floor
            if (overRiskBudget(r))   // widened stop breaches the per-trade risk budget: skipped and logged,
not resized
            {
                if (journalOn()) trace(ETraceLevel.Journal, $"PS-21 {Bars.Time[0]:yyyy-MM-dd HH:mm} skip:
stop {r / tick:F0} ticks breaches risk budget {Cfg.riskBudgetFraction:P2}");
                return;
            }
            m_stop = stop;
            m_target = planned + side * r;                      // 1R from the planned (signal-close) price
            var ctx = new Dictionary<string, string> { { "level", m_level.ToString() } };
            if (journalOn()) { journalArm(side > 0 ? "PS21LE" : "PS21SE", planned, m_stop, m_target, ctx); lo
gSignal("PS-21", side, planned, m_stop, ctx, true); }
            if (side > 0) m_le.Send(Cfg.contracts); else m_se.Send(Cfg.contracts);
        }

        /// <summary>Flat time for a Globex day: Cfg.flat, or on a CME early-close day RthCloseEt -
halfDayExitBefore if earlier
        /// (_CmeHolidays.Rules.Get(day).RthCloseEt; same rule as A263Config.halfDayExitBefore).</summary>
        private TimeSpan flatAt(DateTime day)
        {
            _CmeHolidays.Rules.Day cme = _CmeHolidays.Rules.Get(day);
            TimeSpan f = Cfg.flat;
            if (cme.RthCloseEt.HasValue && cme.RthCloseEt.Value - Cfg.halfDayExitBefore < f) f = cme.RthClose
Et.Value - Cfg.halfDayExitBefore;
            return f;
        }
    }
}
```

Compile status: compiled — type-checked with Roslyn against the repo sources (`origin/main` a07257a) plus MC API stubs (stubs not authoritative). Not run in MultiCharts; no backtest results.

3.24 Example 8 (`_Ex08EntropyDayType`) — A2-29 entropy day type at 11:00 (medium–hard)

Features (direct). F-ROLL (§5.7), F-CAL (§5.8), M-GMM (§5.25), VP helpers (§5.27).

Catalogue rules (A2-29). At 11:00 ET, take the developing RTH profile 09:30–11:00 and its H_norm and n_eff as percentiles versus the same-time values of the trailing 250 sessions. H_norm pct ≥ 80 → trend/dispersed; H_norm pct ≤ 30 **and** n_eff pct ≤ 30 → balance; otherwise no change. It is a classifier (module), so no orders.

Chart. Data1 = @ES 1m, dxFeed unadjusted, 24h Globex template. `RangeProfiles.enabled = true`.

Walk-through.

- At the bar closing at 11:00 ET (skipping NoRth days), `RangeProfileAfter(1, t0, t)` builds the profile of 1m bars ending in `(09:30, 11:00]`: an exclusive start, end-stamped bars.
- `RangeStatsOf(e)` gives the bar count; `ProfileEntropy(e, st.bars)` gives the Miller–Madow `HnormMm` and `Neff`, with N = contributing bars (the VP-4 default).
- Two `_EntropyTracker`s (lookback `ProfileStats.pctLookbackA229` = 250, `pctMinFill` = 0.8) return today's percentile against **earlier** sessions first, and only then push today's value.

- The state (TREND, BALANCE, NEUTRAL, or warm-up/no profile) is exposed as `DayType` and journaled with the raw values.

Pitfalls.

- Percentile first, push second. Doing it the other way includes today in its own baseline.
- `t0 = t − 90 minutes` in chart time; the range is in chart time, not ET.
- Until 80% of 250 sessions are filled, the percentiles are NaN and the line says warm-up.
- **Half-days are in the pool.** Only NoRth days are skipped, so early-close sessions add their 11:00 values to the 250-session baselines (on a half-day, 09:30–11:00 is still complete). M-TOD excludes half-days elsewhere in this manual (`todHalfDay`, §1.7); this example does not.

Full strategy — `_Ex08EntropyDayType.Strategy.CS`:

```
// Manual v4 example 8 - A2-29 developing-profile entropy day type at 11:00 ET (module: journals TREND /
BALANCE / NEUTRAL, no orders).
// Chart: Data1 = @ES 1m, dxFeed continuous UNADJUSTED, 24h CME Globex template. Range profiles ON.
using System;
using System.Collections.Generic;
using PowerLanguage.Function;

namespace PowerLanguage.Strategy
{
    public class _Ex08EntropyDayType : _VolumeProfileFuturesBase
    {
        public enum EDayType { Unknown = 0, Neutral = 1, Trend = 2, Balance = 3 }

        protected sealed class A229Config
        {
            public TimeSpan rthOpen = new TimeSpan(9, 30, 0), decision = new TimeSpan(11, 0, 0);
            public double trendPct = 80, balancePct = 30;
        }
        protected A229Config Cfg { get; } = new A229Config();

        public EDayType DayType { get; private set; }
        private _EntropyTracker m_hnorm, m_neff;
        private DateTime m_day = DateTime.MinValue;
        private int m_slot;

        public _Ex08EntropyDayType(object _ctx) : base(_ctx) { }

        protected override void Create()
        {
            base.Create();
            RollDetection = ERollDetection.Series;
            RangeProfiles.enabled = true;
        }

        protected override void StartCalc()
        {
            base.StartCalc();
            m_slot = Cfg.decision.Hours * 60 + Cfg.decision.Minutes;   // = _EntropyTracker.Slot(11:00) = 660
            m_hnorm = new _EntropyTracker(ProfileStats.pctLookbackA229, ProfileStats.pctMinFill, new[] { m_sl
ot });
            m_neff = new _EntropyTracker(ProfileStats.pctLookbackA229, ProfileStats.pctMinFill, new[] { m_slo
t });
            m_day = DateTime.MinValue; DayType = EDayType.Unknown;
        }

        protected override void CalcBar()
        {
            base.CalcBar();
            if (Bars.Status != EBarState.Close) return;
            DateTime t = Bars.Time[0], et = ToEastern(t), day = GlobexDate(t);
            if (day != m_day) { m_day = day; DayType = EDayType.Unknown; }
            if (et.TimeOfDay != Cfg.decision) return;
            _CmeHolidays.Rules.Day cme = _CmeHolidays.Rules.Get(day);
            if (cme.NoRth) return;

            // developing RTH profile 09:30-11:00: 1m bars ENDING in (09:30, 11:00] -> exclusive t0, in chart
time (t is the 11:00 bar end)
            DateTime t0 = t - (Cfg.decision - Cfg.rthOpen);
            VpExtra e = RangeProfileAfter(1, t0, t);
```

```
            RangeStats st = RangeStatsOf(e);
            if (e == null || st == null || st.bars < 1) { journal(day, "no profile", st); return; }
            EntropyResult r = ProfileEntropy(e, st.bars);   // Miller-Madow N = contributing bars (VP-4
default basis)

            // percentiles vs the prior 250 sessions FIRST, then push today (strictly earlier sessions only)
            double pH = m_hnorm.Percentile(m_slot, r.HnormMm), pN = m_neff.Percentile(m_slot, r.Neff);
            m_hnorm.Push(m_slot, day, r.HnormMm); m_neff.Push(m_slot, day, r.Neff);
            if (double.IsNaN(pH) || double.IsNaN(pN)) { journal(day, "warm-up", st, r, pH, pN); return; }
            DayType = pH >= Cfg.trendPct ? EDayType.Trend : (pH <= Cfg.balancePct && pN <= Cfg.balancePct) ?
EDayType.Balance : EDayType.Neutral;
            journal(day, DayType.ToString(), st, r, pH, pN);
        }

        private void journal(DateTime day, string state, RangeStats st, EntropyResult r = default(EntropyResu
lt), double pH = double.NaN, double pN = double.NaN)
        {
            if (!journalOn()) return;
            trace(ETraceLevel.Journal, $"A2-29 {day:yyyy-MM-dd} 11:00 HnormMm({r.HnormMm:F4})
Neff({r.Neff:F1}) pctH({pH:F1}) pctNeff({pN:F1}) state({state}) stats({st})");
        }
    }
}
```

Compile status: compiled — type-checked with Roslyn against the repo sources (`origin/main` a07257a) plus MC API stubs (stubs not authoritative). Not run in MultiCharts; no backtest results.

3.25 Example 22 (`_Ex22PoorHighRun`) — PS-19 prior-session poor high / poor low run-through (medium–hard)

Features (direct). F-DEV (§5.1), F-PROF (§5.2), F-SESS (§5.6), F-ROLL (§5.7), F-CAL (§5.8), F-TM (§5.12), F-JRNL (§5.13).

What it does (from the file header):

> PS-19 prior-session poor high / poor low run-through. Level: the prior RTH profile's high when IsPoorHigh (low when IsPoorLow). Signal: the first 5m close >= delta beyond it, 09:45-14:30 ET. Entry: market next bar. Stop: 0.5 x ATR(14, 5m) back inside the level, floored. Target: b x R (b = 1; the catalogue sweeps 0.75/1/1.5). Flat 15:55. One trade per day. Chart: Data1 = @ES 5m (24h Globex template), dxFeed continuous UNADJUSTED. Features: F-VP (DevelopingSource = Own, OwnProfileWindow = Rth, PriorRthProfile, IsPoorHigh/IsPoorLow), F-SESS, F-TM (FlatBy).

Code.

```
// Manual v4 example 22 - PS-19 prior-session poor high / poor low run-through.
// Level: the prior RTH profile's high when IsPoorHigh (low when IsPoorLow). Signal: the first 5m close >=
delta beyond it, 09:45-14:30 ET.
// Entry: market next bar. Stop: 0.5 x ATR(14, 5m) back inside the level, floored. Target: b x R (b = 1; the
catalogue sweeps 0.75/1/1.5).
// Flat 15:55. One trade per day.
// Chart: Data1 = @ES 5m (24h Globex template), dxFeed continuous UNADJUSTED.
// Features: F-VP (DevelopingSource = Own, OwnProfileWindow = Rth, PriorRthProfile, IsPoorHigh/IsPoorLow), F-
SESS, F-TM (FlatBy).
using System;
using PowerLanguage.Function;

namespace PowerLanguage.Strategy
{
    public class _Ex22PoorHighRun : _VolumeProfileFuturesBase
    {
        protected sealed class Ps19Config
        {
            public int contracts = 1, deltaTicks = 2, atrLength = 14;
            public double stopAtr = 0.5, targetR = 1.0;
            public TimeSpan from = new TimeSpan(9, 45, 0), to = new TimeSpan(14, 30, 0), flat = new
TimeSpan(15, 55, 0);
            public int costFloorTicks = 14;
            public double riskBudgetFraction = 0.01;
        }
```

```
        protected Ps19Config Cfg { get; } = new Ps19Config();

        private IOrderMarket m_le, m_se, m_lx, m_sx;
        private IOrderPriced m_lxStp, m_sxStp, m_lxLmt, m_sxLmt;
        private DateTime m_day = DateTime.MinValue;
        private double m_hiLvl, m_loLvl, m_stop, m_target;
        private bool m_done;

        public _Ex22PoorHighRun(object _ctx) : base(_ctx) { }

        protected override void Create()
        {
            base.Create();
            RollDetection = ERollDetection.Series;
            DevelopingSource = EDevelopingSource.Own;      // PriorRthProfile() is built from Own profiles
            OwnProfileWindow = EOwnProfileWindow.Rth;
            FlatBy = Cfg.flat;
            m_le = OrderCreator.MarketNextBar(new SOrderParameters(Contracts.UserSpecified, "PS19LE", EOrderA
ction.Buy));
            m_se = OrderCreator.MarketNextBar(new SOrderParameters(Contracts.UserSpecified, "PS19SE", EOrderA
ction.SellShort));
            m_lx = OrderCreator.MarketNextBar(new SOrderParameters(Contracts.Default, "PS19LX",
EOrderAction.Sell));
            m_sx = OrderCreator.MarketNextBar(new SOrderParameters(Contracts.Default, "PS19SX",
EOrderAction.BuyToCover));
            m_lxStp = OrderCreator.Stop(new SOrderParameters(Contracts.Default, "PS19LS", EOrderAction.Sell,
OrderExit.FromAll));
            m_sxStp = OrderCreator.Stop(new SOrderParameters(Contracts.Default, "PS19SS", EOrderAction.BuyToC
over, OrderExit.FromAll));
            m_lxLmt = OrderCreator.Limit(new SOrderParameters(Contracts.Default, "PS19LT",
EOrderAction.Sell, OrderExit.FromAll));
            m_sxLmt = OrderCreator.Limit(new SOrderParameters(Contracts.Default, "PS19ST", EOrderAction.BuyTo
Cover, OrderExit.FromAll));
        }

        protected override void StartCalc() { base.StartCalc(); m_day = DateTime.MinValue; m_done = false; }

        protected override void CalcBar()
        {
            base.CalcBar();
            if (Bars.Status != EBarState.Close) return;
            DateTime day = GlobexDate(Bars.Time[0]);
            if (day != m_day)
            {
                m_day = day; m_done = false; m_hiLvl = m_loLvl = double.NaN;
                VpExtra prior = PriorRthProfile();
                if (prior != null && !prior.crossesRoll)
                {
                    if (IsPoorHigh(prior)) m_hiLvl = prior.high;
                    if (IsPoorLow(prior)) m_loLvl = prior.low;
                }
            }
            if (rollOn(1)) m_done = true;
            int pos = StrategyInfo.MarketPosition;
            if (pos != 0)
            {
                m_done = true;
                if (mustFlatten()) { if (pos > 0) m_lx.Send(); else m_sx.Send(); return; }
                if (pos > 0) { m_lxStp.Send(m_stop); m_lxLmt.Send(m_target); } else { m_sxStp.Send(m_stop); m
_sxLmt.Send(m_target); }
                return;
            }
            TimeSpan tod = ToEastern(Bars.Time[0]).TimeOfDay;
            if (m_done || !IsRth() || tod < Cfg.from || tod > Cfg.to) return;
            double tick = tickSize(), c = Bars.Close[0], d = Cfg.deltaTicks * tick;
            int side = !double.IsNaN(m_hiLvl) && c >= m_hiLvl + d ? +1 : !double.IsNaN(m_loLvl) && c <= m_loL
vl - d ? -1 : 0;
            if (side == 0) return;
            m_done = true;                                                  // the FIRST qualifying close
only
            double atr = averageTrueRange(Bars, Cfg.atrLength);
            if (!(atr > 0)) return;
            double lvl = side > 0 ? m_hiLvl : m_loLvl, stop = lvl - side * Cfg.stopAtr * atr;
            if (side * (c - stop) < Cfg.costFloorTicks * tick) stop = c - side * Cfg.costFloorTicks * tick;
            double r = Math.Abs(c - stop);
            if (overRiskBudget(r)) { if (journalOn()) trace(ETraceLevel.Journal, $"PS-19 {day:yyyy-MM-dd}
skip: stop {r / tick:F0} ticks breaches risk budget"); return; }
```

```
            m_stop = stop; m_target = c + side * Cfg.targetR * r;
            if (side > 0) m_le.Send(Cfg.contracts); else m_se.Send(Cfg.contracts);
            if (journalOn()) journalArm(side > 0 ? "PS19LE" : "PS19SE", c, stop, m_target, null);
        }

        private bool overRiskBudget(double stopDist) { return Cfg.contracts * stopDist * Bars.Info.BigPointVa
lue > equityAvailable(Cfg.riskBudgetFraction); }
    }
}
```

3.26 Example 26 (`_Ex26CompositeBreakPullback`) — PS-01 balance break, acceptance, then the first higher-low pullback (long side shown; short mirrors) (medium–hard)

Features (direct). F-DEV (§5.1), F-PROF (§5.2), F-COMP (§5.3), F-SESS (§5.6), F-ROLL (§5.7), F-CAL (§5.8), F-TM (§5.12), F-JRNL (§5.13), M-SWING (§5.16).

What it does (from the file header):

> PS-01 balance break, acceptance, then the first higher-low pullback (long side shown; short mirrors). Filter: IsBalanced(Own RTH profiles, n = 3, 0.6) at the prior close; V-score >= +1 for longs (<= -1 shorts); break 09:45-14:30 ET. Acceptance: 2 closed 5m bars > composite VAH + delta (W = composite VAH - VAL). Entry: the first pullback that holds above VAH - 0.25 W and prints a PA pivot low labelled HL; buy stop 1 tick above the confirming bar. Stop: max(1 tick below the HL, floor); hard invalidation on a 5m close < VAH - delta. Target: VAH + 1.0 W; stop to break-even at +1 R. Chart: Data1 = @ES 5m (24h Globex template), dxFeed continuous UNADJUSTED. Features: F-COMP (Composite), F-VP (IsBalanced, ValueMigrationScore, OwnProfileWindow), M-PA (HL pivots), F-TM (FlatBy).

Code.

```
// Manual v4 example 26 - PS-01 balance break, acceptance, then the first higher-low pullback (long side
shown; short mirrors).
// Filter: IsBalanced(Own RTH profiles, n = 3, 0.6) at the prior close; V-score >= +1 for longs (<= -1
shorts); break 09:45-14:30 ET.
// Acceptance: 2 closed 5m bars > composite VAH + delta (W = composite VAH - VAL). Entry: the first pullback
that holds above
// VAH - 0.25 W and prints a PA pivot low labelled HL; buy stop 1 tick above the confirming bar. Stop: max(1
tick below the HL, floor);
// hard invalidation on a 5m close < VAH - delta. Target: VAH + 1.0 W; stop to break-even at +1 R.
// Chart: Data1 = @ES 5m (24h Globex template), dxFeed continuous UNADJUSTED.
// Features: F-COMP (Composite), F-VP (IsBalanced, ValueMigrationScore, OwnProfileWindow), M-PA (HL pivots),
F-TM (FlatBy).
using System;
using PowerLanguage.Function;

namespace PowerLanguage.Strategy
{
    public class _Ex26CompositeBreakPullback : _VolumeProfileFuturesBase
    {
        protected sealed class Ps01Config
        {
            public int contracts = 1, n = 3, deltaTicks = 2, acceptCloses = 2;
            public double holdW = 0.25, targetW = 1.0, balanceOverlap = 0.6;
            public TimeSpan from = new TimeSpan(9, 45, 0), to = new TimeSpan(14, 30, 0), flat = new
TimeSpan(15, 55, 0);
            public int costFloorTicks = 14;
            public double riskBudgetFraction = 0.01;
        }
        protected Ps01Config Cfg { get; } = new Ps01Config();

        private IOrderPriced m_leStp, m_seStp, m_lxStp, m_sxStp, m_lxLmt, m_sxLmt;
        private IOrderMarket m_lx, m_sx;
        private DateTime m_day = DateTime.MinValue;
        private bool m_balanced, m_done;
        private int m_vscore, m_up, m_dn, m_side;
        private double m_vah, m_val, m_w, m_trigger = double.NaN, m_stop, m_target, m_entryRisk;
```

```
        public _Ex26CompositeBreakPullback(object _ctx) : base(_ctx) { }

        protected override void Create()
        {
            base.Create();
            RollDetection = ERollDetection.Series;
            DevelopingSource = EDevelopingSource.Own; OwnProfileWindow = EOwnProfileWindow.Rth;
            PriceActionConfig.enabled = true; PriceActionConfig.dataStream = 1;
            FlatBy = Cfg.flat;
            m_leStp = OrderCreator.Stop(new SOrderParameters(Contracts.UserSpecified, "PS01LE",
EOrderAction.Buy));
            m_seStp = OrderCreator.Stop(new SOrderParameters(Contracts.UserSpecified, "PS01SE",
EOrderAction.SellShort));
            m_lx = OrderCreator.MarketNextBar(new SOrderParameters(Contracts.Default, "PS01LX",
EOrderAction.Sell));
            m_sx = OrderCreator.MarketNextBar(new SOrderParameters(Contracts.Default, "PS01SX",
EOrderAction.BuyToCover));
            m_lxStp = OrderCreator.Stop(new SOrderParameters(Contracts.Default, "PS01LS", EOrderAction.Sell,
OrderExit.FromAll));
            m_sxStp = OrderCreator.Stop(new SOrderParameters(Contracts.Default, "PS01SS", EOrderAction.BuyToC
over, OrderExit.FromAll));
            m_lxLmt = OrderCreator.Limit(new SOrderParameters(Contracts.Default, "PS01LT",
EOrderAction.Sell, OrderExit.FromAll));
            m_sxLmt = OrderCreator.Limit(new SOrderParameters(Contracts.Default, "PS01ST", EOrderAction.BuyTo
Cover, OrderExit.FromAll));
        }

        protected override void StartCalc() { base.StartCalc(); m_day = DateTime.MinValue; }

        protected override void CalcBar()
        {
            base.CalcBar();
            if (Bars.Status != EBarState.Close) return;
            DateTime day = GlobexDate(Bars.Time[0]);
            if (day != m_day) newDay(day);
            if (rollOn(1)) m_done = true;
            double tick = tickSize(), c = Bars.Close[0], d = Cfg.deltaTicks * tick;
            int pos = StrategyInfo.MarketPosition;
            if (pos != 0)
            {
                m_done = true;
                if (mustFlatten() || (pos > 0 ? c < m_vah - d : c > m_val + d)) { if (pos > 0) m_lx.Send(); e
lse m_sx.Send(); return; }  // hard invalidation
                double e = StrategyInfo.AvgEntryPrice;
                if (pos > 0 ? Bars.High[0] >= e + m_entryRisk : Bars.Low[0] <= e - m_entryRisk) m_stop = pos
> 0 ? Math.Max(m_stop, e) : Math.Min(m_stop, e);   // BE at +1R
                if (pos > 0) { m_lxStp.Send(m_stop); m_lxLmt.Send(m_target); } else { m_sxStp.Send(m_stop); m
_sxLmt.Send(m_target); }
                return;
            }
            TimeSpan tod = ToEastern(Bars.Time[0]).TimeOfDay;
            if (m_done || !m_balanced || !(m_w > 0) || !IsRth() || tod > Cfg.to) return;
            if (m_side == 0)
            {
                if (tod < Cfg.from) return;
                m_up = c > m_vah + d ? m_up + 1 : 0; m_dn = c < m_val - d ? m_dn + 1 : 0;
                if (m_up >= Cfg.acceptCloses && m_vscore >= 1) m_side = +1;
                else if (m_dn >= Cfg.acceptCloses && m_vscore <= -1) m_side = -1;
                if (m_side != 0 && journalOn()) trace(ETraceLevel.Journal, $"PS-01 {day:yyyy-MM-dd} accepted
{(m_side > 0 ? "above VAH" : "below VAL")} W={m_w:F2} V={m_vscore}");
                return;
            }
            double hold = m_side > 0 ? m_vah - Cfg.holdW * m_w : m_val + Cfg.holdW * m_w;
            if (m_side > 0 ? Bars.Low[0] < hold : Bars.High[0] > hold) { m_done = true; return; }   //
pullback failed to hold
            _PriceActionEngine pa = PriceActionEngine;
            if (pa != null && pa.IsWarm)
            {
                _Pivot p = m_side > 0 ? (pa.NewPivotLow ? pa.LastLow(0) : null) : (pa.NewPivotHigh ? pa.LastH
igh(0) : null);
                if (p != null && p.label == (m_side > 0 ? EPivotLabel.HL : EPivotLabel.LH))
                {
                    m_trigger = m_side > 0 ? Bars.High[0] + tick : Bars.Low[0] - tick;
                    double stop = p.price - m_side * tick;
                    if (m_side * (m_trigger - stop) < Cfg.costFloorTicks * tick) stop = m_trigger - m_side *
Cfg.costFloorTicks * tick;
                    m_entryRisk = Math.Abs(m_trigger - stop);
```

```
                    if (overRiskBudget(m_entryRisk)) { if (journalOn()) trace(ETraceLevel.Journal, $"PS-01
{day:yyyy-MM-dd} skip: stop {m_entryRisk / tick:F0} ticks breaches risk budget"); m_done = true; return; }
                    m_stop = stop; m_target = m_side > 0 ? m_vah + Cfg.targetW * m_w : m_val - Cfg.targetW *
m_w;
                    if (journalOn()) journalArm(m_side > 0 ? "PS01LE" : "PS01SE", m_trigger, m_stop,
m_target, null);
                }
            }
            if (double.IsNaN(m_trigger)) return;
            if (m_side > 0) m_leStp.Send(m_trigger, Cfg.contracts); else m_seStp.Send(m_trigger, Cfg.contract
s);
        }

        private void newDay(DateTime day)
        {
            m_day = day; m_done = false; m_up = m_dn = m_side = 0; m_trigger = double.NaN;
            VpExtra comp = Composite(1, Cfg.n);                        // the n completed RTH profiles before
today
            m_balanced = comp != null && IsBalanced(1, Cfg.n, Cfg.balanceOverlap);
            int? v = ValueMigrationScore(1, Cfg.n, 0.10, 20, false);  // completed sessions only (as of the
prior close)
            m_vscore = v ?? 0;
            if (comp != null) { m_vah = comp.vah; m_val = comp.val; m_w = comp.vah - comp.val; } else m_w = d
ouble.NaN;
        }

        private bool overRiskBudget(double stopDist) { return Cfg.contracts * stopDist * Bars.Info.BigPointVa
lue > equityAvailable(Cfg.riskBudgetFraction); }
    }
}
```

3.27 Example 27 (`_Ex27CompositeBreakMarket`) — PS-02 balance break, market entry after acceptance (long side shown; short mirrors) (medium–hard)

Features (direct). F-DEV (§5.1), F-PROF (§5.2), F-COMP (§5.3), F-SESS (§5.6), F-ROLL (§5.7), F-CAL (§5.8), F-TM (§5.12), F-JRNL (§5.13), M-DAILY (§5.19), M-GMM (§5.25).

What it does (from the file header):

> PS-02 balance break, market entry after acceptance (long side shown; short mirrors). Filter: as PS-01 (IsBalanced(RTH, n = 3, 0.6)) + V-score aligned (>= +1 long, <= -1 short) + composite entropy Hnorm <= 0.93. Entry: at market on the next bar after the 2nd consecutive closed 5m bar beyond composite VAH/VAL + delta, 09:45-14:30 ET. Stop: composite VAH - 0.30 W (long) / VAL + 0.30 W (short), floored at 14 ticks from the signal close. Skip the setup (never resize) if 0.30 W < ATR14(5m) + 2 ticks (noise) or 0.30 W > 1.5 x daily ATR (catalogue "daily ATR"; operationalised as M-DAILY Atr20d). Target: 1.0 W beyond the broken edge. Chart: Data1 = @ES 5m, dxFeed continuous UNADJUSTED. Features: F-COMP, F-VP (IsBalanced, ValueMigrationScore), F-PSTAT (ProfileStats.enabled, ProfileEntropy), M-DAILY (Atr20d), F-TM.

Code.

```
// Manual v4 example 27 - PS-02 balance break, market entry after acceptance (long side shown; short
mirrors).
// Filter: as PS-01 (IsBalanced(RTH, n = 3, 0.6)) + V-score aligned (>= +1 long, <= -1 short) + composite
entropy Hnorm <= 0.93.
// Entry: at market on the next bar after the 2nd consecutive closed 5m bar beyond composite VAH/VAL + delta,
09:45-14:30 ET.
// Stop: composite VAH - 0.30 W (long) / VAL + 0.30 W (short), floored at 14 ticks from the signal close.
Skip the setup (never resize)
// if 0.30 W < ATR14(5m) + 2 ticks (noise) or 0.30 W > 1.5 x daily ATR (catalogue "daily ATR";
operationalised as M-DAILY Atr20d).
// Target: 1.0 W beyond the broken edge. Chart: Data1 = @ES 5m, dxFeed continuous UNADJUSTED.
// Features: F-COMP, F-VP (IsBalanced, ValueMigrationScore), F-PSTAT (ProfileStats.enabled, ProfileEntropy),
M-DAILY (Atr20d), F-TM.
using System;
using PowerLanguage.Function;

namespace PowerLanguage.Strategy
{
```

```
    public class _Ex27CompositeBreakMarket : _VolumeProfileFuturesBase
    {
        protected sealed class Ps02Config
        {
            public int contracts = 1, n = 3, deltaTicks = 2, acceptCloses = 2, atrLength = 14, noiseTicks =
2;
            public double stopW = 0.30, targetW = 1.0, maxEntropy = 0.93, maxDailyAtr = 1.5, balanceOverlap
= 0.6;
            public TimeSpan from = new TimeSpan(9, 45, 0), to = new TimeSpan(14, 30, 0), flat = new
TimeSpan(15, 55, 0);
            public int costFloorTicks = 14;
            public double riskBudgetFraction = 0.01;
        }
        protected Ps02Config Cfg { get; } = new Ps02Config();

        private IOrderMarket m_le, m_se, m_lx, m_sx;
        private IOrderPriced m_lxStp, m_sxStp, m_lxLmt, m_sxLmt;
        private DateTime m_day = DateTime.MinValue;
        private bool m_ok, m_done;
        private int m_vscore, m_up, m_dn;
        private double m_vah, m_val, m_w, m_stop, m_target;

        public _Ex27CompositeBreakMarket(object _ctx) : base(_ctx) { }

        protected override void Create()
        {
            base.Create();
            RollDetection = ERollDetection.Series;
            DevelopingSource = EDevelopingSource.Own; OwnProfileWindow = EOwnProfileWindow.Rth;
            ProfileStats.enabled = true;              // lazy: F-PSTAT helpers (entropy) only exist when
enabled
            FlatBy = Cfg.flat;
            m_le = OrderCreator.MarketNextBar(new SOrderParameters(Contracts.UserSpecified, "PS02LE", EOrderA
ction.Buy));
            m_se = OrderCreator.MarketNextBar(new SOrderParameters(Contracts.UserSpecified, "PS02SE", EOrderA
ction.SellShort));
            m_lx = OrderCreator.MarketNextBar(new SOrderParameters(Contracts.Default, "PS02LX",
EOrderAction.Sell));
            m_sx = OrderCreator.MarketNextBar(new SOrderParameters(Contracts.Default, "PS02SX",
EOrderAction.BuyToCover));
            m_lxStp = OrderCreator.Stop(new SOrderParameters(Contracts.Default, "PS02LS", EOrderAction.Sell,
OrderExit.FromAll));
            m_sxStp = OrderCreator.Stop(new SOrderParameters(Contracts.Default, "PS02SS", EOrderAction.BuyToC
over, OrderExit.FromAll));
            m_lxLmt = OrderCreator.Limit(new SOrderParameters(Contracts.Default, "PS02LT",
EOrderAction.Sell, OrderExit.FromAll));
            m_sxLmt = OrderCreator.Limit(new SOrderParameters(Contracts.Default, "PS02ST", EOrderAction.BuyTo
Cover, OrderExit.FromAll));
        }

        protected override void StartCalc() { NeedsDaily = true; base.StartCalc(); m_day =
DateTime.MinValue; }

        protected override void CalcBar()
        {
            base.CalcBar();
            if (Bars.Status != EBarState.Close) return;
            DateTime day = GlobexDate(Bars.Time[0]);
            if (day != m_day) newDay(day);
            if (rollOn(1)) m_done = true;
            int pos = StrategyInfo.MarketPosition;
            if (pos != 0)
            {
                m_done = true;
                if (mustFlatten()) { if (pos > 0) m_lx.Send(); else m_sx.Send(); return; }
                if (pos > 0) { m_lxStp.Send(m_stop); m_lxLmt.Send(m_target); } else { m_sxStp.Send(m_stop); m
_sxLmt.Send(m_target); }
                return;
            }
            TimeSpan tod = ToEastern(Bars.Time[0]).TimeOfDay;
            if (m_done || !m_ok || !IsRth() || tod < Cfg.from || tod > Cfg.to) return;
            double tick = tickSize(), c = Bars.Close[0], d = Cfg.deltaTicks * tick;
            m_up = c > m_vah + d ? m_up + 1 : 0; m_dn = c < m_val - d ? m_dn + 1 : 0;
            int side = m_up >= Cfg.acceptCloses && m_vscore >= 1 ? +1 : m_dn >= Cfg.acceptCloses && m_vscore
<= -1 ? -1 : 0;
            if (side == 0) return;
            m_done =
```

```
true;                                          // one attempt per day, whatever the skip outcome
            double sw = Cfg.stopW * m_w, atr = averageTrueRange(Bars, Cfg.atrLength);
            double atrD = DailyContext != null ? DailyContext.Atr20d : double.NaN;
            string why = !(atrD > 0) ? "Atr20d NaN" : sw < atr + Cfg.noiseTicks * tick ? "0.30W inside 5m
noise" : sw > Cfg.maxDailyAtr * atrD ? "0.30W > 1.5 daily ATR" : null;
            double stop = side > 0 ? m_vah - sw : m_val + sw;
            if (side * (c - stop) < Cfg.costFloorTicks * tick) stop = c - side * Cfg.costFloorTicks * tick;
            if (why == null && Cfg.contracts * Math.Abs(c - stop) * Bars.Info.BigPointValue >
equityAvailable(Cfg.riskBudgetFraction)) why = "risk budget";
            if (why != null) { if (journalOn()) trace(ETraceLevel.Journal, $"PS-02 {day:yyyy-MM-dd} skip:
{why}"); return; }
            m_stop = stop; m_target = side > 0 ? m_vah + Cfg.targetW * m_w : m_val - Cfg.targetW * m_w;
            if (journalOn()) journalArm(side > 0 ? "PS02LE" : "PS02SE", c, m_stop, m_target, null);
            if (side > 0) m_le.Send(Cfg.contracts); else m_se.Send(Cfg.contracts);
        }

        private void newDay(DateTime day)
        {
            m_day = day; m_done = false; m_up = m_dn = 0;
            VpExtra comp = Composite(1, Cfg.n);
            m_ok = comp != null && IsBalanced(1, Cfg.n, Cfg.balanceOverlap);
            if (m_ok)
            {
                EntropyResult h = ProfileEntropy(comp, comp.endBar - comp.startBar + 1);   // mmN = bars in
the window (EMmN.Bars convention)
                m_ok = h.Hnorm <= Cfg.maxEntropy;
                m_vah = comp.vah; m_val = comp.val; m_w = comp.vah - comp.val;
                m_ok &= m_w > 0;
            }
            m_vscore = ValueMigrationScore(1, Cfg.n, 0.10, 20, false) ?? 0;
        }
    }
}
```

3.28 Example 29 (`_Ex29LvnEdgeRejection`) — PS-22 composite LVN-edge rejection, with the HVN-ceiling veto as a separate (switchable) layer (medium–hard)

Features (direct). F-DEV (§5.1), F-COMP (§5.3), F-HVN (§5.5), F-SESS (§5.6), F-ROLL (§5.7), F-CAL (§5.8), F-TM (§5.12), F-JRNL (§5.13), F-EVT (§5.14).

What it does (from the file header):

> PS-22 composite LVN-edge rejection, with the HVN-ceiling veto as a separate (switchable) layer. Level: LVN edges of the 20-session RTH composite (Composite(1, 20).lvns). Long: a 5m bar trades down into an LVN (low <= lvn.high) from above and closes back above lvn.high + delta in its upper half (rejection close, PS-16 without the slowness filter). Short mirrors at lvn.low. Entry: market next bar. Stop: max(bar extreme -/+ 1 tick, 14-tick floor). Target 1 R. Veto (Cfg.hvnVeto, tested separately): skip a long whose 1 R target lies beyond the low of the nearest composite HVN above. Window 09:45-15:00 ET, one trade per day. Chart: Data1 = @ES 5m, dxFeed continuous UNADJUSTED. Features: F-COMP (lvns/hvns), F-TM.

Code.

```
// Manual v4 example 29 - PS-22 composite LVN-edge rejection, with the HVN-ceiling veto as a separate
(switchable) layer.
// Level: LVN edges of the 20-session RTH composite (Composite(1, 20).lvns). Long: a 5m bar trades down into
an LVN (low <= lvn.high)
// from above and closes back above lvn.high + delta in its upper half (rejection close, PS-16 without the
slowness filter).
// Short mirrors at lvn.low. Entry: market next bar. Stop: max(bar extreme -/+ 1 tick, 14-tick floor). Target
1 R.
// Veto (Cfg.hvnVeto, tested separately): skip a long whose 1 R target lies beyond the low of the nearest
composite HVN above.
// Window 09:45-15:00 ET, one trade per day. Chart: Data1 = @ES 5m, dxFeed continuous UNADJUSTED. Features:
F-COMP (lvns/hvns), F-TM.
using System;
using PowerLanguage.Function;
```

```
namespace PowerLanguage.Strategy
{
    public class _Ex29LvnEdgeRejection : _VolumeProfileFuturesBase
    {
        protected sealed class Ps22Config
        {
            public int contracts = 1, n = 20, deltaTicks = 1;
            public bool hvnVeto = false;              // the veto layer is its own test arm
            public TimeSpan from = new TimeSpan(9, 45, 0), to = new TimeSpan(15, 0, 0), flat = new TimeSpan(1
5, 55, 0);
            public int costFloorTicks = 14;
            public double riskBudgetFraction = 0.01;
        }
        protected Ps22Config Cfg { get; } = new Ps22Config();

        private IOrderMarket m_le, m_se, m_lx, m_sx;
        private IOrderPriced m_lxStp, m_sxStp, m_lxLmt, m_sxLmt;
        private DateTime m_day = DateTime.MinValue;
        private VpExtra m_comp;
        private bool m_done;
        private double m_stop, m_target;

        public _Ex29LvnEdgeRejection(object _ctx) : base(_ctx) { }

        protected override void Create()
        {
            base.Create();
            RollDetection = ERollDetection.Series;
            DevelopingSource = EDevelopingSource.Own; OwnProfileWindow = EOwnProfileWindow.Rth;
            FlatBy = Cfg.flat; MaxTradesPerDay = 1;
            m_le = OrderCreator.MarketNextBar(new SOrderParameters(Contracts.UserSpecified, "PS22LE", EOrderA
ction.Buy));
            m_se = OrderCreator.MarketNextBar(new SOrderParameters(Contracts.UserSpecified, "PS22SE", EOrderA
ction.SellShort));
            m_lx = OrderCreator.MarketNextBar(new SOrderParameters(Contracts.Default, "PS22LX",
EOrderAction.Sell));
            m_sx = OrderCreator.MarketNextBar(new SOrderParameters(Contracts.Default, "PS22SX",
EOrderAction.BuyToCover));
            m_lxStp = OrderCreator.Stop(new SOrderParameters(Contracts.Default, "PS22LS", EOrderAction.Sell,
OrderExit.FromAll));
            m_sxStp = OrderCreator.Stop(new SOrderParameters(Contracts.Default, "PS22SS", EOrderAction.BuyToC
over, OrderExit.FromAll));
            m_lxLmt = OrderCreator.Limit(new SOrderParameters(Contracts.Default, "PS22LT",
EOrderAction.Sell, OrderExit.FromAll));
            m_sxLmt = OrderCreator.Limit(new SOrderParameters(Contracts.Default, "PS22ST", EOrderAction.BuyTo
Cover, OrderExit.FromAll));
        }

        protected override void StartCalc() { base.StartCalc(); m_day = DateTime.MinValue; }

        protected override void CalcBar()
        {
            base.CalcBar();
            if (Bars.Status != EBarState.Close) return;
            DateTime day = GlobexDate(Bars.Time[0]);
            if (day != m_day) { m_day = day; m_done = false; m_comp = Composite(1, Cfg.n); }
            if (rollOn(1)) m_done = true;
            int pos = StrategyInfo.MarketPosition;
            if (pos != 0)
            {
                m_done = true;
                if (mustFlatten()) { if (pos > 0) m_lx.Send(); else m_sx.Send(); return; }
                if (pos > 0) { m_lxStp.Send(m_stop); m_lxLmt.Send(m_target); } else { m_sxStp.Send(m_stop); m
_sxLmt.Send(m_target); }
                return;
            }
            TimeSpan tod = ToEastern(Bars.Time[0]).TimeOfDay;
            if (m_done || m_comp == null || m_comp.crossesRoll || !IsRth() || tod < Cfg.from || tod > Cfg.to
|| !entryAllowed(out string _)) return;
            double tick = tickSize(), d = Cfg.deltaTicks * tick, h = Bars.High[0], l = Bars.Low[0], c =
Bars.Close[0], mid = (h + l) / 2;
            foreach (Node z in m_comp.lvns)
            {
                int side = 0;
                if (l <= z.high && Bars.Close[1] > z.high && c > z.high + d && c >= mid) side = +1;       //
support edge held
                else if (h >= z.low && Bars.Close[1] < z.low && c < z.low - d && c <= mid) side = -1;    //
```

```
resistance edge held
                if (side == 0) continue;
                double stop = side > 0 ? Math.Min(l - tick, c - Cfg.costFloorTicks * tick) : Math.Max(h + tic
k, c + Cfg.costFloorTicks * tick);
                double r = Math.Abs(c - stop), target = c + side * r;
                string why = null;
                if (Cfg.contracts * r * Bars.Info.BigPointValue > equityAvailable(Cfg.riskBudgetFraction)) wh
y = "risk budget";
                else if (Cfg.hvnVeto && side > 0 && hvnCeilingBelow(c, target)) why = "1R target beyond HVN
ceiling";
                m_done = true;
                if (why != null) { logSignal("PS22", side, c, stop, null, false); if (journalOn()) trace(ETra
ceLevel.Journal, $"PS-22 {day:yyyy-MM-dd} skip: {why}"); return; }
                m_stop = stop; m_target = target;
                logSignal("PS22", side, c, stop, null, true);
                if (side > 0) m_le.Send(Cfg.contracts); else m_se.Send(Cfg.contracts);
                return;
            }
        }

        private bool hvnCeilingBelow(double entry, double target)
        {
            double best = double.MaxValue;
            foreach (Node n in m_comp.hvns) if (n.low > entry && n.low < best) best = n.low;
            return best < target;
        }
    }
}
```

3.29 Example 30 (`_Ex30RejectedRepricingOpen`) — PS-05 rejected repricing open (Keller IX-3; gap-up side shown in words, both sides coded) (medium–hard)

Features (direct). F-DEV (§5.1), F-PROF (§5.2), F-SESS (§5.6), F-ROLL (§5.7), F-CAL (§5.8), F-TM (§5.12), F-JRNL (§5.13), M-DAILY (§5.19).

What it does (from the file header):

> PS-05 rejected repricing open (Keller IX-3; gap-up side shown in words, both sides coded). Filter: RTH open outside the prior RTH value area by >= 0.25 x ATR20d, with |open - prior RTH close| <= 1.5 x ATR20d. Signal: by 10:15 ET, two 5m closes back inside the prior VA (gap up: close <= prior VAH). Entry: sell stop 1 tick through the low of the second inside bar (buy stop through the high for a gap down). Stop: max(session extreme + 1 tick, 14-tick floor). Target: prior POC (the "then opposite VA edge" leg is not coded: one target). The prior profile is skipped if it crosses a roll. Chart: Data1 = @ES 5m, dxFeed continuous UNADJUSTED. Features: F-VP (PriorRthProfile), F-SESS (Session), M-DAILY (Atr20d), F-TM.

Code.

```
// Manual v4 example 30 - PS-05 rejected repricing open (Keller IX-3; gap-up side shown in words, both sides
coded).
// Filter: RTH open outside the prior RTH value area by >= 0.25 x ATR20d, with |open - prior RTH close| <=
1.5 x ATR20d.
// Signal: by 10:15 ET, two 5m closes back inside the prior VA (gap up: close <= prior VAH). Entry: sell stop
1 tick through the low
// of the second inside bar (buy stop through the high for a gap down). Stop: max(session extreme + 1 tick,
14-tick floor).
// Target: prior POC (the "then opposite VA edge" leg is not coded: one target). The prior profile is skipped
if it crosses a roll.
// Chart: Data1 = @ES 5m, dxFeed continuous UNADJUSTED. Features: F-VP (PriorRthProfile), F-SESS (Session),
M-DAILY (Atr20d), F-TM.
using System;
using PowerLanguage.Function;

namespace PowerLanguage.Strategy
{
    public class _Ex30RejectedRepricingOpen : _VolumeProfileFuturesBase
    {
        protected sealed class Ps05Config
```

```
        {
            public int contracts = 1, insideCloses = 2;
            public double minOutsideAtr = 0.25, maxGapAtr = 1.5;
            public TimeSpan signalBy = new TimeSpan(10, 15, 0), flat = new TimeSpan(15, 55, 0);
            public int costFloorTicks = 14;
            public double riskBudgetFraction = 0.01;
        }
        protected Ps05Config Cfg { get; } = new Ps05Config();

        private IOrderPriced m_leStp, m_seStp, m_lxStp, m_sxStp, m_lxLmt, m_sxLmt;
        private IOrderMarket m_lx, m_sx;
        private DateTime m_day = DateTime.MinValue;
        private bool m_opened, m_done;
        private int m_side, m_inside;
        private double m_vah, m_val, m_poc, m_trigger = double.NaN, m_stop;

        public _Ex30RejectedRepricingOpen(object _ctx) : base(_ctx) { }

        protected override void Create()
        {
            base.Create();
            RollDetection = ERollDetection.Series;
            DevelopingSource = EDevelopingSource.Own; OwnProfileWindow = EOwnProfileWindow.Rth;   //
PriorRthProfile needs Own
            FlatBy = Cfg.flat;
            m_leStp = OrderCreator.Stop(new SOrderParameters(Contracts.UserSpecified, "PS05LE",
EOrderAction.Buy));
            m_seStp = OrderCreator.Stop(new SOrderParameters(Contracts.UserSpecified, "PS05SE",
EOrderAction.SellShort));
            m_lx = OrderCreator.MarketNextBar(new SOrderParameters(Contracts.Default, "PS05LX",
EOrderAction.Sell));
            m_sx = OrderCreator.MarketNextBar(new SOrderParameters(Contracts.Default, "PS05SX",
EOrderAction.BuyToCover));
            m_lxStp = OrderCreator.Stop(new SOrderParameters(Contracts.Default, "PS05LS", EOrderAction.Sell,
OrderExit.FromAll));
            m_sxStp = OrderCreator.Stop(new SOrderParameters(Contracts.Default, "PS05SS", EOrderAction.BuyToC
over, OrderExit.FromAll));
            m_lxLmt = OrderCreator.Limit(new SOrderParameters(Contracts.Default, "PS05LT",
EOrderAction.Sell, OrderExit.FromAll));
            m_sxLmt = OrderCreator.Limit(new SOrderParameters(Contracts.Default, "PS05ST", EOrderAction.BuyTo
Cover, OrderExit.FromAll));
        }

        protected override void StartCalc() { NeedsDaily = true; base.StartCalc(); m_day =
DateTime.MinValue; }

        protected override void CalcBar()
        {
            base.CalcBar();
            if (Bars.Status != EBarState.Close) return;
            DateTime day = GlobexDate(Bars.Time[0]);
            if (day != m_day) { m_day = day; m_opened = m_done = false; m_side = m_inside = 0; m_trigger = do
uble.NaN; }
            if (rollOn(1)) m_done = true;
            int pos = StrategyInfo.MarketPosition;
            if (pos != 0)
            {
                m_done = true;
                if (mustFlatten()) { if (pos > 0) m_lx.Send(); else m_sx.Send(); return; }
                if (pos > 0) { m_lxStp.Send(m_stop); m_lxLmt.Send(m_poc); } else { m_sxStp.Send(m_stop); m_sx
Lmt.Send(m_poc); }
                return;
            }
            if (m_done || !IsRth()) return;
            SessionStats s = Session;
            double tick = tickSize(), c = Bars.Close[0];
            if (!m_opened)
            {
                m_opened =
true;                                   // first closed RTH bar: classify the open once
                VpExtra prior = PriorRthProfile();
                double atr = DailyContext != null ? DailyContext.Atr20d : double.NaN;
                if (prior == null || prior.crossesRoll || !(atr > 0) || s == null || !(s.rthOpen > 0)) { m_do
ne = true; return; }
                m_vah = prior.vah; m_val = prior.val; m_poc = prior.poc;
                double o = s.rthOpen, gap = Math.Abs(o - s.priorRthClose);
                if (gap > Cfg.maxGapAtr * atr) { m_done = true; return; }
```

```
                m_side = o >= m_vah + Cfg.minOutsideAtr * atr ? -1 : o <= m_val - Cfg.minOutsideAtr * atr ?
+1 : 0;   // fade side
                if (m_side == 0) { m_done = true; return; }
            }
            if (double.IsNaN(m_trigger))
            {
                if (ToEastern(Bars.Time[0]).TimeOfDay > Cfg.signalBy) { m_done = true; return; }
                bool inside = m_side < 0 ? c <= m_vah : c >= m_val;
                m_inside = inside ? m_inside + 1 : 0;
                if (m_inside < Cfg.insideCloses) return;
                double trig = m_side < 0 ? Bars.Low[0] - tick : Bars.High[0] + tick;
                double stop = m_side < 0 ? Math.Max(s.rthHigh + tick, trig + Cfg.costFloorTicks * tick) : Mat
h.Min(s.rthLow - tick, trig - Cfg.costFloorTicks * tick);
                if (m_side * (m_poc - trig) <= 0 || Cfg.contracts * Math.Abs(trig - stop) * Bars.Info.BigPoin
tValue > equityAvailable(Cfg.riskBudgetFraction))
                { if (journalOn()) trace(ETraceLevel.Journal, $"PS-05 {day:yyyy-MM-dd} skip: target behind
trigger or risk budget"); m_done = true; return; }
                m_trigger = trig; m_stop = stop;
                if (journalOn()) journalArm(m_side > 0 ? "PS05LE" : "PS05SE", m_trigger, m_stop, m_poc, null)
;
            }
            if (m_side > 0) m_leStp.Send(m_trigger, Cfg.contracts); else m_seStp.Send(m_trigger, Cfg.contract
s);
        }
    }
}
```

3.30 Example 38 (`_Ex38StackedImbalanceRetest`) — OF-01 stacked-imbalance origin, first retest continuation (long side shown; short mirrors) (medium–hard)

Features (direct). F-SESS (§5.6), F-ROLL (§5.7), F-CAL (§5.8), F-OF (§5.9), F-TM (§5.12), F-JRNL (§5.13).

What it does (from the file header):

> OF-01 stacked-imbalance origin, first retest continuation (long side shown; short mirrors). Zone: a closed 5m bar with >= 3 stacked BUY imbalances (ratio 3:1, VPF-5 StackedImbalances) that closes in its upper 30%; zone = the lowest buy run [lo, hi]. Flow quality must be Exact or High (the catalogue warns TickRule imbalances are artefacts). Arming: price then moves >= 1 x ATR(14, 5m) above the zone top. Entry: first retest only - buy limit at the zone top (catalogue entry arm "limit"; the 1m HL arm is not coded). Stop: zone low - 1 tick, floored at 14 ticks. Target: 1R. Deviation: the catalogue's volume floor P70 is not computed; minVol = 10 contracts is a PLACEHOLDER. Zone expires at session end. Chart: Data1 @ES 5m, dxFeed continuous UNADJUSTED. Features: VPF order flow (OrderFlowEnabled, StackedImbalances, FlowQuality).

Pitfalls.

- Contains PLACEHOLDER values (marked in the code); calibrate them before reading any result.

Code.

```
// Manual v4 example 38 - OF-01 stacked-imbalance origin, first retest continuation (long side shown; short
mirrors).
// Zone: a closed 5m bar with >= 3 stacked BUY imbalances (ratio 3:1, VPF-5 StackedImbalances) that closes in
its upper 30%;
// zone = the lowest buy run [lo, hi]. Flow quality must be Exact or High (the catalogue warns TickRule
imbalances are artefacts).
// Arming: price then moves >= 1 x ATR(14, 5m) above the zone top. Entry: first retest only - buy limit at
the zone top
// (catalogue entry arm "limit"; the 1m HL arm is not coded). Stop: zone low - 1 tick, floored at 14 ticks.
Target: 1R.
// Deviation: the catalogue's volume floor P70 is not computed; minVol = 10 contracts is a PLACEHOLDER. Zone
expires at session end.
// Chart: Data1 @ES 5m, dxFeed continuous UNADJUSTED. Features: VPF order flow (OrderFlowEnabled,
StackedImbalances, FlowQuality).
using System;
using PowerLanguage.Function;
```

```
namespace PowerLanguage.Strategy
{
    public class _Ex38StackedImbalanceRetest : _VolumeProfileFuturesBase
    {
        protected sealed class Of01Config
        {
            public int contracts = 1, minStack = 3, atrLength = 14, costFloorTicks = 14;
            public double ratio = 3.0, minVol = 10 /* PLACEHOLDER for P70 */, closeTopFrac = 0.30, awayAtr =
1.0, targetR = 1.0, riskBudgetFraction = 0.01;
            public TimeSpan flat = new TimeSpan(15, 55, 0);
        }
        protected Of01Config Cfg { get; } = new Of01Config();

        private IOrderPriced m_leLmt, m_lxStp, m_lxLmt;
        private IOrderMarket m_lx;
        private DateTime m_day = DateTime.MinValue;
        private bool m_zone, m_armed, m_used;
        private double m_zLo, m_zHi, m_stop, m_target;

        public _Ex38StackedImbalanceRetest(object _ctx) : base(_ctx) { }

        protected override void Create()
        {
            base.Create();
            RollDetection = ERollDetection.Series;
            OrderFlowEnabled = true;                          // lazy: footprints only exist when enabled
            FlatBy = Cfg.flat;
            m_leLmt = OrderCreator.Limit(new SOrderParameters(Contracts.UserSpecified, "OF01LE",
EOrderAction.Buy));
            m_lx = OrderCreator.MarketNextBar(new SOrderParameters(Contracts.Default, "OF01LX",
EOrderAction.Sell));
            m_lxStp = OrderCreator.Stop(new SOrderParameters(Contracts.Default, "OF01LS", EOrderAction.Sell,
OrderExit.FromAll));
            m_lxLmt = OrderCreator.Limit(new SOrderParameters(Contracts.Default, "OF01LT",
EOrderAction.Sell, OrderExit.FromAll));
        }

        protected override void StartCalc() { base.StartCalc(); m_day = DateTime.MinValue; }

        protected override void CalcBar()
        {
            base.CalcBar();
            if (Bars.Status != EBarState.Close) return;
            DateTime day = GlobexDate(Bars.Time[0]);
            if (day != m_day) { m_day = day; m_zone = m_armed = m_used = false; }
            if (rollOn(1)) m_zone = false;
            int pos = StrategyInfo.MarketPosition;
            if (pos != 0)
            {
                m_used = true;
                if (mustFlatten()) { m_lx.Send(); return; }
                m_lxStp.Send(m_stop); m_lxLmt.Send(m_target);
                return;
            }
            if (!IsRth()) return;
            double tick = tickSize(), h = Bars.High[0], l = Bars.Low[0], c = Bars.Close[0];
            double atr = averageTrueRange(Bars, Cfg.atrLength);
            if (m_zone && !m_used)
            {
                if (!m_armed && h >= m_zHi + Cfg.awayAtr * atr) m_armed = true;
                else if (m_armed && l <= m_zHi) { m_used = true; m_zone = false; return; }   // the retest
happened without a fill: first retest only
                if (m_armed && !mustFlatten()) { m_leLmt.Send(m_zHi, Cfg.contracts); return; }
            }
            if (m_zone || m_used || h - l <= 0 || (c - l) / (h - l) < 1 - Cfg.closeTopFrac) return;
            _OrderFlow.StackedResult s = StackedImbalances(0, Cfg.ratio, Cfg.minVol, Cfg.minStack);
            if (!s.Found || s.Quality > _OrderFlow.FlowQuality.High) return;
            foreach (double[] run in s.Runs)
            {
                if (run[3] <= 0) continue;                    // buy runs first, low -> high: take the
lowest
                double stop = run[0] - tick, entry = run[1];
                if (entry - stop < Cfg.costFloorTicks * tick) stop = entry - Cfg.costFloorTicks * tick;
                double r = entry - stop;
                if (Cfg.contracts * r * Bars.Info.BigPointValue > equityAvailable(Cfg.riskBudgetFraction))
                { if (journalOn()) trace(ETraceLevel.Journal, $"OF-01 {Bars.Time[0]:yyyy-MM-dd HH:mm} skip:
risk budget"); return; }
```

```
                m_zLo = run[0]; m_zHi = run[1]; m_stop = stop; m_target = entry + Cfg.targetR * r; m_zone = t
rue; m_armed = false;
                if (journalOn()) journalArm("OF01LE", entry, m_stop, m_target, null);
                return;
            }
        }
    }
}
```

3.31 Example 31 (`_Ex31MultiTfStack`) — MT-01 multi-timeframe stack (long only, as catalogued) (medium–hard)

Features (direct). F-DEV (§5.1), F-COMP (§5.3), F-SESS (§5.6), F-ROLL (§5.7), F-CAL (§5.8), F-TM (§5.12), F-JRNL (§5.13), M-SWING (§5.16), M-DAILY (§5.19).

What it does (from the file header):

> MT-01 multi-timeframe stack (long only, as catalogued). Filter: M-DAILY Tsmom(20) > 0; last closed 60m structure Up (HH/HL) from a second PRICEACTION engine on Data3; the latest 15m pivot low (third engine, Data2) lies within delta15 of the 20-session composite VAH or the developing RTH POC (delta15 = 0.25 x the 15m engine ATR: PLACEHOLDER, not a catalogue value; swing-AVWAP level omitted). Entry: the next 5m pivot low labelled HL (Data1 engine), buy stop 1 tick above the confirming bar. Stop: max(HL - 1 tick, floor). Target: the last 15m pivot high. RTH 09:45-15:00 ET; one trade per day. Chart: Data1 = @ES 5m, Data2 = @ES 15m, Data3 = @ES 60m (same symbol, dxFeed continuous UNADJUSTED). Features: M-PA (PriceActionConfig + ExtraPriceAction + PriceActionFor, Structure), M-DAILY (Tsmom), F-COMP, F-VP (RthProfile).

Pitfalls.

- Contains PLACEHOLDER values (marked in the code); calibrate them before reading any result.

Code.

```
// Manual v4 example 31 - MT-01 multi-timeframe stack (long only, as catalogued).
// Filter: M-DAILY Tsmom(20) > 0; last closed 60m structure Up (HH/HL) from a second PRICEACTION engine on
Data3; the latest 15m
// pivot low (third engine, Data2) lies within delta15 of the 20-session composite VAH or the developing RTH
POC
// (delta15 = 0.25 x the 15m engine ATR: PLACEHOLDER, not a catalogue value; swing-AVWAP level omitted).
// Entry: the next 5m pivot low labelled HL (Data1 engine), buy stop 1 tick above the confirming bar. Stop:
max(HL - 1 tick, floor).
// Target: the last 15m pivot high. RTH 09:45-15:00 ET; one trade per day.
// Chart: Data1 = @ES 5m, Data2 = @ES 15m, Data3 = @ES 60m (same symbol, dxFeed continuous UNADJUSTED).
// Features: M-PA (PriceActionConfig + ExtraPriceAction + PriceActionFor, Structure), M-DAILY (Tsmom), F-
COMP, F-VP (RthProfile).
using System;
using PowerLanguage.Function;

namespace PowerLanguage.Strategy
{
    public class _Ex31MultiTfStack : _VolumeProfileFuturesBase
    {
        protected sealed class Mt01Config
        {
            public int contracts = 1, tsmomDays = 20, n = 20, ds15 = 2, ds60 = 3;
            public double nearAtr15 = 0.25;   // PLACEHOLDER
            public TimeSpan from = new TimeSpan(9, 45, 0), to = new TimeSpan(15, 0, 0), flat = new TimeSpan(1
5, 55, 0);
            public int costFloorTicks = 14;
            public double riskBudgetFraction = 0.01;
        }
        protected Mt01Config Cfg { get; } = new Mt01Config();

        private IOrderPriced m_leStp, m_lxStp, m_lxLmt;
        private IOrderMarket m_lx;
        private DateTime m_day = DateTime.MinValue;
        private VpExtra m_comp;
        private bool m_done;
```

```
        private double m_trigger = double.NaN, m_stop, m_target;

        public _Ex31MultiTfStack(object _ctx) : base(_ctx) { }

        protected override void Create()
        {
            base.Create();
            RollDetection = ERollDetection.Series;
            DevelopingSource = EDevelopingSource.Own; OwnProfileWindow = EOwnProfileWindow.Rth;
            PriceActionConfig.enabled = true; PriceActionConfig.dataStream = 1;                    // 5m
trigger engine
            ExtraPriceAction.Add(new PriceActionConfig { enabled = true, dataStream = Cfg.ds15, role =
"pb15" });
            ExtraPriceAction.Add(new PriceActionConfig { enabled = true, dataStream = Cfg.ds60, role = "trend
60" });
            FlatBy = Cfg.flat; MaxTradesPerDay = 1;
            m_leStp = OrderCreator.Stop(new SOrderParameters(Contracts.UserSpecified, "MT01LE",
EOrderAction.Buy));
            m_lx = OrderCreator.MarketNextBar(new SOrderParameters(Contracts.Default, "MT01LX",
EOrderAction.Sell));
            m_lxStp = OrderCreator.Stop(new SOrderParameters(Contracts.Default, "MT01LS", EOrderAction.Sell,
OrderExit.FromAll));
            m_lxLmt = OrderCreator.Limit(new SOrderParameters(Contracts.Default, "MT01LT",
EOrderAction.Sell, OrderExit.FromAll));
        }

        protected override void StartCalc() { NeedsDaily = true; base.StartCalc(); m_day =
DateTime.MinValue; }

        protected override void CalcBar()
        {
            base.CalcBar();
            if (Bars.Status != EBarState.Close) return;
            DateTime day = GlobexDate(Bars.Time[0]);
            if (day != m_day) { m_day = day; m_done = false; m_trigger = double.NaN; m_comp = Composite(1, Cf
g.n); }
            if (rollOn(1)) m_done = true;
            if (StrategyInfo.MarketPosition > 0)
            {
                m_done = true;
                if (mustFlatten()) { m_lx.Send(); return; }
                m_lxStp.Send(m_stop); m_lxLmt.Send(m_target); return;
            }
            TimeSpan tod = ToEastern(Bars.Time[0]).TimeOfDay;
            if (m_done || !IsRth() || tod < Cfg.from || tod > Cfg.to) return;
            if (double.IsNaN(m_trigger)) arm(day);
            if (!double.IsNaN(m_trigger)) m_leStp.Send(m_trigger, Cfg.contracts);
        }

        private void arm(DateTime day)
        {
            _PriceActionEngine pa5 = PriceActionEngine, pa15 = PriceActionFor(Cfg.ds15, "pb15"), pa60 = Price
ActionFor(Cfg.ds60, "trend60");
            if (pa5 == null || pa15 == null || pa60 == null || !pa5.IsWarm || !pa15.IsWarm || !pa60.IsWarm
|| !pa5.NewPivotLow) return;
            double ts = DailyContext != null ? DailyContext.Tsmom(Cfg.tsmomDays) : double.NaN;
            if (!(ts > 0) || pa60.Structure(0) != EStructure.Up) return;
            _Pivot pb = pa15.LastLow(0), sh = pa15.LastHigh(0), hl = pa5.LastLow(0);
            if (pb == null || sh == null || hl == null || hl.label != EPivotLabel.HL) return;
            VpExtra dev = RthProfile();
            double near = Cfg.nearAtr15 * pa15.Atr;
            bool atLevel = (m_comp != null && Math.Abs(pb.price - m_comp.vah) <= near) || (dev != null &&
dev.poc > 0 && Math.Abs(pb.price - dev.poc) <= near);
            if (!atLevel) return;
            double tick = tickSize(), trig = Bars.High[0] + tick, stop = Math.Min(hl.price - tick, trig -
Cfg.costFloorTicks * tick);
            string why = sh.price <= trig ? "15m swing high below trigger" : Cfg.contracts * (trig - stop) *
Bars.Info.BigPointValue > equityAvailable(Cfg.riskBudgetFraction) ? "risk budget" : null;
            if (why != null) { if (journalOn()) trace(ETraceLevel.Journal, $"MT-01 {day:yyyy-MM-dd} skip:
{why}"); m_done = true; return; }
            m_trigger = trig; m_stop = stop; m_target = sh.price;
            if (journalOn()) journalArm("MT01LE", m_trigger, m_stop, m_target, null);
        }
    }
}
```

3.32 Example 35 (`_Ex35PocStabilityGate`) — A2-25 POC bootstrap-stability gate + A2-26 KDE bandwidth-sweep veto, as a module on the DV-02 host (Ex32) (medium–hard)

Features (direct). M-GMM (§5.25), VP helpers (§5.27). **Inherited from example 32:** F-DEV, F-PROF, F-SESS, F-ROLL, F-CAL, F-TM, F-JRNL, F-EVT, M-DAILY.

What it does (from the file header):

> A2-25 POC bootstrap-stability gate + A2-26 KDE bandwidth-sweep veto, as a module on the DV-02 host (Ex32). After the prior close, the prior RTH session's Data1 ring bars (SessionBars) feed _ProfileStats.BootstrapPoc (block 5, 500 draws, deterministic BootstrapSeed) and _ProfileStats.KdeSweep. Rules: KDE verdict FLAT or BIMODAL -> no trade (A2-26); stability >= 0.60 -> trade, stop distance = max(host bracket, ci90/2 + 1 tick) (A2-25); 0.25 <= stability < 0.60 -> the catalogue allows the trade only with the POC ZONE as the level; the DV-02 host is line-based, so this module skips that bucket (stated, not a catalogue rule); stability < 0.25 -> skip. Skip if the widened stop makes the target < 1R (the DV-02 bracket is symmetric, so the target is widened with it and stays 1R). Cut-offs 0.60/0.25 and block 5 are catalogue grid points. Features: F-PSTAT (ProfileStats.enabled -> ring, SessionBars, BootstrapPoc, BootstrapSeed, KdeSweep, EKdeVerdict), host hook.

Pitfalls.

- Module on example 32: compile both files; the host's rules, stops and settings apply unless overridden.

Code.

```
// Manual v4 example 35 - A2-25 POC bootstrap-stability gate + A2-26 KDE bandwidth-sweep veto, as a module on
the DV-02 host (Ex32).
// After the prior close, the prior RTH session's Data1 ring bars (SessionBars) feed
_ProfileStats.BootstrapPoc (block 5, 500 draws,
// deterministic BootstrapSeed) and _ProfileStats.KdeSweep. Rules: KDE verdict FLAT or BIMODAL -> no trade
(A2-26);
// stability >= 0.60 -> trade, stop distance = max(host bracket, ci90/2 + 1 tick) (A2-25); 0.25 <= stability
< 0.60 -> the catalogue
// allows the trade only with the POC ZONE as the level; the DV-02 host is line-based, so this module skips
that bucket (stated, not
// a catalogue rule); stability < 0.25 -> skip. Skip if the widened stop makes the target < 1R (the DV-02
bracket is symmetric, so the
// target is widened with it and stays 1R). Cut-offs 0.60/0.25 and block 5 are catalogue grid points.
// Features: F-PSTAT (ProfileStats.enabled -> ring, SessionBars, BootstrapPoc, BootstrapSeed, KdeSweep,
EKdeVerdict), host hook.
using System;
using System.Collections.Generic;
using PowerLanguage.Function;

namespace PowerLanguage.Strategy
{
    public class _Ex35PocStabilityGate : _Ex32FailedLevelFlip
    {
        protected double StabilityHigh { get; set; } = 0.60;
        protected double StabilityLow { get; set; } = 0.25;

        public _Ex35PocStabilityGate(object _ctx) : base(_ctx) { }

        protected override void Create()
        {
            base.Create();
            ProfileStats.enabled = true;                        // lazy: requires the Data1 ring
(ProfileStats.ringBars) and range profiles
            ProfileStats.bootstrapBlock = 5; ProfileStats.bootstrapDraws = 500;
        }

        protected override bool pocGate(DateTime day, double poc, ref double bracket, out string why)
        {
            why = null;
            DateTime prev = priorTradingDay(day);
```

```
            List<RingBar> bars = prev == DateTime.MinValue ? null : SessionBars(prev, true);
            if (bars == null || bars.Count < 20) { why = "no prior RTH ring bars"; return false; }
            double tick = tickSize();
            KdeResult kde = _ProfileStats.KdeSweep(bars, tick, 2, poc, ProfileStats.kdeMedFloorTicks, Profile
Stats.kdeFolds, ProfileStats.kdeFolding);
            if (kde.verdict == EKdeVerdict.Flat || kde.verdict == EKdeVerdict.Bimodal) { why = "KDE " + kde.v
erdict; return false; }
            int block = ProfileStats.bootstrapBlock, draws = ProfileStats.bootstrapDraws;
            BootstrapResult b = _ProfileStats.BootstrapPoc(bars, tick, 2, block, draws, _ProfileStats.Bootstr
apSeed(prev, block, draws));
            if (double.IsNaN(b.stability)) { why = "bootstrap n/a"; return false; }
            if (b.stability < StabilityHigh) { why = $"stability {b.stability:F2} < {StabilityHigh:F2} (zone
bucket not handled by a line host)"; return false; }
            bracket = Math.Max(bracket, b.ci90Ticks / 2 * tick + tick);
            if (journalOn()) trace(ETraceLevel.Journal, $"A2-25 {day:yyyy-MM-dd} stability {b.stability:F2}
ci90 {b.ci90Ticks:F0}t KDE {kde.verdict} bracket {bracket:F2}");
            return true;
        }

        private DateTime priorTradingDay(DateTime day)
        {
            int maxAgo = Math.Min(ProfileStats.ringBars, Bars.CurrentBar - 1);   // ringBarOf reads chart
history: stay inside MaxBarsBack
            for (int ago = 0; ago < maxAgo; ago++)
            {
                RingBar rb = ringBarOf(1, ago);
                if (rb.tradingDay < day) return rb.tradingDay;
            }
            return DateTime.MinValue;
        }
    }
}
```

3.33 Example 7 (`_Ex07RotationVolumeLevel`) — PS-13 / DV-03 rotation POC first-touch retest (hard)

Features (direct). F-ROLL (§5.7), F-CAL (§5.8), F-JRNL (§5.13), F-EVT (§5.14), M-MTF (§5.17), VP helpers (§5.27).

Catalogue rules (PS-13). Filter: a rotation of ≥ 6 bars on 15m with range ≤ 1.5 × ATR15m, then a breakout bar closing outside with range ≥ 1.5 × ATR15m. Level: the highest-volume price in the rotation. Entry: first touch (limit) or first 5m HL after the touch; first touch only; level valid 5 sessions intraday. Stop: 10–20% of the 200-day ATR, floored. Target: before the next cluster. DV-03 (merged) adds a follow-through of ≥ 1 box width before any retest. **G0-d-gated** (family 10).

Chart. Data1 = @ES 1m, Data2 = @ES 15m, dxFeed unadjusted, 24h. `RangeProfiles.enabled = true`, RollDetection Series. This codes the **limit arm** of PS-13.

Walk-through.

- On each new closed 15m bar: ATR15 over the 14 bars before the breakout bar; the box = the high/low of the previous 6 closed 15m bars; reject if the box is taller than 1.5 × ATR15.
- The breakout bar must close outside the box with a range ≥ 1.5 × ATR15.
- `RangeProfileAfter(1, boxStart, boxEnd)` builds the 1m volume profile over the box only, and its POC is the level. `RangeStatsOf` must show coverage ≥ 0.95.
- Stop = level ∓ 0.10 × ATR200d; target = level ± 2R. Risk = max(0.10 × ATR200d, 14-tick cost floor `costFloorTicks`); stop and target use the widened risk. If it breaches the per-trade risk budget (`riskBudgetFraction` = 0.01, 1% of equity, decided by Ettienne 2026-10-23; PF-04's 0.5% cap is not the default for the floor rule), the level is skipped and journaled. Size is never shrunk.
- **Entry window (catalogue §2.1):** the limit is sent only on bar closes from 09:45 to before 15:30 ET (and before an early-close flat time), so it can fill only between 09:45 and 15:30. A position is closed at market at 15:55, or at `RthCloseEt` − 5 min on an early-close day (`flatAt`).
- A buy (sell) limit rests at the level. If a later bar reaches the level and the position is still flat, the level is spent. Once filled, the level is cleared, so it is never traded twice. It also expires after 5

sessions and is cleared at a roll. A touch outside the entry window, with no limit resting (overnight, for example), also spends the level.
- `justArmed` is true only on the bar that armed the level, so that bar's own range can't count as the first touch.
- **ATR200d is computed in the strategy**, not by M-DAILY: per-Globex-day true range, with the roll day's gap dropped, simple average of 200 days. M-DAILY with `atrLength = 200` and RollDetection on would reseed at every roll (about every 63 sessions), so it would never be valid.

Pitfalls.

- **Assumptions (labelled):** ATR15 length 14; coverage 0.95; a fixed 2R target as a stand-in for "before the next cluster".
- **Not coded:** DV-03's follow-through condition, the 5m HL entry arm and the near-HVN-edge level variant.
- The box start is the open time of the oldest box bar. The profile is "after" that time, assuming bars are stamped at their end (Probe P2, **VERIFY**).
- A bar that touches the level without a fill (for example with MC's "fill only when price trades beyond the limit") spends the level.
- **Coding choice (labelled):** a touch outside 09:45–15:30 spends the level (first touch only). The other reading would keep it for the next session's window.
- Stop and target are sent from the bar after the fill (§1.7), so the fill bar has no working exit.

Full strategy — `_Ex07RotationVolumeLevel.Strategy.CS`:

```
// Manual v4 example 7 - PS-13 / DV-03 rotation -> breakout -> first touch of the rotation's highest-volume
price (limit arm).
// Chart: Data1 = @ES 1m, Data2 = @ES 15m (same symbol, 24h Globex), dxFeed continuous UNADJUSTED. Range
profiles ON (RangeProfiles.enabled).
using System;
using System.Collections.Generic;
using PowerLanguage.Function;

namespace PowerLanguage.Strategy
{
    public class _Ex07RotationVolumeLevel : _VolumeProfileFuturesBase
    {
        protected sealed class Ps13Config
        {
            public int contracts = 1;
            public int rotationBars = 6, atr15Length = 14;   // ATR15m length is not in the catalogue: an
assumption
            public double boxAtr = 1.5, breakoutAtr = 1.5;
            public int atrDailyLength = 200;
            public double stopFraction = 0.10;                 // sweep {0.10, 0.20} x ATR200d
            public int costFloorTicks = 14;                     // catalogue 2.1: ES cost floor; stop
distance = max(structural, floor), size never shrunk
            public double riskBudgetFraction = 0.01;  // floor-rule risk budget: Ettienne, 2026-10-23: 1% of
equity (PF-04's 0.5% is not the default here)
            public int validSessions = 5;
            public TimeSpan from = new TimeSpan(9, 45, 0), to = new TimeSpan(15, 30, 0), flat = new
TimeSpan(15, 55, 0);   // catalogue 2.1 intraday rules
            public TimeSpan halfDayExitBefore = new TimeSpan(0, 5, 0);   // early-close days: flat at
RthCloseEt - 5 min
            public double targetR = 2.0;                        // "before the next cluster" is not coded
here: a fixed-R stand-in (coding choice)
        }
        protected Ps13Config Cfg { get; } = new Ps13Config();

        private IOrderPriced m_leLmt, m_seLmt, m_lxStp, m_sxStp, m_lxLmt, m_sxLmt;
        private IOrderMarket m_lx, m_sx;
        private DateTime m_last15 = DateTime.MinValue;
        // own ATR200d: per-Globex-day TR with the roll day's TR dropped (M-DAILY reseeds its ATR at every
roll, see section 4)
        private readonly List<double> m_tr = new List<double>();
        private DateTime m_aggDay = DateTime.MinValue; private double m_dH, m_dL, m_dC = double.NaN, m_prevC
= double.NaN; private bool m_dRoll;
        private double m_level = double.NaN; private int m_side; private DateTime m_levelDay; private int m_l
evelSessions;
        private double m_stop = double.NaN, m_target = double.NaN;
```

```
        /// <summary>Catalogue 2.1 / App F #9 (resolved): true when Cfg.contracts at this stop distance risk
more than the per-trade budget.</summary>
        private bool overRiskBudget(double stopDist) { return Cfg.contracts * stopDist * Bars.Info.BigPointVa
lue > equityAvailable(Cfg.riskBudgetFraction); }
        public _Ex07RotationVolumeLevel(object _ctx) : base(_ctx) { }

        protected override void Create()
        {
            base.Create();
            RollDetection = ERollDetection.Series;
            RangeProfiles.enabled = true;              // M-RANGE: needs the bar ring (ringBars 2880 = two
days of 1m bars)
            m_leLmt = OrderCreator.Limit(new SOrderParameters(Contracts.UserSpecified, "PS13LE",
EOrderAction.Buy));
            m_seLmt = OrderCreator.Limit(new SOrderParameters(Contracts.UserSpecified, "PS13SE",
EOrderAction.SellShort));
            m_lxStp = OrderCreator.Stop(new SOrderParameters(Contracts.Default, "PS13LS", EOrderAction.Sell,
OrderExit.FromAll));
            m_sxStp = OrderCreator.Stop(new SOrderParameters(Contracts.Default, "PS13SS", EOrderAction.BuyToC
over, OrderExit.FromAll));
            m_lxLmt = OrderCreator.Limit(new SOrderParameters(Contracts.Default, "PS13LT",
EOrderAction.Sell, OrderExit.FromAll));
            m_sxLmt = OrderCreator.Limit(new SOrderParameters(Contracts.Default, "PS13ST", EOrderAction.BuyTo
Cover, OrderExit.FromAll));
            m_lx = OrderCreator.MarketNextBar(new SOrderParameters(Contracts.Default, "PS13LX",
EOrderAction.Sell, OrderExit.FromAll));
            m_sx = OrderCreator.MarketNextBar(new SOrderParameters(Contracts.Default, "PS13SX",
EOrderAction.BuyToCover, OrderExit.FromAll));
        }

        protected override void StartCalc()
        {
            HtfStreams = new[] { 2 };
            base.StartCalc();
            m_last15 = m_aggDay = DateTime.MinValue; m_tr.Clear(); m_dC = m_prevC = double.NaN; m_dRoll = fal
se;
            m_level = m_stop = m_target = double.NaN; m_side = 0; m_levelSessions = 0;
        }

        protected override void CalcBar()
        {
            base.CalcBar();
            if (Bars.Status != EBarState.Close) return;
            DateTime t = Bars.Time[0], day = GlobexDate(t);
            TimeSpan tod = ToEastern(t).TimeOfDay, flat = flatAt(day);
            bool roll = rollOn(1);
            stepDailyAtr(day, roll);
            if (roll) { m_level = double.NaN; m_side = 0; }       // multi-day rotations reset at rolls

            int pos = StrategyInfo.MarketPosition;
            if (pos != 0)
            {
                m_level = double.NaN; m_side = 0;                // filled: the level is spent (first touch
only, no re-entry)
                if (tod >= flat && tod < new TimeSpan(17, 0, 0)) { if (pos > 0) m_lx.Send(); else m_sx.Send()
; return; }
                if (pos > 0) { m_lxStp.Send(m_stop); m_lxLmt.Send(m_target); } else { m_sxStp.Send(m_stop); m
_sxLmt.Send(m_target); }
                return;
            }
            if (!double.IsNaN(m_level) && m_side != 0 && m_levelSessions > Cfg.validSessions) { m_level = dou
ble.NaN; m_side = 0; }

            step15();

            // first touch only: the resting limit IS the touch; once price reaches the level without a fill
the level is spent.
            // A touch outside the entry window (no limit resting) also spends it.
            if (!double.IsNaN(m_level) && m_side != 0)
            {
                bool traded = m_side > 0 ? Bars.Low[0] <= m_level : Bars.High[0] >= m_level;
                if (traded && Bars.CurrentBar > 1 && !justArmed) { m_level = double.NaN; m_side = 0;
return; }
                justArmed = false;
                // entries 09:45-15:30 ET only (and before an early-close flat time): the limit sent on this
close works on the next bar
```

```
                bool window = tod >= Cfg.from && tod < Cfg.to && tod < flat;
                if (window) { if (m_side > 0) m_leLmt.Send(m_level, Cfg.contracts); else
m_seLmt.Send(m_level, Cfg.contracts); }
            }
        }
        // true only on the bar that armed the level: that 1m bar belongs to the breakout, so its own range
must not count
        // as the first touch and use up the level it just created
        private bool justArmed;

        /// <summary>New CLOSED 15m bar: rotation of the previous rotationBars bars, breakout on this bar,
level = POC of the box.</summary>
        private void step15()
        {
            DateTime tb; double ob, hb, lb, cb, vb;
            HtfClosedBar(2, 0, out tb, out ob, out hb, out lb, out cb, out vb);
            if (tb == m_last15) return;
            m_last15 = tb;
            int n = Cfg.rotationBars;
            if (BarsOfData(2).CurrentBar <= n + Cfg.atr15Length + 2) return;
            double atr15 = atrClosed(2, Cfg.atr15Length, 1);   // ATR of the bars BEFORE the breakout bar
            double boxH = double.MinValue, boxL = double.MaxValue; DateTime boxStart = DateTime.MinValue, box
End = DateTime.MinValue;
            for (int k = 1; k <= n; k++)
            {
                DateTime tk; double ok, hk, lk, ck, vk;
                HtfClosedBar(2, k, out tk, out ok, out hk, out lk, out ck, out vk);
                boxH = Math.Max(boxH, hk); boxL = Math.Min(boxL, lk);
                if (k == 1) boxEnd = tk;
            }
            { DateTime tk; double ok, hk, lk, ck, vk; HtfClosedBar(2, n + 1, out tk, out ok, out hk, out lk,
out ck, out vk); boxStart = tk; }
            if (!(atr15 > 0) || boxH - boxL > Cfg.boxAtr * atr15) return;
            int side = cb > boxH ? +1 : cb < boxL ? -1 : 0;
            if (side == 0 || hb - lb < Cfg.breakoutAtr * atr15) return;
            // 1m volume-at-price over the box only: bars ending in (boxStart, boxEnd] (exclusive t0, end-
stamped bars, Probe P2)
            VpExtra e = RangeProfileAfter(1, boxStart, boxEnd);
            if (e == null || !(e.poc > 0)) { if (journalOn()) trace(ETraceLevel.Journal, $"PS-13 {tb:yyyy-MM-
dd HH:mm} no range profile"); return; }
            RangeStats st = RangeStatsOf(e);
            if (st == null || !(st.coverage >= 0.95)) { if (journalOn()) trace(ETraceLevel.Journal, $"PS-13
{tb:HH:mm} box coverage {(st == null ? double.NaN : st.coverage):F2} < 0.95: skip"); return; }
            double atrD = atrDaily();
            if (!(atrD > 0)) return;
            double tick = tickSize();
            double risk = Math.Max(Cfg.stopFraction * atrD, Cfg.costFloorTicks * tick);   // catalogue 2.1:
stop = max(structural, cost floor)
            if (overRiskBudget(risk))   // widened stop breaches the per-trade risk budget: skipped and
logged, not resized
            {
                if (journalOn()) trace(ETraceLevel.Journal, $"PS-13 {tb:yyyy-MM-dd HH:mm} skip: stop {risk /
tick:F0} ticks breaches risk budget {Cfg.riskBudgetFraction:P2}");
                return;
            }
            m_level = e.poc; m_side = side; m_levelDay = GlobexDate(Bars.Time[0]); m_levelSessions = 0; justA
rmed = true;
            m_stop = m_level - side * risk; m_target = m_level + side * Cfg.targetR * risk;
            var ctx = new Dictionary<string, string> { { "boxH", boxH.ToString() }, { "boxL",
boxL.ToString() }, { "atr15", atr15.ToString("F2") }, { "atrD", atrD.ToString("F2") } };
            if (journalOn()) { journalArm(side > 0 ? "PS13LE" : "PS13SE", m_level, m_stop, m_target, ctx); lo
gSignal("PS-13", side, m_level, m_stop, ctx, true); }
        }

        private void stepDailyAtr(DateTime day, bool roll)
        {
            if (day != m_aggDay)
            {
                if (m_aggDay != DateTime.MinValue)
                {
                    if (!m_dRoll && !double.IsNaN(m_prevC)) m_tr.Add(Math.Max(m_dH - m_dL,
Math.Max(Math.Abs(m_dH - m_prevC), Math.Abs(m_dL - m_prevC))));
                    else m_tr.Add(m_dH - m_dL);                    // roll day: the gap to the old contract
is not range
                    if (m_tr.Count > Cfg.atrDailyLength) m_tr.RemoveAt(0);
                    m_prevC = m_dC;
                    if (!double.IsNaN(m_level)) m_levelSessions++;
```

```
                }
                m_aggDay = day; m_dH = Bars.High[0]; m_dL = Bars.Low[0]; m_dRoll = false;
            }
            m_dH = Math.Max(m_dH, Bars.High[0]); m_dL = Math.Min(m_dL, Bars.Low[0]); m_dC = Bars.Close[0];
            if (roll) { m_dRoll = true; m_prevC = double.NaN; }
        }
        private double atrDaily()
        {
            if (m_tr.Count < Cfg.atrDailyLength) return double.NaN;
            double s = 0; foreach (double x in m_tr) s += x; return s / m_tr.Count;
        }

        private double atrClosed(int ds, int len, int from)
        {
            double sum = 0;
            for (int k = from; k < from + len; k++)
            {
                DateTime t; double o, h, l, c, v, pc;
                HtfClosedBar(ds, k + 1, out t, out o, out h, out l, out pc, out v);
                HtfClosedBar(ds, k, out t, out o, out h, out l, out c, out v);
                sum += Math.Max(h - l, Math.Max(Math.Abs(h - pc), Math.Abs(l - pc)));
            }
            return sum / len;
        }

        /// <summary>Flat time for a Globex day: Cfg.flat, or on a CME early-close day RthCloseEt -
halfDayExitBefore if earlier
        /// (_CmeHolidays.Rules.Get(day).RthCloseEt; same rule as A263Config.halfDayExitBefore).</summary>
        private TimeSpan flatAt(DateTime day)
        {
            _CmeHolidays.Rules.Day cme = _CmeHolidays.Rules.Get(day);
            TimeSpan f = Cfg.flat;
            if (cme.RthCloseEt.HasValue && cme.RthCloseEt.Value - Cfg.halfDayExitBefore < f) f = cme.RthClose
Et.Value - Cfg.halfDayExitBefore;
            return f;
        }
    }
}
```

Compile status: compiled — type-checked with Roslyn against the repo sources (`origin/main` a07257a) plus MC API stubs (stubs not authoritative). Not run in MultiCharts; no backtest results.

3.34 Example 10 (`_Ex10WeeklyValueAcceptance`) — MT-02 weekly accepted value + 60m trigger (hard)

Features (direct). F-ROLL (§5.7), F-CAL (§5.8), F-JRNL (§5.13), F-EVT (§5.14), M-MTF (§5.17), M-DAILY (§5.19), VP helpers (§5.27).

Catalogue rules (MT-02). Filter: by Tuesday's close, ≥ 2 daily closes above the prior-week VAH. Entry: the first pullback to prior-week VAH ± 0.25 × ATR20d with a 60m higher low, Wednesday–Friday. Stop: back inside the prior-week VA by 0.25 × weekly VA height, floored. Target: prior-week VAH + 1 × weekly VA height. Time stop 7 sessions. Sweep: acceptance days {1, 2} × target {1 × height, trail}. Multi-day levels reset at rolls. **G0-d-gated** (family 7).

Chart. Data1 = @ES 1m, Data2 = @ES 60m, dxFeed unadjusted, 24h. `WeeklyProfiles.enabled = true` (RTH sessions, roll policy Reset, minSessions 3), `NeedsDaily = true`, RollDetection Series. `[Input] AcceptanceDays` 1 or 2 (default 2).

Walk-through.

- The prior week is looked up by **key** (`_WeeklyProfiles.WeekKeyOf`, `WeekKey(i)`, `Week(i)`). A week counts as completed only when the next week's first session completes, so on Monday the prior week may still be `Week(0)`. Weeks that cross a roll, are partial, or have no valid VA are skipped.
- Acceptance: on Monday and Tuesday the RTH close (the 1m bar close-stamped 16:00 ET, or the early close) is compared with the prior-week VAH.

- From Wednesday to Friday during RTH: without enough acceptance closes, the week is done. Otherwise the zone is VAH ± 0.25 × `DailyContext.Atr20d`. The first bar that overlaps it marks the touch.
- Trigger: the first new closed 60m bar after the touch whose low is above the prior 60m low. The entry is a market order; stop = VAH − 0.25 × height; target = VAH + 1 × height. If the R from the signal close (close − stop) is under the 14-tick cost floor (`costFloorTicks`), the stop is widened to close − 14 ticks (the target is unchanged). If the widened stop breaches the per-trade risk budget (`riskBudgetFraction` = 0.01, 1% of equity, decided by Ettienne 2026-10-23; PF-04's 0.5% cap is not the default for the floor rule), the week is skipped and journaled. Size is never shrunk.
- **Zone break:** after the touch and before the 60m trigger, any RTH 1m bar that closes below the zone (VAH − 0.25 × ATR20d) cancels the setup for the rest of the week. That is very strict, since one 1m close is enough. It is a coding choice the catalogue doesn't make (**VERIFY** against intent; Appendix F).
- A roll during the week cancels the setup. If `Atr20d` is NaN the week is skipped. The time stop exits after 7 sessions held.

Pitfalls.

- **Assumptions (labelled):** a close below the zone kills the week's setup (**VERIFY** against intent); "60m HL" read as a 60m bar whose low is above the prior 60m bar's low. The v2 8-tick floor is gone: the stop is the structural 0.25 × height, and the catalogue's 14-tick cost floor widens it when it is shorter; the week is skipped only on a risk-budget breach.
- Stop and target are sent from the bar after the fill (§1.7), so the fill bar has no working exit.
- Acceptance uses the RTH close (16:00 ET or the early close); being a swing idea, the example does not apply the half-day flat.
- `Atr20d` is NaN for 20 sessions after each roll (§4.3), so weeks right after a roll are skipped and journaled.
- The trail target arm is not coded.

Full strategy — `_Ex10WeeklyValueAcceptance.Strategy.CS`:

```
// Manual v4 example 10 - MT-02 weekly accepted value + 60m trigger (swing, long side as written in the
catalogue).
// Chart: Data1 = @ES 1m, Data2 = @ES 60m, dxFeed continuous UNADJUSTED, 24h CME Globex template. Weekly
profiles ON (RTH sessions).
using System;
using System.Collections.Generic;
using PowerLanguage.Function;

namespace PowerLanguage.Strategy
{
    public class _Ex10WeeklyValueAcceptance : _VolumeProfileFuturesBase
    {
        [Input] public int AcceptanceDays { get; set; }   // sweep {1, 2}; catalogue default 2

        protected sealed class Mt02Config
        {
            public int contracts = 1;
            public double zoneAtr = 0.25, stopHeight = 0.25, targetHeight = 1.0;
            public int costFloorTicks =
14;                     // catalogue 2.1: ES cost floor; stop distance = max(structural, floor), size never
shrunk
            public double riskBudgetFraction = 0.01;  // floor-rule risk budget: Ettienne, 2026-10-23: 1% of
equity (PF-04's 0.5% is not the default here)
            public int timeStopSessions = 7;
        }
        protected Mt02Config Cfg { get; } = new Mt02Config();

        private IOrderMarket m_le, m_lxTime; private IOrderPriced m_lxStp, m_lxLmt;
        private DateTime m_day = DateTime.MinValue, m_week = DateTime.MinValue, m_last60 = DateTime.MinValue;
        private double m_vah = double.NaN, m_height = double.NaN;
        private int m_accept; private bool m_touched, m_doneWeek, m_rollThisWeek;
        private double m_stop, m_target; private int m_heldSessions;

        /// <summary>Catalogue 2.1 / App F #9 (resolved): true when Cfg.contracts at this stop distance risk
more than the per-trade budget.</summary>
        private bool overRiskBudget(double stopDist) { return Cfg.contracts * stopDist * Bars.Info.BigPointVa
```

```
lue > equityAvailable(Cfg.riskBudgetFraction); }
        public _Ex10WeeklyValueAcceptance(object _ctx) : base(_ctx) { AcceptanceDays = 2; }

        protected override void Create()
        {
            base.Create();
            RollDetection = ERollDetection.Series;
            NeedsDaily =
true;                               // Atr20d (NaN for ~20 sessions after each roll: those weeks are skipped)
            WeeklyProfiles.enabled = true;                    // source RthSessions, rollPolicy Reset,
minSessions 3
            m_le = OrderCreator.MarketNextBar(new SOrderParameters(Contracts.UserSpecified, "MT02LE", EOrderA
ction.Buy));
            m_lxTime = OrderCreator.MarketNextBar(new SOrderParameters(Contracts.Default, "MT02LX", EOrderAct
ion.Sell, OrderExit.FromAll));
            m_lxStp = OrderCreator.Stop(new SOrderParameters(Contracts.Default, "MT02LS", EOrderAction.Sell,
OrderExit.FromAll));
            m_lxLmt = OrderCreator.Limit(new SOrderParameters(Contracts.Default, "MT02LT",
EOrderAction.Sell, OrderExit.FromAll));
        }

        protected override void StartCalc()
        {
            if (AcceptanceDays < 1 || AcceptanceDays > 2) throw new
ArgumentException("AcceptanceDays must be 1 or 2");
            HtfStreams = new[] { 2 };
            base.StartCalc();
            m_day = m_week = m_last60 = DateTime.MinValue; m_vah = m_height = double.NaN; m_heldSessions = 0;
        }

        protected override void CalcBar()
        {
            base.CalcBar();
            if (Bars.Status != EBarState.Close) return;
            DateTime t = Bars.Time[0], et = ToEastern(t), day = GlobexDate(t);
            bool roll = rollOn(1);
            bool newDay = day != m_day;
            if (newDay) { m_day = day; if (StrategyInfo.MarketPosition > 0) m_heldSessions++; }
            DateTime wk = _WeeklyProfiles.WeekKeyOf(day);
            if (wk != m_week) { m_week = wk; m_vah = double.NaN; m_accept = 0; m_touched = m_doneWeek = m_rol
lThisWeek = false; }
            if (roll) m_rollThisWeek =
true;                  // multi-day levels reset at rolls: no new setup this week

            if (StrategyInfo.MarketPosition > 0)
            {
                if (m_heldSessions >= Cfg.timeStopSessions) { m_lxTime.Send(); return; }
                m_lxStp.Send(m_stop); m_lxLmt.Send(m_target);
                return;
            }
            m_heldSessions = 0;
            if (m_doneWeek || m_rollThisWeek) return;
            if (double.IsNaN(m_vah) && !loadPriorWeek(wk)) return;

            DayOfWeek dow = day.DayOfWeek;
            TimeSpan tod = et.TimeOfDay;
            bool rth = isRthBar(t);
            _CmeHolidays.Rules.Day cme = _CmeHolidays.Rules.Get(day);
            TimeSpan closeBar = cme.RthCloseEt ?? new TimeSpan(16, 0, 0);   // the RTH close bar (close-
stamped 16:00 ET, or the early close)
            // acceptance: RTH closes (the 16:00 ET bar close on 1m) above the prior-week VAH, Monday and
Tuesday
            if ((dow == DayOfWeek.Monday || dow == DayOfWeek.Tuesday) && rth && tod == closeBar &&
Bars.Close[0] > m_vah) m_accept++;
            if (dow < DayOfWeek.Wednesday || dow > DayOfWeek.Friday || cme.NoRth || !rth) return;
            if (m_accept < AcceptanceDays) { m_doneWeek = true; journal("no acceptance"); return; }

            double atr = DailyContext != null ? DailyContext.Atr20d : double.NaN;
            if (!(atr > 0)) { m_doneWeek = true; journal("Atr20d NaN (post-roll seeding)"); return; }
            double zLo = m_vah - Cfg.zoneAtr * atr, zHi = m_vah + Cfg.zoneAtr * atr;
            if (!m_touched && Bars.Low[0] <= zHi && Bars.High[0] >= zLo) m_touched = true;   // first
pullback into the zone
            if (!m_touched) return;
            if (Bars.Close[0] < zLo) { m_doneWeek = true; journal("pullback broke the zone"); return; }   //
coding choice (VERIFY)

            // trigger: the first new CLOSED 60m bar after the touch whose low is above the prior 60m low (a
```

```
60m higher low)
            DateTime t60; double o, h, l, c, v, o1, h1, l1, c1, v1;
            HtfClosedBar(2, 0, out t60, out o, out h, out l, out c, out v);
            if (t60 == m_last60) return;
            m_last60 = t60;
            DateTime tPrev; HtfClosedBar(2, 1, out tPrev, out o1, out h1, out l1, out c1, out v1);
            if (!(l > l1)) return;
            m_stop = m_vah - Cfg.stopHeight * m_height; m_target = m_vah + Cfg.targetHeight * m_height;
            if (Bars.Close[0] <= m_stop || Bars.Close[0] >= m_target) { m_doneWeek = true; return; }
            double r = Bars.Close[0] - m_stop;   // R from the planned (signal-close) entry
            if (r < Cfg.costFloorTicks * tickSize()) { r = Cfg.costFloorTicks * tickSize(); m_stop = Bars.Clo
se[0] - r; }   // catalogue 2.1: widen to the cost floor
            if (overRiskBudget(r)) { m_doneWeek = true; journal($"skip: stop {r / tickSize():F0} ticks
breaches risk budget {Cfg.riskBudgetFraction:P2}"); return; }   // not resized
            m_le.Send(Cfg.contracts); m_doneWeek = true;
            var ctx = new Dictionary<string, string> { { "vah", m_vah.ToString() }, { "height", m_height.ToSt
ring() }, { "atr20d", atr.ToString("F2") }, { "accept", m_accept.ToString() } };
            if (journalOn()) { journalArm("MT02LE", Bars.Close[0], m_stop, m_target, ctx);
logSignal("MT-02", +1, Bars.Close[0], m_stop, ctx, true); }
        }

        /// <summary>Prior-week profile by KEY: a week is completed only when the next week's first session
completes,
        /// so on Monday the prior week is still Week(0) and Week(1) is two weeks back.</summary>
        private bool loadPriorWeek(DateTime wk)
        {
            DateTime prior = wk.AddDays(-7);
            VpExtra e = WeekKey(0) == prior ? Week(0) : WeekKey(1) == prior ? Week(1) : null;
            if (e == null || e.crossesRoll || e.isPartial || !(e.vah > 0) || !(e.val > 0) || e.vah <= e.val)
return false;
            m_vah = e.vah; m_height = e.vah - e.val;
            return true;
        }

        private void journal(string why) { if (journalOn()) trace(ETraceLevel.Journal, $"MT-02 week
{m_week:yyyy-MM-dd} {why}"); }
    }
}
```

Compile status: compiled — type-checked with Roslyn against the repo sources (`origin/main` a07257a) plus MC API stubs (stubs not authoritative). Not run in MultiCharts; no backtest results.

- **v4 note.** This v3 example still exposes `[Input]`s (house rule: protected config); kept unchanged for continuity.

3.35 Example 28 (`_Ex28BreakAvwapPullback`) — AV-11 PS-01 break re-entry at the break-anchored AVWAP (long side shown; short mirrors) (hard)

Features (direct). F-DEV (§5.1), F-PROF (§5.2), F-COMP (§5.3), F-SESS (§5.6), F-ROLL (§5.7), F-CAL (§5.8), F-TM (§5.12), F-JRNL (§5.13), M-SWING (§5.16).

What it does (from the file header):

> AV-11 PS-01 break re-entry at the break-anchored AVWAP (long side shown; short mirrors). Context: the PS-01 filter (IsBalanced(RTH, 3, 0.6), V-score >= +1). Anchor: the FIRST 5m bar that closes > composite VAH + delta (09:45-14:30 ET). The AVWAP and its volume-weighted sigma are accumulated here from that bar (typical price), because the M-AVWAP Breakout anchor is a PA break-of-structure event, not a composite-edge break (see pitfall in section 5). Entry: the first pullback whose low trades into AVWAP +/- 0.25 sigma and that prints a PA HL; buy stop 1 tick above that bar. Stop: max(AVWAP - 1 sigma, 14-tick floor). Target: VAH + 1.0 W (PS-01 target). Invalidation: a 5m close < composite VAH - delta. Chart: Data1 = @ES 5m, dxFeed continuous UNADJUSTED. Features: F-COMP, F-VP, M-PA, F-TM.

Pitfalls.

- Deviation: the catalogue's AVWAP is anchored at the break; `EAvwapAnchor.Breakout` anchors on a PRICEACTION break-of-structure instead, so the example computes its own break-anchored VWAP.

Code.

```
// Manual v4 example 28 - AV-11 PS-01 break re-entry at the break-anchored AVWAP (long side shown; short
mirrors).
// Context: the PS-01 filter (IsBalanced(RTH, 3, 0.6), V-score >= +1). Anchor: the FIRST 5m bar that closes >
composite VAH + delta
// (09:45-14:30 ET). The AVWAP and its volume-weighted sigma are accumulated here from that bar (typical
price), because the
// M-AVWAP Breakout anchor is a PA break-of-structure event, not a composite-edge break (see pitfall in
section 5).
// Entry: the first pullback whose low trades into AVWAP +/- 0.25 sigma and that prints a PA HL; buy stop 1
tick above that bar.
// Stop: max(AVWAP - 1 sigma, 14-tick floor). Target: VAH + 1.0 W (PS-01 target). Invalidation: a 5m close <
composite VAH - delta.
// Chart: Data1 = @ES 5m, dxFeed continuous UNADJUSTED. Features: F-COMP, F-VP, M-PA, F-TM.
using System;
using PowerLanguage.Function;

namespace PowerLanguage.Strategy
{
    public class _Ex28BreakAvwapPullback : _VolumeProfileFuturesBase
    {
        protected sealed class Av11Config
        {
            public int contracts = 1, n = 3, deltaTicks = 2;
            public double zoneSigma = 0.25, stopSigma = 1.0, targetW = 1.0, balanceOverlap = 0.6;
            public TimeSpan from = new TimeSpan(9, 45, 0), to = new TimeSpan(14, 30, 0), flat = new
TimeSpan(15, 55, 0);
            public int costFloorTicks = 14;
            public double riskBudgetFraction = 0.01;
        }
        protected Av11Config Cfg { get; } = new Av11Config();

        private IOrderPriced m_leStp, m_lxStp, m_lxLmt;
        private IOrderMarket m_lx;
        private DateTime m_day = DateTime.MinValue;
        private bool m_ok, m_done, m_anchored, m_touched;
        private double m_vah, m_w, m_spv, m_sp2v, m_sv, m_trigger = double.NaN, m_stop, m_target;

        public _Ex28BreakAvwapPullback(object _ctx) : base(_ctx) { }

        protected override void Create()
        {
            base.Create();
            RollDetection = ERollDetection.Series;
            DevelopingSource = EDevelopingSource.Own; OwnProfileWindow = EOwnProfileWindow.Rth;
            PriceActionConfig.enabled = true; PriceActionConfig.dataStream = 1;
            FlatBy = Cfg.flat;
            m_leStp = OrderCreator.Stop(new SOrderParameters(Contracts.UserSpecified, "AV11LE",
EOrderAction.Buy));
            m_lx = OrderCreator.MarketNextBar(new SOrderParameters(Contracts.Default, "AV11LX",
EOrderAction.Sell));
            m_lxStp = OrderCreator.Stop(new SOrderParameters(Contracts.Default, "AV11LS", EOrderAction.Sell,
OrderExit.FromAll));
            m_lxLmt = OrderCreator.Limit(new SOrderParameters(Contracts.Default, "AV11LT",
EOrderAction.Sell, OrderExit.FromAll));
        }

        protected override void StartCalc() { base.StartCalc(); m_day = DateTime.MinValue; }

        protected override void CalcBar()
        {
            base.CalcBar();
            if (Bars.Status != EBarState.Close) return;
            DateTime day = GlobexDate(Bars.Time[0]);
            if (day != m_day) newDay(day);
            if (rollOn(1)) m_done = true;
            double tick = tickSize(), c = Bars.Close[0], d = Cfg.deltaTicks * tick;
            if (StrategyInfo.MarketPosition > 0)
            {
```

```
                m_done = true;
                if (mustFlatten() || c < m_vah - d) { m_lx.Send(); return; }
                m_lxStp.Send(m_stop); m_lxLmt.Send(m_target); return;
            }
            TimeSpan tod = ToEastern(Bars.Time[0]).TimeOfDay;
            if (m_done || !m_ok || !IsRth()) return;
            if (!m_anchored)
            {
                if (tod < Cfg.from || tod > Cfg.to || !(c > m_vah + d)) return;
                m_anchored = true;
                if (journalOn()) trace(ETraceLevel.Journal, $"AV-11 {day:yyyy-MM-dd} anchor
{Bars.Time[0]:HH:mm} close {c:F2} > VAH {m_vah:F2}");
            }
            double tp = (Bars.High[0] + Bars.Low[0] + c) / 3, v = Bars.Volume[0];
            m_spv += tp * v; m_sp2v += tp * tp * v; m_sv += v;
            if (!(m_sv > 0)) return;
            double av = m_spv / m_sv, sig = Math.Sqrt(Math.Max(0, m_sp2v / m_sv - av * av));
            if (c < m_vah - d) { m_done = true; return; }                       // break failed before the
re-entry
            if (Bars.Low[0] <= av + Cfg.zoneSigma * sig && Bars.High[0] >= av - Cfg.zoneSigma * sig) m_touche
d = true;
            _PriceActionEngine pa = PriceActionEngine;
            if (m_touched && double.IsNaN(m_trigger) && tod <= Cfg.to && pa != null && pa.IsWarm && pa.NewPiv
otLow)
            {
                _Pivot p = pa.LastLow(0);
                if (p != null && p.label == EPivotLabel.HL)
                {
                    double trig = Bars.High[0] + tick, stop = Math.Min(av - Cfg.stopSigma * sig, trig - Cfg.c
ostFloorTicks * tick);
                    if (Cfg.contracts * (trig - stop) * Bars.Info.BigPointValue > equityAvailable(Cfg.riskBud
getFraction))
                    { if (journalOn()) trace(ETraceLevel.Journal, $"AV-11 {day:yyyy-MM-dd} skip: risk
budget"); m_done = true; return; }
                    m_trigger = trig; m_stop = stop; m_target = m_vah + Cfg.targetW * m_w;
                    if (journalOn()) journalArm("AV11LE", m_trigger, m_stop, m_target, null);
                }
            }
            if (!double.IsNaN(m_trigger)) m_leStp.Send(m_trigger, Cfg.contracts);
        }

        private void newDay(DateTime day)
        {
            m_day = day; m_done = m_anchored = m_touched = false; m_trigger = double.NaN; m_spv = m_sp2v = m_
sv = 0;
            VpExtra comp = Composite(1, Cfg.n);
            m_ok = comp != null && IsBalanced(1, Cfg.n, Cfg.balanceOverlap) && (ValueMigrationScore(1,
Cfg.n, 0.10, 20, false) ?? 0) >= 1;
            if (comp != null) { m_vah = comp.vah; m_w = comp.vah - comp.val; m_ok &= m_w > 0; }
        }
    }
}
```

3.36 Example 32 (`_Ex32FailedLevelFlip`) — DV-02 failed level -> reversal (flip) limit, with the CS-8 event log and control layer (hard)

Features (direct). F-DEV (§5.1), F-PROF (§5.2), F-SESS (§5.6), F-ROLL (§5.7), F-CAL (§5.8), F-TM (§5.12), F-JRNL (§5.13), F-EVT (§5.14), M-DAILY (§5.19).

What it does (from the file header):

> DV-02 failed level -> reversal (flip) limit, with the CS-8 event log and control layer. Level L: the prior completed RTH POC (a DV-01 level), passed through controlAdjustLevel() so a ShiftLevels control run moves it. Failure: after the first touch of L, a 5m close >= s x the bracket distance beyond L on the far side within M bars, with "no reaction" (max adverse excursion against the break after the touch < 0.25 x bracket). Entry: limit at L in the break direction, valid to the session end (X = 1 session), one flip per level. Bracket: +/- f x ATR20d (f = 0.10); the stop side is widened to the 14-tick floor if narrower (target unchanged), skipped only on a risk-budget breach, never resized. s = 1.0 and f = 0.10 are catalogue grid points; M = 12 bars is a PLACEHOLDER. Every armed flip is logged with logSignal(taken = true); skipped ones with taken

> = false (needs TraceLevel >= Journal). Chart: Data1 = @ES 5m, dxFeed continuous UNADJUSTED. Features: F-EVT/CS-8 (logSignal, ControlMode, controlAdjustLevel, controlAdjustDelayBars), F-VP (PriorRthProfile), M-DAILY (Atr20d), F-TM.

Pitfalls.

- Contains PLACEHOLDER values (marked in the code); calibrate them before reading any result.

Code.

```
// Manual v4 example 32 - DV-02 failed level -> reversal (flip) limit, with the CS-8 event log and control layer.
// Level L: the prior completed RTH POC (a DV-01 level), passed through controlAdjustLevel() so a ShiftLevels control run moves it.
// Failure: after the first touch of L, a 5m close >= s x the bracket distance beyond L on the far side within M bars, with
// "no reaction" (max adverse excursion against the break after the touch < 0.25 x bracket). Entry: limit at L in the break
// direction, valid to the session end (X = 1 session), one flip per level. Bracket: +/- f x ATR20d (f = 0.10); the stop side is widened to
// the 14-tick floor if narrower (target unchanged), skipped only on a risk-budget breach, never resized. s = 1.0 and f = 0.10 are catalogue grid points; M = 12 bars is a PLACEHOLDER.
// Every armed flip is logged with logSignal(taken = true); skipped ones with taken = false (needs TraceLevel >= Journal).
// Chart: Data1 = @ES 5m, dxFeed continuous UNADJUSTED. Features: F-EVT/CS-8 (logSignal, ControlMode, controlAdjustLevel,
// controlAdjustDelayBars), F-VP (PriorRthProfile), M-DAILY (Atr20d), F-TM.
using System;
using System.Collections.Generic;
using PowerLanguage.Function;

namespace PowerLanguage.Strategy
{
    public class _Ex32FailedLevelFlip : _VolumeProfileFuturesBase
    {
        protected sealed class Dv02Config
        {
            public int contracts = 1, breakWithinBars = 12;   // M: PLACEHOLDER
            public double s = 1.0, f = 0.10, noReaction = 0.25;
            public int costFloorTicks = 14;
            public double riskBudgetFraction = 0.01;
            public TimeSpan flat = new TimeSpan(15, 55, 0);
        }
        protected Dv02Config Cfg { get; } = new Dv02Config();

        private IOrderPriced m_leLmt, m_seLmt, m_lxStp, m_sxStp, m_lxLmt, m_sxLmt;
        private IOrderMarket m_lx, m_sx;
        private DateTime m_day = DateTime.MinValue;
        private double m_level, m_br, m_sd, m_hiSince, m_loSince;
        private int m_touchBar, m_side, m_armBar;
        private bool m_done;

        public _Ex32FailedLevelFlip(object _ctx) : base(_ctx) { }

        protected override void Create()
        {
            base.Create();
            RollDetection = ERollDetection.Series;
            DevelopingSource = EDevelopingSource.Own; OwnProfileWindow = EOwnProfileWindow.Rth;
            FlatBy = Cfg.flat;
            // Placebo run: ControlMode = EControlMode.ShiftLevels; ControlShiftTicks = 8;  (protected properties, never [Input])
            m_leLmt = OrderCreator.Limit(new SOrderParameters(Contracts.UserSpecified, "DV02LE", EOrderAction.Buy));
            m_seLmt = OrderCreator.Limit(new SOrderParameters(Contracts.UserSpecified, "DV02SE", EOrderAction.SellShort));
            m_lx = OrderCreator.MarketNextBar(new SOrderParameters(Contracts.Default, "DV02LX", EOrderAction.Sell));
            m_sx = OrderCreator.MarketNextBar(new SOrderParameters(Contracts.Default, "DV02SX", EOrderAction.BuyToCover));
            m_lxStp = OrderCreator.Stop(new SOrderParameters(Contracts.Default, "DV02LS", EOrderAction.Sell, OrderExit.FromAll));
            m_sxStp = OrderCreator.Stop(new SOrderParameters(Contracts.Default, "DV02SS", EOrderAction.BuyToCover, OrderExit.FromAll));
```

```
            m_lxLmt = OrderCreator.Limit(new SOrderParameters(Contracts.Default, "DV02LT",
EOrderAction.Sell, OrderExit.FromAll));
            m_sxLmt = OrderCreator.Limit(new SOrderParameters(Contracts.Default, "DV02ST", EOrderAction.BuyTo
Cover, OrderExit.FromAll));
        }

        protected override void StartCalc() { NeedsDaily = true; base.StartCalc(); m_day =
DateTime.MinValue; }

        protected override void CalcBar()
        {
            base.CalcBar();
            if (Bars.Status != EBarState.Close) return;
            DateTime day = GlobexDate(Bars.Time[0]);
            if (day != m_day) newDay(day);
            if (rollOn(1)) m_done = true;
            int pos = StrategyInfo.MarketPosition, bar = Bars.FullSymbolData.Current;
            if (pos != 0)
            {
                m_done = true;
                if (mustFlatten()) { if (pos > 0) m_lx.Send(); else m_sx.Send(); return; }
                double e = StrategyInfo.AvgEntryPrice;
                if (pos > 0) { m_lxStp.Send(e - m_sd); m_lxLmt.Send(e + m_br); } else { m_sxStp.Send(e +
m_sd); m_sxLmt.Send(e - m_br); }
                return;
            }
            if (m_done || !(m_br > 0) || !IsRth()) return;
            double h = Bars.High[0], l = Bars.Low[0], c = Bars.Close[0];
            if (m_side != 0)                                          // armed: work the limit at L until
the session ends
            {
                if (bar >= m_armBar + controlAdjustDelayBars())
                { if (m_side > 0) m_leLmt.Send(m_level, Cfg.contracts); else m_seLmt.Send(m_level, Cfg.contra
cts); }
                return;
            }
            if (m_touchBar == 0)
            {
                if (l <= m_level && h >= m_level) { m_touchBar = bar; m_hiSince = h; m_loSince = l; }
                return;
            }
            m_hiSince = Math.Max(m_hiSince, h); m_loSince = Math.Min(m_loSince, l);
            if (bar - m_touchBar > Cfg.breakWithinBars) { m_done = true; return; }    // no failure within M
bars: level held
            int side = c >= m_level + Cfg.s * m_br ? +1 : c <= m_level - Cfg.s * m_br ? -1 : 0;
            if (side == 0) return;
            double mae = side > 0 ? m_level - m_loSince : m_hiSince - m_level;
            bool reaction = mae >= Cfg.noReaction * m_br;
            bool budget = Cfg.contracts * m_sd * Bars.Info.BigPointValue > equityAvailable(Cfg.riskBudgetFrac
tion);
            bool taken = !reaction && !budget;
            logSignal("DV02", side, m_level, m_level - side * m_sd, new Dictionary<string, string> { {
"mae", mae.ToString("F2") }, { "control", ControlMode.ToString() } }, taken);
            if (!taken) { m_done = true; return; }
            m_side = side; m_armBar = bar;
            if (journalOn()) journalArm(side > 0 ? "DV02LE" : "DV02SE", m_level, m_level - side * m_sd, m_lev
el + side * m_br, null);
        }

        /// <summary>Hook for POC gate modules (Ex35: A2-25/A2-26). May widen the bracket; return false to
skip the day.</summary>
        protected virtual bool pocGate(DateTime day, double poc, ref double bracket, out string why) { why =
null; return true; }

        private void newDay(DateTime day)
        {
            m_day = day; m_done = false; m_touchBar = m_side = 0;
            VpExtra prior = PriorRthProfile();
            double atr = DailyContext != null ? DailyContext.Atr20d : double.NaN;
            m_br = atr > 0 ? Cfg.f * atr : double.NaN;
            if (prior == null || prior.crossesRoll || !(prior.poc > 0)) { m_done = true; return; }
            m_level = controlAdjustLevel(prior.poc);
            string gw;
            if (!pocGate(day, prior.poc, ref m_br, out gw)) { if (journalOn()) trace(ETraceLevel.Journal, $"D
V-02 {day:yyyy-MM-dd} gate: {gw}"); m_done = true; return; }
            m_sd = Math.Max(m_br, Cfg.costFloorTicks * tickSize());   // stop = max(structural bracket, 14-
tick floor); the target stays f x ATR20d
```

```
        }
    }
}
```

3.37 Example 33 (`_Ex33FvgHvnAlignment`) — DV-08 Fair Value Gap + heavy-volume alignment as a level layer (bullish side, as catalogued) (hard)

Features (direct). F-HVN (§5.5), F-ROLL (§5.7), F-JRNL (§5.13), M-DAILY (§5.19), VP helpers (§5.27).

What it does (from the file header):

> DV-08 Fair Value Gap + heavy-volume alignment as a level layer (bullish side, as catalogued). Level: a new bullish FVG on closed 30m bars (Fvgs, minAtrFrac = g x ATR30m; g = 0.25 is a PLACEHOLDER). The level is the FVG start edge (high of bar 1, fvgLevel(f, true)) and is kept only if it lies within 0.2 x ATR30m of an HVN edge of the range profile of the last 20 closed 30m bars (RangeProfileBars). Entry (DV-03/04 host style): buy limit at the level while the gap is live and untouched, valid until the gap fills or expires. Bracket: +/- f x ATR20d, f = 0.10 (catalogue grid point); the stop is widened to the 14-tick floor if narrower; skipped only on a risk-budget breach. Chart: Data1 = @ES 30m, dxFeed continuous UNADJUSTED. Features: F-FVG (Fvgs, FvgDetector.NewThisBar/Live, fvgLevel -> LevelRef), F-RANGE (RangeProfiles.enabled, RangeProfileBars), M-DAILY (Atr20d), F-TM.

Pitfalls.

- Contains PLACEHOLDER values (marked in the code); calibrate them before reading any result.

Code.

```
// Manual v4 example 33 - DV-08 Fair Value Gap + heavy-volume alignment as a level layer (bullish side, as
catalogued).
// Level: a new bullish FVG on closed 30m bars (Fvgs, minAtrFrac = g x ATR30m; g = 0.25 is a PLACEHOLDER).
The level is the FVG start
// edge (high of bar 1, fvgLevel(f, true)) and is kept only if it lies within 0.2 x ATR30m of an HVN edge of
the range profile of the
// last 20 closed 30m bars (RangeProfileBars). Entry (DV-03/04 host style): buy limit at the level while the
gap is live and untouched,
// valid until the gap fills or expires. Bracket: +/- f x ATR20d, f = 0.10 (catalogue grid point); the stop
is widened to the 14-tick floor if narrower; skipped only on a risk-budget breach.
// Chart: Data1 = @ES 30m, dxFeed continuous UNADJUSTED. Features: F-FVG (Fvgs, FvgDetector.NewThisBar/Live,
fvgLevel -> LevelRef),
// F-RANGE (RangeProfiles.enabled, RangeProfileBars), M-DAILY (Atr20d), F-TM.
using System;
using PowerLanguage.Function;

namespace PowerLanguage.Strategy
{
    public class _Ex33FvgHvnAlignment : _VolumeProfileFuturesBase
    {
        protected sealed class Dv08Config
        {
            public int contracts = 1, profileBars = 20;
            public double gapAtr = 0.25, alignAtr = 0.2, f = 0.10;   // gapAtr PLACEHOLDER; alignAtr and f
from the catalogue
            public int costFloorTicks = 14;
            public double riskBudgetFraction = 0.01;
        }
        protected Dv08Config Cfg { get; } = new Dv08Config();

        private IOrderPriced m_leLmt, m_lxStp, m_lxLmt;
        private Fvg m_gap;
        private double m_level, m_br, m_sd;

        public _Ex33FvgHvnAlignment(object _ctx) : base(_ctx) { }

        protected override void Create()
        {
            base.Create();
            RollDetection = ERollDetection.Series;
```

```
            Fvgs.enabled = true; Fvgs.dataStream = 1; Fvgs.minAtrFrac = Cfg.gapAtr; Fvgs.atrLen = 14; Fvgs.ma
xAgeSessions = 5;
            RangeProfiles.enabled = true; RangeProfiles.streams = new[] { 1 };
            m_leLmt = OrderCreator.Limit(new SOrderParameters(Contracts.UserSpecified, "DV08LE",
EOrderAction.Buy));
            m_lxStp = OrderCreator.Stop(new SOrderParameters(Contracts.Default, "DV08LS", EOrderAction.Sell,
OrderExit.FromAll));
            m_lxLmt = OrderCreator.Limit(new SOrderParameters(Contracts.Default, "DV08LT",
EOrderAction.Sell, OrderExit.FromAll));
        }

        protected override void StartCalc() { NeedsDaily = true; base.StartCalc(); m_gap = null; }

        protected override void CalcBar()
        {
            base.CalcBar();
            if (Bars.Status != EBarState.Close) return;
            if (rollOn(1)) m_gap = null;
            if (StrategyInfo.MarketPosition > 0)
            {
                m_gap = null;
                double e = StrategyInfo.AvgEntryPrice;
                m_lxStp.Send(e - m_sd); m_lxLmt.Send(e + m_br); return;
            }
            _FvgDetector det = FvgDetector;
            if (det == null) return;
            Fvg nf = det.NewThisBar;
            if (nf != null && nf.dir > 0) consider(nf, det.Atr);
            if (m_gap == null) return;
            if (!m_gap.IsLive || m_gap.touched) { m_gap = null; return; }       // first touch only
            m_leLmt.Send(m_level, Cfg.contracts);
        }

        private void consider(Fvg f, double atr30)
        {
            LevelRef lv = fvgLevel(f, true);
            VpExtra rp = RangeProfileBars(1, 0, Cfg.profileBars);
            double atrD = DailyContext != null ? DailyContext.Atr20d : double.NaN, tick = tickSize();
            if (lv == null || lv.crossesRoll || rp == null || !(atr30 > 0) || !(atrD > 0)) return;
            bool aligned = false;
            foreach (Node n in rp.hvns)
                if (Math.Abs(lv.price - n.high) <= Cfg.alignAtr * atr30 || Math.Abs(lv.price - n.low) <=
Cfg.alignAtr * atr30) { aligned = true; break; }
            double br = Cfg.f * atrD, sd = Math.Max(br, Cfg.costFloorTicks * tick);   // stop =
max(structural, 14-tick floor)
            string why = !aligned ? "no HVN edge within 0.2 ATR30m"
                : Cfg.contracts * sd * Bars.Info.BigPointValue > equityAvailable(Cfg.riskBudgetFraction) ? "r
isk budget" : null;
            if (why != null) { if (journalOn()) trace(ETraceLevel.Journal, $"DV-08 {lv.tag} skip: {why}"); re
turn; }
            m_gap = f; m_level = lv.price; m_br = br; m_sd = sd;
            if (journalOn()) journalArm("DV08LE", m_level, m_level - sd, m_level + br, null);
        }
    }
}
```

3.38 Example 34 (`_Ex34TrendClusterPullback`) — PS-12 trend-cluster pullback, Dale limit variant (long side, as catalogued) (hard)

Features (direct). F-DEV (§5.1), F-COMP (§5.3), F-HVN (§5.5), F-ROLL (§5.7), F-JRNL (§5.13), M-SWING (§5.16), VP helpers (§5.27).

What it does (from the file header):

> PS-12 trend-cluster pullback, Dale limit variant (long side, as catalogued). Filter: 60m PA structure Up (HH/HL; ExtraPriceAction engine on Data2). Cluster: a closed 30m bar whose largest single-price row (BarLadders, bid/ask tiers only) >= 2 x the bar's median row volume, inside a pause (bar range <= 0.7 x ATR30m). Validity: 2 later 30m closes above cluster + delta. Entry: buy limit at the cluster price (first touch). Stop: the next composite LVN low below the cluster, floored at 14 ticks. Target: the last 30m pivot high (skip if not above the entry). One cluster at a time. Chart: Data1 = @ES 30m, Data2 = @ES 60m, dxFeed continuous UNADJUSTED; needs tick/

> bid-ask history for the flow tiers. Features: F-LADDER (BarLadders.enabled -> OrderFlowEnabled, Ladder(ago), LadderBar rows/total/HasBidAsk), M-PA (ExtraPriceAction), F-COMP (lvns), F-TM.

Code.

```
// Manual v4 example 34 - PS-12 trend-cluster pullback, Dale limit variant (long side, as catalogued).
// Filter: 60m PA structure Up (HH/HL; ExtraPriceAction engine on Data2). Cluster: a closed 30m bar whose
largest single-price row
// (BarLadders, bid/ask tiers only) >= 2 x the bar's median row volume, inside a pause (bar range <= 0.7 x
ATR30m).
// Validity: 2 later 30m closes above cluster + delta. Entry: buy limit at the cluster price (first touch).
Stop: the next composite
// LVN low below the cluster, floored at 14 ticks. Target: the last 30m pivot high (skip if not above the
entry). One cluster at a time.
// Chart: Data1 = @ES 30m, Data2 = @ES 60m, dxFeed continuous UNADJUSTED; needs tick/bid-ask history for the
flow tiers.
// Features: F-LADDER (BarLadders.enabled -> OrderFlowEnabled, Ladder(ago), LadderBar rows/total/HasBidAsk),
M-PA (ExtraPriceAction),
// F-COMP (lvns), F-TM.
using System;
using PowerLanguage.Function;

namespace PowerLanguage.Strategy
{
    public class _Ex34TrendClusterPullback : _VolumeProfileFuturesBase
    {
        protected sealed class Ps12Config
        {
            public int contracts = 1, atrLength = 14, validCloses = 2, deltaTicks = 2, n = 20, ds60 = 2;
            public double clusterMult = 2.0, pauseAtr = 0.7;
            public int costFloorTicks = 14;
            public double riskBudgetFraction = 0.01;
        }
        protected Ps12Config Cfg { get; } = new Ps12Config();

        private IOrderPriced m_leLmt, m_lxStp, m_lxLmt;
        private double m_cluster = double.NaN, m_stop, m_target;
        private int m_above;
        private bool m_valid;

        public _Ex34TrendClusterPullback(object _ctx) : base(_ctx) { }

        protected override void Create()
        {
            base.Create();
            RollDetection = ERollDetection.Series;
            DevelopingSource = EDevelopingSource.Own; OwnProfileWindow = EOwnProfileWindow.Rth;
            BarLadders.enabled = true; BarLadders.keepBars = 500;                        // also switches
OrderFlowEnabled on (VP-3)
            PriceActionConfig.enabled = true; PriceActionConfig.dataStream = 1;
            ExtraPriceAction.Add(new PriceActionConfig { enabled = true, dataStream = Cfg.ds60, role = "trend
60" });
            m_leLmt = OrderCreator.Limit(new SOrderParameters(Contracts.UserSpecified, "PS12LE",
EOrderAction.Buy));
            m_lxStp = OrderCreator.Stop(new SOrderParameters(Contracts.Default, "PS12LS", EOrderAction.Sell,
OrderExit.FromAll));
            m_lxLmt = OrderCreator.Limit(new SOrderParameters(Contracts.Default, "PS12LT",
EOrderAction.Sell, OrderExit.FromAll));
        }

        protected override void StartCalc() { base.StartCalc(); m_cluster = double.NaN; }

        protected override void CalcBar()
        {
            base.CalcBar();
            if (Bars.Status != EBarState.Close) return;
            if (rollOn(1)) m_cluster = double.NaN;
            if (StrategyInfo.MarketPosition > 0) { m_cluster = double.NaN; m_lxStp.Send(m_stop);
m_lxLmt.Send(m_target); return; }
            _PriceActionEngine pa60 = PriceActionFor(Cfg.ds60, "trend60");
            bool up = pa60 != null && pa60.IsWarm && pa60.Structure(0) == EStructure.Up;
            if (!up) { m_cluster = double.NaN; return; }
            double tick = tickSize();
            if (double.IsNaN(m_cluster)) { findCluster(); return; }
            if (!m_valid)
```

```
            {
                m_above = Bars.Close[0] > m_cluster + Cfg.deltaTicks * tick ? m_above + 1 : 0;
                if (Bars.Low[0] <= m_cluster) { m_cluster = double.NaN; return; }       // touched before it
was validated
                if (m_above >= Cfg.validCloses) m_valid = arm();
                return;
            }
            m_leLmt.Send(m_cluster, Cfg.contracts);
            if (Bars.Low[0] <= m_cluster) m_cluster = double.NaN;                       // first touch
consumed (filled or not)
        }

        private void findCluster()
        {
            LadderBar lb = Ladder(0);
            double atr = averageTrueRange(Bars, Cfg.atrLength);
            if (lb == null || !lb.HasBidAsk || !lb.HasRows || lb.crossesRoll || !(atr > 0) || Bars.High[0] -
Bars.Low[0] > Cfg.pauseAtr * atr) return;
            double[] t = (double[])lb.total.Clone();
            Array.Sort(t);
            double med = t[t.Length / 2];
            int best = 0;
            for (int i = 1; i < lb.total.Length; i++) if (lb.total[i] > lb.total[best]) best = i;
            if (!(med > 0) || lb.total[best] < Cfg.clusterMult * med) return;
            m_cluster = lb.rows[best] * lb.tick; m_above = 0; m_valid = false;
        }

        private bool arm()
        {
            double tick = tickSize(), lvn = double.NaN;
            VpExtra comp = Composite(1, Cfg.n);
            if (comp != null) foreach (Node z in comp.lvns) if (z.high < m_cluster && (double.IsNaN(lvn) ||
z.low > lvn)) lvn = z.low;
            double stop = double.IsNaN(lvn) ? m_cluster - Cfg.costFloorTicks * tick : Math.Min(lvn - tick, m_
cluster - Cfg.costFloorTicks * tick);
            _PriceActionEngine pa = PriceActionEngine;
            _Pivot sh = pa != null ? pa.LastHigh(0) : null;
            string why = sh == null || sh.price <= m_cluster ? "no swing high above" : Cfg.contracts * (m_clu
ster - stop) * Bars.Info.BigPointValue > equityAvailable(Cfg.riskBudgetFraction) ? "risk budget" : null;
            if (why != null) { if (journalOn()) trace(ETraceLevel.Journal, $"PS-12 cluster {m_cluster:F2}
skip: {why}"); m_cluster = double.NaN; return false; }
            m_stop = stop; m_target = sh.price;
            if (journalOn()) journalArm("PS12LE", m_cluster, m_stop, m_target, null);
            return true;
        }
    }
}
```

3.39 Example 36 (`_Ex36BimodalVoidTraverse`) — A2-27 bimodal-profile void traverse (long side shown; the short mirror uses E2 / mu1) (hard)

Features (direct). F-SESS (§5.6), F-ROLL (§5.7), F-CAL (§5.8), F-TM (§5.12), F-JRNL (§5.13), M-MTF (§5.17), M-GMM (§5.25), VP helpers (§5.27).

What it does (from the file header):

> A2-27 bimodal-profile void traverse (long side shown; the short mirror uses E2 / mu1). Setup (at the first bar of the new Globex day, from the PRIOR RTH session): Data1 ring bars -> _ProfileStats.Histogram -> _ProfileStats.ProfileMixture (kMax 3, BIC, seed MixtureSeed(day), N_eff = bars). Require K == 2, both pi >= 0.30, void depth >= 0.5, mu2 - mu1 >= 1.0 x ATR(14) of Data2 (30m). E1 = mu1 + sd1, E2 = mu2 - sd2. Deviation: the catalogue fits the prior session's profile; here the histogram is built from the 5m Data1 ring (not 1m rows). Entry: price first closes at/below E1, then 2 consecutive closed 5m bars above E1 inside 09:45-14:30 ET -> buy at the next 5m open. Stop: mu1 - 1 tick, capped at 1.0 x ATR(14, 5m) from the signal close, then floored at 14 ticks (user rule). Skip if mu2 - close < 1.2R or if the risk budget (0.01 x equity) is breached. Target: mu2. Time stop: 90 minutes, flat by 15:55. Not coded: T1 50% at E2 (single-contract example). Thresholds are catalogue values, not results. Chart: Data1 @ES 5m, Data2

> @ES 30m, dxFeed continuous UNADJUSTED. Features: F-PSTAT (ProfileStats.enabled ring, SessionBars, Histogram, ProfileMixture, MixtureSeed), F-TM (FlatBy).

Code.

```
// Manual v4 example 36 - A2-27 bimodal-profile void traverse (long side shown; the short mirror uses E2 / mu1).
// Setup (at the first bar of the new Globex day, from the PRIOR RTH session): Data1 ring bars -> _ProfileStats.Histogram ->
// _ProfileStats.ProfileMixture (kMax 3, BIC, seed MixtureSeed(day), N_eff = bars). Require K == 2, both pi >= 0.30, void depth >= 0.5,
// mu2 - mu1 >= 1.0 x ATR(14) of Data2 (30m). E1 = mu1 + sd1, E2 = mu2 - sd2.
// Deviation: the catalogue fits the prior session's profile; here the histogram is built from the 5m Data1 ring (not 1m rows).
// Entry: price first closes at/below E1, then 2 consecutive closed 5m bars above E1 inside 09:45-14:30 ET -> buy at the next 5m open.
// Stop: mu1 - 1 tick, capped at 1.0 x ATR(14, 5m) from the signal close, then floored at 14 ticks (user rule). Skip if mu2 - close < 1.2R
// or if the risk budget (0.01 x equity) is breached. Target: mu2. Time stop: 90 minutes, flat by 15:55.
// Not coded: T1 50% at E2 (single-contract example). Thresholds are catalogue values, not results.
// Chart: Data1 @ES 5m, Data2 @ES 30m, dxFeed continuous UNADJUSTED. Features: F-PSTAT (ProfileStats.enabled ring, SessionBars,
// Histogram, ProfileMixture, MixtureSeed), F-TM (FlatBy).
using System;
using System.Collections.Generic;
using PowerLanguage.Function;

namespace PowerLanguage.Strategy
{
    public class _Ex36BimodalVoidTraverse : _VolumeProfileFuturesBase
    {
        protected sealed class A227Config
        {
            public int contracts = 1, kMax = 3, acceptCloses = 2, atrLength = 14, costFloorTicks = 14, timeStopMin = 90;
            public double minPi = 0.30, minVoidDepth = 0.5, minSepAtr30 = 1.0, stopCapAtr5 = 1.0, minRR = 1.2, riskBudgetFraction = 0.01;
            public TimeSpan from = new TimeSpan(9, 45, 0), to = new TimeSpan(14, 30, 0), flat = new TimeSpan(15, 55, 0);
        }
        protected A227Config Cfg { get; } = new A227Config();

        private IOrderMarket m_le, m_lx;
        private IOrderPriced m_lxStp, m_lxLmt;
        private DateTime m_day = DateTime.MinValue, m_entryTime;
        private bool m_ok, m_done, m_armed;
        private int m_up, m_lastPos;
        private double m_mu1, m_mu2, m_e1, m_stop, m_target;

        public _Ex36BimodalVoidTraverse(object _ctx) : base(_ctx) { }

        protected override void Create()
        {
            base.Create();
            RollDetection = ERollDetection.Series;
            ProfileStats.enabled = true;                        // lazy: Data1 ring for SessionBars
            FlatBy = Cfg.flat;
            m_le = OrderCreator.MarketNextBar(new SOrderParameters(Contracts.UserSpecified, "A227LE", EOrderAction.Buy));
            m_lx = OrderCreator.MarketNextBar(new SOrderParameters(Contracts.Default, "A227LX", EOrderAction.Sell));
            m_lxStp = OrderCreator.Stop(new SOrderParameters(Contracts.Default, "A227LS", EOrderAction.Sell, OrderExit.FromAll));
            m_lxLmt = OrderCreator.Limit(new SOrderParameters(Contracts.Default, "A227LT", EOrderAction.Sell, OrderExit.FromAll));
        }

        protected override void StartCalc() { base.StartCalc(); m_day = DateTime.MinValue; m_lastPos = 0; }

        protected override void CalcBar()
        {
            base.CalcBar();
            if (Bars.Status != EBarState.Close) return;
            DateTime day = GlobexDate(Bars.Time[0]);
            if (day != m_day) newDay(day);
```

```
            if (rollOn(1)) m_done = true;
            int pos = StrategyInfo.MarketPosition;
            if (pos != 0 && m_lastPos == 0) m_entryTime = Bars.Time[0];
            m_lastPos = pos;
            if (pos != 0)
            {
                m_done = true;
                if (mustFlatten() || (Bars.Time[0] - m_entryTime).TotalMinutes >= Cfg.timeStopMin) { m_lx.Sen
d(); return; }
                m_lxStp.Send(m_stop); m_lxLmt.Send(m_target);
                return;
            }
            if (m_done || !m_ok || !IsRth()) return;
            double c = Bars.Close[0], tick = tickSize();
            if (c <= m_e1) { m_armed = true; m_up = 0; return; }   // in or below the lower component
            if (!m_armed) return;
            m_up++;
            TimeSpan tod = ToEastern(Bars.Time[0]).TimeOfDay;
            if (m_up < Cfg.acceptCloses || tod < Cfg.from || tod > Cfg.to) return;
            m_done = true;                                          // one attempt per day
            double atr5 = averageTrueRange(Bars, Cfg.atrLength);
            double stop = Math.Max(m_mu1 - tick, c - Cfg.stopCapAtr5 * atr5);
            if (c - stop < Cfg.costFloorTicks * tick) stop = c - Cfg.costFloorTicks * tick;
            double r = c - stop;
            string why = m_mu2 - c < Cfg.minRR * r ? "target < 1.2R"
                : Cfg.contracts * r * Bars.Info.BigPointValue > equityAvailable(Cfg.riskBudgetFraction) ? "ri
sk budget" : null;
            if (why != null) { if (journalOn()) trace(ETraceLevel.Journal, $"A2-27 {day:yyyy-MM-dd} skip:
{why}"); return; }
            m_stop = stop; m_target = m_mu2;
            if (journalOn()) journalArm("A227LE", c, m_stop, m_target, null);
            m_le.Send(Cfg.contracts);
        }

        private void newDay(DateTime day)
        {
            m_day = day; m_done = false; m_armed = false; m_up = 0; m_ok = false;
            DateTime prev = priorTradingDay(day);
            if (prev == DateTime.MinValue) return;
            List<RingBar> bars = SessionBars(prev, true);
            double tick = tickSize(), atr30 = averageTrueRange(BarsOfData(2), Cfg.atrLength);
            double[] px, vol;
            if (bars.Count < 20 || !_ProfileStats.Histogram(bars, null, tick, 2, out px, out vol)) return;
            ProfileMixtureResult m = _ProfileStats.ProfileMixture(px, vol, Cfg.kMax, _ProfileStats.MixtureSee
d(prev), EGmmNeff.Bars, bars.Count, tick);
            if (journalOn()) trace(ETraceLevel.Journal, m.Journal(prev));
            if (m.K != 2 || m.pi[0] < Cfg.minPi || m.pi[1] < Cfg.minPi || !(m.voidDepth.Length > 0 && m.voidD
epth[0] >= Cfg.minVoidDepth)) return;
            if (!(m.mu[1] - m.mu[0] >= Cfg.minSepAtr30 * atr30)) return;
            m_mu1 = m.mu[0]; m_mu2 = m.mu[1]; m_e1 = m.mu[0] + m.sd[0];
            m_ok = m_e1 < m.mu[1] - m.sd[1];                        // E1 below E2: a real void between the
edges
        }

        private DateTime priorTradingDay(DateTime day)
        {
            int maxAgo = Math.Min(ProfileStats.ringBars, Bars.CurrentBar - 1);
            for (int ago = 0; ago < maxAgo; ago++) { RingBar rb = ringBarOf(1, ago); if (rb.tradingDay <
day) return rb.tradingDay; }
            return DateTime.MinValue;
        }
    }
}
```

3.40 Example 39 (`_Ex39XmktBetaMonitor`) — PF-03 cross-market hidden-beta monitor, RESEARCH PROXY (no orders) (research)

Features (direct). F-SESS (§5.6), F-ROLL (§5.7), F-CAL (§5.8), M-XMKT (§5.22).

What it does (from the file header):

> PF-03 cross-market hidden-beta monitor, RESEARCH PROXY (no orders). The catalogue rule correlates each sleeve's rolling 60-day DAILY P&L with SPX daily returns and flags rho > 0.5.

> Sleeve P&L series do not exist inside one MultiCharts strategy, so this example shows the M-XMKT building block instead: Data2 is a second market (here the leg role "SPX", e.g. an S&P index or another future), XmktLeg aligns it to Data1 on closed bars and keeps a rolling return Correlation / Beta / RsZ over corrLength bars. At the last RTH bar it journals the values and flags |rho| > 0.5. This is NOT the PF-03 statistic; the sleeve-P&L version must be computed offline from the trade journals. Chart: Data1 @ES 30m, Data2 = the reference market at 30m. Features: M-XMKT (XmktLegs, XmktLeg, Correlation, Beta, RsZ, Available).

Pitfalls.

- Research tool: it sends no orders and only journals.

Code.

```
// Manual v4 example 39 - PF-03 cross-market hidden-beta monitor, RESEARCH PROXY (no orders).
// The catalogue rule correlates each sleeve's rolling 60-day DAILY P&L with SPX daily returns and flags rho
> 0.5. Sleeve P&L
// series do not exist inside one MultiCharts strategy, so this example shows the M-XMKT building block
instead: Data2 is a second
// market (here the leg role "SPX", e.g. an S&P index or another future), XmktLeg aligns it to Data1 on
closed bars and keeps a rolling
// return Correlation / Beta / RsZ over corrLength bars. At the last RTH bar it journals the values and flags
|rho| > 0.5.
// This is NOT the PF-03 statistic; the sleeve-P&L version must be computed offline from the trade journals.
// Chart: Data1 @ES 30m, Data2 = the reference market at 30m. Features: M-XMKT (XmktLegs, XmktLeg,
Correlation, Beta, RsZ, Available).
using System;
using PowerLanguage.Function;

namespace PowerLanguage.Strategy
{
    public class _Ex39XmktBetaMonitor : _VolumeProfileFuturesBase
    {
        protected double FlagRho { get; set; } = 0.5;
        private DateTime m_logged = DateTime.MinValue;

        public _Ex39XmktBetaMonitor(object _ctx) : base(_ctx) { }

        protected override void Create()
        {
            base.Create();
            RollDetection = ERollDetection.Series;
            XmktLegs.Add(new XmktLegConfig { role = "SPX", dataStream = 2, returnBars = 1, corrLength = 60, r
sLength = 60 });
        }

        protected override void CalcBar()
        {
            base.CalcBar();
            if (Bars.Status != EBarState.Close || !IsRth()) return;
            DateTime day = GlobexDate(Bars.Time[0]);
            if (day == m_logged || ToEastern(Bars.Time[0]).TimeOfDay < new TimeSpan(15, 59, 0)) return;
            m_logged = day;
            _XmktLeg leg = XmktLeg("SPX");
            if (leg == null || !leg.Available || double.IsNaN(leg.Correlation)) return;
            bool flag = Math.Abs(leg.Correlation) > FlagRho;
            if (journalOn()) trace(ETraceLevel.Journal, $"PF-03proxy {day:yyyy-MM-dd} rho
{leg.Correlation:F2} beta {leg.Beta:F2} rsZ {leg.RsZ:F2}{(flag ? " FLAG" : "")}");
        }
    }
}
```

3.41 Example 41 (`_Ex41LevelPersistenceStudy`) — A2-33 level-persistence vs matched-control study (RESEARCH, no orders) (research)

Features (direct). F-PROF (§5.2), F-NPOC (§5.4), F-ROLL (§5.7), F-CAL (§5.8), A2-33 (§5.28).

What it does (from the file header):

> A2-33 level-persistence vs matched-control study (RESEARCH, no orders). On the first closed Data1 bar of each trading day, registers the prior session's POC / VAH / VAL (age 1) and every still-naked POC up to LevelStudyConfig.maxAge sessions old with levelStudyAdd. The _LevelStudy harness (A2-33) then tracks touches, reactions vs matched random controls (distLo..distHi), and Kaplan-Meier level death, writing its results to the journal / csvPath at the end. Status on main: matched controls and Kaplan-Meier exist; the Cox model in the catalogue spec is NOT implemented (A2-33 partial). Ages assume RecentCompletedProfiles returns newest first (age = index + 1). Chart: Data1 @ES 5m, dxFeed continuous UNADJUSTED. Features: A2-33 (LevelStudyConfig, levelStudyAdd), F-PROF (RecentCompletedProfiles), F-NPOC (TrackNakedPoc, NakedPocs).

Pitfalls.

- Research tool: it sends no orders and only journals.

Code.

```
// Manual v4 example 41 - A2-33 level-persistence vs matched-control study (RESEARCH, no orders).
// On the first closed Data1 bar of each trading day, registers the prior session's POC / VAH / VAL (age 1)
and every still-naked POC
// up to LevelStudyConfig.maxAge sessions old with levelStudyAdd. The _LevelStudy harness (A2-33) then tracks
touches, reactions vs
// matched random controls (distLo..distHi), and Kaplan-Meier level death, writing its results to the
journal / csvPath at the end.
// Status on main: matched controls and Kaplan-Meier exist; the Cox model in the catalogue spec is NOT
implemented (A2-33 partial).
// Ages assume RecentCompletedProfiles returns newest first (age = index + 1). Chart: Data1 @ES 5m, dxFeed
continuous UNADJUSTED.
// Features: A2-33 (LevelStudyConfig, levelStudyAdd), F-PROF (RecentCompletedProfiles), F-NPOC
(TrackNakedPoc, NakedPocs).
using System;
using System.Collections.Generic;
using PowerLanguage.Function;

namespace PowerLanguage.Strategy
{
    public class _Ex41LevelPersistenceStudy : _VolumeProfileFuturesBase
    {
        private DateTime m_day = DateTime.MinValue;

        public _Ex41LevelPersistenceStudy(object _ctx) : base(_ctx) { }

        protected override void Create()
        {
            base.Create();
            RollDetection = ERollDetection.Series;
            TrackNakedPoc = true;                              // VP-base [Input]: set in code, never exposed
            LevelStudyConfig.enabled = true;                   // lazy: harness not created otherwise
            LevelStudyConfig.controls = 20; LevelStudyConfig.reactionBars = 60; LevelStudyConfig.maxAge = 15;
            LevelStudyConfig.csvPath = "";                     // empty = journal only
        }

        protected override void StartCalc() { base.StartCalc(); m_day = DateTime.MinValue; }

        protected override void CalcBar()
        {

base.CalcBar();                                    // the harness steps inside base.CalcBar(); add levels
after it
            if (Bars.Status != EBarState.Close) return;
            DateTime day = GlobexDate(Bars.Time[0]);
            if (day == m_day) return;
            m_day = day;
            int maxAge = LevelStudyConfig.maxAge;
            List<VpExtra> recent = RecentCompletedProfiles(1, maxAge, Bars.CurrentBar);
            if (recent == null || recent.Count == 0) return;
            VpExtra p = recent[0];
            levelStudyAdd("POC", p.poc, 1); levelStudyAdd("VAH", p.vah, 1); levelStudyAdd("VAL", p.val, 1);
            foreach (VpExtra n in NakedPocs(1))
            {
                int age = recent.IndexOf(n) + 1;
                if (age >= 1 && age <= maxAge) levelStudyAdd("nPOC", n.poc, age, double.NaN, "nPOC@" + n.star
```

```
tBar);
            }
        }
    }
}
```

3.42 Example 42 (`_Ex42ForwardConeDiagnostic`) — KD-08 forward-cone diagnostic, RESEARCH SNIPPET (no orders; KD-08 itself is not buildable on main) (research)

Features (direct). F-ROLL (§5.7), M-DVP (§5.31).

What it does (from the file header):

> KD-08 forward-cone diagnostic, RESEARCH SNIPPET (no orders; KD-08 itself is not buildable on main). Status (origin/main 628542f): M-DVP is hosted on Data1 (PR-D3, #36) with the Fib lattice (PR-D4, #37). This file uses the host API (DvpConfig, dvpSnapshot, dvpZ, dvpRailPrice, dvpRails). The engine itself measures forward coverage: with journalFits = true it writes one DVP FIT line per fit and, on the last historical bar, a DVP RUN line with meanCovIn / meanCovFwd / meanCovFwdFlat (cov_fwd over projBars, roll-crossing horizons dropped and counted in fwdDropped). KD-08 still needs PR-D8 (Calibrate study) and PR-D9 (frozen d*): until then DevCoverageFrozen falls back to Calibrate with reported degree 1 (one "DVP DEGREE not frozen" warning). Every Step closed bars it journals the as-of t-1 snapshot, z of the close and the +-2 sigma rail prices. winLen 240 / projBars 60 / degMax 3 are DvpConfig defaults, not tuned values; Step 12 is a journal-thinning choice, not a parameter of the measurement. Chart: Data1 @ES 5m, dxFeed continuous UNADJUSTED. Features: M-DVP host API (+ F-ROLL via the snapshot's roll skip).

Note (800bce7): the header above predates D5–D7; the confluence, acceptance, HTF and A2-33 hook APIs are now on main (§5.31). The example itself is unchanged and uses only the D3/D4 host API.

Pitfalls.

- Research tool: it sends no orders and only journals.

Code.

```
// Manual v4 example 42 - KD-08 forward-cone diagnostic, RESEARCH SNIPPET (no orders; KD-08 itself is not
buildable on main).
// Status (origin/main 628542f): M-DVP is hosted on Data1 (PR-D3, #36) with the Fib lattice (PR-D4, #37).
This file uses the host API
// (DvpConfig, dvpSnapshot, dvpZ, dvpRailPrice, dvpRails). The engine itself measures forward coverage: with
journalFits = true it
// writes one DVP FIT line per fit and, on the last historical bar, a DVP RUN line with meanCovIn /
meanCovFwd / meanCovFwdFlat
// (cov_fwd over projBars, roll-crossing horizons dropped and counted in fwdDropped). KD-08 still needs PR-D8
(Calibrate study) and
// PR-D9 (frozen d*): until then DevCoverageFrozen falls back to Calibrate with reported degree 1 (one "DVP
DEGREE not frozen" warning).
// Every Step closed bars it journals the as-of t-1 snapshot, z of the close and the +-2 sigma rail prices.
winLen 240 / projBars 60 /
// degMax 3 are DvpConfig defaults, not tuned values; Step 12 is a journal-thinning choice, not a parameter
of the measurement.
// Chart: Data1 @ES 5m, dxFeed continuous UNADJUSTED. Features: M-DVP host API (+ F-ROLL via the snapshot's
roll skip).
using System;
using System.Collections.Generic;
using PowerLanguage.Function;

namespace PowerLanguage.Strategy
{
    public class _Ex42ForwardConeDiagnostic : _VolumeProfileFuturesBase
    {
        protected int Step { get; set; } = 12;

        public _Ex42ForwardConeDiagnostic(object _ctx) : base(_ctx) { }
```

```
        protected override void Create()
        {
            base.Create();
            DvpConfig.enabled = true;        // lazy engine, Data1 only (DvpConfig.streams default { 1 })
            DvpConfig.journalFits = true;    // DVP FIT per fit + DVP RUN (cov_fwd) at run end - the KD-08
measurement
            DvpConfig.coneMode = EDvpConeMode.Prediction;
        }

        protected override void CalcBar()
        {
            base.CalcBar();                  // stepDvp runs inside; the snapshot below is as of t-1
            if (Bars.Status != EBarState.Close || Bars.CurrentBar % Step != 0 || rollOn(1)) return;
            DvpSnapshot s = dvpSnapshot(1);
            if (s == null || !s.valid) return;   // warm-up, or the window crosses a roll
            double z = dvpZ(1, Bars.Close[0]);
            double up2 = dvpRailPrice(1, ELevelKind.DvpSigma, 2), dn2 = dvpRailPrice(1, ELevelKind.DvpSigma,
-2);
            IReadOnlyList<LevelRef> rails = dvpRails(1);   // reused by the next call; only counted here
            if (journalOn()) trace(ETraceLevel.Journal, $"KD-08sketch {Bars.Time[0]:yyyy-MM-dd HH:mm} deg
{s.deg} calibrating {s.calibrating} R2 {s.r2:F2} sigmaAtr {s.sigmaAtr:F2} drift {s.endpointDrift:F2}
cov2s(in) {s.inSampleCov2:F2} z {z:F2} band {dn2:F2}..{up2:F2} rails {rails.Count}");
        }
    }
}
```

4 — Engines and helpers

> Each chapter follows the same order: **why it exists · when to use it · enable · settings · API (with plain meaning) · timing · journal · tests · snippet**.
>
> **Snippets** in this part are the bodies of small compiled classes (`_S4_01` … `_S4_14`, each deriving from `_CustomSignal`; type-checked against `origin/main`). Every block starts with a comment that names where the code lives: `// class field(s)`, `// Create`, `// StartCalc`, `// CalcBar`. Two rules from §1 apply everywhere: - **Configure in `Create()`, or in `StartCalc()` before `base.StartCalc()`.** `base.StartCalc()` builds the engines from the settings it finds. Changing config after that is **unsupported**: engines hold the config objects by reference, so a mid-run change can take partial effect. **Exception:** call `requireRing(...)` after `base.StartCalc()`, because `startVpHelpers` (`_CustomSignal` L1440) clears earlier ring requests (§4.14). - **Read in `CalcBar()` after `base.CalcBar()`.** `base.CalcBar()` steps every engine on the closed bar; before that call you read the previous bar's values.
>
> Full public signatures, with file and class, are in **Appendix E**; complete strategies are in §3.

4.1 `_CustomSignal` core services

Why it exists. Every strategy needs the same plumbing: a log you can audit, a trade journal, position sizing from a stop, daily trading limits ("no entries before 09:35, flat by 15:55, at most two trades, stop after two losers"), time-zone conversion and a trading-day key. `_CustomSignal` provides these once, so a strategy only contains its own entry and exit logic.

When to use it. Always: every strategy in §3 derives from it (directly, or through `_VolumeProfileFuturesBase`, §4.15). Trade-management rules (`entryAllowed`, `mustFlatten`) appear in examples 12, 13, 17, 18, 20, 22, 23, 25–32, 36, 37, 38 and 40; `logSignal` in examples 1–3, 5–7, 10–16, 20, 24, 29 and 32.

Enable. Always on. The only `[Input]` is `TraceLevel` (default `None`, set in the constructor since PR #49; `base.Create()` no longer resets it). The settings below are properties, set in `Create()`.

API.

Member	Plain meaning
`traceOn(ETraceLevel)`, `trace(ETraceLevel, string)`	Is this log level on? Write one line to the trace file at that level.
`journalOn()`	True when the journal level is on; guard expensive journal text with it.
`engineSink(ETraceLevel)`	The callback the engines use to write into your trace; pass it to engines you build yourself.
`journalArm(string entryOrderName, double plannedEntry, double initialStop, double target, IDictionary<string,string> context = null)`	Record what you knew when you sent the entry (planned price, stop, target, context). The CS-1 trade row is written when the trade with that exact order name completes. Call it next to `Send()`.
`logSignal(string tag, int direction, double price, double stop, IDictionary<string,string> context, bool taken)`	Log a setup whether or not you traded it, so taken and skipped signals can be compared later. Forward returns are measured at `SignalHorizons` (default `{5,15,30,60}` bars).
`NoEntryBefore`, `NoEntryAfter`, `FlatBy` (`TimeSpan?`)	Daily entry window and the forced-flat time, compared with `sessionClock()`.
`MaxTradesPerDay`, `DailyLossStopCount`, `DailyLossStopR`, `RiskUnitDollars`	Daily limits: max entries, max losing trades, max loss in R (R = `RiskUnitDollars`). 0 = off.
`TimeStopBars`	Exit a position after this many bars. 0 = off.

Member	Plain meaning
`entryAllowed(out string reason)`	May I enter now? False with the rule that blocked (traced as `entry blocked: …` at Info).
`mustFlatten()`	Should the open position be closed now (FlatBy reached or TimeStopBars elapsed)? Always false when flat.
`TradesToday`, `LosersToday`, `NetProfitToday`	Today's counters, per `tradingDayKey`.
`sizeForRisk(IInstrument bars, double entry, double stop, double riskFraction, SizingLimits limits)`	Contracts so that the entry-to-stop loss ≈ `riskFraction` of equity, then capped by `SizingLimits` (`maxQty`, `maxNotionalFraction`, `unitNotional`, `minStopTicks`, `roundLots`). 0 = do not trade (reason traced). Also publishes the arm to M-PORT for Data1.
`equityAvailable(double)`	Equity available for one position: the whole chart equity, or the portfolio per-position share in Portfolio Trader, unless a custom percentage is given.
`costFilterPasses(IInstrument, entry, stop, out reason)`	Is the round-trip cost small enough relative to R? Cost = 2 × (Commission + Slippage) from Strategy Properties; fails when cost / R > `CostFilterMaxFractionOfR` (default 0 = filter off, always passes).
`ToZone(DateTime barTime, string tzId)`, `BarTimeZone`, `resolveBarTimeZone(IInstrumentSettings)`	Convert a bar time to another zone; the chart's zone.
`inTimeWindow(TimeSpan start, TimeSpan end, string tzId)`	Is the current bar time in `[start, end)` in that zone? Wraps past midnight.
`TradingDayRoll` (`Calendar` default, or `Auto`), `tradingDayKey(DateTime)` (virtual)	Which trading day a bar belongs to; all daily counters and engines use it.
`sessionClock(DateTime)` (virtual)	The time of day the CS-7 rules compare against: bar time on `_CustomSignal`, US Eastern on the futures base.
`Holidays` (`_NyseHolidays`)	NYSE holiday calendar.
`tickSize()`	Minimum price move of Data1 (resolved once per run).

Cost example. With the manual's cost assumptions ($2.25 commission + 0.5 tick = $6.25 slippage per side, $17.00 round trip on ES), a 14-tick stop is R = $175 and the cost is 0.097 R, so `CostFilterMaxFractionOfR = 0.10` lets it through; a 13-tick stop ($162.50, 0.105 R) is rejected.

Timing. The CS-7 counters reset on a new `tradingDayKey`. `mustFlatten` is true at or after `FlatBy`, or once `TimeStopBars` have elapsed.

Journal.

- `RUN id(..) build(..) strategy(..) symbol(..) category(..) resolution(..) timeZone(..) firstBar(..) barNumber(..) app(..) inputs[..]`, plus `event[canon] journalSchema(CS-1+evtDayTier,evtMinSince)` when Event is on.
- `RUN-END id(..) lastBar(..) barNumber(..) realTime(..)`.
- `entry blocked: <reason>` at Info.

Snippet. A long breakout entry that uses every core service: daily rules in `Create`, then on each closed bar: flatten if the rules say so, ask whether an entry is allowed, size from the stop, check costs, journal and send. Sizing follows the house rule (cost floor, §5.13): risk 1% of equity on the structural stop, widen the stop to at least 14 ticks, skip the setup if the widened risk exceeds 1%, and never shrink the size. The floor is strategy code; `sizeForRisk` has no floor of its own. `sizeForRisk` also arms M-PORT (§4.16) with the structural stop, so after widening the snippet calls `portfolioStop(stop)` (`protected void portfolioStop(double stop)`, `_CustomSignal` L1594; a no-op outside Portfolio Trader) so `pm.riskOpen` uses

the widened stop once the order fills. `pm.R$` and `pm.riskNew` stay on the structural stop. The widened-risk key `pm.R$w` exists only in the draft M-PORT risk-profiles spec and is not on main, so for now the pending-entry risk the money manager sees is understated. Do not call `portfolioArm` with the widened stop: it would overwrite `pm.R$` and the money manager would cut the size. A result of `qty = 0`, a budget breach or a failed cost filter means "skip this setup"; the reason is in the trace.

```
// class fields
private IOrderPriced m_le;                      // buy-stop entry
private IOrderMarket m_lx;                      // market exit

// Create
protected override void Create()
{
    base.Create();
    m_le = OrderCreator.Stop(new SOrderParameters(Contracts.UserSpecified, "LE", EOrderAction.Buy));
    m_lx = OrderCreator.MarketNextBar(new SOrderParameters(Contracts.Default, "LX", EOrderAction.Sell));
    NoEntryBefore = new TimeSpan(9, 35, 0);      // compared with sessionClock(): bar time on _CustomSignal,
ET on the futures base
    NoEntryAfter = new TimeSpan(15, 30, 0);
    FlatBy = new TimeSpan(15, 55, 0);            // mustFlatten() turns true at or after this time
    MaxTradesPerDay = 2;                         // counted per tradingDayKey()
    DailyLossStopCount = 2;                      // no new entries after 2 losing trades today
    CostFilterMaxFractionOfR = 0.10;             // skip trades whose round-trip cost exceeds 0.10 R
}

// CalcBar
protected override void CalcBar()
{
    base.CalcBar();                              // engines and CS-7 counters update here
    if (Bars.Status != EBarState.Close) return;
    if (mustFlatten()) { m_lx.Send(); return; }  // FlatBy reached or TimeStopBars elapsed
    string why;
    if (StrategyInfo.MarketPosition != 0 || !entryAllowed(out why)) return;   // why names the blocking rule
(traced at Info)
    double entry = Bars.High[0] + tickSize();
    double stop = Bars.Low[0] - tickSize();                    // structural stop
    int qty = sizeForRisk(Bars, entry, stop, 0.01,                   // house rule: 1% of equity at risk
        new SizingLimits { maxQty = 5 });                            // 0 = do not trade (reason traced)
    if (qty <= 0) return;
    stop = Math.Min(stop, entry - 14 * tickSize());            // house floor: stop distance =
max(structural, 14 ticks)
    if (qty * (entry - stop) * Bars.Info.BigPointValue > equityAvailable(0.01)) return;   // widened risk >
1%: skip, never shrink qty
    portfolioStop(stop);                        // M-PORT: the stop the order carries (pm.riskOpen once
filled); pm.R$/pm.riskNew stay on the structural stop
    if (!costFilterPasses(Bars, entry, stop, out why)) return;
    journalArm("LE", entry, stop, double.NaN);   // CS-1 row for the entry named "LE"
    m_le.Send(entry, qty);
}
```

4.2 `_CmeSessionClock` and the CME/NYSE calendars

Why it exists. CME futures trade nearly 24 hours, and the chart's time zone is rarely US Eastern. A strategy that says "trade the first 30 minutes of the cash session" must know, for any bar, the Eastern time, which session it falls in (Asia, Europe, New York, RTH), which Globex trading day it belongs to (an 18:00 ET bar is the next day's session), and whether today is a holiday, a half-day or a day with no cash session.

When to use it. Any rule tied to session times. Examples 1, 2, 4, 5, 12, 13, 18 and 23 build the clock themselves; strategies on the futures base use the built-in `SessionClock` and the wrappers listed below.

Enable. `new _CmeSessionClock(resolveBarTimeZone(Bars.Info))` (as in `_CmeSessionClockTest`). On the futures bases it is built in and exposed as `SessionClock`.

Defaults (ET).

- Globex open 18:00, close 17:00; halt 17:00–18:00.
- Asia 18:00–03:00; Europe 03:00–11:30; NewYork 08:00–17:00.

- `RthStart` 09:30, `RthEnd` 16:15 (session flag), `RthValueEnd` 16:00 (value window).
- `UseCmeHolidayTable = true`.

API.

Member	Plain meaning
`ToEastern`, `EasternOf(barTime)`	The bar time in US Eastern (bar time is in the chart zone).
`State(barTime)` → `GlobexState {Closed, Halt, Open}`	Is Globex open, in the daily halt, or closed?
`Sessions(barTime)` → `[Flags] MarketSession`; `InSession`, `InAny`	Every session that contains the bar (they overlap: 10:05 ET is Europe, NewYork and Rth).
`GlobexDate(barTime)`, `GlobexDateEt`	The Globex trading date (Sunday 18:30 ET belongs to Monday).
`PrimarySessionEt`	The single main session of an Eastern time.
`TimeSinceOpenEt` / `MinutesSinceOpenEt(et, session)`, `TimeUntilCloseEt` / `MinutesUntilCloseEt`, `SessionLength`	How far into, or how far from the end of, a session.
`IsOpeningRangeEt(et, session, minutes)`, `IsRthAfterInitialBalanceEt(et, ibMinutes)`	Inside the first N minutes of a session? After the RTH initial balance?
`TrySessionBoundsEt(et, session, out open, out close)`, `TryWindow`	Open and close times of the session containing `et`; false for `None` or combined flags.
`RthValueEndEt(globexDate)`, `IsRthValueBarEt(et)`	End of the RTH value window (earlier on early-close days); is a close-stamped bar inside `(RthStart, RthValueEnd]`?
`IsGlobexClosedDay`, `IsNoRthDay`	Full Globex closure? NYSE holiday with Globex open (no cash session)?
`HolidayHaltEt`, `RthEarlyCloseEt`	Holiday halt time and early RTH close (null when none).
Hooks `GlobexClosedDayHook`, `NoRthDayHook`, `HolidayHaltHook`, `RthEarlyCloseHook`	Replace the built-in holiday rules.
Statics `DefaultGlobexClosedDay`, `InWindow`, `IsSingleFlag`	The default closure rule; a `[start, end)` window test that wraps midnight; is a flag a single session?
Static `GlobexTradingDayEt(DateTime et)`, `GlobexTradingDayEt(DateTime et, TimeSpan globexOpen)` (PR #46)	The Globex trading day of an ET time without a clock instance: at or after the open (18:00) it is the next date. `GlobexDateEt` and `_Test6.tradingDayKey` use it.

`_CmeHolidays.Rules.` `Get(date)` returns a `Day` with `Closed`, `NoRth`, `HaltEt`, `RthCloseEt` and `Name`. Constants: `HolidayHaltEt` 13:00, `EarlyCloseEt` 13:15, `GoodFridayCloseEt` 09:15, `MourningCloseEt` 09:30. Helpers: `ClosedDates(year)`, `Observed`, `NthWeekday`, `LastWeekday`, `Easter`.

Futures-base wrappers (`_VolumeProfileFuturesBase`, no clock argument needed): `IsRth()`, `IsAsia()`, `IsEurope()`, `IsNewYork()`, `IsLondonNyOverlap()`, `GetGlobexState()`, `GetSessions()`, `MinutesSinceOpen(session)`, `MinutesUntilClose(session)`, `IsOpeningRange(session, minutes)`, `IsRthAfterInitialBalance(ib)`, `TrySessionBounds(...)`, `IsNewGlobexDay()`, `Session` (SessionStats: RTH/ETH/IB, prior-day H/L/C, `IbRange`, `Gap`), `IbMinutes` (60).

Timing. Bars are close-stamped. A bar is RTH when its ET close is in `(09:30, value end]`.

Journal. The test signal prints `VPF-7 clock globexDate(..) firstBarEt(..) state(..) sessions(..) noRth(..) rth(..)`.

Tests. `tests/SessionClockTests`, `tests/CalendarTests` (with `gen.sh`).

Snippet. Builds the clock on first use (as examples 1 and 12 do), then on each closed bar finds the Eastern time and Globex date, skips days without a cash session, and measures how far the bar is into RTH. `mins` is minutes since 09:30 ET; `openingRange` is true for bars inside the first 30 RTH minutes.

```
// class field: built lazily because Bars.Info is needed for the chart time zone
private _CmeSessionClock m_clock;
private _CmeSessionClock Clock { get { if (m_clock == null) m_clock = new
_CmeSessionClock(resolveBarTimeZone(Bars.Info)); return m_clock; } }

// StartCalc
protected override void StartCalc()
{
    base.StartCalc();
    m_clock = null;                              // rebuilt on the first bar of every run
}

// CalcBar
protected override void CalcBar()
{
    base.CalcBar();
    if (Bars.Status != EBarState.Close) return;
    DateTime et = Clock.EasternOf(Bars.Time[0]);       // close stamp in US Eastern
    DateTime gx =
Clock.GlobexDate(Bars.Time[0]);      // Globex trading date (an 18:00 ET bar belongs to the next day)
    if (Clock.IsNoRthDay(gx)) return;                  // NYSE holiday with Globex open: no RTH session today
    DateTime open, close;
    if (Clock.TrySessionBoundsEt(et, _CmeSessionClock.MarketSession.Rth, out open, out close))
    {
        int mins = Clock.MinutesSinceOpenEt(et, _CmeSessionClock.MarketSession.Rth);
        bool openingRange = Clock.IsOpeningRangeEt(et, _CmeSessionClock.MarketSession.Rth,
30);   // first 30 RTH minutes
    }
}
```

4.3 M-DAILY `_DailyContext`

Why it exists. Intraday strategies usually need daily context: yesterday's high, low and close, a daily ATR for stop sizes, volatility (HV), momentum (TSMOM), the overnight gap. Computing these from intraday bars is error-prone around rolls and holidays, and it is easy to peek at today's unfinished day. M-DAILY builds them from **completed** days only and adjusts its history when the contract rolls.

When to use it. Daily levels, ATR-based stops or targets, and volatility filters: examples 9, 10, 17, 20, 27 and 30–33. Regime (§4.4) turns it on automatically.

Enable. `NeedsDaily = true` in `Create()` (or in `StartCalc()` before `base.StartCalc()`).

`DailyConfig` defaults.

- `dataStream 0` (aggregate Data1 per `tradingDayKey`), `weeklyStream 0`;
- `atrLength 20`, `smaLength 200`, `rsiLength 2`, `tsmomLookbacks {252,126,63}`;
- `hvLengths {}`, `hvAnnualisation 252`, `hvPctLookback 0`, `lbrRsiLength 0`;
- `closeBasis EDailyClose.RthValue`; `HvOn`; `Validate()`.

API.

Member	Plain meaning
`Atr20d`	Wilder ATR of daily true range, length `atrLength` (the name is historical). NaN until seeded.
`Sma200`, `Rsi2`	Daily SMA (`smaLength`) and RSI (`rsiLength`) of the basis close.
`Tsmom(n)`	Time-series momentum: adjusted close / close n days ago − 1.
`Hv(n)`, `HvPercentile(n)`	Annualised historical volatility over n days (0.25 = 25%); its percentile (0–100) over the last `hvPctLookback` days.
`Roc1`, `Lbr`	One-day change in points; RSI of that change (LBR).
`RthCloseAdj`, `RthClosePrev`, `PrevClose`	Roll-adjusted RTH close; unadjusted RTH close; last bar's close of the prior day.
`PrevHigh` / `PrevLow` / `PrevDate` / `PrevRange`, `PrevWeekHigh` / `PrevWeekLow`	Prior day's and prior week's levels.

Member	Plain meaning
`Valid`, `Days`	ATR seeded and a prior day exists; number of completed days.
`GapPct(todayOpen)`, host `DailyGapPct`	Today's gap vs the adjusted prior close, as a fraction.

Timing. A day is added when the next trading day's first bar arrives, so on every bar `PrevDate` is a completed prior session. With a daily data stream, only the closed daily bar (`HtfClosedIndex`) whose day is before today is used. On a roll (only with RollDetection on) the host calls `OnRoll(day, rollBarOpen - PrevClose)` (`_CustomSignal` line 991): the stored o/h/l/c, RTH closes and weekly values are difference-adjusted, and, by default, the ATR restarts its seed: **`Atr20d` is NaN and `Valid` is false for `atrLength` sessions after every roll.** With `atrLength = 200` and quarterly @ES rolls (about every 63 sessions) the ATR never becomes valid. With RollDetection Off nothing is adjusted and the roll gap enters the ATR.

ATR across rolls (`DailyConfig.atrRestartOnRoll`, PR #46). True ranges do not change when a whole history is shifted by the roll offset, so the ATR does not have to restart. Set `DailyConfig.atrRestartOnRoll = false` (default `true`, the old reseed) and the running ATR carries on; only the splice day's true range is skipped, so there is no NaN blackout. Use it when a strategy needs `Atr20d` every day (stops, filters, PS-13-style long ATRs); keep the default for a strict post-roll warm-up. `_Test6` sets it to false (a RegimeGate hash field, `host.atrRestartOnRoll`). Set it before `base.StartCalc()` (lives in `StartCalc` or `Create`):

```
// StartCalc (before base.StartCalc)
protected override void StartCalc()
{
    NeedsDaily = true;
    RollDetection = ERollDetection.Table;          // _Test6 style: fixed roll dates
    for (int y = 2016; y <= 2026; y++)
        for (int m = 3; m <= 12; m += 3) RollDates.Add(EventRules.RollDate(y, m));
    DailyConfig.atrRestartOnRoll = false;          // ATR carries on; only the splice day's TR is skipped
    base.StartCalc();                              // M-DAILY is built here with this config
}
```

Journal.

- `DAILY {date} return spans a gap of N business days since {prev};`
- `DAILY hv{n} first valid {date} …; DAILY hv{n}pct first valid …;`
- `DAILY roll reset {date}: history adjusted by {d}, ATR reseeding` (or `…, ATR carried on (splice-day TR skipped)` with `atrRestartOnRoll = false`);
- `DAILY closeBasis=Bar (P3 unverified): …;`
- a Warning: `DAILY Data{n} advanced k bars between Data1 bars.`

Tests. `tests/DailyTests` (`gen.sh` generates the calendars).

Snippet. Turns M-DAILY on with HV(20) and looks for a close above yesterday's high while volatility is low. `d == null` means M-DAILY is off; `!d.Valid` means warm-up or ATR reseeding after a roll, so the strategy waits.

```
// Create
protected override void Create()
{
    base.Create();
    NeedsDaily = true;                          // lazy: M-DAILY is built in base.StartCalc only when asked
for
    DailyConfig.hvLengths = new[] { 20 };       // also compute HV(20)
}

// CalcBar
protected override void CalcBar()
{
    base.CalcBar();
    if (Bars.Status != EBarState.Close) return;
    _DailyContext d = DailyContext;
    if (d == null || !d.Valid) return;          // warm-up, or ATR reseeding after a roll
    bool priorDaysOnly = d.PrevDate < tradingDayKey(Bars.Time[0]);   // always true by design; documents the
no-lookahead rule
```

```
    if (priorDaysOnly && Bars.Close[0] > d.PrevHigh && d.Hv(20) < 0.25)
    {
        /* breakout above yesterday's high in a calm (HV20 < 25%) market */
    }
}
```

4.4 G-R regime engine `_RegimeEngine`

Why it exists. The same setup behaves differently in calm and in volatile markets. The regime engine labels each day as one of K market states (0 = calmest) with a hidden-Markov (or Gaussian-mixture) model on daily return, volatility and momentum features. Hysteresis stops the label flipping on noise, and an ATR-tercile fallback covers the warm-up.

When to use it. As a filter or a sizing bucket, never as an entry trigger: example 9 (via RegimeGate, §4.5) and `_Test6`.

Enable. `RegimeConfig.enabled = true` in `Create()`. This also turns on M-DAILY with HV {20, 63}.

`RegimeConfig` defaults.

- Model: `method Hmm, K 3, kSelect Fixed, features {Return, LogHv20, Tsmom63z}`.
- Fitting: `fitSessions 1000`, `minSessions 500`, `refit Monthly`, `refitEveryN 21`, `restarts 10`, `maxIter 200`, `tol 1e-6`, `regCovar 1e-6`, `transPrior 1`.
- Filtering and switching: `warmFilter 63`, `switchProb 0.60`, `switchMargin 0.10`.
- Other: `seed 20261021`, `fallback AtrTercile`, `publish false`, `atrPctLookback 250`, `parkinsonBars 10`, `bicDiagnostic true`, `rollSuspectZ 5`, `gapBusinessDays 4`.
- Methods: `Validate()`, `Copy()`, `IndexOf`.

Enums. `ERegimeMethod {Hmm, Gmm}`, `ERegimeKSelect {Fixed, Bic}`, `ERegimeRefit {Monthly, EveryNSessions, Quarterly}`, `ERegimeFallback {AtrTercile, Unknown}`, `ERegimeFeature {Return, LogHv20, Tsmom63z, Parkinson10}`.

API. The `Regime` property returns a copy of `RegimeState` (neutral, `Valid=false`, `Label=-1`, when the engine is off).

Field / method	Plain meaning
`Valid`, `Label`	Is there a usable label? The state, 0 = calmest … K−1, −1 = none.
`RawArgmax`, `Probs`	Most likely state before hysteresis; the state probabilities.
`AsOf`, `FittedThrough`	The day whose close produced the label (usable from the next session); the last day of the fit that produced the model (always before `AsOf`).
`K`, `LogLik`, `Refits`, `RelabelConflicts`	Number of states, fit quality, refit count, a model-health count.
`UsingFallback`	The label comes from the ATR-tercile fallback, not the model.
`Copy()`, `Neutral(K)`	Copy; an empty state.
Engine statics `AtrPercentile`, `AtrBucket`, `Hysteresis`, `Argmax`	Building blocks, exposed for tests.

Timing. The engine is fed once per completed M-DAILY day. `AsOf` is that day, and the value is usable from the **next** session. Refits on a roll day are deferred.

M-PORT. With `publish = true`, `pm.regime = Label+1` and `pm.regimeP = Probs` are published at session close.

Journal. `REGIME {date} label() raw() p() fb()`; `REGIME switch a->b {date}`; `REGIME refit {date} window() K() ll() iters() conv() means()`; `REGIME diag {date} bicK…`; `REGIME fallback reason(…)`; `REGIME refit deferred {date}: roll day`; `REGIME gap …`; `REGIME roll adjust suspect …`; `REGIME relabel-conflict …`.

Tests. `tests/RegimeTests` (`reference.py`; `run.sh` can `--export gv`), `tests/GmmTests`.

Snippet. Turns the engine on and turns its output into a bucket number. `bucket = -1` means "no usable regime today" (warm-up, fallback, or not yet from a completed day); treat that as its own case rather than as calm.

```
// Create
protected override void Create()
{
    base.Create();
    RegimeConfig.enabled = true;                // also turns on M-DAILY with HV {20, 63}
}

// CalcBar
protected override void CalcBar()
{
    base.CalcBar();
    if (Bars.Status != EBarState.Close) return;
    RegimeState r = Regime;                     // a copy; neutral (Valid = false, Label = -1) when disabled
    bool usable = r.Valid
        && !r.UsingFallback                     // HMM label, not the ATR-tercile fallback
        && r.AsOf < tradingDayKey(Bars.Time[0]); // labelled from completed prior sessions only
    int bucket = usable ? r.Label :
-1;          // 0 = calmest; use as a filter or sizing bucket, never as a trigger
}
```

4.5 RegimeGate / A2-63 helpers (`_RegimeGate.Function.CS`)

Why it exists. To test "does skipping volatile days improve this strategy?" fairly, the rule must be fixed in advance and the gated and ungated runs must be identical apart from the gate. RegimeGate is that pre-registered rule: three arms (U = ungated, G = skip days whose prior-session HMM label is 2, A = skip ATR tercile 2), one decision function, and hashes that prove which configuration produced a run.

When to use it. Only for the A2-63 regime experiment: `_Test6` and example 9 (`_Ex09OpenAtrBreakout`). For other filters read `Regime` directly (§4.4).

Enable. Build the configs yourself and call `RegimeGate.ApplyArm(RegimeGateConfig, RegimeConfig)` in `StartCalc()` **before** `base.StartCalc()`.

Defaults.

- `RegimeGateConfig { arm = ERegimeGateArm.U, placeboSeed = RegimeGate.PlaceboSeed (20261022) }`.
- `A263Config { k 0.5, atrLength 20, rthOpen 09:30, activeEnd 15:30, exitTime 15:55, rthClose 16:00, contracts 1, halfDayExitBefore 5m }`.

API.

Member	Plain meaning
`RegimeGate.Decide(arm, RegimeState, session, atrTercile) → RegimeGateDecision`	Should this arm trade this session? `allow` is the answer; `reason` (`ERegimeGateReason {Ok, Warmup, Invalid, Fallback}`) says whether the regime was usable; also `session`, `asOf`, `arm`, `label`, `raw`, `atrTercile`, `pFilt`, `fb`, `usable`, `wouldBlock`.
`RegimeGate.ApplyArm(gate, regimeConfig)`	Write the arm's settings into `RegimeConfig` (arm A disables the model with `fitSessions = minSessions = 100000`).
`RegimeGate.Line`, `TradeFields`, `HashFields`, `Canon`, `Hash`, `Fnv1a64`; `BlockedLabel = 2`, `Version = "A263-RG-2"`	Journal line and canonical hash of the configuration (proves which arm ran).
`AtrTercileTracker(lookback, K)`: `.AddDay(date, atr)`, `.Tercile`, `.LastDate`	Which third of its recent range today's daily ATR is in (input for arm A).
`A263Core(cfg)`: `IsLatchBar(etClose)`, `Latch(...)`, `Step(etClose, isLong) → A263Action {latched,`	The A2-63 host strategy (a buy stop at the RTH open + k × daily ATR) as a reusable core.

Member	Plain meaning
`sendStop, stop, exit}, HasSignal, Threshold, Suffix()`	
`RegimeGateCounts.Add(d, noRth), .Suffix()`	Running counts of usable, warm-up, invalid, fallback, blocked and no-RTH sessions.

Hash pins. U `26b38655261cc0bf`, G `9c1da7b123ca9441`, A `3d9814a5f264c353`.

Journal. Header `REGGATE header version() arm() cfg() seed() symbol() bar() barDeviation() firstBar() canon[]`. Per session `REGGATE {date} arm() asOf() label() raw() pFilt() fb() usable() allow() atrT() wouldBlock() why() signal() cfg() seed()`, then `O() atr20d() atrAsOf() atrOk() thr() exitAt() halfDay() noRth()` and the counts `nSess() nUsable() nWarm() nInv() nFb() nBlock() nNoRth()`.

Tests. `tests/RegimeGateTests` (`gen.sh`).

Snippet. Applies arm G before the engines are built and asks the gate once per bar. `d.allow == false` means "do not trade this session"; the full host (ATR tercile, latch, counts) is example 9.

```
// class field
protected RegimeGateConfig GateCfg { get; } = new RegimeGateConfig();

// StartCalc (the arm must be applied BEFORE base.StartCalc, where the engines are built)
protected override void StartCalc()
{
    GateCfg.arm = ERegimeGateArm.G;              // U = ungated, G = skip HMM label 2, A = skip ATR tercile 2
    string err = GateCfg.Validate();
    if (err != null) throw new ArgumentException(err);
    RegimeGate.ApplyArm(GateCfg, RegimeConfig);  // writes the arm's settings into RegimeConfig
    base.StartCalc();
}

// CalcBar
protected override void CalcBar()
{
    base.CalcBar();
    if (Bars.Status != EBarState.Close) return;
    RegimeGateDecision d = RegimeGate.Decide(GateCfg.arm, Regime, Bars.Time[0].Date, -1);   // -1: no ATR
tercile (arm A needs one)
    if (!d.allow) return;                        // d.reason says why: Warmup, Invalid, Fallback, or Ok with
a blocked label
}
```

4.6 M-EVENT `_EventCalendar`

Why it exists. Scheduled releases (FOMC, CPI, NFP ...) move index futures in seconds, and calendar days (expiry, roll week, half-days, index rebalances) change normal behaviour. A backtest that trades through them without knowing it mixes two different markets. M-EVENT is a calendar with **as-of visibility**: an event is only visible once it was publicly scheduled, so a backtest cannot "know" an unscheduled meeting in advance.

When to use it. Blackouts around releases, flattening before a release, or classifying days ("FOMC day", "roll week") for filters or for splitting results: examples 2, 4 and 24.

Enable. `EventConfig.enabled = true` in `Create()`.

`EventConfig` defaults.

- Scope: `classes Macro|Calendar`, `minTier T3`, `tierOverride {}`, `csvPath ""`, `allowPartialCoverage false`.
- Blackout: `blackoutPreMin 0`, `blackoutPostMin 0` (0/0 = no blackout), `blackoutTier T1`, `blackoutFlatten false`.
- Other: `publish false`, `includeRussell false`; methods `Validate()`, `Canon()`.

Coverage. Macro: FOMC, CPI, PPI, NFP, JOLTS, Claims, PCE, GDP, Retail, ISM, Jackson Hole, testimony. Energy: EIA. Calendar: quarterly expiry, roll week, monthly opex, index rebalance, half-day, VIX expiry, Russell recon.

Enums.

- `EEventTier {None=0, T3=1, T2=2, T1=3, C=8}` (importance; T1 is the highest);
- `EEventClassMask {None, Macro=1, Calendar=2, Energy=4, All=7}`;
- `EEventKind` (FomcDecision=1 … EiaPetroleum=18, QuarterlyExpiry=30 … RussellRecon=36, Custom=62);
- `EEventFlags` (Sep, Quad, Unscheduled, Scheduled, Delayed, Canceled, Csv, RuleOnly, Pm).

Records.

- `EventRec` (one event): `kind`, `tier`, `flags`, `etTime`, `etEnd`, `utc`, `tradingDay`, `knownSinceUtc` (when it became public), `canceledSinceUtc`, `refPeriod`, `name`, `source`; plus `IsSpan`, `Class`, `KindName`.
- `EventDay` (one day's summary): `day`, `maxTier`, `kindsMask`, `hasFomc`, `hasSep`, `isHalfDay`, `isRollWeek`, `isQuarterlyExpiry`, `isOpex`, `isRebalance`, `firstReleaseEt` (TimeSpan?), `count`, `eiaRuleOnly`; plus `Has(kind)`.

API.

Member	Plain meaning
Host `EventBlackout`, `EventFlattenNow`	Inside the configured blackout? Should you flatten now (only with `blackoutFlatten`)? False when the engine is off.
`TradingDay`, `NowUtc`, `Disabled`	Current trading day and bar-close instant; true only when the table is missing or fails its hash check (all answers are then neutral).
`NextEvent(minTier, out e)`, `PrevEvent(minTier, out e)` (+ overloads `(nowUtc, minTier, mask, out e)`)	Next / last known event of at least that tier.
`MinutesTo(minTier, mask = Macro)`, `MinutesSince`, `MinutesSinceAt`	Minutes from the bar close to the next event, or since the last one; NaN when none.
`IsInWindow(minTier, pre, post, mask = Macro)`, `IsInWindowAt`	Inside `[event − pre, event + post]` of a qualifying event?
`EventsOn(day, mask = All)`, `EventsOnExPost`, `EventsInRange`	Events of a day or range (ExPost = with hindsight, for research only).
`Classify(day)`, `Classify(day, asOfUtc)`, `ClassifyExPost(day)`	The day's `EventDay` summary as known at its session open (or at a chosen instant, or with hindsight).
`IsRollWeek()`, `IsRollDay()`, `SessionsToExpiry()`, `InSpan(kind)`, `LastT1Instant(out et)`	Roll and expiry position; inside a multi-day span (e.g. roll week)? Latest T1 release at or before now.
`ToEt`, `TradingDayOf`, `SessionOpenUtc`	Time conversions.
Statics `RuleRollDate`, `RollDateFor`, `EventRules` (`IsNyseBusinessDay`, `ThirdFriday`, `OpexDate`, `RollDate`, `VixExpiry`, `TableSha`)	Calendar rules behind the table.

Timing.

- Stepped once per closed bar with `[open, close)` in UTC.
- An event is visible only once it is known (`knownSinceUtc <= now`) or has happened. Cancellations are honoured as of now.
- The blackout uses the Macro class only, so Energy consumers call `IsInWindow(..., EEventClassMask.Energy)` explicitly.
- Window queries on daily bars return false.
- **The base never blocks or sends orders.** Acting on `EventBlackout` / `EventFlattenNow` is your job.

M-PORT keys (`publish = true`): `pm.evtTier`, `pm.evtMask` (daily); `pm.evtWin`, `pm.evtNext` (per bar). **CS-1 extra columns:** `evtDayTier`, `evtMinSince`.

Journal.

- `EVENT created classes() minTier() blackout(pre/post tier) …; EVENT table missing: …; EVENT coverage rules(…) …; EVENT correction …;`
- `EVENT day {yyyy-MM-dd} at {bar} tier() kinds(Kind@HH:mm,…) firstRelease() [rollWeek] [halfDay] [EiaRuleOnly];`
- `EVENT blackout on|off at {bar} {Kind} {et} ET pre() post() [flatten];`
- an Error line ending `…; engine disabled` **only** when the table is missing or its hash mismatches (EV-4.6b);
- `COVGAP {Kind} {from}..{to} Unknown …` once at start, and `COVGAP day {yyyy-MM-dd} Unknown Kind(blocks|belowMinTier|NoEvent)` per affected day.

EV-4.6b (PRs #28/#29). `Coverage(kind, day)`, `CoverageBlocks(day, mask, minTier, out kind)`, `CoverageBlocks(day, mask)`, `CoverageReport(from, to, mask[, minTier], out flagged)`, `CoverageStats()`, `EventRules.CoverageHoles()`, `EventRules.TableSha()`, `EventRules.ExpectedTableSha`, `EventConfig.tierOverride`: where the table has no data for a kind, so a "no event" answer would be unknown rather than "none". Details in Appendix G.

Tests. `tests/EventTests` (`gen.sh`). Tooling lives in `tools/m-event`.

Snippet. A 5-minutes-before / 15-minutes-after blackout around T1 releases, with flattening. On each bar the strategy exits if told to, skips entries inside the blackout, and reads the day's classification and the distance to the next T1 event. `toT1` is NaN when no event is ahead in the table's coverage.

```
// class field
private IOrderMarket m_lx;

// Create
protected override void Create()
{
    base.Create();
    m_lx = OrderCreator.MarketNextBar(new SOrderParameters(Contracts.Default, "LX", EOrderAction.Sell));
    EventConfig.enabled = true;
    EventConfig.blackoutPreMin = 5;             // blackout from 5 min before ...
    EventConfig.blackoutPostMin = 15;           // ... to 15 min after each T1 (blackoutTier) Macro event
    EventConfig.blackoutFlatten = true;         // also raise EventFlattenNow during the blackout
}

// CalcBar
protected override void CalcBar()
{
    base.CalcBar();
    if (Bars.Status != EBarState.Close) return;
    if (EventFlattenNow && StrategyInfo.MarketPosition != 0) { m_lx.Send(); return; }   // the base never
sends orders itself
    if (EventBlackout) return;                  // no new entries inside the window
    _EventCalendar ev = EventCalendar;
    if (ev == null) return;                     // disabled, or the table was missing / failed its hash
check
    EventDay today = ev.Classify(ev.TradingDay); // as known at today's session open
    bool fomcDay = today.hasFomc;
    double toT1 = ev.MinutesTo(EEventTier.T1);   // minutes to the next T1 Macro event
    EventRec next;
    if (ev.NextEvent(EEventTier.T2, out next) && journalOn())
        trace(ETraceLevel.Journal, $"next {next.KindName} {next.etTime:HH:mm} ET");
}
```

4.7 M-AVWAP `_AvwapManager`

Why it exists. An anchored VWAP is the volume-weighted average price since a specific event (a swing low, a gap open, the week's open). It answers "where is the average buyer since that event?", and traders use it, with its σ-bands, as dynamic support and resistance. The manager creates anchors

automatically on the events you choose, keeps a few alive, and expires them, without lookahead: a swing anchor exists only once the swing is confirmed.

When to use it. Pullback-to-VWAP or band-reversion entries, and "price above or below the event VWAP" filters: example 16 (`AvwapEvents = SwingLow`, `maxAgeSessions 60`, set in `Create`).

Enable. `AvwapEvents = …` (flags) in `Create()`; `None` = no manager.

`EAvwapAnchor` flags. `SwingHigh=1`, `SwingLow=2`, `Breakout=4`, `GapOpen=8`, `VolumeClimax=16`, `WeekOpen=32`, `MonthOpen=64`, `QuarterOpen=128`, `NewsBar=256` (inert), and `Swing` (= SwingHigh | SwingLow).

`AvwapLimits` defaults.

- Anchors: `maxAnchors 6`, `maxAgeSessions 10`, `replaceSameType true` (a new anchor of a type expires the old one).
- Bands and slope: `bandSigmas {1,2,3}`, `slopeBars 5`, `dataStream 1`.
- Detection thresholds: `gapAtrFraction 0.5`, `gapFallbackTrMultiple 3`, `climaxZ 3`, `fallbackLength 20`, `backfillBars 2`.
- Placebos: `placebos EAvwapPlacebo.None`, `p2MinPool 10`.

API.

Member	Plain meaning
`Anchors (IReadOnlyList<_Avwap>)`	The live anchors, oldest first.
`Latest(EAvwapAnchor type)`	Newest live anchor of a type, or null.
`LastBar`, `FallbackTrAvg`	Last bar fed; the SMA of true range used when M-DAILY ATR is missing.
`_Avwap.Id`, `Type`, `AnchorTime`, `AnchorBar`	Which event, and where the VWAP starts.
`UsableFrom`	The first time you may use it (pivot confirmation for swings, the anchor time otherwise).
`Direction`	+1 up event (swing low, breakout up, gap up), −1 down, 0 neutral.
`Value`, `Sigma`, `Band(k)`	The VWAP, its volume-weighted standard deviation, and `Value + k × Sigma` (`Band(-1)` is the −1σ band).
`SlopeTicks`, `SlopeAtr`	Slope per bar over `slopeBars`, in ticks and in ATR.
`Valid`	Not expired and has a value.
`AgeBars`, `AgeSessions`, `Touched`, `TouchCount`, `Expired`, `ExpireReason`	Life-cycle state.
`Tag (AV:{Type}:{yyyyMMddHHmm})`, `TagFor(arm)`	Journal tag; tag of a placebo arm.
`P1`, `P1Sigma`, `P2`, `P2Sigma`, `p2Degraded`	Placebo twins for research: unweighted mean price (P1) and volume-shuffled VWAP (P2).

Timing. Swing anchors become usable at PA pivot confirmation (`UsableFrom`); other anchors at the anchor time. GapOpen uses `isRthBar` and the M-DAILY ATR when available. All anchors are invalidated on a roll.

Dependencies. Swing and Breakout force PriceAction on. VolumeClimax and P2 force TOD Volume on.

Journal. `AV created {Tag} usable={t} value={v}`; `AV expired ({why}) {Tag} at {t}`; `AV invalidated (roll) {Tag} at {t}`; `AV p2Degraded …`.

Tests. `tests/AvwapTests`.

Snippet. Anchors VWAPs at confirmed swings and gap opens, then each bar looks for a long pullback: the bar's low reaches the −1σ band of an up-anchored VWAP (one started at a swing low or a gap up). `usable` false means the anchor is expired, has no value yet, or is not yet confirmed; skip it.

```
// Create
protected override void Create()
{
    base.Create();
    AvwapEvents = EAvwapAnchor.Swing             // anchor at confirmed swing highs and lows (forces
PRICEACTION on)
        | EAvwapAnchor.GapOpen;                  // and at RTH gap opens
    AvwapLimits.maxAgeSessions = 5;              // drop anchors older than 5 completed sessions
}

// CalcBar
protected override void CalcBar()
{
    base.CalcBar();                              // the manager steps here, after PRICEACTION
    if (Bars.Status != EBarState.Close) return;
    _AvwapManager am = AvwapManager;
    if (am == null) return;                      // AvwapEvents == None
    foreach (_Avwap a in am.Anchors)             // live anchors, oldest first
    {
        bool usable = a.Valid                        // not expired, value defined
            && a.UsableFrom <= Bars.Time[0];         // no lookahead: swing anchors only from pivot
confirmation
        bool upAnchor = a.Direction > 0;             // swing low or gap up
        bool atMinus1Sigma = Bars.Low[0] <= a.Band(-1);   // Band(-1) = Value - 1 x Sigma
        if (usable && upAnchor && atMinus1Sigma)
        {
            /* long pullback to the -1 sigma band of an up-anchored VWAP */
        }
    }
}
```

4.8 PRICEACTION `_PriceActionEngine`

Why it exists. Market structure (swing highs and lows, higher-highs / lower-lows, breaks of structure) is the vocabulary of discretionary traders but is easy to code with lookahead, because a swing is only known some bars after its extreme. The engine finds pivots on several layers (minor and major swings), labels them, detects breaks of structure (BOS) and changes of character (CHoCH), and flags candle and volume patterns, each only from the bar on which it is confirmed.

When to use it. Structural stops (below the last higher-low), trend filters from the higher layer, breakout triggers: examples 19, 26, 28, 31 and 34 (31 and 34 also run extra engines on other streams).

Enable. `PriceActionConfig.enabled = true` in `Create()`. Extra instances: `ExtraPriceAction.Add(new PriceActionConfig { enabled = true, dataStream = 2, role = "htf" })`.

`PriceActionConfig` defaults.

- Stream and pivots: `dataStream 1`, `pivotMode Fractal`, `tieRule Strict`, `layers 2`, `strength 1`.
- ATR and volume: `atrLength 20`, `volumeLength 20`, `volumeIncludesCurrent true`, `atrMultiple 1.5`.
- Candle factors: `factorDoji 0.1`, `factorClose 0.1`, `factorStrong 1.0`, `factorEpsilon 0.05`, `equalFactor 0.1`.
- Breakouts and history: `lookbackPivots 5`, `minBreakoutLayer 1` (main engine; the DVP-hosted engines use 0, §5.31), `breakoutNeedsStrong true`, `history 50`, `candles true`.
- Presets: `preset Library`, `role ""`; `ApplyPreset(EPriceActionPreset)`, `ApplyPresetDefaults`. The Futures preset uses `LeftInclusive` and `volumeIncludesCurrent = false`; the futures bases apply it unless you called `ApplyPreset` yourself.

API.

Member	Plain meaning
`IsWarm`	Enough bars for ATR and volume statistics.
`Atr`, `VolumeMean`, `VolumeSigma`	The engine's own ATR and volume statistics.
`Candle` (`ECandle` flags), `Has(ECandle)`	Candle pattern flags of the last closed bar (doji, strong close, breakout candle ...).

Member	Plain meaning
`Changed`	Candle flags that differ from the previous bar's flags.
`LastBarTime`, `LastBar`	Which bar was last processed.
`NewPivotHigh`, `NewPivotLow`	A layer-0 pivot was confirmed on this bar.
`BosUp`, `BosDown`, `ChochUp`, `ChochDown`	This bar closed beyond the last swing high / low (break of structure); CHoCH = a break against the structure before the break.
`LastHigh(layer = 0)`, `LastLow(layer = 0)`, `High(layer, ago)`, `Low(layer, ago)`, `PivotCount(layer, isHigh)`	Confirmed pivots, newest first.
`Structure(layer = 0)` → `EStructure {Unknown, Up, Down, Range}`	Trend state of a layer.
`DoubleTop(layer)`, `DoubleBottom(layer)`	The last pivot high (low) is equal, within `equalFactor`, to an earlier one (`isEqual`).
`_Pivot` fields `isHigh`, `layer`, `seq`, `price`, `extremeTime`, `extremeBar`, `confirmTime`, `confirmBar`, `label` (`EPivotLabel` HH/LH/HL/LL), `isEqual`, `equalToSeq`, `legSizeAtr`	Where the extreme was, when it was confirmed, and how it relates to the previous pivot.
`PriceActionFor(int dataStream, string role = null)`	Find an extra engine by stream and role.

Timing. Steps on its own stream's closed bars. A pivot exists only from `confirmTime` (after `strength` bars) and is never relabelled. The engine resets on a roll.

Journal. `PA{ds} L{layer} PivotHigh|PivotLow #{seq} {price} {label} [EQ#n] ext={t} conf={t}`; … `BreakoutUp|BreakoutDown L{layer} of #{seq} {price} at {t}`; … `BOS up|down [CHoCH] at {t}`; … `roll reset at {t}`; `PA{ds}-{role} created strength() layers() shared()`.

Tests. `tests/PriceActionTests` (`McShim.cs`).

Snippet. A bullish break of structure in an up-trending higher layer, with a structural stop 2 ticks under the last confirmed swing low. `stop = NaN` means no swing low exists yet; do not trade.

```
// Create
protected override void Create()
{
    base.Create();
    PriceActionConfig.ApplyPreset(EPriceActionPreset.Futures);   // the futures bases do this for you
    PriceActionConfig.enabled = true;
}

// CalcBar
protected override void CalcBar()
{
    base.CalcBar();                              // the engine steps on its stream's closed bar here
    if (Bars.Status != EBarState.Close) return;
    _PriceActionEngine pa = PriceActionEngine;
    if (pa == null || !pa.IsWarm) return;
    bool bullishBreak = pa.BosUp                 // this bar broke the last pivot high
        && pa.Structure(1) == EStructure.Up;     // and the higher layer is in an up-trend
    if (!bullishBreak) return;
    _Pivot hl = pa.LastLow(0);                   // last confirmed layer-0 pivot low
    double stop = hl != null ? hl.price - 2 * tickSize() : double.NaN;   // structural stop 2 ticks under it
}
```

4.9 M-TOD `_TodNormaliser`

Why it exists. Volume and range follow a strong daily pattern: the first minutes after 09:30 ET are always busy, lunch is always quiet. "Volume is high" only means something compared with the same time on other days. M-TOD keeps, for each minute-of-session bucket, the values of prior completed sessions and reports how unusual this bar is for its time of day.

When to use it. Volume-climax or quiet-market filters, and "is this opening range wide for an opening range?": examples 4 (volume pool), 19 (range median) and 25 (true range and volume z).

Enable. `TodMetrics = ETodMetric.Volume | ETodMetric.Range …` in `Create()` (`ETodMetric {None, Volume, Delta, Range, TrueRange}`).

Defaults. `TodSessions 20` (sessions in the norm), `TodMinSessions 10` (needed before `Valid`), `TodBucketMinutes 0`. The host resolves the bucket in `_CustomSignal` (line 882): 0 → the Data1 bar size when Data1 is a minute chart, else 1 minute.

API.

Member	Plain meaning
`Valid(m)`	Enough prior sessions in this bar's bucket.
`Z(m)`	z-score of this bar vs the same bucket in prior sessions (3 = three σ above normal for this time).
`Median(m)`, `Pct(m)`	The bucket's median; the fraction (0–1) of prior sessions at or below this bar.
`ZCum(m)`	Session-to-date cumulative value vs the cumulative norm (e.g. "volume so far today is unusually high").
`Sessions(m)`	Sessions currently in the norm.
`VolumePool(minuteOfSession)`	The raw prior-session volumes of a bucket (copy, oldest first).

Timing. Norms come from prior completed sessions. Half-days are excluded (via `todHalfDay`). Range and TR are dropped on the roll-splice bar. Delta requires the futures base order flow.

Journal. `TOD delta tier change a -> b: norm reset at session {day}.`

Tests. `tests/TodTests.`

Snippet. Flags a volume climax: this bar's volume is more than 3 σ above the normal volume for this minute of the session. `climax` is false during warm-up (fewer than `TodMinSessions` sessions).

```
// Create
protected override void Create()
{
    base.Create();
    TodMetrics = ETodMetric.Volume;             // norm volume per minute-of-session bucket
}

// CalcBar
protected override void CalcBar()
{
    base.CalcBar();
    if (Bars.Status != EBarState.Close) return;
    _TodNormaliser tod = TodNormaliser;
    bool climax = tod != null
        && tod.Valid(ETodMetric.Volume)         // enough prior sessions for this bucket
        && tod.Z(ETodMetric.Volume) > 3;        // volume > 3 sigma above normal for this time of day
}
```

4.10 M-XMKT `_XmktLeg`

Why it exists. Related markets lead or confirm each other (NQ vs ES, ES vs bonds). M-XMKT reads another market from an extra chart stream, aligns it to Data1 on closed bars only, and gives you returns, rolling correlation, beta and a relative-strength z-score. Orders stay on Data1.

When to use it. Relative-strength or divergence filters: example 39.

Enable. `XmktLegs.Add(new XmktLegConfig { role = "NQ", dataStream = 2 })` in `Create()`.

Defaults. `role "leg"`, `dataStream 2`, `maxStaleSec 0` (= 2× the leg bar; off for non-time bars), `returnBars 1`, `corrLength 60`, `rsLength 60`, `leaderSwing false`.

API.

Member	Plain meaning
`XmktLeg(role)` → `_XmktLeg`	The leg, or null when not configured.
`Available`, `LastLegTime`, `LastClose`	The leg has a fresh closed bar (not stale); its time and close.
`LogReturn`, `Data1LogReturn`	Log returns of the leg and of Data1 over `returnBars`.
`Correlation`, `Beta`	Rolling correlation and beta of the leg vs Data1 (`corrLength` bars).
`RsZ`	z-score of ln(leg / Data1) vs its rolling window: negative = the leg is weak relative to Data1.
`XmktLeaderSwing(role)`	A PRICEACTION engine on the leg (with `leaderSwing = true`), or null.

Timing. The leg is fed its last closed bar with `Time <= Data1 time` (`htfMapped`). It becomes unavailable when stale. Leg rolls are not inherited from Data1.

Journal. `XMKT {role} Data{n} available at {t} (staleness limit …)`; Info `XMKT leg … symbol() resolution() dataFeed() maxStale()`.

Tests. `tests/XmktTests`.

Snippet. NQ on Data2 as a leg; the strategy reacts when NQ is unusually weak against ES. `RsZ` is NaN until `rsLength` bars are in the window.

```
// Create
protected override void Create()
{
    base.Create();
    XmktLegs.Add(new XmktLegConfig { role = "NQ", dataStream = 2 });   // NQ on Data2; orders stay on Data1
}

// CalcBar
protected override void CalcBar()
{
    base.CalcBar();
    if (Bars.Status != EBarState.Close) return;
    _XmktLeg nq = XmktLeg("NQ");
    if (nq == null || !nq.Available) return;     // no closed leg bar yet, or the leg is stale
    if (nq.RsZ < -2)
    {
        /* ln(NQ / Data1) is 2 sigma below its rolling mean: NQ unusually weak vs Data1 */
    }
}
```

4.11 M-TICK `_TickStream`

Why it exists. A minute bar hides how it traded: 2,000 contracts in one burst of prints reads differently from the same volume spread evenly. M-TICK loads the 1-tick Trade data behind each Data1 bar (through a DataLoader request, so the chart itself stays on minute bars) and summarises it per bar: print count, peak prints per second, gaps between prints, and a burst score.

When to use it. Filters on print intensity ("only take the breakout if the tape is bursting"): example 37.

Enable. `TickConfig.enabled = true` in `Create()`. Nothing is requested while it is off.

Defaults. `historyTicks 2,000,000`, `realTime true`, `burstZ 3`, `burstNormLength 200`, `burstMinSamples 30`, `sizeBuckets {1,2,5,10,20,50}`, `largePrintSize 20`, `sizeOutputsVerified false`.

API. `TickStats` returns the last closed bar's `TickBarStats`, or null.

Field	Plain meaning
`valid`, `quality`	The tick data for this bar arrived complete and on time (`quality` "Tick"); otherwise do not use the other fields.
`prints`, `maxPrintsPerSecond`	Number of trades in the bar; the busiest second.
`interPrintMedianMs`, `interPrintP10Ms`	Typical and short (10th percentile) gaps between trades.
`burstZ`, `burst`	z-score of this bar's busiest-second print count vs the previous `burstNormLength` bars (NaN until `burstMinSamples` bars); `burst` = z ≥ `TickConfig.burstZ`.
`sizeHistogram`, `largePrints`, `volume`	Trade-size counts and large prints; null / −1 / NaN until `sizeOutputsVerified` is set.

Timing. Aggregated over `(Time[1], Time[0]]` at bar close. Late ticks are dropped.

Journal. `TICK DataLoader 1-tick Trade request for {sym}, N ticks back, RT=…`; `TICK n late tick(s) dropped …`; `TICK data missing/late … outputs invalid`; `TICK data valid again …`.

VERIFY. The `Bar` member names are read by reflection (`Time`, `Close`, `TotalVolume`/`Volume`/`Ticks`); the code flags this as "verify in the editor".

Tests. `tests/TickTests`.

Snippet. Turns the tick request on and reads the burst flag of the bar that just closed. `ts == null` or `!ts.valid` means the tick data did not cover this bar; the strategy treats that as "no information", not as "no burst".

```
// Create
protected override void Create()
{
    base.Create();
    TickConfig.enabled = true;                  // starts a DataLoader 1-tick Trade request
    TickConfig.burstZ = 3.0;                    // burst = prints/s z-score >= 3
}

// CalcBar
protected override void CalcBar()
{
    base.CalcBar();
    if (Bars.Status != EBarState.Close) return;
    TickBarStats ts = TickStats;                // stats of the bar that just closed, or null
    if (ts == null || !ts.valid) return;        // tick data missing or late: do not trade on it
    bool burst = ts.burst;                      // a print-rate burst inside this bar
}
```

4.12 M-MTF (`_MtfMath`, host helpers)

Why it exists. Reading a higher-timeframe stream (Data2 = 60 m) from a 5 m chart is the classic source of lookahead: the 60 m bar is still forming, so its high is not final. M-MTF gives indices and values of the last **closed** higher-timeframe bar only, and checks at start that the stream is the same instrument.

When to use it. Higher-timeframe levels and trend filters on the same instrument: examples 2, 5, 6, 7, 10, 11, 13 and 14. For a different market use M-XMKT (§4.10).

Enable. `HtfStreams = new[] { 2 }` before `base.StartCalc()`. StartCalc throws if the stream is missing or carries another root symbol.

API.

Member	Plain meaning
`HtfClosedIndex(ds, ago = 0)`	Bars-ago index of the ago-th closed bar of stream `ds` (0 if bar 0 has closed, else 1).

Member	Plain meaning
`HtfClosed(ds, ago = 0)`	Close of that bar.
`HtfClosedBar(ds, ago, out time, out o, out h, out l, out c, out v)`	All fields of that bar.
`htfMapped(ds, maxBack = 2)`	Index of the last closed bar of `ds` whose time is not after Data1's; −1 = none.
`HtfForming(ds)`	Deliberate access to the still-forming bar (values change intrabar; journaled once).
Statics `_MtfMath.ClosedIndex`, `MapIndex`, `SymbolRoot`, `IsStale`	The same rules, MC-free, for tests.

Journal. `MTF Data{n} first mapping {t} -> {htf t} (ago i, status s)` (once per session); `MTF forming-bar access on Data{n} (not closedBarsOnly)`.

VERIFY. Whether MC stamps HTF bars with their end time ("Probe P2" in the code).

Tests. `tests/MtfTests`.

Snippet (as in `_MtfProbe`). Declares Data2 as a higher timeframe and reads the high of its last completed bar. `i < 0` means no Data2 bar has closed yet.

```
// StartCalc (HtfStreams must be set before base.StartCalc, which validates the streams)
protected override void StartCalc()
{
    HtfStreams = new[] { 2 };                    // Data2 = the same instrument on a higher timeframe
    base.StartCalc();
}

// CalcBar
protected override void CalcBar()
{
    base.CalcBar();
    if (Bars.Status != EBarState.Close) return;
    int i = htfMapped(2);                        // index of the last CLOSED Data2 bar, -1 = none
    if (i < 0) return;
    double h60 = BarsOfData(2).High[i];          // high of the last completed 60m bar
}
```

4.13 Rolls (`_RollDetector`, `_RollWindow`)

Why it exists. A continuous, unadjusted futures chart (@ES) jumps when it switches from the expiring contract to the next one. That jump is not a market move, but ATRs, gaps, VWAPs and profiles would treat it as one. Roll detection finds the splice bar so that every engine can reset or adjust, and so that your strategy can skip it.

When to use it. On every continuous futures chart (almost all §3 examples set it); without it a Warning is traced.

Enable. `RollDetection = ERollDetection.Series | Table | SeriesOrTable` (default `Off`) in `Create()`.

Settings. `RollGapTicks` (8); `RollDates` (a HashSet of trading days, for `Table`).

Series or Table. `Series` reads the splice from the prices and needs no date list, but it is not currently reliable on intraday charts (it over-detects rolls; see App F, Known issues: fix pending in PR #51, so use `Table` with `EventRules.RollDate` meanwhile). On an intraday chart it compares trading-day opens, so it only works when `tradingDayKey` is a Globex (or other evening-rolling) key. `Table` uses fixed dates and is reproducible: `_Test6` (PR #46) fills `RollDates` with `EventRules.RollDate(y, 3/6/9/12)` for 2000–2035, keys days with `_CmeSessionClock.GlobexTradingDayEt` so a splice at the 18:00 ET open falls on the roll day's first bar, and journals `roll(Table n dates from-to digest)` with `_RollDetector.DatesDigest(RollDates)` (FNV-1a 64 of the sorted distinct dates) in the REGGATE header. `EventRules.RollDate` uses the Monday-

before-expiry rule from `RollRuleMondayFrom` = 2022-04-01 (ESM22 onward), the Thursday eight days before the third Friday earlier. See §4.3 for a `Table` snippet.

Series rule. A trading-day open that gaps ≥ `RollGapTicks` from the prior close on days 1–21 of Mar/Jun/Sep/Dec, or a change of `Bars.Info.Name`.

API.

Member	Plain meaning
`rollOn(ds)` (virtual)	This bar of stream `ds` is a roll splice.
`RollBars`	1-based Data1 bar numbers of the splices seen so far.
`IsRollWindow(tradingDay)`	The day is a roll session or the session just before / after one. False unless rings, FVGs or RollDetection created the roll window.

Journal. Info `VPF-4 roll splice detected tradingDay() bar() name()`; Warning `RollDetection is Off on a futures symbol …`; once per run, a Warning from `_RollDetector.SeriesKeyWarning(series, resolutionType, calendarKeyed)` when `Series` is used on an intraday chart whose `tradingDayKey` is the calendar date (the 18:00 ET splice is never tested); with M-DAILY on, also `DAILY roll offset on a calendar-keyed intraday futures chart …`.

Tests. `tests/RollTests`. Event-calendar roll dates: `_EventCalendar.RuleRollDate` / `RollDateFor`.

Snippet. Turns on Series detection and skips the splice bar. `nearRoll` is true on the roll session and the sessions on either side, a common "do not trust levels today" flag.

```
// Create
protected override void Create()
{
    base.Create();
    RollDetection = ERollDetection.Series;       // detect splices from the price series
    RollGapTicks = 8;                            // a quarterly-window open gap of >= 8 ticks counts
}

// CalcBar
protected override void CalcBar()
{
    base.CalcBar();                              // engines that read Data1 have already reset on a roll bar
    if (Bars.Status != EBarState.Close) return;
    if (rollOn(1)) return;                       // splice bar: its gap is the contract change, not a market
move
    bool nearRoll = IsRollWindow(tradingDayKey(Bars.Time[0]));   // roll session +-1
}
```

4.14 Bar rings, fair-value gaps and `LevelRef` (`_BarRing`, `_FvgDetector`, `LevelRef`)

These three helpers work together: **rings** keep recent closed bars so helpers can look back over time windows, the **FVG detector** finds one kind of level from those bars, and **`LevelRef`** is the common format in which every level-producing helper hands a level to your strategy.

Bar rings — closed-bar history per stream

Why it exists. Profile and level helpers need to re-read past windows ("the bars of yesterday's RTH session", "the last 30 minutes before 10:00"). `Bars.High[n]` only reaches back as far as MaxBarsBack and is indexed by bars-ago, not by time. A ring keeps a copy of every closed bar of a stream (time, trading day, OHLCV, up/down volume, RTH flag, roll flag) in a fixed-size buffer, searchable by time.

When to use it. Mostly indirectly: DVP (§5.31), bar ladders, range profiles and profile stats request their own rings. Request one yourself when your code needs time-window queries over past bars, or call `SessionBars` on the VP base, as examples 35 and 36 do.

Enable. `requireRing(ds, bars, consumer)` in `StartCalc()` **after** `base.StartCalc()` (which clears earlier requests). Capacity is the **sum** of all requests on that stream (`m_ringNeed[ds] = cur + Math.Max(1, bars)`, `_CustomSignal` L1410); `BarRingConfig.capacityBars 0` = automatic, capped at `MaxAutoBars` 40000 (about 29 Globex sessions of 1-minute bars).

API.

Member	Plain meaning
`Ring(ds)` → `_BarRing`	The ring of stream `ds`, or null when none was requested.
`Capacity`, `Count`, `Added`, `Dropped`, `LastBarNumber`	Size, bars held, bars ever added, bars pushed out, last bar number stored.
`this[ago]`	Bar by bars-ago (0 = newest).
`At(idx)`	Bar by chronological index (0 = oldest held).
`IndexAtOrBefore(t)`, `FirstIndexAtOrAfter(t)`	Search by time.
`Covers(t0)`	Does the ring still reach back to `t0`?
`Window(t0, t1, t0Exclusive)`	The bars in a time window (allocates a list; call once per session, not per bar).
`Session(day, rthOnly)`	The bars of one trading day, optionally RTH only.
`RingBar` fields `bar`, `time`, `tradingDay`, `o h l c v up down`, `rth`, `rollBar`	One stored bar.
`ringBarOf(ds, ago)`	Shortcut for one ring bar.

Journal. `RING Data{n} capacity() consumers()`.

Fair-value gaps (FVG)

Why it exists. A fair-value gap is a three-bar imbalance: price moved so fast that bar 3's low stays above bar 1's high (bullish) or bar 3's high stays below bar 1's low (bearish). Traders expect price to return to that unfilled zone, and use it as a pullback entry or target. The detector finds such gaps without lookahead, tracks touches and fills, and expires them.

When to use it. Pullback entries into a gap, or gaps as targets: example 33.

Enable. `Fvgs.enabled = true` in `Create()`. When disabled, `FvgDetector` is null (one Warning on first query).

Defaults. `dataStream 1`, `minTicks 1`, `minAtrFrac 0`, `atrLen 14`, `crossSessions false` (all three bars on one trading day, so an opening gap is never an FVG), `rthOnly false`, `maxLive 20`, `maxAgeSessions 5`, `fillOn EFvgFillOn.Close`.

Rules. Bullish zone = [bar-1 high, bar-3 low], start edge = bar-1 high; bearish is the mirror. Size must be ≥ max(`minTicks` × tick, `minAtrFrac` × ATR), where ATR is the detector's own Wilder ATR(`atrLen`). A gap is created at bar 3's close and usable from the next bar. A roll invalidates every live gap.

API.

Member	Plain meaning
`FvgDetector.Live`	Gaps not yet filled, expired or invalidated.
`NewThisBar`	A gap completed on this bar, or null.
`Atr`	The detector's ATR.
`Nearest(price, dir)`	

Member	Plain meaning
	Closest live gap of a direction (distance 0 when price is inside it).
`ToLevel(f, startEdge, stream)`	The gap as a `LevelRef`.
`Fvg` fields `id`, `dir` (+1 bullish, −1 bearish), `lo`, `hi`, `start`, `createdTime`, `createdDay`, `touched`, `filled`, `expired`, `invalid`, `fillPct`, `ageSessions`, `Size`, `IsLive`, `Tag` (`FVG:U|D:yyyyMMddHHmm`)	One gap: its zone, timing and state.
Host `fvgLevel(f, startEdge)`	`ToLevel` plus the `isRollWindow` flag.

`LevelRef` — one format for every level

Why it exists. Levels come from many helpers (range profiles, weekly profiles, DVP, profile stats, FVGs, Gate 0). `LevelRef` gives them one shape, so the same strategy or study code can test "price reached a level" regardless of the source, and always knows **from when** the level may be used.

When to use it. Whenever you consume levels from more than one helper, or feed them to the level study (§4.17): examples 33 and 42. Producers: `RangeLevels`, `WeekLevels` (VP base), `fvgLevel`, DVP, ProfileStats, `_TestGate0`.

Field / method	Plain meaning
`kind` (`ELevelKind`), `price`, `lo`, `hi`	What the level is; a single price, or a zone from `lo` to `hi`.
`createdTime`	Usable from this time (no lookahead).
`sourceT0`, `sourceT1`, `dataStream`	The time window and stream that produced it.
`tag`	Source id, e.g. `RP:…`, `WK:2026W42`, `FVG:U:…`.
`quality`	Helper-specific quality score.
`crossesRoll`, `isRollWindow`	The source window spans a roll; the level was born around a roll.
`Line(...)`, `Zone(...)`	Build a line or zone level.
`StudyType`, `StudyKey`	Type and key passed to `_LevelStudy.AddLevel`.

Tests. `tests/VpHelperTests`.

Snippet. Enables FVGs (ignoring gaps smaller than 0.25 × ATR), requests a 400-bar ring on Data1, and on each bar reads the previous bar from the ring and turns a new bullish gap into a level. `lv.price` is the gap's start edge (bar-1 high) and is usable from the next bar; levels born around a roll are skipped.

```
// Create
protected override void Create()
{
    base.Create();
    RollDetection = ERollDetection.Series;
    Fvgs.enabled = true;                        // fair-value-gap detector on Data1
    Fvgs.minAtrFrac = 0.25;                     // ignore gaps smaller than 0.25 x ATR(14)
}

// StartCalc (rings are requested after base.StartCalc, which clears earlier requests)
protected override void StartCalc()
{
    base.StartCalc();
    requireRing(1, 400, "my-study");            // keep the last >= 400 closed Data1 bars
}

// CalcBar
protected override void CalcBar()
{
    base.CalcBar();                             // rings and the detector are filled here
    if (Bars.Status != EBarState.Close) return;
    _BarRing ring = Ring(1);
    RingBar prev = ring != null && ring.Count > 1 ? ring[1] : default(RingBar);   // the bar before this one
```

```
    _FvgDetector det = FvgDetector;
    Fvg gap = det != null ? det.NewThisBar : null;   // a gap completed on this bar, or null
    if (gap == null || gap.dir <= 0) return;
    LevelRef lv = fvgLevel(gap, true);           // start edge (bar-1 high) as a level, usable from the next
bar
    if (lv.isRollWindow) return;                 // skip levels born around a roll
}
```

4.15 Volume-profile bases (`_VolumeProfileBase`, `_VolumeProfileFuturesBase`)

Why it exists. Volume-profile trading reads where volume traded, not just where price went: the point of control (POC, the busiest price), the value area (VA, about 70 % of volume), high- and low-volume nodes (HVN/LVN), and POCs that price has not revisited ("naked"). The VP bases build these profiles from MC data, add up to three VWAP slots, combine levels into confluence zones and targets, and provide the Phase 3 helpers (range, weekly and bar-ladder profiles, profile stats). The futures base adds the CME clock, IB/RTH stats, VPF-5 order flow and the CME holiday hooks.

When to use it. Any strategy that needs profile levels or order flow. 31 of the 42 §3 examples run on `_VolumeProfileFuturesBase`: 28 derive from it directly (3, 6–8, 10, 11, 14, 15, 17, 19, 20, 22, 25–34, 36–39, 41, 42) and examples 21, 35 and 40 inherit it through another example. The other 11 run on `_CustomSignal`: 10 (1, 2, 4, 5, 9, 12, 13, 16, 18, 23) derive from it directly and example 24 derives from example 18 (`_Ex18FirstHourBreak`); move one onto the VP base before calling anything below.

Enable. Derive from `_VolumeProfileFuturesBase`. Unlike `_CustomSignal`, these bases expose `[Input]`s:

- `HvnPercentile .8`, `LvnPercentile .2`, `MinNodeTicks 3`, `TrackNakedPoc true`, `NakedPocTickTolerance 1`;
- `Vwap1..3 Type/Length/FromDate/DataStream` (Vwap1 = Session on DS1; 2 and 3 Disabled);
- `DevelopingSource`, `DevProfileSourceMode`, `DevProfileDataStream`, `ValueAreaMethod`, `LookaheadAudit false`.

Main API.

Group	Members	Plain meaning
Profiles	`ExtraVolumeProfileForBar([ds,] barIndex)` (1-based), `RecentCompletedProfiles(ds, count, asOfBar)`, `NakedPocs(ds[, asOf])`, `IsNewVolumeProfile(...)`	The profile a bar belongs to; the last finished profiles; POCs not yet revisited; a new profile started on this bar.
Confluence	`GetConfluenceZones`, `GetTopConfluenceZones([n])`, `GetConfluenceTargets(price, EVpDirection[, n])`, `GetConfluenceTargetZones`	Price zones where several levels (POC, VA edges, VWAPs, naked POCs) cluster; the strongest ones; the next zones in a trade's direction, for targets.
VWAP slots	`CurrentVwap1..3`, `CurrentVwap1Sigma`, `Vwap1Active`	The configured VWAPs, slot 1's standard deviation, whether slot 1 has a value.
Auction and value	`ValueRelationship`, `IsPoorHigh/IsPoorLow`, `HasUnfinishedAuction`, `SinglePrints`, `ProfileShape`, `ValueMigrationScore/ValueMigrationState`, `Composite(stream, n)`, `IsBalanced`, `BalanceRange`	Where today's value sits vs yesterday's; extremes without a clean auction (likely revisited); thin single-print areas; profile shape; whether value is moving; an n-session composite; whether the market is balancing and its range.
Futures profiles	`RthProfile()`, `GlobexProfile()`, `PriorRthProfile()`	Today's RTH and overnight profiles; yesterday's RTH profile.
Range profiles	`RangeProfile(stream, t0, t1)`, `RangeProfileAfter`, `RangeProfileBars`, `OpenRange/CloseRange/RangeHandleProfile`, `RangeStatsOf`, `RangeLevels`	A profile over any time window (e.g. the opening hour), its stats, and its levels as `LevelRef`s.
Weekly	`Week(weeksAgo)`, `WeeklyComposite(n)`, `CompletedWeeks`, `WeekKey`, `WeekLevels`	Weekly profiles and their levels.
Ladders	`Ladder(ago[, minQuality])`, `LadderAt(time)`, `HtfLadder(ds, ago)`, `LadderProfile(b)`	A per-bar price ladder (volume at each price inside one bar).

Group	Members	Plain meaning
Other	`ProfileEntropy(e, mmN)`, `SessionBars(day, rthOnly)`	How spread-out a profile is; the ring bars of one session (§4.14).

Order flow (futures base). Why: bid/ask volume shows who was aggressive inside a bar. Example 38 sets `OrderFlowEnabled = true` itself; example 34 only enables `BarLadders`, which switches `OrderFlowEnabled` on (`_VolumeProfileFuturesBase` L133, VP-3). Enable with `OrderFlowEnabled = true`; configure `FlowCfg` (`_OrderFlow.FlowConfig`) and `FlowTickSource`.

Member	Plain meaning
`Flow`	The order-flow engine.
`Footprint(barsAgo)`	Volume at each price split into bid and ask for one bar.
`BarDelta(barsAgo)`, `CumDelta(barsAgo)`	Ask minus bid volume for the bar; running total since the reset key (session or RTH).
`DiagonalImbalances(barsAgo, ratio = 3.0, minVol = 10)`, `StackedImbalances(barsAgo, ratio = 3.0, minVol = 10, minStack = 3)`	Prices where one side dominates the diagonal neighbour; several such prices in a row.
`Absorption(barsAgo, side, edgeTicks = 2, volMult = 2.0, holdTicks = 1)`	Heavy aggression at the bar's edge that did not move price.

Phase 3 configs. `RangeProfiles`, `WeeklyProfiles`, `ProfileStats`, `BarLadders` (`enabled false` by default; defaults in the `configs` of `_RangeProfile`/`_WeeklyProfiles`/`_ProfileStats`/`_BarLadder`).

Journal. `VPB-1 own profile: source() mode() approximate() valueArea() audit()`; `RP build {tag} Data{n} [t0..t1] bars(a/b) poc() va() approx() roll()`; `RP coverage …`; `LADDER HTF Data{n} {t} coverage() -> totals-only`.

Tests. `tests/ProfileTests`, `tests/VpHelperTests`, `tests/OrderFlowTests`.

4.16 M-PORT publisher and `_PortfolioRiskManager` (`_PortMM`)

Why it exists. In Portfolio Trader several strategies trade at once, and their risks add up: three long equity-index trades are one big bet. M-PORT has each strategy publish its state (`pm.*` keys in `MyPortfolioData`); a separate money-management signal reads all of them and sizes and limits risk across the book (risk parity, heat and cluster caps, drawdown steps, daily loss stops).

When to use it. Only in Portfolio Trader with more than one strategy: examples 37 and 40. On a single chart the publisher is inactive.

Enable.

- The publisher is automatic in PT (`PortfolioPublish`). Override as needed: `PortfolioFamily`, `PortfolioScore`, `PortfolioHalted`, `PortfolioCluster` (default from the symbol root: ES→EQIDX…), `PortfolioRegime`.
- Apply `_PortfolioRiskManager` as the portfolio's Money Management signal. It has no `[Input]`s; subclass it and override `Config` or `ConfigPath` (JSON).

Keys (`_PortKeys`).

Group	Keys	Plain meaning
Identity	`pm.ver`, `pm.id`, `pm.family`, `pm.cluster`	Who is publishing, and which correlated group it belongs to.
Equity and returns	`pm.eq`, `pm.pnl`, `pm.ret`, `pm.uRet`, `pm.vol`	Equity, P&L, realised and unrealised return, volatility.

Group	Keys	Plain meaning
Risk	`pm.R$`, `pm.qtyCode`, `pm.riskNew`, `pm.riskOpen`, `pm.openR`	Dollar risk per R, requested size, new and open risk.
State	`pm.state`, `pm.side`, `pm.regime`, `pm.regimeP`, `pm.score`, `pm.stamp`, `pm.day`, `pm.pos`	`EPortState {Flat, Armed, Long, Short, Exiting, Halted}`, direction, regime, score, freshness, position.
Events	`pm.evt*`	Event flags.
Written by the MM	`pm.mult`, `pm.heat`, `pm.dd`, `pm.rho`	Size multiplier set by the MM; book heat; drawdown; correlation.

`PortMmConfig` defaults.

- Sizing: `sizing RiskParity`, `bookVolTarget 0.10`, `riskPerTrade 0.005` (library default; the house rule is 1% risk per trade).
- Risk caps: `heatCap 0.04`, `clusterCap 0.02`, `rhoMerge 0.9`, `rhoWin 60`, `rhoMinDays 40`, `maxPositions 6`.
- Drawdown: `ddSteps {5%→0.5, 10%→0.25, 15%→0}`, `ddRecoverHighDays 20`.
- Loss stops and kills: `dailyLossStop 0.02`, `dailyLossFlatten 0.03`, `stratKillR -8`, `stratKillTrades 20`.
- Other: `staleBars 3`, `minContracts 1`, `idmCap 2.5`.

API in your strategy. `portfolioArm(int side, double entry, double stop, int qtyCode)` announces a planned entry (called for you by `sizeForRisk` for Data1); `portfolioStop(double)` only sets the stop used for `pm.riskOpen` while the position is open; it publishes nothing by itself (`pm.R$` and `pm.riskNew` keep the armed values).

Journal. Info `PORT publish on: id() family() cluster()`; Journal `PORT book multiplier x at t`. MM lines go to Output (`PORT Warning/Info`).

Tests. `tests/PortMMTests`. Live checks use `_TestPortArm` + `_TestPortMM`. Worked snippet: §5.23.

4.17 A2-33 level study (`_LevelStudy`) and Gate 0 (`_Gate0`)

Why it exists. Before trading a level type ("prior POC", "naked VWAP"), you want evidence that price reacts there more than at a random price. The level study compares each registered level with matched control prices (random prices at a similar distance) and measures reaction and survival ("death" = price trades through by `deathR`). Gate 0 is the research harness that runs this for whole level families against seeded placebos.

When to use it. Research, not live trading: example 41, and the Gate 0 runs in §2.

Level study.

- Enable with `LevelStudyConfig.enabled = true`.
- Defaults: `controls 20`, `distLo .9`, `distHi 1.1`, `reactionBars 60`, `sigmaBars 60`, `touchTicks 0`, `maxAge 15`, `deathR 1`, `seed 33`, `from/to`, `csvPath ""`, `controlsEitherSide false`, `reactionAfterTouch false`.
- Register prior-session levels on the session's first bar with `levelStudyAdd(type, price, ageSessions, quality = NaN, key = null)`.
- Results: Info `A2-33 RESULT events() types() window()` plus a summary per type at RUN-END, and an optional CSV.
- No strategy on `main` enables `LevelStudyConfig`: `_Gate0` drives `_LevelStudy` directly. The catalogue and the `LevelStudyConfig` comment in `_CustomSignal` name `_Test1`, but `_Test1` is a plain `SignalObject` (discrepancy, Appendix F).

Gate 0 (`Gate0Engine`, `EGate0Mode/Family/Canary`, `Gate0Bar/Intent/Profile/Vwap/Atr/Session`): MC-free, hosted by `_TestGate0`; use in **§2**, snippet in §5.29.

Tests. `tests/LevelStudyTests`, `tests/Gate0Tests`. Snippet: §5.28.

4.18 Utility engines

Why they exist. Shared maths so that strategies do not reimplement statistics. All are MC-free and deterministic.

When to use them. Profile-shape and bimodality filters (worked snippet §5.25): example 35 uses `_ProfileStats.KdeSweep`, `BootstrapPoc` and `BootstrapSeed`; example 36 uses `_ProfileStats.Histogram`, `ProfileMixture` and `MixtureSeed` (the mixture fit runs on `_Gmm` internally; neither example calls `_Gmm` directly); `_Rng` wherever a reproducible random choice is needed (used by Gate 0, the regime engine, AVWAP and profile stats).

Engine	Members	Plain meaning
`_Gmm`	`Fit`, `Fit1D`, `FitK`, `FitOnce`, `LogDensity`, `Density1D`, `Order`, `ParamCount`; `GmmModel`, `GmmFitConfig`	Gaussian-mixture fits: does a session's volume form one hump or two (bimodal = two value areas)?
`_Rng`	`Mix`, `Seed(params long[])`, `Hash(string)`	Deterministic random numbers: the same inputs give the same draws on every run.
`_ProfileStats`	`Entropy`, `MmN`, `KL`, `KLKde*`, `KdeOnGrid`	How spread-out a profile is; how different two profiles are; smoothed profile.
	`KdeSweep`, `BootstrapPoc`, `BootstrapSeed`	KDE bandwidth sweep with a Stable/Flat/Bimodal verdict; bootstrap POC stability (example 35).
	`Histogram`, `ProfileMixture`, `MixtureSeed`	Profile histogram from ring bars; Gaussian-mixture fit of the profile (example 36).
	`ParkinsonPerBar`, `ParkinsonSessionPrice`, `EwmaForecast`, `WilderAtr`	Range-based volatility estimates and forecasts; ATR.
	`AggregateBars`; `GmmSession.FromBars`	Combine bars; build a GMM input from session bars.
`_SignalJournal`	`PROBE_ID`, `Describe()`	A CS-0 probe description only; no runtime role.

5 — Feature reference (catalogue §2.4 base-feature map)

One subsection per row of the catalogue's §2.4 table (31 rows), in the catalogue's order. **Status** is what `origin/main` @ `a07257a` contains, checked against the code (where it differs from the catalogue's Status column, that is said). **Signatures** are the key members, pulled by script from the tree (the full list is in Appendix E). **Snippets** are minimal strategy fragments; each was compiled as a small class (`_SnNN`) with the same checker as the examples (0 errors). The comment on each method says where the code lives: `Create()` for configuration, `StartCalc()` for per-run state, `CalcBar()` for per-bar logic. Snippets omit order objects; see the examples for complete strategies.

§	Row	Status on main
5.1	F-DEV — Own developing profile, VA methods, lookahead audit (VPB-1/2/3/4)	✓ Done on main
5.2	F-PROF — Completed-profile analytics (VPB-6)	✓ Done on main.
5.3	F-COMP — Composites of N sessions, IsBalanced / BalanceRange (VPB-7)	✓ Done on main.
5.4	F-NPOC — Naked-POC tracking	✓ Done on main.
5.5	F-HVN — KDE HVN/LVN nodes on a 1-tick grid	✓ Done on main.
5.6	F-SESS — Session stats (VPF-2) and RTH/ETH profile streams (VPF-3)	✓ Done on main.
5.7	F-ROLL — Roll-splice flag (VPF-4)	✓ Done on main.
5.8	F-CAL — CME holidays / half-days (VPF-6), CmeSessionClock (VPF-7)	✓ Done on main.
5.9	F-OF — Order-flow helpers (VPF-5) and the FlowQuality chain	✓ Done on main
5.10	F-VWAP — VWAP engine, 3 slots with σ	✓ Done on main
5.11	F-CONF — Confluence engine	✓ Done on main
5.12	F-TM — Trade-management primitives (CS-7)	✓ Done on main.
5.13	F-JRNL — Journal: R, MFE/MAE, run id (CS-1/CS-3); sizeForRisk (CS-4); cost filter	✓ Done on main.
5.14	F-EVT — logSignal forward MFE/MAE + ControlMode (CS-8)	✓ Done on main
5.15	M-AVWAP — AVWAP anchor manager	✓ Available on main
5.16	M-SWING — PRICEACTION swing / structure engine	✓ Available on main
5.17	M-MTF — Multi-TF via native BarsOfData(n)	✓ Available on main
5.18	M-TOD — Time-of-day normalisers	✓ Available on main
5.19	M-DAILY — Daily context: ATR20d, SMA200, RSI(2), TSMOM, HV	✓ Available on main
5.20	M-EVENT — Scheduled-event calendar (M-NEWS / M-EVENT)	◐ Scheduled-event calendar available on main
5.21	M-TICK — Seconds-level burst / trade-size stream	◐ Available on main for counts/timing
5.22	M-XMKT — Second-instrument stream	✓ Available on main
5.23	M-PORT — Portfolio-level sizing/throttles	◐ Coded on main
5.24	M-COST — Style-dependent slippage, p90 stop slippage, haircut	◐ Not a module: MC native Commission/Slippage + the optional cost filter
5.25	M-GMM — Bootstrap POC, KDE sweep, GMM/BIC, entropy, KL	✓ Available on main

§	Row	Status on main
5.26	M-GEX — External daily GEX file	✗ Deferred: no code on main
5.27	VP helpers — Range profile, weekly profiles, per-bar ladders, entropy/Parkinson/V(p)/T(p), FVG detector (VP-1…VP-6)	✓ Available on main
5.28	A2-33 — Matched-control level study; Kaplan–Meier / Cox level death	◐ Available except Cox
5.29	Gate0 — Gate-0 measurement: level families vs seeded placebos; G0-b canaries	✓ Available on main
5.30	_RegimeEngine — Market regime (G-R): HMM K = 3, walk-forward, hysteresis; AtrTercile fallback	◐ Partial, code complete
5.31	M-DVP — Detrended volume-profile engine (Keller B5)	◐ Partially on main a07257a: spec rev 5.3 PR-D0a–D7 (fit core, rails, Data1 host, Fib lattice, D5 confluence/acceptance, D6 HTF streams + P2 gate, D7 A2-33 hook); D8–D9 pending

The catalogue's G-R row covers `_RegimeEngine`; `_RegimeGate` has no row of its own and is documented in §5.30 and §4.5.

5.1 F-DEV — Own developing profile, VA methods, lookahead audit (VPB-1/2/3/4)

Status on main a07257a. ✓ Done on main (MC verification of the truncation audit still owed).

Why it exists. MC's built-in developing profile cannot be restarted at the RTH open or audited for lookahead; the own profile gives a per-session POC/VA you can trust intraday.

Where. `_VolumeProfileBase.Strategy.CS` · `_VolumeProfileBase` (`DevelopingSource`, `DevelopingExtra`); `_VolumeProfileFuturesBase.Strategy.CS` (`OwnProfileWindow`, `RthProfile`)

Key signatures.

```
public enum EDevelopingSource { MultiCharts = 0, Own = 1 }   // _VolumeProfileBase · _VolumeProfileBase:744
[Input] public EDevelopingSource DevelopingSource { get; set; }   // _VolumeProfileBase · _VolumeProfileBase:756
public VpExtra DevelopingExtra(int stream)   // _VolumeProfileBase · _VolumeProfileBase:936
public enum EOwnProfileWindow { MultiChartsProfile = 0, GlobexDay = 1, Rth = 2 }   // _VolumeProfileFuturesBase · _VolumeProfileFuturesBase:467
protected EOwnProfileWindow OwnProfileWindow { get; set; }   // _VolumeProfileFuturesBase · _VolumeProfileFuturesBase:473
public VpExtra RthProfile()   // _VolumeProfileFuturesBase · _VolumeProfileFuturesBase:524
```

Snippet (compiled as `_Sn01`, base `_VolumeProfileFuturesBase`).

Switches the VP base to its own RTH-anchored profile and journals the developing POC and value area each bar. `dev` is null before the first RTH bar.

```
// Create
protected override void Create()
{
    base.Create();
    DevelopingSource = EDevelopingSource.Own;      // VP-base [Input]: set in code, never exposed
    OwnProfileWindow = EOwnProfileWindow.Rth;      // the developing profile restarts at 09:30 ET
}

// CalcBar
protected override void CalcBar()
{
    base.CalcBar();                                // the profile is updated inside base.CalcBar()
    if (Bars.Status != EBarState.Close) return;
    VpExtra dev = RthProfile();                    // developing RTH profile (null before the first RTH bar)
    if (dev != null && journalOn())
```

```
        trace(ETraceLevel.Journal, $"dev POC {dev.poc} VA {dev.val}-{dev.vah}");
}
```

Used in examples. example 21 (§3.17), example 22 (§3.25), example 26 (§3.26), example 27 (§3.27), example 29 (§3.28), example 30 (§3.29), example 31 (§3.31), example 28 (§3.35), example 32 (§3.36), example 34 (§3.38); inherited by example 35.

Pitfalls.

- `DevelopingSource` and `OwnProfileWindow` are VP-base properties; `DevelopingSource` is an `[Input]` on the base. Set both in `Create()`; do not add your own `[Input]`.
- `RthProfile()` and `PriorRthProfile()` return null unless `DevelopingSource = Own` and `OwnProfileWindow = Rth`.
- Read the profile only after `base.CalcBar()` and only on a closed bar; the developing profile includes the current bar.

5.2 F-PROF — Completed-profile analytics (VPB-6)

Status on main a07257a. ✓ Done on main.

Why it exists. Yesterday's profile tells you where the market accepted value; poor highs, single prints and value migration flag auctions likely to be revisited or extended.

Where. `_VolumeProfileBase.Strategy.CS` · `_VolumeProfileBase` (`ValueRelationship`, `IsPoorHigh/Low`, `SinglePrints`, `ProfileShape`, `IsNakedVah/Val`, `AcceptanceOutsideValue`, `ValueMigrationScore`, `RecentCompletedProfiles`); `_VolumeProfileFuturesBase.PriorRthProfile`

Key signatures.

```
public ProfileMath.ValueRel ValueRelationship(VpExtra prev, VpExtra cur)   // _VolumeProfileBase · _VolumeProfileBase:1081
public bool IsPoorHigh(VpExtra e, double fraction = 0.5, int minRows = 2)   // _VolumeProfileBase · _VolumeProfileBase:1088
public bool IsPoorLow(VpExtra e, double fraction = 0.5, int minRows = 2)   // _VolumeProfileBase · _VolumeProfileBase:1095
public List<double[]> SinglePrints(VpExtra e, int minTicks, double pct = 0.2)   // _VolumeProfileBase · _VolumeProfileBase:1115
public ProfileMath.Shape ProfileShape(VpExtra e)   // _VolumeProfileBase · _VolumeProfileBase:1127
public bool IsNakedVah(VpExtra e, int asOfBar)   // _VolumeProfileBase · _VolumeProfileBase:1157
public bool IsNakedVal(VpExtra e, int asOfBar)   // _VolumeProfileBase · _VolumeProfileBase:1158
public int AcceptanceOutsideValue(VpExtra reference, bool above, int minCloses)   // _VolumeProfileBase · _VolumeProfileBase:1164
public int? ValueMigrationScore(int stream, int n = 3, double minChangeAtrFraction = 0.10, int atrLen = 20, bool includeDeveloping = true, double dailyAtr = 0)   // _VolumeProfileBase · _VolumeProfileBase:1198
public int ValueMigrationScore(int stream, double atr20Daily, int n = 3, double minChangeFraction = 0.10, bool includeDeveloping = true)   // _VolumeProfileBase · _VolumeProfileBase:1309
public List<VpExtra> RecentCompletedProfiles(int datastream, int count, int asOfBar)   // _VolumeProfileBase · _VolumeProfileBase:1495
public VpExtra PriorRthProfile()   // _VolumeProfileFuturesBase · _VolumeProfileFuturesBase:548
public static List<int[]> SinglePrints(double[] vols, int vaLo, int vaHi, double pct, int minRows)   // _ProfileMath · _ProfileMath:144
public static Shape ProfileShape(double[] vols, double vaPercent)   // _ProfileMath · _ProfileMath:164
public static bool ValueMigrationScore(IList<double> completedPocs, int n, double developingPoc, double minChange, out int score)   // _ProfileMath · _ProfileMath:247
public static List<double[]> TpoSinglePrints(IDictionary<long, int> tpo, int minRows, double tick)   // Gate0Profile · _Gate0:140
```

Snippet (compiled as `_Sn02`, base `_VolumeProfileFuturesBase`).

Reads yesterday's completed RTH profile: is its high poor, how many closes are above its VAH, and how value has migrated over 3 sessions. `v` is null until enough sessions exist; ≥ +2 = value migrating up, ≤ −2 = down.

```
// Create
protected override void Create()
{
```

```
    base.Create();
    DevelopingSource = EDevelopingSource.Own;      // build our own profile ...
    OwnProfileWindow = EOwnProfileWindow.Rth;      // ... per RTH session, so PriorRthProfile() exists
}

// CalcBar
protected override void CalcBar()
{
    base.CalcBar();
    if (Bars.Status != EBarState.Close) return;
    VpExtra prior = PriorRthProfile();             // last COMPLETED RTH profile
    if (prior == null) return;
    bool poorHigh = IsPoorHigh(prior);             // fraction 0.5, minRows 2 (defaults)
    int closesAbove = AcceptanceOutsideValue(prior, true, 2);
    int? v = ValueMigrationScore(1, 3);            // null until 4 completed sessions (20 without a daily
ATR)
    if (journalOn())
        trace(ETraceLevel.Journal, $"poorHigh {poorHigh} closesAboveVAH {closesAbove} V {v}");
}
```

Used in examples. example 11 (§3.14), example 21 (§3.17), example 6 (§3.23), example 22 (§3.25), example 26 (§3.26), example 27 (§3.27), example 30 (§3.29), example 28 (§3.35), example 32 (§3.36), example 41 (§3.41); inherited by example 35.

Pitfalls.

- `ValueMigrationScore` has two overloads: `(stream, n, ...)` returns `int?` (null when history is short); `(stream, atr20Daily, n, ...)` returns `int`.
- Analytics take a completed `VpExtra`; passing the developing profile gives values that change during the session.

5.3 F-COMP — Composites of N sessions, IsBalanced / BalanceRange (VPB-7)

Status on main a07257a. ✓ Done on main.

Why it exists. One session is noisy; a composite of several sessions shows the value area the market has agreed on over days, and whether it is balancing.

Where. `_VolumeProfileBase.Strategy.CS` · `_VolumeProfileBase.Composite`, `IsBalanced`, `BalanceRange`

Key signatures.

```
public VpExtra Composite(int stream, int n)   // _VolumeProfileBase · _VolumeProfileBase:1249
public bool IsBalanced(int stream, int n, double minVaOverlap = 0.6)   // _VolumeProfileBase ·
_VolumeProfileBase:1275
public bool BalanceRange(int stream, int n, bool union, out double high, out double low)   //
_VolumeProfileBase · _VolumeProfileBase:1285
public static bool IsBalanced(IList<double> vahs, IList<double> vals, double minOverlap)   // _ProfileMath ·
_ProfileMath:226
```

Snippet (compiled as `_Sn03`, base `_VolumeProfileFuturesBase`).

Builds a 3-session composite and checks whether the sessions overlap (≥ 60 %), i.e. the market is balancing; `BalanceRange` returns the balance high and low.

```
// CalcBar
protected override void CalcBar()
{
    base.CalcBar();
    if (Bars.Status != EBarState.Close) return;
    VpExtra comp = Composite(1, 3);                // last 3 completed sessions on Data1
    if (comp == null) return;
    bool balanced = IsBalanced(1, 3, 0.6);         // value areas overlap >= 60 %
    double hi, lo;
    if (BalanceRange(1, 3, true, out hi, out lo) && journalOn())
```

```
        trace(ETraceLevel.Journal, $"comp VA {comp.val}-{comp.vah} balanced {balanced} range {lo}-{hi}");
}
```

Used in examples. example 21 (§3.17), example 26 (§3.26), example 27 (§3.27), example 29 (§3.28), example 31 (§3.31), example 28 (§3.35), example 34 (§3.38).

Pitfalls.

- A composite has `crossesRoll`; on unadjusted continuous data, a composite across a roll mixes two contracts. Check it or set `RollDetection`.
- Compute the composite once per session (first bar), not on every bar; it is rebuilt on each call.

5.4 F-NPOC — Naked-POC tracking

Status on main a07257a. ✓ Done on main.

Why it exists. A POC that price has not traded back to is an unfinished auction that price often returns to, a common target or reaction level.

Where. `_VolumeProfileBase.Strategy.CS` · `TrackNakedPoc`, `NakedPocTickTolerance` ([Input]s), `NakedPocs(ds[, asOfBar])`, `VpExtra.IsNakedPoc`

Key signatures.

```
public bool IsNakedPoc(int asOfBar)   // _VolumeProfileBase.VpExtra · _VolumeProfileBase:181
[Input] public bool TrackNakedPoc { get; set; }   // _VolumeProfileBase · _VolumeProfileBase:501
public List<VpExtra> NakedPocs(int datastream)   // _VolumeProfileBase · _VolumeProfileBase:1513
public List<VpExtra> NakedPocs(int datastream, int asOfBar)   // _VolumeProfileBase · _VolumeProfileBase:1524
```

Snippet (compiled as `_Sn04`, base `_VolumeProfileFuturesBase`).

Walks the naked POCs and logs when price is within 2 ticks of one, a revisit of an unfinished auction.

```
// Create
protected override void Create()
{
    base.Create();
    TrackNakedPoc = true;                          // VP-base [Input]: set in code
}

// CalcBar
protected override void CalcBar()
{
    base.CalcBar();
    if (Bars.Status != EBarState.Close) return;
    foreach (VpExtra p in NakedPocs(1))            // newest unfinished auction first
    {
        bool atPoc = Math.Abs(Bars.Close[0] - p.poc) <= 2 * tickSize();   // within 2 ticks
        if (atPoc && journalOn())
            trace(ETraceLevel.Journal, $"at naked POC {p.poc}");
    }
}
```

Used in examples. example 6 (§3.23), example 41 (§3.41).

Pitfalls.

- `NakedPocs` is empty unless `TrackNakedPoc = true`.
- Age the levels yourself (e.g. by index in `RecentCompletedProfiles`) if a rule caps age in sessions.

5.5 F-HVN — KDE HVN/LVN nodes on a 1-tick grid

Status on main a07257a. ✓ Done on main.

Why it exists. High-volume nodes act as magnets and low-volume nodes as fast-move zones; price inside an LVN tends to travel through it.

Where. `_VolumeProfileBase.Strategy.CS` · `VpExtra.hvns` / `VpExtra.lvns` (`List<Node>`: low, high, peak, peakVol, strength, isPoc); `MaxLvnFractionOfRange`

Key signatures.

```
public double MaxLvnFractionOfRange { get; set; }   // _VolumeProfileBase · _VolumeProfileBase:647
```

Snippet (compiled as `_Sn05`, base `_VolumeProfileFuturesBase`).

Takes the LVNs of a 20-session composite and logs when the close sits inside one; inside an LVN, a fast move through is more likely.

```
// CalcBar
protected override void CalcBar()
{
    base.CalcBar();
    if (Bars.Status != EBarState.Close) return;
    VpExtra comp = Composite(1, 20);
    if (comp == null || comp.lvns == null) return;
    foreach (Node n in comp.lvns)                  // low-volume nodes: [low, high], peak = node centre
    {
        bool inside = Bars.Close[0] >= n.low && Bars.Close[0] <= n.high;
        if (inside && journalOn())
            trace(ETraceLevel.Journal, $"inside LVN {n.low}-{n.high}");
    }
}
```

Used in examples. example 29 (§3.28), example 33 (§3.37), example 34 (§3.38).

Pitfalls.

- Nodes come from a smoothed (KDE) profile; their edges are not exact price prints.
- HVN/LVN lists can be empty on thin profiles; check for null and Count.

5.6 F-SESS — Session stats (VPF-2) and RTH/ETH profile streams (VPF-3)

Status on main a07257a. ✓ Done on main.

Why it exists. Initial balance, gap and prior close are the reference points most opening strategies key off.

Where. `_VolumeProfileFuturesBase.Strategy.CS` · `SessionStats Session` (globex/eth/rth OHLC, IB, prior RTH, Gap), `IsRth()`, `IbMinutes`

Key signatures.

```
protected bool IsRth()   // _VolumeProfileFuturesBase · _VolumeProfileFuturesBase:238
protected int IbMinutes { get; set; }   // _VolumeProfileFuturesBase · _VolumeProfileFuturesBase:390
public SessionStats Session { get; }   // _VolumeProfileFuturesBase · _VolumeProfileFuturesBase:399
```

Snippet (compiled as `_Sn06`, base `_VolumeProfileFuturesBase`).

During RTH, logs closes above the completed initial balance together with the gap and prior close. `ibComplete` is false until the initial balance has finished.

```
// CalcBar
protected override void CalcBar()
{
    base.CalcBar();
    if (Bars.Status != EBarState.Close || !IsRth()) return;
    SessionStats s = Session;
```

```
    if (s.ibComplete && Bars.Close[0] > s.ibHigh && journalOn())
        trace(ETraceLevel.Journal, $"above IB {s.ibLow}-{s.ibHigh}, gap {s.Gap}, prior close
{s.priorRthClose}");
}
```

Used in examples. example 20 (§3.9), example 19 (§3.16), example 25 (§3.20), example 37 (§3.21), example 22 (§3.25), example 26 (§3.26), example 27 (§3.27), example 29 (§3.28), example 30 (§3.29), example 38 (§3.30), example 31 (§3.31), example 28 (§3.35), example 32 (§3.36), example 36 (§3.39), example 39 (§3.40); inherited by example 40, example 35.

Pitfalls.

- `Session` values are for the current Globex day; `priorRth*` is the prior completed RTH session.
- `ibComplete` is false until `IbMinutes` (default 60) after 09:30 ET.

5.7 F-ROLL — Roll-splice flag (VPF-4)

Status on main a07257a. ✓ Done on main.

Why it exists. On a continuous chart the contract switch looks like a gap; without this flag multi-day levels and ATRs would treat it as a real move.

Where. `_CustomSignal.Strategy.CS` · `rollOn(ds)` (virtual; overridden in `_VolumeProfileFuturesBase`); `RollDetection` (`ERollDetection`); `_RollDetector.Function.CS`

Key signatures.

```
public enum ERollDetection { Off = 0, Series = 1, Table = 2, SeriesOrTable = 3 }   // _CustomSignal ·
_CustomSignal:1719
protected ERollDetection RollDetection { get; set; }   // _CustomSignal · _CustomSignal:1722
protected virtual bool rollOn(int ds)   // _CustomSignal · _CustomSignal:1794
```

Snippet (compiled as `_Sn07`, base `_VolumeProfileFuturesBase`).

Enables Series roll detection and gives you the bar to reset multi-day state. `rollOn(1)` is true only on the splice bar.

```
// Create
protected override void Create()
{
    base.Create();
    RollDetection = ERollDetection.Series;         // detect the dxFeed continuous-contract splice
}

// CalcBar
protected override void CalcBar()
{
    base.CalcBar();
    if (Bars.Status != EBarState.Close) return;
    if (rollOn(1))
    {
        /* reset multi-day state: prior levels belong to the old contract */
    }
}
```

Used in examples. example 1 (§3.1), example 13 (§3.3), example 12 (§3.4), example 16 (§3.5), example 3 (§3.6), example 14 (§3.7), example 18 (§3.8), example 20 (§3.9), example 17 (§3.10), example 4 (§3.11), example 5 (§3.12), example 11 (§3.14), example 15 (§3.15), example 19 (§3.16), example 23 (§3.18), example 25 (§3.20), example 37 (§3.21), example 6 (§3.23), example 8 (§3.24), example 22 (§3.25), example 26 (§3.26), example 27 (§3.27), example 29 (§3.28), example 30 (§3.29), example 38 (§3.30), example 31 (§3.31), example 7 (§3.33), example 10 (§3.34), example 28 (§3.35), example 32 (§3.36), example 33 (§3.37), example 34 (§3.38), example 36 (§3.39), example 39 (§3.40), example 41 (§3.41), example 42 (§3.42); inherited by example 21, example 24, example 40, example 35.

Pitfalls.

- On unadjusted continuous data the splice is a price jump; any multi-day level (prior POC, composite, swing) is stale after it.
- Engines with their own roll handling (M-DAILY, AVWAP) reseed after a roll and are NaN for a warm-up period.

5.8 F-CAL — CME holidays / half-days (VPF-6), CmeSessionClock (VPF-7)

Status on main a07257a. ✓ Done on main.

Why it exists. Globex days start at 18:00 ET and CME holidays shorten sessions; dates and windows must follow the exchange clock, not the calendar date.

Where. `_CmeHolidays.Function.CS`, `_NyseHolidays.Function.CS`, `_CmeSessionClock.Function.CS` · `_CmeSessionClock`; host `GlobexDate`, `ToEastern`, `tradingDayKey`, `sessionClock`

Key signatures.

```
public sealed class _CmeSessionClock   // _CmeSessionClock · _CmeSessionClock:24
protected DateTime ToEastern(DateTime barTime)   // _VolumeProfileFuturesBase · _VolumeProfileFuturesBase:204
protected DateTime GlobexDate(DateTime barTime)   // _VolumeProfileFuturesBase · _VolumeProfileFuturesBase:255
protected override DateTime tradingDayKey(DateTime barTime)   // _VolumeProfileFuturesBase · _VolumeProfileFuturesBase:356
protected override TimeSpan sessionClock(DateTime barTime)   // _VolumeProfileFuturesBase · _VolumeProfileFuturesBase:359
```

Snippet (compiled as `_Sn08`, base `_VolumeProfileFuturesBase`).

Converts the bar time to the Globex trading day and Eastern time, and logs the RTH open. A bar at 18:00 ET already belongs to the next day.

```
// CalcBar
protected override void CalcBar()
{
    base.CalcBar();
    if (Bars.Status != EBarState.Close) return;
    DateTime day = GlobexDate(Bars.Time[0]);       // 18:00 ET bars belong to the NEXT Globex day
    TimeSpan et = ToEastern(Bars.Time[0]).TimeOfDay;
    TimeSpan clock = sessionClock(Bars.Time[0]);   // the clock CS-7 windows use
    if (journalOn() && et == new TimeSpan(9, 30, 0))
        trace(ETraceLevel.Journal, $"{day:yyyy-MM-dd} RTH open, session clock {clock}");
}
```

Used in examples. example 1 (§3.1), example 2 (§3.2), example 13 (§3.3), example 12 (§3.4), example 3 (§3.6), example 14 (§3.7), example 18 (§3.8), example 20 (§3.9), example 17 (§3.10), example 4 (§3.11), example 5 (§3.12), example 11 (§3.14), example 15 (§3.15), example 19 (§3.16), example 23 (§3.18), example 24 (§3.19), example 25 (§3.20), example 37 (§3.21), example 6 (§3.23), example 8 (§3.24), example 22 (§3.25), example 26 (§3.26), example 27 (§3.27), example 29 (§3.28), example 30 (§3.29), example 38 (§3.30), example 31 (§3.31), example 7 (§3.33), example 10 (§3.34), example 28 (§3.35), example 32 (§3.36), example 36 (§3.39), example 39 (§3.40), example 41 (§3.41); inherited by example 21, example 40, example 35.

Pitfalls.

- A check like `tod >= 15:55` is also true at 19:00 on a Globex chart; bound it (`&& tod < 17:00`) or use `IsRth()`.
- Half-days close RTH at 13:00 ET (13:15 for equity futures); fixed-time exits must respect them.

5.9 F-OF — Order-flow helpers (VPF-5) and the FlowQuality chain

Status on main a07257a. ✓ Done on main (MC-BidAsk tier on main since PR #19, off by default; per-stream earliest-tick tagging partial, VERIFY).

Why it exists. Bid/ask volume shows who was aggressive inside a bar; stacked imbalances mark one-sided pressure across several prices.

Where. `_VolumeProfileFuturesBase.Strategy.CS` · `OrderFlowEnabled`, `BarDelta`, `CumDelta`, `DiagonalImbalances`, `StackedImbalances`, `Absorption`; `_OrderFlow.Function.CS` · `FlowValue`, `FlowQuality`, `FlowSource`, `StackedResult`

Key signatures.

```
public enum FlowQuality { Exact = 0, High = 1, Medium = 2, Low = 3, Unknown = 4 }   // _OrderFlow · _OrderFlow:38
protected bool OrderFlowEnabled { get; set; }   // _VolumeProfileFuturesBase · _VolumeProfileFuturesBase:588
public _OrderFlow.FlowValue BarDelta(int barsAgo)   // _VolumeProfileFuturesBase · _VolumeProfileFuturesBase:874
public _OrderFlow.FlowValue CumDelta(int barsAgo)   // _VolumeProfileFuturesBase · _VolumeProfileFuturesBase:882
public _OrderFlow.StackedResult StackedImbalances(int barsAgo, double ratio = 3.0, double minVol = 10, int minStack = 3)   // _VolumeProfileFuturesBase · _VolumeProfileFuturesBase:898
public _OrderFlow.FlowValue Absorption(int barsAgo, _OrderFlow.Side side, int edgeTicks = 2, double volMult = 2.0, int holdTicks = 1)   // _VolumeProfileFuturesBase · _VolumeProfileFuturesBase:906
protected _OrderFlow.FlowConfig FlowCfg { get; }   // _VolumeProfileFuturesBase · _VolumeProfileFuturesBase:591
protected _OrderFlow.FlowEngine Flow { get; }   // _VolumeProfileFuturesBase · _VolumeProfileFuturesBase:612
```

Bounded window (PR #48). The engine keeps at most `FlowCfg.MaxBars` footprint bars (default 5000, minimum 10; `Flow.BarCap` is the effective cap) and drops the oldest beyond that; `Flow.DroppedBars` is the running total of dropped bars over the whole run (it never resets at a session/RTH reset). `CumDelta` sums its reset period (the Globex session, or RTH with `FlowCfg.ResetMode = Rth`) from those bars, so when that period started before the oldest kept bar the value is short and `FlowValue.Truncated` is true (`ToString()` adds `trunc`; only CumDelta sets it). Use the flag to skip or down-weight session-delta signals on long sessions with fine bars; raise `MaxBars` in `Create()` if it fires often. The engine also journals the first quality downgrade per session (`OF quality downgrade session() bar() from->to source() …`, Journal) and warns once when the cap is first hit (`OF MaxBars cap N hit at bar …`, Warning).

Snippet (compiled as `_Sn09`, base `_VolumeProfileFuturesBase`).

Turns order flow on (with the default 5000-bar window set explicitly), reads the bar delta, and logs stacked imbalances (≥ 3 prices, ratio 3:1, ≥ 10 contracts). Bars whose flow quality is below `High` (Medium, Low, Unknown) are skipped. It then refuses a truncated session `CumDelta` and journals how many bars the window has dropped.

```
// Create
protected override void Create()
{
    base.Create();
    OrderFlowEnabled = true;                        // lazy: no footprints otherwise
    FlowCfg.MaxBars =
5000;                          // footprint window (default 5000, min 10); older bars are dropped
}

// CalcBar
protected override void CalcBar()
{
    base.CalcBar();
    if (Bars.Status != EBarState.Close) return;
    _OrderFlow.FlowValue d = BarDelta(0);
    if (!d.IsAvailable || d.Quality > _OrderFlow.FlowQuality.High) return;   // Exact/High only for signals
    _OrderFlow.StackedResult st = StackedImbalances(0, 3.0, 10, 3);
    if (st.Found && journalOn())
        trace(ETraceLevel.Journal, $"delta {d.Value} stacked runs {st.Runs.Count} longest {st.Longest}");
    _OrderFlow.FlowValue cd = CumDelta(0);
    if (cd.Truncated) return;                       // session CumDelta lost its oldest bars to the
```

```
FlowCfg.MaxBars cap
    if (Flow != null && Flow.DroppedBars > 0 && journalOn())
        trace(ETraceLevel.Journal, $"flow window full: {Flow.DroppedBars} bars dropped (cap {Flow.BarCap})");
}
```

Used in examples. example 38 (§3.30).

Pitfalls.

- Before the tick window only BarUpDown (Low) flow exists; never trade on it. Report results per FlowQuality tier.
- `FlowQuality` is ordered Exact < High < Medium < Low < Unknown; "at least High" is `Quality <= High`.
- Absorption and single-price delta need tier >= 3 (catalogue data-quality rule).
- A `CumDelta` with `Truncated = true` is missing the session's oldest bars; do not compare it with untruncated values.

5.10 F-VWAP — VWAP engine, 3 slots with σ

Status on main a07257a. ✓ Done on main; `EVwapType` adds Week, Month, Quarter, RollDate (M-AVWAP-lite).

Why it exists. VWAP is the volume-weighted average price institutions benchmark against; its σ bands measure how stretched price is.

Where. `_VolumeProfileBase.Strategy.CS` · `Vwap1Type…Vwap3Type` ([Input]s), `Vwap1Length`, `Vwap1FromDate`, `CurrentVwap1…3`, `CurrentVwap1Sigma…`; `_VpVwapEngine.Function.CS`

Key signatures.

```
public enum EVwapType { Disabled = 0, Session = 1, Length = 2, AdaptiveFromDate = 3, Week = 4, Month = 5,
Quarter = 6, RollDate = 7 }   // _VolumeProfileBase · _VolumeProfileBase:59
[Input] public EVwapType Vwap1Type { get; set; }   // _VolumeProfileBase · _VolumeProfileBase:505
public double CurrentVwap1 { get; }   // _VolumeProfileBase · _VolumeProfileBase:1708
public double CurrentVwap1Sigma { get; }   // _VolumeProfileBase · _VolumeProfileBase:1717
```

Snippet (compiled as `_Sn10`, base `_VolumeProfileFuturesBase`).

Sets slot 1 to the session VWAP and slot 2 to the weekly VWAP, and logs closes more than 2σ above the session VWAP: price is stretched.

```
// Create
protected override void Create()
{
    base.Create();
    Vwap1Type = EVwapType.Session;                // VP-base [Input]: set in code
    Vwap2Type = EVwapType.Week;
}

// CalcBar
protected override void CalcBar()
{
    base.CalcBar();
    if (Bars.Status != EBarState.Close) return;
    double v = CurrentVwap1;                      // session VWAP
    double sd = CurrentVwap1Sigma;                // its standard deviation
    double w = CurrentVwap2;                      // weekly VWAP
    if (!double.IsNaN(v) && Bars.Close[0] > v + 2 * sd && journalOn())
        trace(ETraceLevel.Journal, $"> +2σ session VWAP; week VWAP {w}");
}
```

Used in examples. example 3 (§3.6), example 14 (§3.7), example 20 (§3.9).

Pitfalls.

- The slots are `[Input]`s on the VP base; set them in `Create()` so the strategy still exposes no inputs of its own.
- Session VWAP resets on the chart session; on a Globex chart that is 18:00 ET, not 09:30.

5.11 F-CONF — Confluence engine

Status on main a07257a. ✓ Done on main (existing).

Why it exists. A price where several independent levels cluster is more likely to react than any single level.

Where. `_VolumeProfileBase.Strategy.CS · GetConfluenceZones`, `GetTopConfluenceZones`, `GetConfluenceTargets`, `ConfluenceZone`, `Confluence*` settings; `_VpConfluenceEngine.Function.CS`

Key signatures.

```
public List<ConfluenceZone> GetConfluenceZones()   // _VolumeProfileBase · _VolumeProfileBase:1602
public List<ConfluenceZone> GetTopConfluenceZones(int count)   // _VolumeProfileBase · _VolumeProfileBase:1623
public List<ConfluenceTarget> GetConfluenceTargets(double price, EVpDirection direction)   // _VolumeProfileBase · _VolumeProfileBase:1640
protected bool UsesConfluenceZones { get; set; }   // _VolumeProfileBase · _VolumeProfileBase:636
protected internal double ConfluenceSplitAtrFraction { get; set; }   // _VolumeProfileBase · _VolumeProfileBase:626
public static double SplitWidth(double tol, double atr20d, double atrFraction)   // VpConfluenceRules · _VpConfluenceEngine:38
```

Zone width (PR #47). Anchors within `ConfluenceAlignTicks` (tol) of each other chain into one cluster. A cluster of ≥ 3 anchors wider than `VpConfluenceRules.SplitWidth` = max(4 × tol, `ConfluenceSplitAtrFraction` × Atr20d) (default fraction 0.03; 4 × tol alone while Atr20d is not ready) is split at its largest internal gap (the lowest one on a tie). The ATR term keeps zones proportional to the market's daily range: on a quiet day 4 × tol rules, on a volatile day a zone may be up to 3% of the daily ATR wide. Atr20d comes only from M-DAILY, so a strategy that reads zones or targets sets `UsesConfluenceZones = true` in `Create()` (or before `base.StartCalc()`); the base then turns on `NeedsDaily` when a confluence slot is enabled. Without it the split uses 4 × tol and a one-time Warning says so (`VPCONF: confluence zones used but UsesConfluenceZones is false …`). Each split is journaled `VPCONF split anchors() range() width() limit() at() gap() atr20d() basis(atr|tol-fallback)`. Enabling two confluence slots on the same data stream now throws in `StartCalc`.

Developing profile. Labels from today's unfinished profile are tagged `ConfluenceHit.developing` and also listed in `ConfluenceZone.developingHits` (null when there are none; `HasDeveloping`). `developingScore` is their credit, already included in `rawScore` (report only). `independentSources` counts distinct independent volume masses (stream, profile window); `profileCount` the distinct profiles among them.

Snippet (compiled as `_Sn11`, base `_VolumeProfileFuturesBase`).

Declares that it reads zones (so the ATR split is used), then logs the three strongest confluence zones, skipping a zone whose whole score comes from the developing profile. A higher `zScore` and more `independentSources` = a stronger zone.

```
// Create
protected override void Create()
{
    base.Create();
    UsesConfluenceZones = true;                    // builds M-DAILY so the zone split can use 0.03 x Atr20d
}

// CalcBar
protected override void CalcBar()
{
    base.CalcBar();
```

```
    if (Bars.Status != EBarState.Close) return;
    foreach (ConfluenceZone z in GetTopConfluenceZones(3))
    {
        if (z.HasDeveloping && z.developingScore >= z.rawScore) continue;   // only today's unfinished
profile backs it
        if (journalOn())
            trace(ETraceLevel.Journal, $"zone {z.low}-{z.high} z {z.zScore:F2} sources
{z.independentSources}");
    }
}
```

Used in examples. none.

Pitfalls.

- Zones score several level sources together; their weights are settings, not fitted values. Treat them as a level list for Gate-0, not as evidence.
- Forgetting `UsesConfluenceZones = true` gives narrower zones (4 × tol only) than the hosts on main (`_DeltaDivRevFuturesLELX`, `_VolumeProfileTest1` with `TraceConfluence`).
- A developing-profile label can move until the session ends; check `HasDeveloping` before treating a zone as settled.

5.12 F-TM — Trade-management primitives (CS-7)

Status on main a07257a. ✓ Done on main.

Why it exists. Time windows, trade caps and time stops keep a strategy out of the hours and situations it was not designed for.

Where. `_CustomSignal.Strategy.CS · NoEntryBefore, NoEntryAfter, FlatBy, MaxTradesPerDay, DailyLossStopCount, DailyLossStopR, RiskUnitDollars, TimeStopBars, entryAllowed(out string), mustFlatten()`

Key signatures.

```
protected TimeSpan? NoEntryAfter { get; set; }   // _CustomSignal · _CustomSignal:2664
protected TimeSpan? FlatBy { get; set; }   // _CustomSignal · _CustomSignal:2666
protected int MaxTradesPerDay { get; set; }   // _CustomSignal · _CustomSignal:2668
protected int DailyLossStopCount { get; set; }   // _CustomSignal · _CustomSignal:2670
protected double DailyLossStopR { get; set; }   // _CustomSignal · _CustomSignal:2672
protected double RiskUnitDollars { get; set; }   // _CustomSignal · _CustomSignal:2674
protected int TimeStopBars { get; set; }   // _CustomSignal · _CustomSignal:2676
protected bool entryAllowed(out string reason)   // _CustomSignal · _CustomSignal:2730
protected bool mustFlatten()   // _CustomSignal · _CustomSignal:2749
```

Snippet (compiled as `_Sn12`, base `_VolumeProfileFuturesBase`).

Sets the entry window, flat time, trade cap and time stop, then gates entries. When in a position, `mustFlatten()` says it is time to exit; when flat, `reason` names the rule that blocked.

```
// Create
protected override void Create()
{
    base.Create();
    NoEntryAfter = new TimeSpan(14, 30, 0);        // no new entries after 14:30
    FlatBy = new TimeSpan(15, 55, 0);              // flat by 15:55
    MaxTradesPerDay = 2;
    TimeStopBars = 18;                             // exit after 18 bars in the trade
}

// CalcBar
protected override void CalcBar()
{
    base.CalcBar();
    if (Bars.Status != EBarState.Close) return;
    if (StrategyInfo.MarketPosition != 0)
    {
```

```
        if (mustFlatten()) { /* send the market exit */ }
        return;
    }
    string reason;
    if (!entryAllowed(out reason)) return;         // reason says which rule blocked
    /* entry logic */
}
```

Used in examples. example 13 (§3.3), example 12 (§3.4), example 18 (§3.8), example 20 (§3.9), example 17 (§3.10), example 23 (§3.18), example 25 (§3.20), example 37 (§3.21), example 40 (§3.22), example 22 (§3.25), example 26 (§3.26), example 27 (§3.27), example 29 (§3.28), example 30 (§3.29), example 38 (§3.30), example 31 (§3.31), example 28 (§3.35), example 32 (§3.36), example 36 (§3.39); inherited by example 24, example 35.

Pitfalls.

- The CS-7 rules act only where you call them: `entryAllowed` before entries and `mustFlatten` while in a position.
- `DailyLossStopCount` counts all losing closed trades of the day (not consecutive ones); `DailyLossStopR` needs `RiskUnitDollars` > 0 and uses closed P&L.
- Windows use `sessionClock`, which on the futures base is ET wall time.

5.13 F-JRNL — Journal: R, MFE/MAE, run id (CS-1/CS-3); sizeForRisk (CS-4); cost filter

Status on main a07257a. ✓ Done on main.

Why it exists. A per-trade record in R (risk units) with MFE/MAE shows how good the entries and exits actually were.

Where. `_CustomSignal.Strategy.CS` · `journalOn()`, `journalArm`, `trace`, `sizeForRisk`, `equityAvailable`; `_SignalJournal.Function.CS`

Key signatures.

```
protected void journalArm(string entryOrderName, double plannedEntry, double initialStop, double target,
IDictionary<string, string> context = null)   // _CustomSignal · _CustomSignal:2198
protected double equityAvailable(double maxCustomRiskEquityPerPosPercent = 0.0d)   // _CustomSignal ·
_CustomSignal:2355
protected int sizeForRisk(IInstrument bars, double entry, double stop, double riskFraction, SizingLimits limi
ts)   // _CustomSignal · _CustomSignal:2791
protected double equityAvailable(double fraction, bool includeOpenPnl)   // _CustomSignal ·
_CustomSignal:2829
```

Snippet (compiled as `_Sn13`, base `_VolumeProfileFuturesBase`).

Turns the journal on, skips trades whose dollar risk exceeds 1 % of available equity, and records the planned entry, stop and 1R target next to the Send.

```
// Create
protected override void Create()
{
    base.Create();
    TraceLevel = ETraceLevel.Journal;              // journal on; no [Input]
}

// CalcBar
protected override void CalcBar()
{
    base.CalcBar();
    if (Bars.Status != EBarState.Close) return;
    double entry = Bars.Close[0];
    double stop = entry - 14 * tickSize();         // 14-tick stop
    double riskDollars = Bars.Info.BigPointValue * (entry - stop);
    if (riskDollars > equityAvailable(0.01)) return;   // skip, never resize
```

```
    if (journalOn())
        journalArm("LE", entry, stop, entry + (entry - stop), null);   // call next to the entry Send
}
```

Used in examples. example 1 (§3.1), example 2 (§3.2), example 13 (§3.3), example 12 (§3.4), example 16 (§3.5), example 3 (§3.6), example 14 (§3.7), example 18 (§3.8), example 20 (§3.9), example 17 (§3.10), example 5 (§3.12), example 9 (§3.13), example 11 (§3.14), example 15 (§3.15), example 19 (§3.16), example 23 (§3.18), example 25 (§3.20), example 37 (§3.21), example 6 (§3.23), example 22 (§3.25), example 26 (§3.26), example 27 (§3.27), example 29 (§3.28), example 30 (§3.29), example 38 (§3.30), example 31 (§3.31), example 7 (§3.33), example 10 (§3.34), example 28 (§3.35), example 32 (§3.36), example 33 (§3.37), example 34 (§3.38), example 36 (§3.39); inherited by example 21, example 24, example 40, example 35.

Pitfalls.

- `journalArm` must be called on the bar the entry order is sent, with the planned stop, so R and MFE/MAE are measured against it.
- User rule: a trade that breaches the 0.01 × equity risk budget is skipped, never resized; `sizeForRisk` is shown in §4 but not used by the examples.
- **The floor is example code, not library behaviour.** The stop floor (stop = max(structural, 14 ticks); skip if q × widened stop × $50 > 0.01 × equity; never shrink q) is Ettienne's decided rule (2026-10-23), implemented **per strategy**: main (a07257a) has no stop floor or widening, and each example implements it locally (`costFloorTicks = 14`, `riskBudgetFraction = 0.01`, a local `overRiskBudget` or an inline check using `equityAvailable(fraction)`). The library part is **pending**: a library `CostConfig` is only specified in `instrument-profile-spec.md` IP-3.2/IP-8.1 (not built). The only library cost rule is `CostFilterMaxFractionOfR` (default 0 = off), which skips and never widens. Consequence: a strategy that sizes at exactly 1% risk skips every stop under 14 ticks (any widening breaches the budget); the floor widens without a skip only for fixed size or sizing below 1%. The examples trade a fixed 1 contract, so a 14-tick stop ($175) is skipped only when equity < $17,500.

5.14 F-EVT — logSignal forward MFE/MAE + ControlMode (CS-8)

Status on main a07257a. ✓ Done on main; this is the Gate-3 event-study harness.

Why it exists. Measuring what happens after every signal, traded or not, separates edge from execution, and control modes test it against randomised signals.

Where. `_CustomSignal.Strategy.CS` · `logSignal`, `ControlMode`, `ControlShiftTicks`, `ControlDelayBars`, `ControlSeed`, `ControlRandomProbability`, `controlAdjustLevel`, `controlAdjustDelayBars`, `controlRandomEntry`, `SignalHorizons`

Key signatures.

```
public enum EControlMode { None = 0, RandomEntry = 1, ShiftLevels = 2, DelayEntryBars = 3 }   //
_CustomSignal · _CustomSignal:2518
protected EControlMode ControlMode { get; set; }   // _CustomSignal · _CustomSignal:2523
protected int ControlShiftTicks { get; set; }   // _CustomSignal · _CustomSignal:2528
protected int ControlDelayBars { get; set; }   // _CustomSignal · _CustomSignal:2530
protected void logSignal(string tag, int direction, double price, double stop, IDictionary<string, string> co
ntext, bool taken)   // _CustomSignal · _CustomSignal:2552
protected bool controlRandomEntry()   // _CustomSignal · _CustomSignal:2642
protected double controlAdjustLevel(double level)   // _CustomSignal · _CustomSignal:2648
protected int controlAdjustDelayBars()   // _CustomSignal · _CustomSignal:2654
```

Snippet (compiled as `_Sn14`, base `_VolumeProfileFuturesBase`).

Logs every breakout of the prior high as a signal with a 14-tick stop; the journal then reports its forward MFE/MAE at the configured horizons whether or not it was traded.

```
// Create
protected override void Create()
{
    base.Create();
    TraceLevel = ETraceLevel.Journal;              // logSignal is a no-op below Journal
}

// CalcBar
protected override void CalcBar()
{
    base.CalcBar();
    if (Bars.Status != EBarState.Close) return;
    double level = controlAdjustLevel(Bars.High[1]);   // identity unless ControlMode = ShiftLevels
    if (Bars.Close[0] > level)
        logSignal("BRK", +1, Bars.Close[0], Bars.Close[0] - 14 * tickSize(), null, false);   // forward MFE/
MAE at SignalHorizons
}
```

Used in examples. example 1 (§3.1), example 2 (§3.2), example 13 (§3.3), example 12 (§3.4), example 16 (§3.5), example 3 (§3.6), example 14 (§3.7), example 20 (§3.9), example 5 (§3.12), example 11 (§3.14), example 15 (§3.15), example 24 (§3.19), example 6 (§3.23), example 29 (§3.28), example 7 (§3.33), example 10 (§3.34), example 32 (§3.36); inherited by example 21, example 35.

Pitfalls.

- `logSignal` writes `MCEvents_<runId>.csv`; it records every event, taken or not, so log the event before skip filters if you study the filter.
- Controls (random entry, shifted levels, delayed entries) are the placebo arms; run them with the same code path.

5.15 M-AVWAP — AVWAP anchor manager

Status on main a07257a. ✓ Available on main (PR #7): NewsBar anchor reserved and inert; P1/P2 placebo twins built in (PR #19); roll invalidation only on the futures base.

Why it exists. A VWAP anchored at a swing or gap shows the average price of everyone who traded since that event, a common pullback level.

Where. `_AvwapManager.Function.CS · _AvwapManager`, `_Avwap`, `EAvwapAnchor`, `AvwapLimits`, `EAvwapPlacebo`; `_CustomSignal.AvwapEvents`, `AvwapLimits`, `AvwapManager`

Key signatures.

```
protected EAvwapAnchor AvwapEvents { get; set; }   // _CustomSignal · _CustomSignal:820
protected AvwapLimits AvwapLimits { get; }   // _CustomSignal · _CustomSignal:822
internal _AvwapManager AvwapManager   // _CustomSignal · _CustomSignal:828
public enum EAvwapAnchor { None = 0, SwingHigh = 1 << 0, SwingLow = 1 << 1, Breakout = 1 << 2, GapOpen = 1
<< 3, VolumeClimax = 1 << 4, WeekOpen = 1 << 5, MonthOpen = 1 << 6, QuarterOpen = 1 << 7, NewsBar = 1 << 8, S
wing = SwingHigh | SwingLow }   // _AvwapManager · _AvwapManager:15
```

Snippet (compiled as `_Sn15`, base `_VolumeProfileFuturesBase`).

Anchors VWAPs at swing lows and gap opens and logs a touch of the newest swing-low VWAP. `Band(1)` is the +1σ band.

```
// Create
protected override void Create()
{
    base.Create();
    AvwapEvents = EAvwapAnchor.SwingLow            // lazy: no anchors unless set
        | EAvwapAnchor.GapOpen;
    AvwapLimits.maxAgeSessions = 5;
}

// CalcBar
protected override void CalcBar()
```

```
{
    base.CalcBar();
    if (Bars.Status != EBarState.Close) return;
    _AvwapManager am = AvwapManager;
    _Avwap a = am != null ? am.Latest(EAvwapAnchor.SwingLow) : null;   // newest swing-low anchor
    if (a != null && a.Valid && Bars.Low[0] <= a.Value && journalOn())
        trace(ETraceLevel.Journal, $"touch {a.Tag} {a.Value:F2} ±1σ {a.Band(1):F2}");
}
```

Used in examples. example 16 (§3.5).

Pitfalls.

- The `Breakout` anchor fires on a PRICEACTION break-of-structure, not on a profile/composite edge break; Ex28 hand-rolls a break-anchored VWAP for that reason.
- Swing anchors are usable only from the pivot's confirmation bar (`UsableFrom`); earlier values would be lookahead.

5.16 M-SWING — PRICEACTION swing / structure engine

Status on main a07257a. ✓ Available on main (PR #6); extra streams via `ExtraPriceAction` (PR #19); MC acceptance trace (Q3) not yet run.

Why it exists. Swing pivots and break of structure (BOS) define trend in price-action terms; a second engine on a higher timeframe gives the bigger trend.

Where. `_PriceActionEngine.Function.CS · _PriceActionEngine, PriceActionConfig, EStructure; _CustomSignal.PriceActionConfig, PriceActionEngine, ExtraPriceAction, PriceActionFor`

Key signatures.

```
protected PriceActionConfig PriceActionConfig { get; }   // _CustomSignal · _CustomSignal:380
internal _PriceActionEngine PriceActionEngine   // _CustomSignal · _CustomSignal:385
protected List<PriceActionConfig> ExtraPriceAction { get; }   // _CustomSignal · _CustomSignal:724
internal _PriceActionEngine PriceActionFor(int dataStream, string role = null)   // _CustomSignal · _CustomSignal:735
```

Snippet (compiled as `_Sn16`, base `_VolumeProfileFuturesBase`).

Runs PRICEACTION on Data1 and Data2 and logs a Data1 break of structure up while the Data2 structure is up: a pullback-continuation setup.

```
// Create
protected override void Create()
{
    base.Create();
    PriceActionConfig.enabled = true;              // Data1 engine (futures preset applied by the base)
    ExtraPriceAction.Add(new PriceActionConfig     // a second engine on Data2 (must be on the chart)
    {
        enabled = true,
        dataStream = 2,
        role = "htf"
    });
}

// CalcBar
protected override void CalcBar()
{
    base.CalcBar();
    if (Bars.Status != EBarState.Close) return;
    _PriceActionEngine pa = PriceActionEngine;     // Data1
    _PriceActionEngine htf = PriceActionFor(2, "htf");
    if (pa == null || htf == null) return;
    if (htf.Structure(0) == EStructure.Up && pa.BosUp && journalOn())
        trace(ETraceLevel.Journal, $"BOS up, last swing low {pa.LastLow().price}");
}
```

Used in examples. example 19 (§3.16), example 26 (§3.26), example 31 (§3.31), example 28 (§3.35), example 34 (§3.38).

Pitfalls.

- Pivots are confirmed `strength` bars after the extreme; use the engine's confirmed values, never `Bars.Low[k]` of an unconfirmed pivot.
- Call `PriceActionFor` with the same role string used in `ExtraPriceAction`.
- The snippet requires Data2 on the chart (the `dataStream = 2` engine); without it `StartCalc` throws `ArgumentException` (`requireDataStream`, `_CustomSignal` L764/L2333).

5.17 M-MTF — Multi-TF via native BarsOfData(n)

Status on main a07257a. ✓ Available on main (PR #8); Probe P2 still to be run in MC.

Why it exists. Higher-timeframe levels must come from closed bars only, or a backtest sees the future.

Where. `_CustomSignal.Strategy.CS` · `HtfStreams`, `HtfClosed`, `HtfClosedBar`, `HtfForming`; `_MtfMath.Function.CS`; probe `_MtfProbe.Strategy.CS`

Key signatures.

```
protected int[] HtfStreams { get; set; }   // _CustomSignal · _CustomSignal:1658
protected double HtfClosed(int ds, int ago = 0)   // _CustomSignal · _CustomSignal:1665
protected void HtfClosedBar(int ds, int ago, out DateTime time, out double open, out double high, out double
low, out double close, out double volume)   // _CustomSignal · _CustomSignal:1667
protected double HtfForming(int ds)   // _CustomSignal · _CustomSignal:1674
```

Snippet (compiled as `_Sn17`, base `_VolumeProfileFuturesBase`).

Reads the last closed Data2 bar and logs a Data1 close above its high. The forming Data2 bar is never used.

```
// Create
protected override void Create()
{
    base.Create();
    HtfStreams = new[] { 2 };                    // Data2 is read as closed bars only
}

// CalcBar
protected override void CalcBar()
{
    base.CalcBar();
    if (Bars.Status != EBarState.Close) return;
    DateTime t;
    double o, h, l, c, v;
    HtfClosedBar(2, 0, out t, out o, out h, out l, out c, out v);   // last CLOSED Data2 bar
    if (Bars.Close[0] > h && journalOn())
        trace(ETraceLevel.Journal, $"above last closed HTF high {h} ({t:HH:mm})");
}
```

Used in examples. example 2 (§3.2), example 13 (§3.3), example 14 (§3.7), example 5 (§3.12), example 11 (§3.14), example 6 (§3.23), example 7 (§3.33), example 10 (§3.34), example 36 (§3.39).

Pitfalls.

- `BarsOfData(2).Close[0]` is the FORMING higher-TF bar during the HTF bar; use `HtfClosed`/`HtfClosedBar` for rules.
- Orders go to Data1 only.

5.18 M-TOD — Time-of-day normalisers

Status on main a07257a. ✓ Available on main (PR #9); delta fed only by the futures base (BarUpDown → NaN).

Why it exists. Volume and range follow a strong daily pattern; comparing a bar with the same minute of prior sessions shows whether it is truly unusual.

Where. `_TodNormaliser.Function.CS` · `_TodNormaliser` (`Z`, `Median`, `Pct`, `ZCum`), `ETodMetric`; `_CustomSignal.TodMetrics`, `TodSessions`, `TodMinSessions`, `TodNormaliser`

Key signatures.

```
protected ETodMetric TodMetrics { get; set; }   // _CustomSignal · _CustomSignal:843
protected int TodSessions { get; set; }   // _CustomSignal · _CustomSignal:845
internal _TodNormaliser TodNormaliser   // _CustomSignal · _CustomSignal:855
public enum ETodMetric { None = 0, Volume = 1 << 0, Delta = 1 << 1, Range = 1 << 2, TrueRange = 1 <<
3 }   // _TodNormaliser · _TodNormaliser:15
```

Snippet (compiled as `_Sn18`, base `_VolumeProfileFuturesBase`).

Enables volume and range normalisation over 20 sessions and logs bars whose volume is ≥ 2σ above the same minute's norm.

```
// Create
protected override void Create()
{
    base.Create();
    TodMetrics = ETodMetric.Volume                // lazy
        | ETodMetric.Range;
    TodSessions = 20;
}

// CalcBar
protected override void CalcBar()
{
    base.CalcBar();
    if (Bars.Status != EBarState.Close || TodNormaliser == null) return;
    double vz = TodNormaliser.Z(ETodMetric.Volume);       // vs the same clock minute over the prior sessions
    if (vz >= 2 && journalOn())
        trace(ETraceLevel.Journal, $"volume z {vz:F2}, range pct {TodNormaliser.Pct(ETodMetric.Range):F2}");
}
```

Used in examples. example 4 (§3.11), example 19 (§3.16), example 25 (§3.20).

Pitfalls.

- Z is NaN until `TodMinSessions` sessions exist for that clock minute.
- Half-days are excluded from the norms by the futures base.

5.19 M-DAILY — Daily context: ATR20d, SMA200, RSI(2), TSMOM, HV

Status on main a07257a. ✓ Available on main (PR #10; HV and LBR since PR #19); Probe P3 (dxFeed daily = settlement?) open.

Why it exists. Daily ATR, momentum and volatility scale stops and filters to the current market.

Where. `_DailyContext.Function.CS` · `_DailyContext` (`Atr20d`, `Sma200`, `Rsi2`, `Tsmom(n)`, `Hv(n)`, `HvPercentile`, `Lbr`, `PrevHigh/Low/Close`, `Valid`), `DailyConfig`; `_CustomSignal.NeedsDaily`, `DailyConfig`, `DailyContext`

Key signatures.

```
protected bool NeedsDaily { get; set; }   // _CustomSignal · _CustomSignal:908
protected DailyConfig DailyConfig { get; }   // _CustomSignal · _CustomSignal:910
internal _DailyContext DailyContext   // _CustomSignal · _CustomSignal:920
```

Snippet (compiled as `_Sn19`, base `_VolumeProfileFuturesBase`).

Requests the daily stream before `base.StartCalc()` and logs 20-day ATR, momentum and volatility once `Valid`.

```
// StartCalc (before base.StartCalc)
protected override void StartCalc()
{
    NeedsDaily = true;                              // must be set BEFORE base.StartCalc()
    DailyConfig.hvLengths = new[] { 20 };
    base.StartCalc();
}

// CalcBar
protected override void CalcBar()
{
    base.CalcBar();
    if (Bars.Status != EBarState.Close || DailyContext == null || !DailyContext.Valid) return;
    double atr = DailyContext.Atr20d;               // 20-day ATR
    double mom = DailyContext.Tsmom(20);            // 20-day time-series momentum
    double hv = DailyContext.Hv(20);                // 20-day historical volatility
    if (journalOn())
        trace(ETraceLevel.Journal, $"ATR20d {atr:F2} TSMOM20 {mom:P1} HV20 {hv:F3}");
}
```

Used in examples. example 20 (§3.9), example 17 (§3.10), example 9 (§3.13), example 27 (§3.27), example 30 (§3.29), example 31 (§3.31), example 10 (§3.34), example 32 (§3.36), example 33 (§3.37); inherited by example 35.

Pitfalls.

- Set `NeedsDaily` in `StartCalc()` before `base.StartCalc()` (or in `Create()`); later is too late.
- With RollDetection on, ATR is reseeded at each roll and is NaN for `atrLength` sessions, unless `DailyConfig.atrRestartOnRoll = false` (PR #46; §4.3), which carries it on and skips only the splice day's TR.

5.20 M-EVENT — Scheduled-event calendar (M-NEWS / M-EVENT)

Status on main a07257a. ◐ Scheduled-event calendar available on main (PRs #25, #27–#29); unscheduled news not covered; `EAvwapAnchor.NewsBar` inert; GDP/PCE coverage holes 2016–24 (fail-closed).

Why it exists. Tier-1 releases (CPI, FOMC, NFP) cause jumps a normal strategy cannot handle; the calendar blocks entries and can flatten around them.

Where. `_EventCalendar.Function.CS · _EventCalendar, EventConfig, EEventKind, EEventTier; _CustomSignal.EventConfig, EventCalendar, EventBlackout, EventFlattenNow`

Key signatures.

```
protected EventConfig EventConfig { get; }   // _CustomSignal · _CustomSignal:1071
internal _EventCalendar EventCalendar   // _CustomSignal · _CustomSignal:1078
protected bool EventBlackout { get; }   // _CustomSignal · _CustomSignal:1087
protected bool EventFlattenNow { get; }   // _CustomSignal · _CustomSignal:1089
public bool EventBlackout { get; }   // _EventCalendar · _EventCalendar:1748
public bool EventFlattenNow { get; }   // _EventCalendar · _EventCalendar:1749
```

Snippet (compiled as `_Sn20`, base `_VolumeProfileFuturesBase`).

Enables a 15-minute blackout around tier-1 events: no new entries inside it, and flatten when `EventFlattenNow` is set.

```
// Create
protected override void Create()
{
    base.Create();
    EventConfig.enabled = true;                    // lazy
    EventConfig.blackoutPreMin = 15;               // no entries 15 min before ...
    EventConfig.blackoutPostMin = 15;              // ... and 15 min after a release
}

// CalcBar
protected override void CalcBar()
{
    base.CalcBar();
    if (Bars.Status != EBarState.Close) return;
    if (EventFlattenNow)
    {
        /* flatten */
        return;
    }
    if (EventBlackout) return;                     // no new entries around tier-1 releases
}
```

Used in examples. example 2 (§3.2), example 4 (§3.11), example 24 (§3.19).

Pitfalls.

- With `allowPartialCoverage = false` (default) the calendar blocks days with unknown coverage; about 15–18 % of 2016–22 days under the Macro mask (catalogue EV-4.6b).
- Unscheduled news is not covered.

5.21 M-TICK — Seconds-level burst / trade-size stream

Status on main a07257a. ◐ Available on main for counts/timing (PR #12); trade-size outputs off until `sizeOutputsVerified`.

Why it exists. A burst of prints inside a bar signals urgency that a minute bar alone does not show.

Where. `_TickStream.Function.CS` · `_TickStream`, `TickConfig`, `TickBarStats`; `_CustomSignal.TickConfig`, `TickStats`

Key signatures.

```
protected TickConfig TickConfig { get; }   // _CustomSignal · _CustomSignal:1247
internal TickBarStats TickStats { get; }   // _CustomSignal · _CustomSignal:1254
```

Snippet (compiled as `_Sn21`, base `_VolumeProfileFuturesBase`).

Requests tick data and logs bars with a print burst (z ≥ 3), with print count and the busiest second.

```
// Create
protected override void Create()
{
    base.Create();
    TickConfig.enabled = true;                     // lazy DataLoader 1-tick Trade request for Data1's symbol
    TickConfig.burstZ = 3.0;
}

// CalcBar
protected override void CalcBar()
{
    base.CalcBar();
    if (Bars.Status != EBarState.Close) return;
    TickBarStats t = TickStats;                    // aggregate of the last closed Data1 bar
    if (t != null && t.valid && t.burst && journalOn())
```

```
        trace(ETraceLevel.Journal, $"burst z {t.burstZ:F1}, {t.prints} prints, max {t.maxPrintsPerSecond}/s")
;
}
```

Used in examples. example 37 (§3.21); inherited by example 40.

Pitfalls.

- Tick history is limited (W-T); earlier bars have `valid = false`.
- Size fields (histogram, large prints, volume) stay NaN/null until `TickConfig.sizeOutputsVerified`.

5.22 M-XMKT — Second-instrument stream

Status on main a07257a. ✓ Available on main (PR #11); VIX/ZN depend on dxFeed bundles; orders on Data1 only.

Why it exists. A related market (NQ for ES) confirms or contradicts a move; correlation, beta and relative strength quantify that.

Where. `_XmktLeg.Function.CS` · `_XmktLeg` (`Available`, `LogReturn`, `Correlation`, `Beta`, `RsZ`), `XmktLegConfig`; `_CustomSignal.XmktLegs`, `XmktLeg(role)`

Key signatures.

```
protected List<XmktLegConfig> XmktLegs { get; }   // _CustomSignal · _CustomSignal:1160
internal _XmktLeg XmktLeg(string role)   // _CustomSignal · _CustomSignal:1166
```

Snippet (compiled as `_Sn22`, base `_VolumeProfileFuturesBase`).

Adds NQ on Data2 as a cross-market leg and logs its rolling correlation, beta and relative-strength z. `rsZ` negative = NQ weak vs Data1.

```
// Create
protected override void Create()
{
    base.Create();
    XmktLegs.Add(new XmktLegConfig
    {
        role = "NQ",                                // name used by XmktLeg("NQ")
        dataStream = 2,                             // NQ on Data2
        corrLength = 60                             // bars in the rolling correlation
    });
}

// CalcBar
protected override void CalcBar()
{
    base.CalcBar();
    if (Bars.Status != EBarState.Close) return;
    _XmktLeg nq = XmktLeg("NQ");
    if (nq != null && nq.Available && journalOn())
        trace(ETraceLevel.Journal, $"rho {nq.Correlation:F2} beta {nq.Beta:F2} rsZ {nq.RsZ:F2}");
}
```

Used in examples. example 39 (§3.40).

Pitfalls.

- Check `Available` (staleness) before reading values.
- The correlation is of bar returns, not of strategy P&L (see Ex39).

5.23 M-PORT — Portfolio-level sizing/throttles

Status on main a07257a. ◐ Coded on main (PRs #14, #17, #22); Probe P5b live checks (L1–L9) not yet run.

Why it exists. Several strategies in Portfolio Trader share one risk budget; publishing each strategy's state lets one money manager size the whole book.

Where. `_CustomSignal.Strategy.CS` · `PortfolioPublish`, `PortfolioFamily`, `PortfolioRegime`, `PortfolioScore`, `PortfolioHalted`, `PortfolioCluster` (virtuals), `portfolioArm`, `portfolioStop`; `_PortMM.Function.CS` · `_PortMM`, `PortMmConfig`; `_PortfolioRiskManager.Strategy.CS`

Key signatures.

```
protected virtual bool PortfolioPublish { get; }   // _CustomSignal · _CustomSignal:1513
protected virtual string PortfolioFamily { get; }   // _CustomSignal · _CustomSignal:1515
protected virtual bool PortfolioHalted { get; }   // _CustomSignal · _CustomSignal:1530
protected void portfolioArm(int side, double entry, double stop, int qtyCode)   // _CustomSignal ·
_CustomSignal:1579
protected void portfolioStop(double stop)   // _CustomSignal · _CustomSignal:1594
```

Snippet (compiled as `_Sn23`, base `_VolumeProfileFuturesBase`).

Names the strategy's family and announces each planned long entry to the portfolio money manager; outside Portfolio Trader `portfolioArm` does nothing.

```
// class member (property override)
protected override string PortfolioFamily { get { return "PS"; } }

// CalcBar
protected override void CalcBar()
{
    base.CalcBar();
    if (Bars.Status != EBarState.Close) return;
    double entry = Bars.Close[0];
    double stop = entry - 14 * tickSize();
    portfolioArm(+1, entry, stop, 1);              // declare the pending entry (no-op outside Portfolio
Trader)
    /* m_le.Send(1); while in position: portfolioStop(currentStop); */
}
```

Used in examples. example 37 (§3.21), example 40 (§3.22).

Pitfalls.

- Publishing is on only in Portfolio Trader; chart runs publish nothing.
- The book-level daily kill and flatten live in `_PortfolioRiskManager` (`PortMmConfig.dailyLossStop`, `dailyLossFlatten`), not in the strategy.

5.24 M-COST — Style-dependent slippage, p90 stop slippage, haircut

Status on main a07257a. ◐ Not a module: MC native Commission/Slippage + the optional cost filter; style, p90 and haircut are not coded.

Why it exists. Short-stop trades can lose most of their edge to commission and slippage; the filter skips trades whose round-trip cost is too large a share of R.

Where. `_CustomSignal.Strategy.CS` · `CostFilterMaxFractionOfR`, `roundTripCostPerContract()` (virtual), `costFilterPasses`

Key signatures.

```
protected double CostFilterMaxFractionOfR { get; set; }   // _CustomSignal · _CustomSignal:2846
protected virtual double roundTripCostPerContract()   // _CustomSignal · _CustomSignal:2852
```

```
protected bool costFilterPasses(IInstrument bars, double entry, double stop, out string reason)   //
_CustomSignal · _CustomSignal:2858
```

Snippet (compiled as `_Sn24`, base `_VolumeProfileFuturesBase`).

Skips any trade whose round-trip cost exceeds 0.10 R. With ES costs ($17.00 round trip) a 14-tick stop ($175) costs ≈ 0.097 R and passes.

```
// Create
protected override void Create()
{
    base.Create();
    CostFilterMaxFractionOfR = 0.10;              // illustrative: skip if the round trip costs > 0.10 R
}

// CalcBar
protected override void CalcBar()
{
    base.CalcBar();
    if (Bars.Status != EBarState.Close) return;
    double entry = Bars.Close[0];
    double stop = entry - 14 * tickSize();         // 14-tick stop
    string why;
    if (!costFilterPasses(Bars, entry, stop, out why)) return;   // why = cost and R, traced
}
```

Used in examples. none.

Pitfalls.

- The filter reads MC's Commission and Slippage settings ($ per contract per side); set them in the strategy properties to $4.50 commission and 0.5 tick slippage so the ES round trip is $17.00.
- `CostFilterMaxFractionOfR` only skips; it never widens a stop. At 0.10 with a 14-tick stop the trade passes: $17.00 / (14 × $12.50 = $175) = 0.097 R ≤ 0.10. A stop of 13 ticks ($162.50, 0.105 R) would be skipped.
- **The floor is example code, not library behaviour.** The stop floor (stop = max(structural, 14 ticks); skip if q × widened stop × $50 > 0.01 × equity; never shrink q) is Ettienne's decided rule (2026-10-23), implemented **per strategy**: main (a07257a) has no stop floor or widening, and each example implements it locally (`costFloorTicks = 14`, `riskBudgetFraction = 0.01`, a local `overRiskBudget` or an inline check using `equityAvailable(fraction)`). The library part is **pending**: a library `CostConfig` is only specified in `instrument-profile-spec.md` IP-3.2/IP-8.1 (not built). The only library cost rule is `CostFilterMaxFractionOfR` (default 0 = off), which skips and never widens. Consequence: a strategy that sizes at exactly 1% risk skips every stop under 14 ticks (any widening breaches the budget); the floor widens without a skip only for fixed size or sizing below 1%. The examples trade a fixed 1 contract, so a 14-tick stop ($175) is skipped only when equity < $17,500.

5.25 M-GMM — Bootstrap POC, KDE sweep, GMM/BIC, entropy, KL

Status on main a07257a. ✓ Available on main (PRs #18, #21, #24).

Why it exists. Whether a POC is stable and a profile one- or two-humped decides whether its levels can be trusted.

Where. `_ProfileStats.Function.CS` · `_ProfileStats` (`BootstrapPoc`, `BootstrapSeed`, `KdeSweep`, `Entropy`, `Histogram`, `ProfileMixture`, `MixtureSeed`, `MixtureLevels`), `_GmmProfileTracker`, `BootstrapResult`, `KdeResult`, `ProfileMixtureResult`; `_Gmm.Function.CS`; host `ProfileStats`, `ProfileEntropy`, `SessionBars`

Key signatures.

```
public EntropyResult ProfileEntropy(VpExtra e, double mmN)   // _VolumeProfileBase · _VolumeProfileBase:3458
public List<RingBar> SessionBars(DateTime tradingDay, bool rthOnly)   // _VolumeProfileBase ·
_VolumeProfileBase:3468
```

```
public static ulong BootstrapSeed(DateTime tradingDay, int block, int draws)   // _ProfileStats · _ProfileStats:495
public static BootstrapResult BootstrapPoc(IList<RingBar> bars, double tick, int distribution, int block, int draws, ulong seed)   // _ProfileStats · _ProfileStats:503
public static KdeResult KdeSweep(IList<RingBar> bars, double tick, int distribution, double mcPoc = double.NaN, double medFloorTicks = 1, int folds = 5, ECvFolding folding = ECvFolding.Contiguous)   // _ProfileStats · _ProfileStats:559
public static ulong MixtureSeed(DateTime tradingDay)   // _ProfileStats · _ProfileStats:724
public static ProfileMixtureResult ProfileMixture(double[] prices, double[] vols, int kMax, ulong seed, EGmmNeff neff, int bars, double tick, GmmFitConfig cfg = null)   // _ProfileStats · _ProfileStats:742
```

Snippet (compiled as `_Sn25`, base `_VolumeProfileFuturesBase`).

Enables profile stats, takes yesterday's RTH bars and logs POC stability (bootstrap) and the KDE verdict (`Bimodal` = two humps, `Stable`, `Flat`). How `KdeSweep` decides (`_ProfileStats` L594–609): it refits the KDE POC over a sweep of bandwidths and measures the POC jumps between neighbouring bandwidths, with the median jump floored at `medFloorTicks` ticks. σ = 0 (one traded row) → `Stable`; largest jump > 5 × median → `Bimodal`; largest jump < 2 × median **and** POC range < 0.15σ → `Stable`; otherwise `Flat`.

```
// Create
protected override void Create()
{
    base.Create();
    ProfileStats.enabled = true;                    // lazy: Data1 ring (ringBars) + range profiles
}

// CalcBar
protected override void CalcBar()
{
    base.CalcBar();
    if (Bars.Status != EBarState.Close) return;
    DateTime day = Bars.Time[0].Date.AddDays(-1);  // illustrative: use the real prior trading day (see Ex35)
    List<RingBar> bars = SessionBars(day, true);
    if (bars.Count < 20) return;
    double tick = tickSize();
    BootstrapResult b = _ProfileStats.BootstrapPoc(bars, tick, 2, 5, 500,
        _ProfileStats.BootstrapSeed(day, 5, 500));   // deterministic seed per day
    KdeResult k = _ProfileStats.KdeSweep(bars, tick, 2);
    if (journalOn())
        trace(ETraceLevel.Journal, $"POC stability {b.stability:F2} ci90 {b.ci90Ticks}t KDE {k.verdict}");
}
```

Used in examples. example 8 (§3.24), example 27 (§3.27), example 35 (§3.32), example 36 (§3.39).

Pitfalls.

- Use the deterministic seeds (`BootstrapSeed`, `MixtureSeed`) so reruns give the same draws.
- `SessionBars` is empty unless `ProfileStats.enabled` (or `RangeProfiles.enabled`) and the ring holds that day.
- Compute session statistics once per day; they are not cheap.

5.26 M-GEX — External daily GEX file

Status on main a07257a. ✗ Deferred: no code on main (data-blocked).

Where. —

Snippet. None: there is no code on main.

Used in examples. none.

Pitfalls.

- No API exists; no example uses it.

5.27 VP helpers — Range profile, weekly profiles, per-bar ladders, entropy/Parkinson/V(p)/T(p), FVG detector (VP-1…VP-6)

Status on main a07257a. ✓ Available on main (PR #18).

Why it exists. Profiles over any window (opening hour, last N bars, last week) and fair-value gaps supply levels beyond the daily session profile.

Where. `_RangeProfile.Function.CS` (`RangeProfileConfig`; host `RangeProfiles`, `RangeProfile`, `RangeProfileAfter`, `RangeProfileBars`, `OpenRange`, `CloseRange`); `_WeeklyProfiles.Function.CS` (`WeeklyProfiles`, `Week`, `WeeklyComposite`); `_BarLadder.Function.CS` (`BarLadders`, `Ladder`, `HtfLadder`, `LadderBar`); `_FvgDetector.Function.CS` (`Fvgs`, `FvgDetector`, `Fvg`, `fvgLevel`); `_BarRing.Function.CS` (`RingBar`, `ringBarOf`); `_LevelRef.Function.CS` (`LevelRef`, `ELevelKind`)

Key signatures.

```
protected FvgConfig Fvgs { get; }   // _CustomSignal · _CustomSignal:1390
protected RingBar ringBarOf(int ds, int ago = 0)   // _CustomSignal · _CustomSignal:1436
protected LevelRef fvgLevel(Fvg f, bool startEdge)   // _CustomSignal · _CustomSignal:1502
protected RangeProfileConfig RangeProfiles { get; }   // _VolumeProfileBase · _VolumeProfileBase:2982
protected WeeklyProfileConfig WeeklyProfiles { get; }   // _VolumeProfileBase · _VolumeProfileBase:2984
protected BarLadderConfig BarLadders { get; }   // _VolumeProfileBase · _VolumeProfileBase:2988
public VpExtra RangeProfileBars(int stream, int barsBack, int count)   // _VolumeProfileBase ·
_VolumeProfileBase:3126
public RangeProfileHandle OpenRange(int stream, DateTime t0, string tag)   // _VolumeProfileBase ·
_VolumeProfileBase:3136
public VpExtra Week(int weeksAgo)   // _VolumeProfileBase · _VolumeProfileBase:3317
public LadderBar Ladder(int ago)   // _VolumeProfileBase · _VolumeProfileBase:3391
```

Snippet (compiled as `_Sn27`, base `_VolumeProfileFuturesBase`).

Enables range, weekly and FVG helpers and logs the POC of the last 20 bars and last week's value area.

```
// Create
protected override void Create()
{
    base.Create();
    RangeProfiles.enabled = true;                  // VP-1 (ring on Data1)
    WeeklyProfiles.enabled = true;                 // VP-2
    Fvgs.enabled = true;                           // VP-5
    Fvgs.minAtrFrac = 0.25;                        // illustrative
}

// CalcBar
protected override void CalcBar()
{
    base.CalcBar();
    if (Bars.Status != EBarState.Close) return;
    VpExtra last20 = RangeProfileBars(1, 0, 20);   // profile of the last 20 closed Data1 bars
    VpExtra lastWeek = Week(1);
    if (last20 != null && lastWeek != null && journalOn())
        trace(ETraceLevel.Journal, $"20-bar POC {last20.poc} last-week VA {lastWeek.val}-{lastWeek.vah}");
}
```

Used in examples. example 24 (§3.19), example 8 (§3.24), example 35 (§3.32), example 7 (§3.33), example 10 (§3.34), example 33 (§3.37), example 34 (§3.38), example 36 (§3.39).

Pitfalls.

- Every helper is lazy and needs its `enabled` flag in `Create()`; the ring length (`ringBars`) bounds how far back range profiles reach.
- Enabling `BarLadders` turns `OrderFlowEnabled` on; the BarUpDown tier spreads volume uniformly, so ladder cluster logic must require `HasBidAsk`.
- Range and weekly profiles can cross a roll (`crossesRoll`); the roll policy decides whether they reject, split or reset.

5.28 A2-33 — Matched-control level study; Kaplan–Meier / Cox level death

Status on main a07257a. ◐ Available except Cox (PRs #15, #16; Gate-0 verdict PR #20): matched controls, paired edge, Holm, terciles, half-life, Kaplan–Meier; Cox regression not implemented. DVP levels feed the study through the M-DVP D7 hook (PR #42, §5.31): with DVP and `LevelStudyConfig` both on, the G0-d DVP rails and confluence levels are added once per session. `_LevelStudy` itself is unchanged.

Why it exists. Tests whether a level type gets more reactions than matched random prices before you trade it.

Where. `_LevelStudy.Function.CS` · `_LevelStudy`, `LevelStudyConfig`, `LevelTypeResult`; `_CustomSignal.LevelStudyConfig`, `levelStudyAdd`

Key signatures.

```
protected LevelStudyConfig LevelStudyConfig { get; }   // _CustomSignal · _CustomSignal:1328
protected bool levelStudyAdd(string type, double price, int ageSessions, double quality = double.NaN, string
key = null)   // _CustomSignal · _CustomSignal:1359
```

Snippet (compiled as `_Sn28`, base `_VolumeProfileFuturesBase`).

Registers yesterday's POC with the level study at the RTH open; results appear at the end of the run.

```
// Create
protected override void Create()
{
    base.Create();
    LevelStudyConfig.enabled = true;              // lazy; harness steps inside base.CalcBar()
}

// CalcBar
protected override void CalcBar()
{
    base.CalcBar();                               // add levels AFTER base.CalcBar(), on the first bar of
the session
    if (Bars.Status != EBarState.Close) return;
    VpExtra p = PriorRthProfile();
    bool rthOpen = ToEastern(Bars.Time[0]).TimeOfDay == new TimeSpan(9, 30, 0);   // first bar of the session
    if (p != null && rthOpen)
        levelStudyAdd("POC", p.poc, 1);           // prior POC, 1 session old
}
```

Used in examples. example 41 (§3.41).

Pitfalls.

- The reference price of a level is the close of the bar `levelStudyAdd` is called on; call it once per level per session.
- No Cox model: use the Kaplan–Meier output and age buckets.

5.29 Gate0 — Gate-0 measurement: level families vs seeded placebos; G0-b canaries

Status on main a07257a. ✓ Available on main (PR #20); not yet run in MultiCharts.

Why it exists. Gate 0 measures level families against seeded placebos under a frozen, pre-registered protocol.

Where. `_Gate0.Function.CS` · `Gate0Engine`, `EGate0Family`; `_TestGate0.Strategy.CS` (host); see §2

Key signatures.

```
public enum EGate0Family { All = 0, PriorProfile = 1, NakedPoc = 2, IbEdge = 3, Overnight = 4, Avwap = 5,
Vwap = 6, Weekly = 7, CompositeVa = 8, SinglePrint = 9, DvSource = 10, HvnLvn = 11, Mixture = 12, SigmaMixtur
```

```
e = 13 }   // _Gate0 · _Gate0:25
public sealed class Gate0Engine   // _Gate0 · _Gate0:273
public static string PreRegistration()   // Gate0Engine · _Gate0:290
```

Snippet (compiled as `_Sn29`, base `_VolumeProfileFuturesBase`).

Writes Gate 0's frozen pre-registration text to the journal at start.

```
// StartCalc (after base.StartCalc)
protected override void StartCalc()
{
    base.StartCalc();
    if (journalOn())
        trace(ETraceLevel.Journal, Gate0Engine.PreRegistration());   // the frozen pre-registration text
}
```

Used in examples. none.

Pitfalls.

- Run Gate 0 through `_TestGate0` as described in §2; the engine is not meant to be hosted ad hoc inside a trading strategy.
- Examples trading G0-d-gated level types must wait for their family's verdict (§2.6).

5.30 _RegimeEngine — Market regime (G-R): HMM K = 3, walk-forward, hysteresis; AtrTercile fallback

Status on main a07257a. ◐ Partial, code complete (PR #23); consumers need pre-registration, consumer C1 has not run, and the HMM must beat RG-03. `_RegimeGate` (PR #26) has the A2-63 arm switch.

Why it exists. Trend, range and high-volatility regimes need different rules; the regime label lets one strategy switch or stand aside.

Where. `_RegimeEngine.Function.CS · _RegimeEngine, RegimeConfig, RegimeState; _CustomSignal.RegimeConfig, Regime; _RegimeGate.Function.CS · RegimeGate, RegimeGateConfig, ERegimeGateArm`

Key signatures.

```
protected RegimeConfig RegimeConfig { get; }   // _CustomSignal · _CustomSignal:1019
protected RegimeState Regime   // _CustomSignal · _CustomSignal:1025
public enum ERegimeGateArm { U = 0, G = 1, A = 2 }   // _RegimeGate · _RegimeGate:25
public static void ApplyArm(RegimeGateConfig g, RegimeConfig r)   // RegimeGate · _RegimeGate:117
```

Snippet (compiled as `_Sn30`, base `_VolumeProfileFuturesBase`).

Enables the regime engine and logs the regime label (0 = calmest) and whether the ATR-tercile fallback is in use.

```
// Create
protected override void Create()
{
    base.Create();
    RegimeConfig.enabled = true;                    // lazy; needs the daily stream (M-DAILY)
}

// CalcBar
protected override void CalcBar()
{
    base.CalcBar();
    if (Bars.Status != EBarState.Close) return;
    RegimeState s = Regime;
    if (s != null && s.Valid && journalOn())
```

```
        trace(ETraceLevel.Journal, $"regime {s.Label} fallback {s.UsingFallback}");
}
```

Used in examples. example 9 (§3.13).

Pitfalls.

- The state is valid only after `minSessions` daily bars; before that `Valid = false`.
- Regime consumers must be pre-registered; do not pick the regime filter after looking at results.

5.31 M-DVP — Detrended volume-profile engine (Keller B5)

Status on main a07257a. ◐ Partially on main a07257a. The spec is m-dvp-spec rev 5.3; main implements the rev 5–5.3 amendments through PR-D0a–D7 (the rev 5.2 rules: `HtfAccepted` with explicit `countTrigger`/`deltaAtr` and N ≥ 2, `NextRailDistAtr` strictly beyond or NaN, `Tradeable` never reads `diagnosticOnly`; the rev 5.3 rules: the per-session `DVP A2-33` Journal line joined by `tradingDayKey`, levels first evaluated on the bar after creation):

- the fit core (D1) and the manifold and rails (D2);
- the Data1 host wiring with the `DVP CREATED/FIT/RUN` journal and cov_fwd (D3, PR #36), and the Fib lattice with the PA bridge (D4, PR #37);
- **D5 (PRs #39, #40):** confluence (`dvpConfluence`, `dvpFamiliesAt`, `DvpConf` rails), acceptance (`dvpTriggered`/`dvpRunBeyond`/`dvpBackInside`/`dvpAccepted`), the 8-bar rail history, the `DVP RAIL` journal and `dvpZClose`;
- **D6 (PR #41):** Data2/Data3 HTF streams of the same root, the Probe P2 gate (`dvpTradeable`), `dvpHtfAccepted` and `dvpHtfNextRailDistAtr`.
- **D7 (PR #42):** the A2-33 hook. With `LevelStudyConfig.enabled` and DVP on, the first closed Data1 bar of each new `tradingDayKey` sends every G0-d level (`DvpPoc/Vah/Val/Hvn/Lvn/Sigma/Conf`) of `dvpRails(1)` and `dvpConfluence(1)` to `levelStudyAdd` (age 1, the rail's quality and tag). Roll-crossing and roll-window levels are skipped, and `DvpFib` is counted but not added. Each level is first evaluated on the next bar. At Journal level one `DVP A2-33 runId() tradingDayKey(yyyy-MM-dd) halfDay() rails() conf() rollSkipped() fibSkipped()` line per session joins the LevelStudy CSV by day.

Pending: D8 the KD02 canary and Calibrate study; D9 the frozen d* (`DvpFrozenDegree.Table` is empty, so `DevCoverageFrozen` falls back to Calibrate with reported d = 1 and one `DVP DEGREE not frozen` warning); `rollMode` (DV-7.0, waits for the instrument profile); Probe P2 itself (`DvpConst.P2_PASSED = false`, so HTF streams are not tradeable). The host still has no `NextRail(ds, …)` or `Cone(ds, …)` wrapper. The catalogue row still says "not yet coded".

Why it exists. A profile fitted around a trend line (detrended) gives value rails that move with the trend instead of a flat VA.

Where. `_CustomSignal.Strategy.CS`:

- **Configuration and engine:** `DvpConfig` (protected property, L397), `Dvp` (internal, L405), `rthKnown()` (L415, overridden to true in `_VolumeProfileFuturesBase` L515).
- **Fit and rails:** `dvpSnapshot` (L425), `dvpRails` (L434), `dvpRailPrice` (L610), `dvpZ` (L619).
- **D5 confluence and acceptance:** `dvpConfluence` (L485), `dvpFamiliesAt` (L499/L510), `dvpTriggered` (L526), `dvpRunBeyond` (L528), `dvpBackInside` (L530), `dvpAccepted` (L533), `dvpZClose` (L539).
- **D6 HTF:** `dvpTradeable` (L582), `dvpHtfAccepted` (L588), `dvpHtfNextRailDistAtr` (L597).
- **D7 A2-33 hook (private):** `dvpLevelStudyStep` (L467, called from `CalcBar` L2100 after `stepLevelStudy`).
- **Private wiring:** `startDvp` (L665, called from `startEngines` L1842), `stepDvp` (L698, from `stepEngines` L1906, after the rings), `dvpRailStep` (L551, `DVP RAIL`, from `stepEngines` L1914 after every PA step), and `m_dvp.Finish()` (L2101, `DVP RUN` lines on the last historical bar).

`_DvpEngine.Function.CS`:

- `DvpConfig`, `DvpFit`, `DvpManifold`, `DvpRails`, `DvpSnapshot`, `DvpStream`, `_DvpEngine` (incl. `Tradeable` L1249).
- The D4 classes `DvpPivots`, `DvpLattice` and `DvpPaBridge.Fill` (internal).
- The D5 classes `DvpConfluence` (L1513), `DvpAccept` (L1665), `DvpRailHistory` (L1728) and `DvpRailJournal` (L1842).
- The D6 class `DvpHtf` (L1983).
- The D7 class `DvpA233` (L1947: `Line`, `IsG0dType`, `Sample`).

`_LevelRef.Function.CS`: `ELevelKind.DvpPoc…DvpConf`.

Key signatures.

```
protected DvpConfig DvpConfig { get; }   // _CustomSignal · _CustomSignal:397
internal _DvpEngine Dvp   // _CustomSignal · _CustomSignal:405
protected virtual bool rthKnown()   // _CustomSignal · _CustomSignal:415
protected DvpSnapshot dvpSnapshot(int ds)   // _CustomSignal · _CustomSignal:425
protected IReadOnlyList<LevelRef> dvpRails(int ds)   // _CustomSignal · _CustomSignal:434
protected double dvpRailPrice(int ds, ELevelKind kind, double z)   // _CustomSignal · _CustomSignal:610
protected double dvpZ(int ds, double price)   // _CustomSignal · _CustomSignal:619
protected IReadOnlyList<LevelRef> dvpConfluence(int ds)   // _CustomSignal · _CustomSignal:485
protected void dvpFamiliesAt(int ds, double price, double tolAtr, out int families, out int nRanges)   //
_CustomSignal · _CustomSignal:499
protected bool dvpTriggered(int ds, LevelRef rl, int dir)   // _CustomSignal · _CustomSignal:526
protected int dvpRunBeyond(int ds, LevelRef rl, int dir, double deltaAtr)   // _CustomSignal ·
_CustomSignal:528
protected bool dvpBackInside(int ds, LevelRef rl, int dir)   // _CustomSignal · _CustomSignal:530
protected bool dvpAccepted(int ds, LevelRef rl, int dir, int N, bool countTrigger, double deltaAtr)   //
_CustomSignal · _CustomSignal:533
protected double dvpZClose(int ds, int k)   // _CustomSignal · _CustomSignal:539
protected bool dvpTradeable(int ds)   // _CustomSignal · _CustomSignal:582
protected bool dvpHtfAccepted(int ds, LevelRef rl, int dir, int N, bool countTrigger, double deltaAtr)   //
_CustomSignal · _CustomSignal:588
protected double dvpHtfNextRailDistAtr(int ds, int dir)   // _CustomSignal · _CustomSignal:597
protected override bool rthKnown()   // _VolumeProfileFuturesBase · _VolumeProfileFuturesBase:515
public sealed class DvpConfig   // _DvpEngine · _DvpEngine:42
public sealed class DvpFit   // _DvpEngine · _DvpEngine:208
public bool Fit(double[] y, double[] v, int n, int degMax, bool weightVol)   // DvpFit · _DvpEngine:243
public sealed class DvpRails   // _DvpEngine · _DvpEngine:680
public double Z(double price)   // DvpRails · _DvpEngine:727
public double RailPrice(ELevelKind kind, double z)   // DvpRails · _DvpEngine:730
public int NextRail(double fromZ, int dir)   // DvpRails · _DvpEngine:739
public sealed class DvpSnapshot   // _DvpEngine · _DvpEngine:780
public int ds, endBar, anchorBar, sinceRefit, n, deg, dCap, manRowsEff;   // DvpSnapshot · _DvpEngine:782
public DateTime tPrev, t0;   // DvpSnapshot · _DvpEngine:783
public bool valid, crossesRoll, lowR2, refitBar, calibrating, manValid, diagnosticOnly;   // DvpSnapshot ·
_DvpEngine:784
public double atr, sigma, sigmaAtr, r2, dw, skew, yhatEnd, slopeEnd, slope1, slope1SigmaPerBar,
endpointDrift, uT, yhatT,   // DvpSnapshot · _DvpEngine:785
public sealed class DvpPivots   // _DvpEngine · _DvpEngine:1361
public sealed class DvpLattice   // _DvpEngine · _DvpEngine:1385
public static int Strength(DvpConfig c, int j)   // DvpLattice · _DvpEngine:1414
public static bool Anchor(DvpPivots p, int endBar, int look, out double a, out double b, out int aBar, out in
t bBar)   // DvpLattice · _DvpEngine:1417
public void Build(DvpConfig c, DvpPivots[] pivots, int endBar, double atr, double close)   // DvpLattice ·
_DvpEngine:1441
public const bool P2_PASSED = false;   // DvpConst · _DvpEngine:38
public bool Tradeable(int ds)   // _DvpEngine · _DvpEngine:1249
public sealed class DvpConfluence   // _DvpEngine · _DvpEngine:1513
public bool Qualifies(int cl)   // DvpConfluence · _DvpEngine:1612
public static class DvpAccept   // _DvpEngine · _DvpEngine:1665
public const double Delta = 0.10;   // DvpAccept · _DvpEngine:1667
public const int MaxN = 6;   // DvpAccept · _DvpEngine:1668
public static bool Accepted(double[] c, double[] p, double[] a, int len, int dir, int N, bool countTrigger, d
ouble deltaAtr)   // DvpAccept · _DvpEngine:1711
public sealed class DvpRailHistory   // _DvpEngine · _DvpEngine:1728
public static class DvpHtf   // _DvpEngine · _DvpEngine:1983
public static double NextRailDistAtr(DvpRails rails, DvpLattice lat, double c, int dir, double atr)   //
DvpHtf · _DvpEngine:2010
internal static class DvpPaBridge   // _DvpEngine · _DvpEngine:2033
```

```
public static void Fill(_PriceActionEngine pa, int barOffset, int endBar, DvpPivots into)   // DvpPaBridge · _DvpEngine:2035
```

Snippet (compiled as `_Sn31`, base `_VolumeProfileFuturesBase`).

Enables DVP with Fibonacci rails and logs the fit quality, the close's z-distance from the trend-adjusted value, the VAH and +2σ rails, and the rail count.

```
// Create: configuration only. DvpConfig is a protected get-only property; set its fields, never an [Input].
protected override void Create()
{
    base.Create();
    DvpConfig.enabled = true;                       // lazy: engine, ring and rail lists exist only when enabled
    DvpConfig.useFib =
true;                        // D4 lattice: hosts fibRanges PA pivot engines (strengths 20/40/80)
}
// StartCalc: nothing to do. base.StartCalc() -> startEngines() -> startDvp() validates DvpConfig and builds the engine.

// CalcBar: query after base.CalcBar() (stepDvp has run); the fit is as of t-1 for bar t (no lookahead).
protected override void CalcBar()
{
    base.CalcBar();
    if (Bars.Status != EBarState.Close) return;
    DvpSnapshot s = dvpSnapshot(1);                 // null: disabled, warm-up, or a window that crosses a roll
    if (s == null || !s.valid) return;
    double z = dvpZ(1, Bars.Close[0]);              // (price - yhat(u_t)) / sigma
    double vah = dvpRailPrice(1, ELevelKind.DvpVah, 0);     // NaN if absent
    double up2 = dvpRailPrice(1, ELevelKind.DvpSigma, 2);   // +2 sigma rail, NaN if absent
    IReadOnlyList<LevelRef> rails = dvpRails(1);   // reused list: copy what you keep
    int fib = 0;
    for (int i = 0; i < rails.Count; i++)
    {
        if (rails[i].kind == ELevelKind.DvpFib) fib++;
    }
    if (journalOn())
        trace(ETraceLevel.Journal, $"dvp deg {s.deg} r2 {s.r2:F2} z {z:F2} vah {vah} +2s {up2} rails {rails.Count} fib {fib}");
}
```

Used in examples. example 42 (§3.42).

Pitfalls.

- **Streams.** `DvpConfig.streams` may list Data1 and the HTF streams Data2/Data3. Each must exist (`requireDataStream`) and carry the Data1 root, or `StartCalc` throws `ArgumentException` (`DvpHtf.RootError`, DV-7.0).
- **HTF streams are not tradeable yet.** `dvpTradeable(ds)` is false for ds ≥ 2 (one `used before Probe P2` warning) until `DvpConst.P2_PASSED`, which is `false` on main. MT-07 trades the 60m stream only; 240m is diagnostic by consumer rule (DV-7.5). `Tradeable` does not hard-code a timeframe.
- **When to set the config.** Set `DvpConfig` fields in `Create()`, or in `StartCalc()` before `base.StartCalc()`. `startDvp` validates once in `StartCalc` and throws `ArgumentException` on a bad config.
- **`fibAnchor`.** `fibAnchor = ExtremeInLookback` is rejected: `Validate()` reports "fibAnchor ... not supported (MostRecentPivot only, spec rev 5 §2)" (`_DvpEngine` L107, PR #39 prelude 334dc0b). Leave it at `MostRecentPivot`.
- **When to query.** Query only after `base.CalcBar()`. The host picks the window end itself (`dvpEndAgo` → `DvpHtf.EndAgo`), so a snapshot on bar t is the fit as of t−1. On an HTF stream only closed HTF bars are read; a forming HTF bar never reaches a snapshot.
- **Reused lists.** `dvpRails` and `dvpConfluence` return lists (and `LevelRef` objects) reused by the next call. Copy anything you keep across bars. `LevelRef.quality` on a rail is the number of families in its cluster (DV-6.5).

- **Rail lookup.** `dvpRailPrice` matches z exactly for `DvpSigma`/`DvpHvn`/`DvpLvn`/`DvpFib`; for `DvpPoc`/`DvpVah`/`DvpVal` z is ignored. Acceptance history matches HVN/LVN to the nearest same-kind rail within |dz| ≤ 0.25 (`DvpRailHistory.NodeZTol`).
- **Confluence.** A `DvpConf` level is a cluster with at least `DvpConst.MinFamilies` = 3 families (manifold, sigma, lattice) within `confTol` (0.30 ATR) (`DvpConfluence.Qualifies`).
- **Acceptance semantics.** `dvpAccepted(ds, rl, dir, N, countTrigger, deltaAtr)` is true on the completing bar.
- Trigger: a close beyond its own rail by more than `DvpAccept.Delta` = 0.10 ATR, where the bar before closed not beyond (`Triggered`).
- countTrigger = true: the trigger close **plus N further closes**, all beyond the rail by more than deltaAtr·ATR. False: N closes in total, including the trigger.
- N must be 1..6 (`MaxN`), else `ArgumentOutOfRangeException`. There is no default convention (DV-8.3), so pass both explicitly. PS-25 uses N = 2, countTrigger = true, deltaAtr = 0. Per the spec, KD-06/KD-10 pass countTrigger = false.
- It returns false for `DvpFib` and `DvpConf` levels, which have no rail history (DV-10.4).
- **HTF acceptance.** `dvpHtfAccepted` is the same test on an HTF stream's closed bars: it needs ds ≥ 2 and N in 2..6, else it throws. `dvpHtfNextRailDistAtr(ds, dir)` is the distance from Data1's C(t−1) to the nearest HTF rail or lattice level strictly beyond it, in the HTF ATR. It returns NaN when there is no fit or no such rail.
- **d* is not frozen (D9).** With the default `DevCoverageFrozen`, the reported degree is 1 and `s.calibrating` is true. Do not read results by degree until PR-D9.
- **No trade verdict yet.** Since D7 the A2-33 study can score DVP level types (G0-d), but no DVP type has a verdict yet, and nothing gates trades on one. The rails, confluence and acceptance are research levels and primitives, not validated trade triggers. The A2-33 `halfDay` split needs Journal trace level (the `DVP A2-33` line is written only then). The `DVP RAIL` journal logs interactions to `logSignal` with `taken: false`.
- **Disabled engine.** Calling `Dvp` while it is disabled returns null and logs one Warning; the host queries then return null, empty lists, NaN or false. The direct `DvpFit` API (fit core) is still public for offline sketches.
- **`diagnosticOnly` never gates `Tradeable` (by design).** Rev 5.2 (DV-7.0, DV-7.4) makes `diagnosticOnly` a run-level verdict, reported in the `DVP RUN` line only; a stream stays tradeable under rollMode None. Do not expect `dvpTradeable` to turn false on it (`_DvpEngine` L1249).
- **Still pending (m-dvp-spec rev 5.3; not on main a07257a).**
- D8 the KD02 canary and Calibrate study, and D9 the frozen d*.
- Probe P2 (which flips `P2_PASSED`) and `rollMode` (DV-7.0) are also pending. `rollMode` None waits for the instrument profile.
- There is no winLen sweep, and no 240m stream for trading (DV-1.1, DV-7.5).
- The DVP-hosted PA engines use `minBreakoutLayer = 0` (`_CustomSignal` L647), unlike the main engine's default 1 (§4.8).

Appendices

A. `ETraceLevel`

Level	Value	Enables
None	0	nothing (the default after `Create`)
Error	1	errors (for example the Event coverage failure)
Warning	2	+ one-time "engine queried but not enabled" warnings, config warnings
Info	3	+ RUN / RUN-END, `entry blocked:`, roll detected, XMKT leg, PORT publish
Journal	4	+ engine facts, `MCJournal_<runId>.csv`, `MCEvents_<runId>.csv`
Verbose	5	+ equity and account details
Debug	6	+ engine debug sinks (for example PA `trace`), sizing reasons

B. Files and formats

All files go to `%LOCALAPPDATA%\MCSignalLogs\` (fallback: `%TEMP%`).

- **Trace log** `MCSignal{yyyyMMdd-HHmmss}-{pid}.csv`, one per process. Columns: `level,yyyy-MM-dd HH:mm:ss {zone},signalName,symbol,runId,message`.
- **Trade journal** `MCJournal_<runId>.csv`, with the header: `runId,strategy,symbol,tag,side,qty,entryTime,entryPrice,exitTime,exitPrice,exitOrder,barsHeld,profit,com` plus `,evtDayTier,evtMinSince` when M-EVENT is on. The `context` column holds the `journalArm` pairs as `k=v|k=v`.
- **Signal events** `MCEvents_<runId>.csv`: `runId,strategy,symbol,tag,direction,time,bar,price,stop,riskPerUnit,taken,controlMode`, then `,mfeR_{h},maeR_{h}` for each horizon h, then `,completedBy,context` (exact header checked in `_CustomSignal`). The context is `k=v;`.
- **How lines are written (PR #49).** Every trace, journal and event line is one `File.AppendAllText` (UTF-8, no BOM) under a lock; no file handle stays open between lines. A log deleted, moved or emptied while MultiCharts runs is recreated on the next line (the journal and event CSVs get their header again when the file is missing). Write errors (e.g. the CSV open in Excel) go to the calling signal's Output window and that line is dropped; they never reach the strategy. There is nothing to close in `Destroy()`.
- **Which file is mine?** The trace log is process-wide, so its `Log file created …` line appears once, in the Output of whichever signal traced first. Each signal instance also prints its own path once to its own Output when it first traces: `{SignalClass} {Symbol}: trace log {path}`.
- **Engine line prefixes:** `DAILY`, `REGIME`, `REGGATE`, `EVENT`, `AV`, `PA{ds}[-role]`, `TOD`, `XMKT`, `TICK`, `MTF`, `RING`, `RP`, `LADDER`, `VPB-1`, `VPF-4`, `VPF-7`, `PORT`, `A2-33`.

C. Running the tests

All tests need the .NET 8 SDK. Each `tests/<Name>/run.sh` does `dotnet run -c Release` in its folder. Where a `gen.sh` exists, it generates `Generated/*.cs` from the real study sources (for example it extracts the `// <calendar-rules …>` blocks), and the build runs it itself.

```
cd MultiCharts.Net
for t in tests/*/run.sh; do echo "== $t"; bash "$t" || break; done
bash tests/RegimeTests/run.sh          # also supports the reference.py export path
bash tests/EventTests/gen.sh            # regenerate only
```

Suites: Avwap, Calendar, Daily, Event, Gate0, Gmm, LevelStudy, Mtf, OrderFlow, PortMM, PriceAction, Profile, RegimeGate, Regime, Roll, SessionClock, Tick, Tod, VpHelper, Xmkt. `gen.sh` exists in Calendar, Daily, Event, Gate0, Gmm, OrderFlow, Profile and RegimeGate.

D. Glossary

- **AsOf**: the date whose completed data produced a value. It is usable strictly after that date.
- **AVWAP**: anchored VWAP from an event bar.
- **BOS / CHoCH**: break of structure / change of character (a BOS against the prior structure).
- **Closed bar**: `Bars.Status == EBarState.Close`. Engines commit only on these.
- **Globex date**: the CME trading date. The 18:00 ET evening session belongs to the next date.
- **Half-day**: a CME early RTH close (`RthCloseEt`, 13:15 ET) or a holiday halt.
- **HTF**: a higher-timeframe stream of the same instrument (`HtfStreams`).
- **Leg**: a cross-market stream (`XmktLegs`).
- **No-RTH day**: Globex is open but there is no cash session (an NYSE holiday).
- **Placebo (P1/P2)**: an AVWAP twin with neutralised volume, used for falsification.
- **Roll splice**: the first bar of a new contract on a continuous unadjusted series.
- **RTH value window**: `(09:30, 16:00]` ET, or up to the early close.
- **Tier**: event importance (T1 > T2 > T3, C = calendar).
- **TOD**: time-of-day normalisation per session-minute bucket.

E. Public signatures by file and class (origin/main @ a07257a)

Generated by script from every `.CS` file in the tree: public and protected members, grouped by file and class, with line numbers (`/* line */`).

_CustomSignal.Strategy.CS

```
// --- (namespace)
/* 18 */ public abstract class _CustomSignal : PortfolioSignalObject
// --- _CustomSignal
/* 24 */ public const double DEFAULT_ACCOUNT_RECALC_INTERVAL = 5;
/* 25 */ public const int DEFAULT_ENTRY_EXIT_LOOKBACK = 63;
/* 28 */ public const string BUILD_TAG = "base-class-rev2";
/* 35 */ public enum ETraceLevel { None = 0, Error, Warning, Info, Journal, Verbose, Debug, }
/* 50 */ public enum EUseFill { First, Last, }
/* 57 */ [Input] public virtual ETraceLevel TraceLevel { get; set; }
/* 221 */ public int IntradayBarCount
/* 226 */ public double IntradayLow
/* 231 */ public DateTime IntradayLowTime
/* 236 */ public double IntradayHigh
/* 241 */ public DateTime IntradayHighTime
/* 247 */ public double IntradayClose
/* 252 */ public DateTime IntradayCloseTime
/* 257 */ public double IntradayOpen
/* 262 */ public DateTime IntradayOpenTime
/* 268 */ public double IntradayVolume
/* 273 */ public Function._Holidays Holidays { get; }
/* 285 */ public string RunId { get; }
/* 288 */ protected virtual string BuildTag { get; }
/* 290 */ protected ITradingData tradingData
/* 340 */ public _CustomSignal(object _ctx)
/* 349 */ protected bool traceOn(ETraceLevel level)
/* 359 */ public void trace(ETraceLevel traceLevel, string message)
/* 380 */ protected PriceActionConfig PriceActionConfig { get; }
/* 385 */ internal _PriceActionEngine PriceActionEngine
/* 397 */ protected DvpConfig DvpConfig { get; }
/* 405 */ internal _DvpEngine Dvp
/* 415 */ protected virtual bool rthKnown()
/* 425 */ protected DvpSnapshot dvpSnapshot(int ds)
/* 434 */ protected IReadOnlyList<LevelRef> dvpRails(int ds)
/* 485 */ protected IReadOnlyList<LevelRef> dvpConfluence(int ds)
/* 499 */ protected void dvpFamiliesAt(int ds, double price, double tolAtr, out int families, out int
nRanges)
/* 510 */ protected void dvpFamiliesAt(int ds, double price, out int families, out int nRanges)
```

```
/* 526 */ protected bool dvpTriggered(int ds, LevelRef rl, int dir)
/* 528 */ protected int dvpRunBeyond(int ds, LevelRef rl, int dir, double deltaAtr)
/* 530 */ protected bool dvpBackInside(int ds, LevelRef rl, int dir)
/* 533 */ protected bool dvpAccepted(int ds, LevelRef rl, int dir, int N, bool countTrigger, double deltaAtr)
/* 539 */ protected double dvpZClose(int ds, int k)
/* 582 */ protected bool dvpTradeable(int ds)
/* 588 */ protected bool dvpHtfAccepted(int ds, LevelRef rl, int dir, int N, bool countTrigger, double deltaA
tr)
/* 597 */ protected double dvpHtfNextRailDistAtr(int ds, int dir)
/* 610 */ protected double dvpRailPrice(int ds, ELevelKind kind, double z)
/* 619 */ protected double dvpZ(int ds, double price)
/* 724 */ protected List<PriceActionConfig> ExtraPriceAction { get; }
/* 731 */ protected virtual void presetExtraPriceAction(PriceActionConfig c)
/* 735 */ internal _PriceActionEngine PriceActionFor(int dataStream, string role = null)
/* 817 */ protected virtual bool priceActionRequired()
/* 820 */ protected EAvwapAnchor AvwapEvents { get; set; }
/* 822 */ protected AvwapLimits AvwapLimits { get; }
/* 828 */ internal _AvwapManager AvwapManager
/* 838 */ protected virtual bool isRthBar(DateTime barTime)
/* 843 */ protected ETodMetric TodMetrics { get; set; }
/* 845 */ protected int TodSessions { get; set; }
/* 847 */ protected int TodMinSessions { get; set; }
/* 849 */ protected int TodBucketMinutes { get; set; }
/* 855 */ internal _TodNormaliser TodNormaliser
/* 866 */ protected virtual int todMinuteOfSession(DateTime barTime)
/* 868 */ protected virtual bool todHalfDay(DateTime sessionDate)
/* 871 */ protected virtual double todDelta(out int tier)
/* 908 */ protected bool NeedsDaily { get; set; }
/* 910 */ protected DailyConfig DailyConfig { get; }
/* 920 */ internal _DailyContext DailyContext
/* 930 */ protected double DailyGapPct { get; }
/* 1019 */ protected RegimeConfig RegimeConfig { get; }
/* 1025 */ protected RegimeState Regime
/* 1071 */ protected EventConfig EventConfig { get; }
/* 1078 */ internal _EventCalendar EventCalendar
/* 1087 */ protected bool EventBlackout { get; }
/* 1089 */ protected bool EventFlattenNow { get; }
/* 1160 */ protected List<XmktLegConfig> XmktLegs { get; }
/* 1166 */ internal _XmktLeg XmktLeg(string role)
/* 1174 */ internal _PriceActionEngine XmktLeaderSwing(string role)
/* 1247 */ protected TickConfig TickConfig { get; }
/* 1254 */ internal TickBarStats TickStats { get; }
/* 1328 */ protected LevelStudyConfig LevelStudyConfig { get; }
/* 1359 */ protected bool levelStudyAdd(string type, double price, int ageSessions, double quality = double.N
aN, string key = null)
/* 1388 */ protected BarRingConfig BarRings { get; }
/* 1390 */ protected FvgConfig Fvgs { get; }
/* 1406 */ protected void requireRing(int ds, int bars, string consumer)
/* 1419 */ protected _BarRing Ring(int ds)
/* 1433 */ protected bool IsRollWindow(DateTime tradingDay)
/* 1436 */ protected RingBar ringBarOf(int ds, int ago = 0)
/* 1492 */ internal _FvgDetector FvgDetector
/* 1502 */ protected LevelRef fvgLevel(Fvg f, bool startEdge)
/* 1513 */ protected virtual bool PortfolioPublish { get; }
/* 1515 */ protected virtual string PortfolioFamily { get; }
/* 1518 */ protected virtual int PortfolioRegime
/* 1528 */ protected virtual double PortfolioScore { get; }
/* 1530 */ protected virtual bool PortfolioHalted { get; }
/* 1532 */ protected virtual string PortfolioCluster
/* 1579 */ protected void portfolioArm(int side, double entry, double stop, int qtyCode)
/* 1594 */ protected void portfolioStop(double stop)
/* 1658 */ protected int[] HtfStreams { get; set; }
/* 1663 */ protected int HtfClosedIndex(int ds, int ago = 0)
/* 1665 */ protected double HtfClosed(int ds, int ago = 0)
/* 1667 */ protected void HtfClosedBar(int ds, int ago, out DateTime time, out double open, out double high,
out double low, out double close, out double volume)
/* 1674 */ protected double HtfForming(int ds)
/* 1684 */ protected int htfMapped(int ds, int maxBack = 2)
/* 1719 */ public enum ERollDetection { Off = 0, Series = 1, Table = 2, SeriesOrTable = 3 }
/* 1722 */ protected ERollDetection RollDetection { get; set; }
/* 1727 */ protected int RollGapTicks { get; set; }
/* 1729 */ protected HashSet<DateTime> RollDates { get; }
/* 1736 */ public IList<int> RollBars { get; }
/* 1760 */ protected void stepRollDetector()
/* 1794 */ protected virtual bool rollOn(int ds)
/* 1806 */ protected Action<string> engineSink(ETraceLevel level)
/* 1940 */ protected override void StopCalc()
```

```
/* 1948 */ protected override void Create()
/* 1958 */ protected override void Destroy()
/* 1966 */ protected override void StartCalc()
/* 2019 */ protected override void CalcBar()
/* 2124 */ protected string runLine()
/* 2134 */ protected string inputSnapshot()
/* 2165 */ protected double tickSize()
/* 2191 */ protected bool journalOn()
/* 2198 */ protected void journalArm(string entryOrderName, double plannedEntry, double initialStop, double t
arget, IDictionary<string, string> context = null)
/* 2309 */ protected static void requireUnitInterval(string name, double value)
/* 2315 */ protected static void requireAtLeast(string name, int value, int min)
/* 2321 */ protected static void requireNonNegative(string name, double value)
/* 2327 */ protected static void requireNonNegative(string name, int value)
/* 2333 */ protected void requireDataStream(string name, int stream)
/* 2344 */ protected virtual DateTime tradingDayKey(DateTime barTime)
/* 2355 */ protected double equityAvailable(double maxCustomRiskEquityPerPosPercent = 0.0d)
/* 2427 */ public enum ETradingDayRoll { Calendar = 0, Auto = 1 }
/* 2433 */ protected ETradingDayRoll TradingDayRoll { get; set; }
/* 2438 */ protected TimeZoneInfo BarTimeZone
/* 2444 */ protected DateTime ToZone(DateTime barTime, string tzId)
/* 2456 */ protected bool inTimeWindow(TimeSpan start, TimeSpan end, string tzId)
/* 2463 */ protected DateTime autoTradingDay(DateTime barTime, string category)
/* 2474 */ protected TimeZoneInfo resolveBarTimeZone(IInstrumentSettings info)
/* 2496 */ protected virtual string exchangeTimeZoneId(string exchange)
/* 2507 */ protected static TimeZoneInfo findTz(string windowsId)
/* 2518 */ public enum EControlMode { None = 0, RandomEntry = 1, ShiftLevels = 2, DelayEntryBars = 3 }
/* 2521 */ protected int[] SignalHorizons { get; set; }
/* 2523 */ protected EControlMode ControlMode { get; set; }
/* 2524 */ protected int ControlSeed { get; set; }
/* 2526 */ protected double ControlRandomProbability { get; set; }
/* 2528 */ protected int ControlShiftTicks { get; set; }
/* 2530 */ protected int ControlDelayBars { get; set; }
/* 2552 */ protected void logSignal(string tag, int direction, double price, double stop,
IDictionary<string, string> context, bool taken)
/* 2629 */ protected double controlRandom(int bar)
/* 2642 */ protected bool controlRandomEntry()
/* 2648 */ protected double controlAdjustLevel(double level)
/* 2654 */ protected int controlAdjustDelayBars()
/* 2662 */ protected TimeSpan? NoEntryBefore { get; set; }
/* 2664 */ protected TimeSpan? NoEntryAfter { get; set; }
/* 2666 */ protected TimeSpan? FlatBy { get; set; }
/* 2668 */ protected int MaxTradesPerDay { get; set; }
/* 2670 */ protected int DailyLossStopCount { get; set; }
/* 2672 */ protected double DailyLossStopR { get; set; }
/* 2674 */ protected double RiskUnitDollars { get; set; }
/* 2676 */ protected int TimeStopBars { get; set; }
/* 2685 */ protected virtual TimeSpan sessionClock(DateTime barTime)
/* 2688 */ protected int TradesToday { get; }
/* 2689 */ protected int LosersToday { get; }
/* 2690 */ protected double NetProfitToday { get; }
/* 2730 */ protected bool entryAllowed(out string reason)
/* 2749 */ protected bool mustFlatten()
/* 2777 */ public struct SizingLimits
// --- _CustomSignal.SizingLimits
/* 2779 */ public int maxQty;
/* 2780 */ public double maxNotionalFraction;
/* 2781 */ public double unitNotional;
/* 2782 */ public double minStopTicks;
/* 2783 */ public bool roundLots;
// --- _CustomSignal
/* 2791 */ protected int sizeForRisk(IInstrument bars, double entry, double stop, double riskFraction, Sizing
Limits limits)
/* 2829 */ protected double equityAvailable(double fraction, bool includeOpenPnl)
/* 2846 */ protected double CostFilterMaxFractionOfR { get; set; }
/* 2852 */ protected virtual double roundTripCostPerContract()
/* 2858 */ protected bool costFilterPasses(IInstrument bars, double entry, double stop, out string reason)
/* 2877 */ protected static int applyRoundLots(int qty, bool useRoundLots)
/* 2896 */ protected int capByNotional(int qty, double unitNotional, double maxEquityFraction)
/* 2908 */ protected int equityBasedPositionSize(double entryPrice, double maxEquityPerPosPercent = 0.0d, boo
l useRoundLots = false)
/* 2933 */ protected int equityBasedPositionSize(IInstrument bars, double maxEquityPerPosPercent = 0.0d,
bool useRoundLots = false)
/* 2954 */ protected int riskBasedPositionSize(double risk, double maxRiskEquityPerPosPercent, bool useRoundL
ots = false)
/* 2986 */ protected int stopLossBasedPositionSize(IInstrument bars, double stopLossPrice, double maxRiskEqui
tyPerPosPercent = 0.0d, bool useRoundLots = false)
```

```
/* 2992 */ protected int stopLossBasedPositionSize(IInstrument bars, double entryPrice, double
stopLossPrice, double maxRiskEquityPerPosPercent, bool useRoundLots = false)
/* 3009 */ protected int volumeBasedPositionSize(IInstrument bars, int volumePeriod, int volumePeriodOffset,
double maxAvgVolumePercent, bool useRoundLots = false)
/* 3039 */ protected double averageTrueRange(IInstrument bars, int length)
/* 3056 */ protected bool isWithinAverageRange(IInstrument bars, int maLength, int atrLength, double atrFacto
r)
/* 3065 */ protected int calendarDayCount(DateTime entryDate, DateTime currentDateTime)
/* 3079 */ protected int tradingDayCount(DateTime entryDate, DateTime currentDateTime)
/* 3104 */ protected virtual bool isTradingHoliday(DateTime date)
/* 3114 */ protected Tuple<DateTime, double> recentLow(int daysBack, DateTime? earliestDateTime = null, bool
includeToday = true)
/* 3156 */ protected Tuple<DateTime, double> recentHigh(int daysBack, DateTime? earliestDateTime = null,
bool includeToday = true)
/* 3200 */ public DateTime orderEntryTime(params String[] entryOrderNames)
/* 3227 */ public DateTime orderEntryTimeUncached(params String[] entryOrderNames)
/* 3233 */ public enum EOrderNameMatch { Contains = 0, Exact = 1, Prefix = 2 }
/* 3236 */ protected static bool orderNameMatches(string orderName, string wanted, EOrderNameMatch mode)
/* 3248 */ public DateTime orderEntryTime(EOrderNameMatch matchMode, params String[] entryOrderNames)
/* 3309 */ public DateTime orderExitTime(EMarketPositionSide side, EUseFill useFill = EUseFill.Last)
/* 3321 */ public DateTime orderExitTime(EMarketPositionSide side, EUseFill useFill, DateTime notBefore, para
ms string[] exitOrderNames)
/* 3420 */ protected void refreshTradeManagerIfNeeded()
```

_CmeSessionClock.Function.CS

```
// --- (namespace)
/* 24 */ public sealed class _CmeSessionClock
// --- _CmeSessionClock
/* 29 */ public enum MarketSession { None = 0, Asia = 1 << 0, Europe = 1 << 1, NewYork = 1 << 2, Rth = 1 <<
3, Premarket = 1 << 4, PostMarket = 1 << 5 }
/* 41 */ public enum GlobexState { Closed, Halt, Open }
/* 49 */ public static readonly TimeSpan DefaultAsiaStart = new TimeSpan(18, 0, 0);
/* 50 */ public static readonly TimeSpan DefaultAsiaEnd = new TimeSpan(3, 0, 0);
/* 51 */ public static readonly TimeSpan DefaultEuropeStart = new TimeSpan(3, 0, 0);
/* 52 */ public static readonly TimeSpan DefaultEuropeEnd = new TimeSpan(11, 30, 0);
/* 53 */ public static readonly TimeSpan DefaultNewYorkStart = new TimeSpan(8, 0, 0);
/* 54 */ public static readonly TimeSpan DefaultNewYorkEnd = new TimeSpan(17, 0, 0);
/* 55 */ public static readonly TimeSpan DefaultRthStart = new TimeSpan(9, 30, 0);
/* 56 */ public static readonly TimeSpan DefaultRthEnd = new TimeSpan(16, 15, 0);
/* 57 */ public static readonly TimeSpan DefaultRthValueEnd = new TimeSpan(16, 0, 0);
/* 58 */ public static readonly TimeSpan DefaultHaltStart = new TimeSpan(17, 0, 0);
/* 59 */ public static readonly TimeSpan DefaultHaltEnd = new TimeSpan(18, 0, 0);
/* 60 */ public static readonly TimeSpan DefaultGlobexOpen = new TimeSpan(18, 0, 0);
/* 61 */ public static readonly TimeSpan DefaultGlobexClose = new TimeSpan(17, 0, 0);
/* 71 */ public TimeSpan AsiaStart = DefaultAsiaStart, AsiaEnd = DefaultAsiaEnd;
/* 72 */ public TimeSpan EuropeStart = DefaultEuropeStart, EuropeEnd = DefaultEuropeEnd;
/* 73 */ public TimeSpan NewYorkStart = DefaultNewYorkStart, NewYorkEnd = DefaultNewYorkEnd;
/* 74 */ public TimeSpan RthStart = DefaultRthStart, RthEnd = DefaultRthEnd;
/* 76 */ public TimeSpan RthValueEnd = DefaultRthValueEnd;
/* 77 */ public TimeSpan HaltStart = DefaultHaltStart, HaltEnd = DefaultHaltEnd;
/* 78 */ public TimeSpan GlobexOpen = DefaultGlobexOpen, GlobexClose = DefaultGlobexClose;
/* 81 */ public bool UseCmeHolidayTable = true;
/* 84 */ public Func<DateTime, bool> GlobexClosedDayHook;
/* 86 */ public Func<DateTime, bool> NoRthDayHook;
/* 88 */ public Func<DateTime, TimeSpan?> HolidayHaltHook;
/* 90 */ public Func<DateTime, TimeSpan?> RthEarlyCloseHook;
/* 106 */ public _CmeSessionClock(TimeZoneInfo barTimeZone = null)
/* 120 */ public TimeZoneInfo Eastern { get; }
/* 123 */ public TimeZoneInfo BarTimeZone
/* 136 */ public void Invalidate()
/* 142 */ public DateTime ToEastern(DateTime barTime)
/* 156 */ public DateTime EasternOf(DateTime barTime)
/* 157 */ public GlobexState State(DateTime barTime)
/* 159 */ public MarketSession Sessions(DateTime barTime)
/* 160 */ public bool InSession(DateTime barTime, MarketSession session)
/* 161 */ public bool InAny(DateTime barTime, MarketSession sessions)
/* 164 */ public DateTime GlobexDate(DateTime barTime)
/* 171 */ public GlobexState StateEt(DateTime et)
/* 200 */ public MarketSession SessionsEt(DateTime et)
/* 229 */ public DateTime GlobexDateEt(DateTime et)
/* 234 */ public static DateTime GlobexTradingDayEt(DateTime et)
/* 235 */ public static DateTime GlobexTradingDayEt(DateTime et, TimeSpan globexOpen)
/* 241 */ public MarketSession PrimarySessionEt(DateTime et)
```

```
/* 254 */ public TimeSpan TimeSinceOpenEt(DateTime et, MarketSession session)
/* 262 */ public int MinutesSinceOpenEt(DateTime et, MarketSession session)
/* 268 */ public TimeSpan TimeUntilCloseEt(DateTime et, MarketSession session)
/* 276 */ public int MinutesUntilCloseEt(DateTime et, MarketSession session)
/* 282 */ public TimeSpan SessionLength(MarketSession session)
/* 291 */ public bool IsOpeningRangeEt(DateTime et, MarketSession session, int minutes)
/* 297 */ public bool IsRthAfterInitialBalanceEt(DateTime et, int ibMinutes)
/* 306 */ public bool TrySessionBoundsEt(DateTime et, MarketSession session, out DateTime open, out DateTime
close)
/* 333 */ public bool TryWindow(MarketSession session, out TimeSpan start, out TimeSpan end, out bool wraps)
/* 352 */ public TimeSpan RthValueEndEt(DateTime globexDate)
/* 359 */ public bool IsRthValueBarEt(DateTime et)
/* 368 */ public bool IsGlobexClosedDay(DateTime globexDate)
/* 373 */ public bool IsNoRthDay(DateTime globexDate)
/* 379 */ public TimeSpan? HolidayHaltEt(DateTime globexDate)
/* 385 */ public TimeSpan? RthEarlyCloseEt(DateTime globexDate)
/* 395 */ public static bool DefaultGlobexClosedDay(DateTime globexDate, bool useCmeHolidayTable)
/* 407 */ public static bool InWindow(TimeSpan t, TimeSpan start, TimeSpan end, bool wraps)
/* 413 */ public static bool IsSingleFlag(MarketSession session)
```

_CmeHolidays.Function.CS

```
// --- (namespace)
/* 13 */ public sealed class _CmeHolidays : _Holidays
// --- _CmeHolidays
/* 32 */ public static class Rules
// --- _CmeHolidays.Rules
/* 34 */ public struct Day
// --- _CmeHolidays.Rules.Day
/* 36 */ public bool Closed;
/* 37 */ public bool NoRth;
/* 38 */ public TimeSpan? HaltEt;
/* 39 */ public TimeSpan? RthCloseEt;
/* 40 */ public string Name;
// --- _CmeHolidays.Rules
/* 43 */ public static readonly TimeSpan HolidayHaltEt = new TimeSpan(13, 0, 0);
/* 44 */ public static readonly TimeSpan EarlyCloseEt = new TimeSpan(13, 15, 0);
/* 45 */ public static readonly TimeSpan GoodFridayCloseEt = new TimeSpan(9, 15, 0);
/* 46 */ public static readonly TimeSpan MourningCloseEt = new TimeSpan(9, 30, 0);
/* 68 */ public static Day Get(DateTime date)
/* 107 */ public static List<DateTime> ClosedDates(int year)
/* 119 */ public static DateTime SundayToMonday(DateTime d)
/* 121 */ public static DateTime Observed(DateTime d)
/* 127 */ public static DateTime NthWeekday(int year, int month, DayOfWeek day, int n)
/* 134 */ public static DateTime LastWeekday(int year, int month, DayOfWeek day)
/* 141 */ public static DateTime Easter(int year)
// --- _CmeHolidays
/* 153 */ public _CmeHolidays(CStudyControl _master)
/* 154 */ public _CmeHolidays(CStudyControl _master, int _ds)
/* 156 */ protected override void refreshHolidays(int year)
```

_NyseHolidays.Function.CS

```
// --- (namespace)
/* 14 */ public sealed class _NyseHolidays : _Holidays
// --- _NyseHolidays
/* 26 */ public static class Rules
// --- _NyseHolidays.Rules
/* 44 */ public static List<DateTime> Holidays(int year)
/* 65 */ public static DateTime Observed(DateTime holiday)
/* 72 */ public static DateTime NthWeekday(int year, int month, DayOfWeek day, int n)
/* 79 */ public static DateTime LastWeekday(int year, int month, DayOfWeek day)
/* 86 */ public static DateTime Easter(int year)
// --- _NyseHolidays
/* 104 */ public _NyseHolidays(CStudyControl _master)
/* 105 */ public _NyseHolidays(CStudyControl _master, int _ds)
/* 112 */ protected override void refreshHolidays(int year)
```

_DailyContext.Function.CS

```
// --- (namespace)
/* 17 */ public enum EDailyClose { Bar = 0, RthValue = 1 }
/* 20 */ public sealed class DailyConfig
// --- DailyConfig
/* 22 */ public int dataStream = 0;
/* 23 */ public int weeklyStream = 0;
/* 24 */ public int atrLength = 20;
/* 27 */ public bool atrRestartOnRoll = true;
/* 28 */ public int smaLength = 200;
/* 29 */ public int rsiLength = 2;
/* 30 */ public int[] tsmomLookbacks = { 252, 126, 63 };
/* 32 */ public int[] hvLengths = { };
/* 33 */ public int hvAnnualisation = 252;
/* 34 */ public int hvPctLookback = 0;
/* 35 */ public int lbrRsiLength = 0;
/* 36 */ public EDailyClose closeBasis = EDailyClose.RthValue;
/* 37 */ public bool HvOn { get; }
/* 39 */ public string Validate()
// --- (namespace)
/* 56 */ internal sealed class _DailyContext
// --- _DailyContext
/* 58 */ public DailyConfig config = new DailyConfig();
/* 59 */ public string label = "DAILY";
/* 60 */ public Action<string> journal;
/* 61 */ public Action<string> warnOnce;
/* 72 */ public bool Configured { get; private set; }
/* 89 */ public double Hv(int n)
/* 91 */ public double HvPercentile(int n)
/* 102 */ public double Roc1 { get; }
/* 104 */ public double Lbr
/* 114 */ public double RthCloseAdj { get; }
/* 116 */ public double RthClosePrev { get; }
/* 120 */ public double Atr20d { get; }
/* 121 */ public double Sma200 { get; }
/* 122 */ public double Rsi2 { get; }
/* 123 */ public double PrevHigh { get; }
/* 124 */ public double PrevLow { get; }
/* 125 */ public double PrevClose { get; }
/* 126 */ public DateTime PrevDate { get; }
/* 127 */ public double PrevRange { get; }
/* 128 */ public double PrevWeekHigh { get; }
/* 129 */ public double PrevWeekLow { get; }
/* 131 */ public bool Valid { get; }
/* 133 */ public double Tsmom(int n)
/* 135 */ public double GapPct(double todayOpen)
/* 136 */ public int Days { get; }
/* 138 */ public string Configure()
/* 154 */ public void Reset()
/* 168 */ public void SetPrevWeek(double high, double low)
/* 174 */ public bool AddDay(DateTime date, double o, double h, double l, double c, bool firstDayOfNewContrac
t = false, double rollOffset = double.NaN, double rthClose = double.NaN)
/* 277 */ public static double HvOf(double[] ring, int next, int count, int n, int annualisation)
/* 291 */ public void OnRoll(DateTime date, double offset)
// --- (namespace)
/* 313 */ internal sealed class _RthCloseSelector
// --- _RthCloseSelector
/* 315 */ public DateTime Day { get; private set; }
/* 316 */ public double Close { get; private set; }
/* 318 */ public double Observe(DateTime day, bool isRthValueBar, double close)
```

_RollDetector.Function.CS

```
// --- (namespace)
/* 16 */ public sealed class _RollDetector
// --- _RollDetector
/* 18 */ public bool series;
/* 19 */ public bool table;
/* 20 */ public int gapTicks = 8;
/* 21 */ public ICollection<DateTime> dates;
/* 29 */ public IList<int> RollBars { get; }
/* 30 */ public int LastRollBar { get; }
```

```
/* 32 */ public void Reset()
/* 36 */ public static string DatesDigest(IEnumerable<DateTime> dates)
/* 51 */ public static string SeriesKeyWarning(bool series, string resolutionType, bool calendarKeyed)
/* 65 */ public bool Step(int bar, DateTime tradingDay, double open, double prevClose, string name, double ti
ck)
// --- (namespace)
/* 87 */ public sealed class _RollPending
// --- _RollPending
/* 91 */ public bool Armed { get; }
/* 92 */ public void Arm(DateTime data1RollTime)
/* 93 */ public bool Fire(int bar, DateTime time)
/* 100 */ public void Reset()
```

_BarRing.Function.CS

```
// --- (namespace)
/* 14 */ public struct RingBar
// --- RingBar
/* 16 */ public int bar; public DateTime time; public DateTime tradingDay;
/* 17 */ public double o, h, l, c, v, up, down;
/* 18 */ public bool rth; public bool rollBar;
// --- (namespace)
/* 21 */ public sealed class BarRingConfig
// --- BarRingConfig
/* 24 */ public int capacityBars = 0;
/* 25 */ public const int MaxAutoBars = 40000;
/* 27 */ public string Canon()
// --- (namespace)
/* 30 */ public sealed class _BarRing
// --- _BarRing
/* 37 */ public _BarRing(int capacity)
/* 39 */ public int Capacity { get; }
/* 40 */ public int Count { get; }
/* 42 */ public long Added { get; }
/* 43 */ public bool Dropped { get; }
/* 44 */ public int LastBarNumber { get; }
/* 47 */ public bool Add(RingBar b)
/* 62 */ public void Clear()
/* 65 */ public RingBar this[int ago]
/* 76 */ public RingBar At(int idx)
/* 79 */ public int IndexAtOrBefore(DateTime t)
/* 91 */ public int FirstIndexAtOrAfter(DateTime t, bool exclusive = false)
/* 104 */ public bool Covers(DateTime t0)
/* 112 */ public List<RingBar> Window(DateTime t0, DateTime t1, bool t0Exclusive = false)
/* 121 */ public List<RingBar> Session(DateTime tradingDay, bool rthOnly = false)
```

_MtfMath.Function.CS

```
// --- (namespace)
/* 11 */ public static class _MtfMath
// --- _MtfMath
/* 14 */ public static int ClosedIndex(bool barZeroClosed, int ago)
/* 21 */ public static int MapIndex(Func<int, DateTime> timeAt, int first, DateTime t, int maxBack)
/* 28 */ public static string SymbolRoot(string name)
/* 41 */ public static bool IsStale(DateTime streamTime, DateTime data1Time, TimeSpan maxStale)
```

_TickStream.Function.CS

```
// --- (namespace)
/* 15 */ public sealed class TickConfig
// --- TickConfig
/* 17 */ public bool enabled = false;
/* 18 */ public int historyTicks = 2000000;
/* 19 */ public bool realTime = true;
/* 20 */ public double burstZ = 3.0;
/* 21 */ public int burstNormLength = 200;
/* 22 */ public int burstMinSamples = 30;
/* 23 */ public double[] sizeBuckets = { 1, 2, 5, 10, 20, 50 };
/* 24 */ public double largePrintSize = 20;
```

```
/* 25 */ public bool sizeOutputsVerified = false;
/* 27 */ public string Validate()
// --- (namespace)
/* 42 */ public sealed class TickBarStats
// --- TickBarStats
/* 44 */ public bool valid;
/* 45 */ public string quality = "Unknown";
/* 46 */ public int prints;
/* 47 */ public int maxPrintsPerSecond;
/* 48 */ public double interPrintMedianMs = double.NaN, interPrintP10Ms = double.NaN;
/* 49 */ public double burstZ = double.NaN; public bool burst;
/* 50 */ public int[] sizeHistogram;
/* 51 */ public int largePrints = -1;
/* 52 */ public double volume = double.NaN;
// --- (namespace)
/* 55 */ internal sealed class _TickStream
// --- _TickStream
/* 57 */ public TickConfig config = new TickConfig();
/* 58 */ public Action<string> journal;
/* 70 */ public bool Configured { get; private set; }
/* 71 */ public bool HistoryComplete { get; }
/* 72 */ public TickBarStats Last { get; private set; }
/* 74 */ public string Configure()
/* 80 */ public void Reset()
/* 88 */ public void Enqueue(DateTime time, double price, double size)
/* 93 */ public void MarkHistoryComplete()
/* 114 */ public TickBarStats Aggregate(DateTime barStart, DateTime barEnd)
/* 174 */ public static double Quantile(double[] sorted, double q)
```

_TodNormaliser.Function.CS

```
// --- (namespace)
/* 15 */ public enum ETodMetric { None = 0, Volume = 1 << 0, Delta = 1 << 1, Range = 1 << 2, TrueRange = 1
<< 3 }
/* 18 */ public struct TodSample
// --- TodSample
/* 20 */ public DateTime sessionDate;
/* 21 */ public int minuteOfSession;
/* 22 */ public bool halfDay;
/* 23 */ public bool splice;
/* 24 */ public double volume, delta, range, trueRange;
/* 25 */ public int deltaTier;
// --- (namespace)
/* 28 */ internal sealed class _TodNormaliser
// --- _TodNormaliser
/* 30 */ public ETodMetric metrics = ETodMetric.None;
/* 31 */ public int sessions = 20;
/* 32 */ public int minSessions = 10;
/* 33 */ public int bucketMinutes = 1;
/* 34 */ public string label = "TOD";
/* 35 */ public Action<string> journal;
/* 36 */ public Action<string> warnOnce;
/* 53 */ public bool Configured { get; private set; }
/* 55 */ public string Validate()
/* 65 */ public string Configure()
/* 79 */ public void Reset()
/* 96 */ public bool Valid(ETodMetric m)
/* 98 */ public double Z(ETodMetric m)
/* 99 */ public double Median(ETodMetric m)
/* 101 */ public double Pct(ETodMetric m)
/* 103 */ public double ZCum(ETodMetric m)
/* 105 */ public int Sessions(ETodMetric m)
/* 109 */ public IList<double> VolumePool(int minuteOfSession)
/* 120 */ public bool Add(int barNumber, TodSample s)
```

_XmktLeg.Function.CS

```
// --- (namespace)
/* 14 */ public sealed class XmktLegConfig
// --- XmktLegConfig
/* 16 */ public string role = "leg";
/* 17 */ public int dataStream = 2;
```

```
/* 18 */ public int maxStaleSec = 0;
/* 19 */ public int returnBars = 1;
/* 20 */ public int corrLength = 60;
/* 21 */ public int rsLength = 60;
/* 22 */ public bool leaderSwing = false;
/* 24 */ public string Validate()
// --- (namespace)
/* 36 */ internal sealed class _XmktLeg
// --- _XmktLeg
/* 38 */ public XmktLegConfig config = new XmktLegConfig();
/* 39 */ public TimeSpan maxStale = TimeSpan.Zero;
/* 40 */ public Action<string> journal;
/* 50 */ public bool Configured { get; private set; }
/* 52 */ public bool Available { get; }
/* 53 */ public DateTime LastLegTime { get; }
/* 54 */ public double LastClose { get; }
/* 56 */ public string Configure()
/* 66 */ public void Reset()
/* 83 */ public void Step(DateTime d1Time, double d1Close, int legBar, DateTime legTime, double legClose)
/* 110 */ public void OnRoll()
/* 120 */ public double LogReturn { get; }
/* 122 */ public double Data1LogReturn { get; }
/* 124 */ public double Correlation { get; }
/* 126 */ public double Beta { get; }
/* 128 */ public double RsZ { get; }
```

_AvwapManager.Function.CS

```
// --- (namespace)
/* 15 */ public enum EAvwapAnchor { None = 0, SwingHigh = 1 << 0, SwingLow = 1 << 1, Breakout = 1 << 2, GapOp
en = 1 << 3, VolumeClimax = 1 << 4, WeekOpen = 1 << 5, MonthOpen = 1 << 6, QuarterOpen = 1 << 7, NewsBar = 1
<< 8, Swing = SwingHigh | SwingLow }
/* 30 */ public enum EAvwapPeriod { Week, Month, Quarter }
/* 33 */ public static class _AvwapCalendar
// --- _AvwapCalendar
/* 36 */ public static long PeriodKey(EAvwapPeriod p, DateTime sessionDate)
// --- (namespace)
/* 51 */ public enum EAvwapPlacebo { None = 0, P1 = 1, P2 = 2 }
/* 54 */ public sealed class AvwapLimits
// --- AvwapLimits
/* 56 */ public int maxAnchors = 6;
/* 57 */ public int maxAgeSessions = 10;
/* 58 */ public bool replaceSameType = true;
/* 59 */ public double[] bandSigmas = { 1, 2, 3 };
/* 60 */ public int slopeBars = 5;
/* 61 */ public int dataStream = 1;
/* 62 */ public double gapAtrFraction = 0.5;
/* 63 */ public double gapFallbackTrMultiple = 3;
/* 64 */ public double climaxZ = 3;
/* 65 */ public int fallbackLength = 20;
/* 66 */ public int backfillBars = 2;
/* 67 */ public EAvwapPlacebo placebos = EAvwapPlacebo.None;
/* 68 */ public int p2MinPool = 10;
/* 70 */ public string Validate()
// --- (namespace)
/* 90 */ public struct AvwapBar
// --- AvwapBar
/* 92 */ public int bar;
/* 93 */ public DateTime time;
/* 94 */ public double o, h, l, c, v;
/* 95 */ public DateTime sessionDate;
/* 96 */ public bool rth;
/* 97 */ public int slot;
// --- (namespace)
/* 101 */ public struct AvwapPaEvents
// --- AvwapPaEvents
/* 103 */ public bool swingHigh, swingLow;
/* 104 */ public int swingHighBar, swingLowBar;
/* 105 */ public DateTime swingHighTime, swingLowTime;
/* 106 */ public bool breakoutUp, breakoutDown;
// --- (namespace)
/* 110 */ public sealed class _Avwap
// --- _Avwap
/* 112 */ internal double sPV, sP2V, sV;
```

```
/* 113 */ internal double tS, tS2; internal int tN;
/* 114 */ internal double qPV, qP2V, qV;
/* 115 */ internal bool p2Journaled;
/* 117 */ public double P1 { get; internal set; }
/* 118 */ public double P1Sigma { get; internal set; }
/* 120 */ public double P2 { get; internal set; }
/* 121 */ public double P2Sigma { get; internal set; }
/* 123 */ public bool p2Degraded { get; internal set; }
/* 125 */ public string TagFor(EAvwapPlacebo arm)
/* 126 */ internal double[] hist; internal int histCount, histNext;
/* 127 */ public int Id { get; internal set; }
/* 128 */ public EAvwapAnchor Type { get; internal set; }
/* 129 */ public DateTime AnchorTime { get; internal set; }
/* 130 */ public int AnchorBar { get; internal set; }
/* 131 */ public DateTime UsableFrom { get; internal set; }
/* 132 */ public int Direction { get; internal set; }
/* 133 */ public double Value { get; internal set; }
/* 134 */ public double Sigma { get; internal set; }
/* 135 */ public double SlopeTicks { get; internal set; }
/* 136 */ public double SlopeAtr { get; internal set; }
/* 137 */ public int AgeBars { get; internal set; }
/* 138 */ public int AgeSessions { get; internal set; }
/* 139 */ public bool Touched { get; internal set; }
/* 140 */ public int TouchCount { get; internal set; }
/* 141 */ public bool Expired { get; internal set; }
/* 142 */ public string ExpireReason { get; internal set; }
/* 144 */ public string Tag { get; }
/* 145 */ public double Band(double k)
/* 146 */ public bool Valid { get; }
// --- (namespace)
/* 149 */ internal sealed class _AvwapManager
// --- _AvwapManager
/* 151 */ public AvwapLimits limits = new AvwapLimits();
/* 152 */ public EAvwapAnchor events = EAvwapAnchor.None;
/* 153 */ public double tickSize = 0.25;
/* 154 */ public string label = "AV";
/* 155 */ public Action<string> journal;
/* 156 */ public Action<string> trace;
/* 157 */ public Action<string> warnOnce;
/* 158 */ public Func<double> dailyAtr;
/* 159 */ public Func<double> volumeZ;
/* 161 */ public Func<int, IList<double>> volumePool;
/* 163 */ public bool Configured { get; private set; }
/* 164 */ public int LastBar { get; }
/* 183 */ public static string P2PoolError(EAvwapPlacebo placebos, int dataStream, bool todRunning)
/* 191 */ public string Configure()
/* 204 */ public void Reset()
/* 216 */ public IReadOnlyList<_Avwap> Anchors { get; }
/* 218 */ public _Avwap Latest(EAvwapAnchor type)
/* 225 */ public double FallbackTrAvg { get; }
/* 228 */ public void OnRoll(DateTime barTime)
/* 239 */ public bool Add(AvwapBar b, AvwapPaEvents pa)
```

_PriceActionEngine.Function.CS

```
// --- (namespace)
/* 24 */ public enum EPivotMode { Fractal = 1, AtrZigZag = 2 }
/* 25 */ public enum EPivotTieRule { Strict = 0, LeftInclusive = 1 }
/* 26 */ public enum EPivotLabel { None = 0, HH, LH, HL, LL }
/* 27 */ public enum EStructure { Unknown = 0, Up, Down, Range }
/* 31 */ public enum EPriceActionPreset { Library = 0, Futures = 1 }
/* 35 */ public enum ECandle { None = 0, Bull = 1 << 0, Bear = 1 << 1, Doji = 1 << 2, BodyAtHigh = 1 << 3, Bo
dyAtLow = 1 << 4, Inside = 1 << 5, Outside = 1 << 6, Engulfing = 1 << 7, BullReversal = 1 << 8, BearReversal
= 1 << 9, Strong = 1 << 10, VolAboveAvg = 1 << 11, VolPlus1SD = 1 << 12, VolPlus2SD = 1 << 13, VolPlus3SD =
1 << 14, VolBelowAvg = 1 << 15, VolMinus1SD = 1 << 16, VolMinus2SD = 1 << 17, VolMinus3SD = 1 << 18, Breakout
Up = 1 << 19, BreakoutDown = 1 << 20 }
/* 49 */ public sealed class PriceActionConfig
// --- PriceActionConfig
/* 51 */ public bool enabled = false;
/* 52 */ public int dataStream = 1;
/* 53 */ public EPivotMode pivotMode = EPivotMode.Fractal;
/* 54 */ public EPivotTieRule tieRule = EPivotTieRule.Strict;
/* 55 */ public int layers = 2;
/* 56 */ public int strength = 1;
```

```
/* 57 */ public int atrLength = 20;
/* 58 */ public int volumeLength = 20;
/* 59 */ public bool volumeIncludesCurrent = true;
/* 60 */ public double atrMultiple = 1.5;
/* 61 */ public double factorDoji = 0.1;
/* 62 */ public double factorClose = 0.1;
/* 63 */ public double factorStrong = 1.0;
/* 64 */ public double factorEpsilon = 0.05;
/* 65 */ public double equalFactor = 0.1;
/* 66 */ public int lookbackPivots = 5;
/* 67 */ public int minBreakoutLayer = 1;
/* 68 */ public bool breakoutNeedsStrong = true;
/* 69 */ public int history = 50;
/* 70 */ public bool candles = true;
/* 71 */ public EPriceActionPreset preset = EPriceActionPreset.Library;
/* 73 */ public string role = "";
/* 75 */ public bool presetApplied;
/* 78 */ public PriceActionConfig ApplyPreset(EPriceActionPreset p)
/* 91 */ public PriceActionConfig ApplyPresetDefaults(EPriceActionPreset p)
/* 101 */ public static PriceActionConfig Futures()
/* 103 */ public static PriceActionConfig Library()
/* 107 */ public string Canon()
/* 116 */ public string Validate()
// --- (namespace)
/* 139 */ public sealed class _Pivot
// --- _Pivot
/* 141 */ public readonly bool isHigh;
/* 142 */ public readonly int layer;
/* 143 */ public readonly long seq;
/* 144 */ public readonly double price;
/* 145 */ public readonly DateTime extremeTime;
/* 146 */ public readonly int extremeBar;
/* 147 */ public readonly DateTime confirmTime;
/* 148 */ public readonly int confirmBar;
/* 149 */ public readonly EPivotLabel label;
/* 150 */ public readonly bool isEqual;
/* 151 */ public readonly long equalToSeq;
/* 152 */ public readonly double legSizeAtr;
/* 153 */ public _Pivot(bool isHigh, int layer, long seq, double price, DateTime extremeTime, int
extremeBar, DateTime confirmTime, int confirmBar, EPivotLabel label, bool isEqual, long equalToSeq, double le
gSizeAtr)
// --- (namespace)
/* 162 */ internal sealed class _Ring<T>
// --- _Ring
/* 165 */ public _Ring(int capacity)
/* 166 */ public int Count { get; }
/* 167 */ public int Capacity { get; }
/* 168 */ public void Clear()
/* 169 */ public void Add(T v)
/* 171 */ public T Ago(int ago)
/* 172 */ public T Oldest()
// --- (namespace)
/* 175 */ internal struct _PaBar
/* 177 */ internal sealed class _PriceActionEngine
// --- _PriceActionEngine
/* 180 */ public PriceActionConfig config = new PriceActionConfig();
/* 181 */ public string label = "PA";
/* 182 */ public Action<string> trace;
/* 183 */ public Action<string> journal;
/* 184 */ public Action<string> warnOnce;
/* 185 */ public Func<double> volumeZOverride;
/* 188 */ public bool Configured { get; private set; }
/* 189 */ public bool IsWarm { get; private set; }
/* 190 */ public double Atr { get; }
/* 191 */ public double VolumeMean { get; private set; }
/* 192 */ public double VolumeSigma { get; private set; }
/* 193 */ public ECandle Candle { get; private set; }
/* 194 */ public ECandle Changed { get; private set; }
/* 195 */ public bool NewPivotHigh { get; private set; }
/* 196 */ public bool NewPivotLow { get; private set; }
/* 197 */ public bool BosUp { get; private set; }
/* 198 */ public bool BosDown { get; private set; }
/* 199 */ public bool ChochUp { get; private set; }
/* 200 */ public bool ChochDown { get; private set; }
/* 201 */ public DateTime LastBarTime { get; private set; }
/* 203 */ public int LastBar { get; }
/* 224 */ public string Configure()
```

```
/* 242 */ public void Reset()
/* 252 */ public void OnRoll(DateTime barTime)
/* 262 */ public void Step(IInstrument bars)
/* 270 */ public bool Add(int barNumber, DateTime time, double o, double h, double l, double c, double v)
/* 295 */ public _Pivot LastHigh(int layer = 0)
/* 296 */ public _Pivot LastLow(int layer = 0)
/* 297 */ public _Pivot High(int layer, int ago)
/* 298 */ public _Pivot Low(int layer, int ago)
/* 299 */ public int PivotCount(int layer, bool isHigh)
/* 300 */ public EStructure Structure(int layer = 0)
/* 301 */ public bool DoubleTop(int layer = 0)
/* 302 */ public bool DoubleBottom(int layer = 0)
/* 303 */ public bool Has(ECandle f)
/* 306 */ public const double SMALL_BODY_RATIO = 1.0 / 3.0;
/* 307 */ public const double TAIL_MIN_RATIO = 1.0 / 3.0;
/* 308 */ public const double TAIL_MAX_RATIO = 1.0 / 2.0;
/* 311 */ internal static ECandle ClassifyCandle(_PaBar b, _PaBar p, double atr, PriceActionConfig k)
```

_FvgDetector.Function.CS

```
// --- (namespace)
/* 16 */ public enum EFvgFillOn { Close = 0, Trade = 1 }
/* 18 */ public sealed class FvgConfig
// --- FvgConfig
/* 20 */ public bool enabled = false;
/* 21 */ public int dataStream = 1;
/* 22 */ public int minTicks = 1;
/* 23 */ public double minAtrFrac = 0;
/* 24 */ public int atrLen = 14;
/* 25 */ public bool crossSessions = false;
/* 26 */ public bool rthOnly = false;
/* 27 */ public int maxLive = 20;
/* 28 */ public int maxAgeSessions = 5;
/* 29 */ public EFvgFillOn fillOn = EFvgFillOn.Close;
/* 31 */ public string Validate()
/* 42 */ public string Canon()
// --- (namespace)
/* 48 */ public sealed class Fvg
// --- Fvg
/* 50 */ public int id, dir;
/* 51 */ public double lo, hi, start;
/* 52 */ public DateTime createdTime, bar1Time, firstTouchTime;
/* 53 */ public DateTime createdDay;
/* 54 */ public bool touched, filled, expired, invalid;
/* 55 */ public double fillPct;
/* 56 */ public int ageSessions;
/* 57 */ public double Size { get; }
/* 58 */ public bool IsLive { get; }
/* 59 */ public string Tag { get; }
// --- (namespace)
/* 62 */ public sealed class _FvgDetector
// --- _FvgDetector
/* 64 */ public readonly FvgConfig config;
/* 65 */ public readonly double tick;
/* 66 */ public readonly TimeSpan resolution;
/* 67 */ public Action<string> journal;
/* 75 */ public _FvgDetector(FvgConfig cfg, double tick, TimeSpan resolution)
/* 80 */ public IList<Fvg> Live { get; }
/* 81 */ public Fvg NewThisBar { get; private set; }
/* 82 */ public double Atr { get; }
/* 85 */ public void Add(RingBar b)
/* 164 */ public Fvg Nearest(double price, int dir)
/* 177 */ public LevelRef ToLevel(Fvg f, bool startEdge, int stream = 1)
```

_LevelRef.Function.CS

```
// --- (namespace)
/* 15 */ public enum ELevelKind { Poc, Vah, Val, Hvn, Lvn, RangePoc, RangeVah, RangeVal, WeekPoc, WeekVah, We
ekVal, FvgUpper, FvgLower, GmmMu, GmmEdge, GmmVoid, SigmaMu, VtPrice, BootstrapPocZoneLo,
BootstrapPocZoneHi, DvpPoc, DvpVah, DvpVal, DvpHvn, DvpLvn, DvpSigma, DvpFib, DvpConf }
/* 20 */ public sealed class LevelRef
// --- LevelRef
```

```
/* 22 */ public ELevelKind kind; public double price, lo, hi;
/* 23 */ public DateTime createdTime;
/* 24 */ public DateTime sourceT0, sourceT1;
/* 25 */ public int dataStream; public string tag;
/* 26 */ public double quality = double.NaN;
/* 27 */ public bool crossesRoll;
/* 28 */ public bool isRollWindow;
/* 30 */ public static LevelRef Line(ELevelKind kind, double price, DateTime created, DateTime t0, DateTime t
1, int stream, string tag, double quality = double.NaN)
/* 35 */ public static LevelRef Zone(ELevelKind kind, double price, double lo, double hi, DateTime created, D
ateTime t0, DateTime t1, int stream, string tag, double quality = double.NaN)
/* 41 */ public string StudyType { get; }
/* 43 */ public string StudyKey { get; }
/* 45 */ public override string ToString()
// --- (namespace)
/* 57 */ public sealed class _RollWindow
// --- _RollWindow
/* 64 */ public void Observe(DateTime tradingDay, bool roll)
/* 75 */ public bool IsRollDay(DateTime tradingDay)
/* 78 */ public bool IsRollWindow(DateTime tradingDay)
/* 88 */ public int Days { get; }
```

_OrderFlow.Function.CS

```
// --- (namespace)
/* 26 */ public sealed class _OrderFlow
// --- _OrderFlow
/* 32 */ public enum FlowSource { Aggressor = 0, McBidAsk = 1, Quote = 2, TickRule = 3, BarUpDown = 4, Unavai
lable = 5 }
/* 34 */ public enum EMcBidAsk { Off = 0, Prefer = 1, CrossCheckOnly = 2 }
/* 36 */ public const string McBidAskMethod = "McBidAsk(quote-classified, not aggressor; MC rule: inside-
spread trade -> last recorded side, UM p35/p159)";
/* 38 */ public enum FlowQuality { Exact = 0, High = 1, Medium = 2, Low = 3, Unknown = 4 }
/* 39 */ public enum Side { Unknown = 0, Buy = 1, Sell = -1 }
/* 41 */ public enum ResetMode { Session = 0, Rth = 1 }
/* 43 */ public enum VolumeBasis { Unknown = 0, TradeVolume = 1, TickCount = 2 }
/* 45 */ public enum Regime { TickStream = 0, IogRealTime = 1, IogMagnifierOneTick = 2,
IogHistoryNoMagnifier = 3, BarsOnly = 4 }
/* 54 */ public struct FlowValue
// --- _OrderFlow.FlowValue
/* 56 */ public double Value;
/* 57 */ public FlowSource Source;
/* 58 */ public FlowQuality Quality;
/* 59 */ public bool Developing;
/* 62 */ public bool Truncated;
/* 63 */ public static FlowValue Unavailable(bool developing = false)
/* 67 */ public bool IsAvailable { get; }
/* 68 */ public override string ToString()
// --- _OrderFlow
/* 71 */ public sealed class FlowConfig
// --- _OrderFlow.FlowConfig
/* 73 */ public double MaxQuoteAgeMs = 1000;
/* 74 */ public ResetMode ResetMode = ResetMode.Session;
/* 76 */ public bool AllowMixed = false;
/* 77 */ public bool QuotesExpected = true;
/* 78 */ public VolumeBasis BuildVolumeOn = VolumeBasis.Unknown;
/* 79 */ public bool ExcludeExtremesTickRule = true;
/* 80 */ public int MaxBars = 5000;
/* 83 */ public EMcBidAsk McBidAsk = EMcBidAsk.Off;
/* 85 */ public double mcVolTolerance = 0.005;
// --- _OrderFlow
/* 89 */ public struct BarInput
// --- _OrderFlow.BarInput
/* 91 */ public int BarNumber;
/* 92 */ public DateTime CloseTime;
/* 93 */ public DateTime SessionKey;
/* 94 */ public DateTime ResetKey;
/* 95 */ public Regime Regime;
/* 96 */ public double High, Low, Close, Volume, UpTicks, DownTicks;
// --- _OrderFlow
/* 99 */ public struct Imbalance
// --- _OrderFlow.Imbalance
/* 101 */ public double Price;
/* 102 */ public Side Side;
```

```
/* 103 */ public double Ratio;
/* 104 */ public FlowQuality Quality;
// --- _OrderFlow
/* 107 */ public sealed class StackedResult
// --- _OrderFlow.StackedResult
/* 109 */ public readonly List<double[]> Runs = new List<double[]>();
/* 110 */ public FlowQuality Quality = FlowQuality.Unknown;
/* 111 */ public int Longest { get; }
/* 112 */ public bool Found { get; }
// --- _OrderFlow
/* 116 */ public sealed class FootprintBar
// --- _OrderFlow.FootprintBar
/* 118 */ public int BarNumber;
/* 119 */ public DateTime CloseTime, SessionKey, ResetKey;
/* 120 */ public FlowSource Source = FlowSource.Unavailable;
/* 121 */ public FlowQuality Quality = FlowQuality.Unknown;
/* 122 */ public double Buy, Sell, Volume;
/* 123 */ public int Trades, StaleQuoteTrades;
/* 124 */ public double McDelta = double.NaN;
/* 125 */ public double ChainDelta = double.NaN;
/* 126 */ public string McMiss;
/* 127 */ public bool FirstAfterConnect;
/* 128 */ public double TradeHigh = double.NaN, TradeLow = double.NaN, BarHigh, BarLow, BarClose;
/* 129 */ internal readonly SortedDictionary<long, double[]> Ladder = new SortedDictionary<long, double[]>();
/* 130 */ public double Tick;
/* 131 */ public bool HasLadder { get; }
/* 132 */ public double Delta { get; }
/* 133 */ public long Row(double price)
/* 134 */ public double Bid(double price)
/* 135 */ public double Ask(double price)
/* 137 */ public List<double[]> Rows()
/* 144 */ internal void add(double price, double size, Side side)
// --- _OrderFlow
/* 158 */ public sealed class TickRule
// --- _OrderFlow.TickRule
/* 163 */ public TickRule(double tick)
/* 164 */ public Side Peek(double price)
/* 171 */ public void Observe(double price, Side side)
/* 172 */ public Side Classify(double price)
// --- _OrderFlow
/* 193 */ public sealed class FlowEngine
// --- _OrderFlow.FlowEngine
/* 195 */ public readonly FlowConfig Cfg;
/* 196 */ public readonly double Tick;
/* 213 */ public Action<string> journal;
/* 215 */ public Action<string> warnOnce;
/* 217 */ public int DroppedBars { get; }
/* 219 */ public int BarCap { get; }
/* 223 */ public int McUsed, McMissing, McVolMismatch;
/* 226 */ public void AddMcLevels(DateTime barTime, double[] price, double[] ask, double[] bid)
/* 231 */ public int PendingMcSets { get; }
/* 272 */ public FlowEngine(double tick, FlowConfig cfg)
/* 274 */ public void AddTrade(DateTime time, double price, double size)
/* 275 */ public void AddQuote(DateTime time, bool isBid, double price)
/* 276 */ public int PendingTrades { get; }
/* 277 */ public int Count { get; }
/* 280 */ public FootprintBar Closed(int ago)
/* 281 */ public FootprintBar ByNumber(int bar)
/* 294 */ public FootprintBar CloseBar(BarInput b)
/* 384 */ public static bool BarUpDownQualifies(VolumeBasis basis, double volume, double up, double down)
/* 390 */ public FlowValue BarDelta(int ago)
/* 398 */ public FlowValue CumDelta(int ago)
/* 417 */ public List<Imbalance> DiagonalImbalances(int ago, double ratio = 3.0, double minVol = 10)
/* 422 */ public StackedResult StackedImbalances(int ago, double ratio = 3.0, double minVol = 10, int minStac
k = 3)
/* 433 */ public FlowValue Absorption(int ago, Side side, int edgeTicks = 2, double volMult = 2.0, int holdTi
cks = 1)
// --- _OrderFlow
/* 468 */ public static List<Imbalance> Imbalances(FootprintBar fp, double ratio, double minVol, bool exclude
Extremes)
/* 497 */ public static StackedResult Stacked(FootprintBar fp, double ratio, double minVol, int minStack, boo
l excludeExtremes)
```

_RegimeEngine.Function.CS

```
// --- (namespace)
/* 27 */ public enum ERegimeMethod { Hmm = 0, Gmm = 1 }
/* 28 */ public enum ERegimeKSelect { Fixed = 0, Bic = 1 }
/* 29 */ public enum ERegimeRefit { Monthly = 0, EveryNSessions = 1, Quarterly = 2 }
/* 30 */ public enum ERegimeFallback { AtrTercile = 0, Unknown = 1 }
/* 31 */ public enum ERegimeFeature { Return = 0, LogHv20 = 1, Tsmom63z = 2, Parkinson10 = 3 }
/* 34 */ public sealed class RegimeConfig
// --- RegimeConfig
/* 36 */ public bool enabled = false;
/* 37 */ public ERegimeMethod method = ERegimeMethod.Hmm;
/* 38 */ public int K = 3;
/* 39 */ public ERegimeKSelect kSelect = ERegimeKSelect.Fixed;
/* 40 */ public ERegimeFeature[] features = { ERegimeFeature.Return, ERegimeFeature.LogHv20, ERegimeFeature.T
smom63z };
/* 41 */ public int fitSessions = 1000;
/* 42 */ public int minSessions = 500;
/* 43 */ public ERegimeRefit refit = ERegimeRefit.Monthly;
/* 44 */ public int refitEveryN = 21;
/* 45 */ public int restarts = 10;
/* 46 */ public int maxIter = 200;
/* 47 */ public double tol = 1e-6;
/* 48 */ public double regCovar = 1e-6;
/* 49 */ public double transPrior = 1;
/* 50 */ public int warmFilter = 63;
/* 51 */ public double switchProb = 0.60;
/* 52 */ public double switchMargin = 0.10;
/* 53 */ public ulong seed = 20261021;
/* 54 */ public ERegimeFallback fallback = ERegimeFallback.AtrTercile;
/* 55 */ public bool publish = false;
/* 57 */ public int atrPctLookback = 250;
/* 58 */ public int parkinsonBars = 10;
/* 59 */ public bool bicDiagnostic = true;
/* 60 */ public double rollSuspectZ = 5;
/* 61 */ public int gapBusinessDays = 4;
/* 63 */ public int D { get; }
/* 65 */ public string Validate()
/* 87 */ public RegimeConfig Copy()
/* 89 */ public int IndexOf(ERegimeFeature f)
// --- (namespace)
/* 93 */ public sealed class RegimeState
// --- RegimeState
/* 95 */ public bool Valid;
/* 96 */ public int Label = -1;
/* 97 */ public int RawArgmax = -1;
/* 98 */ public double[] Probs = new double[0];
/* 99 */ public DateTime AsOf = DateTime.MinValue;
/* 100 */ public DateTime FittedThrough = DateTime.MinValue;
/* 101 */ public int K;
/* 102 */ public double LogLik = double.NaN;
/* 103 */ public int Refits;
/* 104 */ public bool UsingFallback;
/* 105 */ public int RelabelConflicts;
/* 107 */ public RegimeState Copy()
/* 109 */ public static RegimeState Neutral(int K)
// --- (namespace)
/* 113 */ public sealed class RegimeFit
// --- RegimeFit
/* 115 */ public ERegimeMethod method;
/* 116 */ public int K, D;
/* 117 */ public double[] pi;
/* 118 */ public double[,] A;
/* 119 */ public double[][] mu;
/* 120 */ public double[][,] cov;
/* 121 */ public double[][,] L;
/* 122 */ public double[] zMean, zSd;
/* 123 */ public double logLik = double.NaN, bic = double.NaN;
/* 124 */ public int iters, gmmIters;
/* 125 */ public bool converged;
/* 126 */ public DateTime fittedThrough = DateTime.MinValue, windowStart = DateTime.MinValue;
/* 127 */ public int window;
/* 128 */ public string failReason;
/* 130 */ public GmmModel gmmInit, gmm;
/* 131 */ public double[] hmmInitPi; public double[,] hmmInitA; public double[][] hmmInitMu; public double[]
```

```
[,] hmmInitCov;
/* 132 */ public double[][] gamma;
/* 134 */ public static int ParamCount(ERegimeMethod m, int K, int D)
/* 137 */ public void Raw(int k, out double[] m, out double[,] c)
// --- (namespace)
/* 144 */ public sealed class _RegimeEngine
// --- _RegimeEngine
/* 146 */ public RegimeConfig config = new RegimeConfig();
/* 147 */ public string label = "REGIME";
/* 148 */ public Action<string> journal;
/* 149 */ public Action<string> warnOnce;
/* 151 */ public Action<RegimeFit> onRefit;
/* 166 */ public bool Configured { get; private set; }
/* 168 */ public RegimeState State { get; }
/* 169 */ public RegimeFit Model { get; }
/* 170 */ public int Refits { get; }
/* 171 */ public int RelabelConflicts { get; }
/* 172 */ public int ValidObservations { get; }
/* 174 */ public string Configure()
/* 182 */ public void Reset()
/* 207 */ public void AddDay(DateTime date, double adjClose, double hv20, double hv63, double high, double lo
w, double atr20d, bool rollDay)
/* 299 */ public static double AtrPercentile(double[] ring, int next, int count, double cur)
/* 308 */ public static int AtrBucket(double pct, int K)
/* 311 */ public static int Hysteresis(int current, double[] p, double switchProb, double switchMargin)
/* 320 */ public static int Argmax(double[] p)
/* 434 */ public static void ZStats(double[][] x, out double[] mean, out double[] sd)
/* 445 */ public static double[] Standardise(RegimeFit m, double[] raw)
/* 453 */ public static double[] FilterStep(RegimeFit m, double[] alpha, double[] z)
/* 475 */ public static RegimeFit FitWindow(double[][] z, int K, RegimeConfig cfg, ulong seed, int orderFeatu
re, int tieFeature)
/* 609 */ public static void OrderStates(RegimeFit r, int of, int tf)
/* 625 */ public static double Bhattacharyya(double[] m1, double[,] c1, double[] m2, double[,] c2)
/* 642 */ public static int[] BestMatch(RegimeFit oldFit, RegimeFit newFit)
```

_RegimeGate.Function.CS

```
// --- (namespace)
/* 25 */ public enum ERegimeGateArm { U = 0, G = 1, A = 2 }
/* 28 */ public sealed class RegimeGateConfig
// --- RegimeGateConfig
/* 30 */ public ERegimeGateArm arm = ERegimeGateArm.U;
/* 32 */ public ulong placeboSeed = RegimeGate.PlaceboSeed;
/* 33 */ public string Validate()
// --- (namespace)
/* 41 */ public sealed class A263Config
// --- A263Config
/* 43 */ public double k = 0.5;
/* 44 */ public int atrLength = 20;
/* 45 */ public TimeSpan rthOpen = new TimeSpan(9, 30, 0);
/* 46 */ public TimeSpan activeEnd = new TimeSpan(15, 30, 0);
/* 47 */ public TimeSpan exitTime = new TimeSpan(15, 55, 0);
/* 48 */ public TimeSpan rthClose = new TimeSpan(16, 0, 0);
/* 49 */ public int contracts = 1;
/* 51 */ public TimeSpan halfDayExitBefore = new TimeSpan(0, 5, 0);
/* 55 */ public string rollDetection = "Table";
/* 56 */ public int rollFromYear = 2000, rollToYear = 2035;
/* 59 */ public string rollDatesDigest = RollDatesDigest2000To2035;
/* 60 */ public const string RollDatesDigest2000To2035 = "fdd63bd8f179a8d4";
/* 62 */ public string tradingDay = "Globex18ET";
/* 63 */ public bool atrRestartOnRoll = false;
// --- (namespace)
/* 67 */ public struct RegimeGateDecision
// --- RegimeGateDecision
/* 69 */ public DateTime session, asOf;
/* 70 */ public ERegimeGateArm arm;
/* 71 */ public int label, raw, atrTercile;
/* 72 */ public double pFilt;
/* 73 */ public bool fb, usable, wouldBlock, allow;
/* 77 */ public ERegimeGateReason reason;
// --- (namespace)
/* 80 */ public enum ERegimeGateReason { Ok = 0, Warmup = 1, Invalid = 2, Fallback = 3 }
/* 84 */ public sealed class RegimeGateCounts
// --- RegimeGateCounts
```

```
/* 86 */ public int sessions, usable, warmup, invalid, fallback, blocked, noRth;
/* 89 */ public int atrNaN, rollAtrNaN;
/* 90 */ public void AddAtr(bool atrIsNaN, bool afterRoll, bool noRthDay = false)
/* 96 */ public void Add(RegimeGateDecision d, bool noRthDay = false)
/* 106 */ public string Suffix()
// --- (namespace)
/* 109 */ public static class RegimeGate
// --- RegimeGate
/* 111 */ public const int BlockedLabel = 2;
/* 112 */ public const ulong PlaceboSeed = 20261022;
/* 113 */ public const int ArmAFitSessions = 100000;
/* 114 */ public const string Version = "A263-RG-2";
/* 117 */ public static void ApplyArm(RegimeGateConfig g, RegimeConfig r)
/* 129 */ public static RegimeGateDecision Decide(ERegimeGateArm arm, RegimeState s, DateTime session, int at
rTercile)
/* 151 */ public static string Line(RegimeGateDecision d, bool entrySignal, string cfgHash, ulong
placeboSeed)
/* 159 */ public static IDictionary<string, string> TradeFields(RegimeGateDecision d, string cfgHash, ulong p
laceboSeed)
/* 174 */ public static readonly string[] HashFields =
/* 186 */ public static Dictionary<string, string> CanonFields(RegimeConfig r, RegimeGateConfig g,
A263Config h)
/* 210 */ public static string Canon(IDictionary<string, string> fields)
/* 222 */ public static string Canon(RegimeConfig r, RegimeGateConfig g, A263Config h)
/* 225 */ public static string Fnv1a64(string s)
/* 232 */ public static string Hash(RegimeConfig r, RegimeGateConfig g, A263Config h)
/* 233 */ public static string Hash(IDictionary<string, string> fields)
// --- (namespace)
/* 240 */ public sealed class AtrTercileTracker
// --- AtrTercileTracker
/* 243 */ public DateTime LastDate { get; private set; }
/* 244 */ public int Tercile { get; private set; }
/* 245 */ public AtrTercileTracker(int lookback, int K)
/* 246 */ public void AddDay(DateTime date, double atr20d)
// --- (namespace)
/* 256 */ public struct A263Action
// --- A263Action
/* 258 */ public bool latched;
/* 259 */ public bool sendStop;
/* 260 */ public double stop;
/* 261 */ public bool exit;
// --- (namespace)
/* 274 */ public sealed class A263Core
// --- A263Core
/* 276 */ public readonly A263Config config;
/* 277 */ public A263Core(A263Config c)
/* 278 */ public DateTime Session { get; private set; }
/* 279 */ public double Open { get; private set; }
/* 280 */ public double Atr { get; private set; }
/* 281 */ public DateTime AtrAsOf { get; private set; }
/* 282 */ public double Threshold { get; private set; }
/* 283 */ public TimeSpan ExitAt { get; private set; }
/* 284 */ public TimeSpan? EarlyClose { get; private set; }
/* 285 */ public bool NoRth { get; private set; }
/* 286 */ public bool AtrRejected { get; private set; }
/* 287 */ public bool Entered { get; private set; }
/* 288 */ public bool Allowed { get; private set; }
/* 291 */ public bool IsLatchBar(DateTime etClose)
/* 299 */ public void Latch(DateTime etClose, double barOpen, double atr20d, DateTime atrAsOf, bool allow, Ti
meSpan? earlyCloseEt = null, bool noRth = false)
/* 309 */ public bool HasSignal { get; }
/* 311 */ public A263Action Step(DateTime etClose, bool isLong)
/* 323 */ public string Suffix()
```

_EventCalendar.Function.CS

```
// --- (namespace)
/* 27 */ public enum EEventKind : byte { None = 0, FomcDecision = 1, FomcPress = 2, FomcMinutes = 3, Cpi =
4, Ppi = 5, Nfp = 6, Jolts = 7, Claims = 8, Pce = 9, GdpAdvance = 10, GdpSecond = 11, GdpThird = 12, RetailSa
les = 13, IsmMfg = 14, IsmSvc = 15, JacksonHole = 16, ChairTestimony = 17, EiaPetroleum = 18,
QuarterlyExpiry = 30, RollWeek = 31, MonthlyOpex = 32, IndexRebalance = 33, HalfDay = 34, VixExpiry = 35, Rus
sellRecon = 36, Custom = 62 }
/* 42 */ public enum EEventTier : byte { None = 0, T3 = 1, T2 = 2, T1 = 3, C = 8 }
/* 46 */ public enum EEventCoverage : byte { NoEvent = 0, Event = 1, Unknown = 2 }
```

```
/* 48 */ public enum EEventFlags : ushort { None = 0, Sep = 1, Quad = 2, Unscheduled = 4, Scheduled = 8, Dela
yed = 16, Canceled = 32, Csv = 64, RuleOnly = 128, Pm = 256 }
/* 55 */ public enum EEventClassMask : byte { None = 0, Macro = 1, Calendar = 2, Energy = 4, All = 7 }
/* 58 */ public struct EventRec
// --- EventRec
/* 60 */ public EEventKind kind;
/* 61 */ public EEventTier tier;
/* 62 */ public EEventFlags flags;
/* 63 */ public DateTime etTime;
/* 64 */ public DateTime etEnd;
/* 65 */ public DateTime utc;
/* 66 */ public DateTime tradingDay;
/* 67 */ public DateTime knownSinceUtc;
/* 68 */ public DateTime canceledSinceUtc;
/* 69 */ public string refPeriod;
/* 70 */ public string name;
/* 71 */ public string source;
/* 73 */ public bool IsSpan { get; }
/* 74 */ public EEventClassMask Class { get; }
/* 76 */ public static EventRec Make(EEventKind kind, DateTime et, EEventTier tier, EEventFlags flags = EEven
tFlags.None)
/* 84 */ public string KindName { get; }
// --- (namespace)
/* 88 */ public struct EventDay
// --- EventDay
/* 90 */ public DateTime day;
/* 91 */ public EEventTier maxTier;
/* 92 */ public ulong kindsMask;
/* 93 */ public bool hasFomc, hasSep, isHalfDay, isRollWeek, isQuarterlyExpiry, isOpex, isRebalance;
/* 94 */ public TimeSpan? firstReleaseEt;
/* 95 */ public int count;
/* 96 */ public bool eiaRuleOnly;
/* 98 */ public bool Has(EEventKind k)
// --- (namespace)
/* 102 */ public struct EventCorrection
// --- EventCorrection
/* 104 */ public EEventKind kind;
/* 105 */ public string refPeriod;
/* 106 */ public long originalEt;
/* 107 */ public long actualEt;
/* 108 */ public long announcedEt;
/* 109 */ public long canceledSinceEt;
/* 110 */ public EEventFlags flags;
/* 111 */ public string source;
// --- (namespace)
/* 115 */ public sealed class EventConfig
// --- EventConfig
/* 117 */ public bool enabled = false;
/* 118 */ public EEventClassMask classes = EEventClassMask.Macro | EEventClassMask.Calendar;
/* 119 */ public EEventTier minTier = EEventTier.T3;
/* 120 */ public Dictionary<EEventKind, EEventTier> tierOverride = new Dictionary<EEventKind, EEventTier>();
/* 121 */ public string csvPath = "";
/* 122 */ public bool allowPartialCoverage = false;
/* 123 */ public int blackoutPreMin = 0, blackoutPostMin = 0;
/* 124 */ public EEventTier blackoutTier = EEventTier.T1;
/* 125 */ public bool blackoutFlatten = false;
/* 126 */ public bool publish = false;
/* 128 */ public bool includeRussell = false;
/* 129 */ internal string csvSha = null;
/* 131 */ public static bool IsMacroTier(EEventTier t)
/* 134 */ public string Validate()
/* 149 */ public string Canon()
// --- (namespace)
/* 165 */ public static partial class EventRules
// --- EventRules
/* 167 */ public static readonly TimeSpan T0830 = new TimeSpan(8, 30, 0), T0930 = new TimeSpan(9, 30, 0), T10
00 = new TimeSpan(10, 0, 0),
/* 179 */ public static readonly DateTime RollRuleMondayFrom = new DateTime(2022, 4, 1);
/* 219 */ public static EEventTier DefaultTier(EEventKind k)
/* 233 */ public static EEventClassMask ClassOf(EEventKind k, EEventTier tier)
/* 241 */ public static readonly EEventKind[] TableKinds =
/* 246 */ public static bool IsTableKind(EEventKind k)
/* 249 */ public static DateTime ThirdFriday(int y, int m)
/* 254 */ public static bool IsNyseHoliday(DateTime d)
/* 260 */ public static bool IsNyseBusinessDay(DateTime d)
/* 261 */ public static DateTime PrevBusinessDayOrSelf(DateTime d)
/* 262 */ public static DateTime NthBusinessDay(int y, int m, int n)
```

```
/* 266 */ public static bool IsFederalHoliday(DateTime d)
/* 279 */ public static DateTime OpexDate(int y, int m)
/* 284 */ public static DateTime ExpiryDate(int y, int m)
/* 286 */ public static DateTime RuleExpiryDate(int y, int m)
/* 289 */ public static DateTime RollDate(int y, int m)
/* 298 */ public static DateTime VixExpiry(int y, int m)
/* 306 */ public static DateTime FromYmdhm(long v)
/* 311 */ public static long ToYmdhm(DateTime d)
/* 314 */ public static List<EventRec> Generate(int y, bool includeRussell)
/* 381 */ public static long[] CoverageHoles()
/* 382 */ public static string HoleWhyFor(long kind, long fromEt, long toEt)
/* 388 */ public static List<EventRec> Table()
/* 400 */ public static EventCorrection[] Corrections { get; }
/* 403 */ public static string TableSha()
/* 408 */ public static string ExpectedTableSha { get; }
/* 409 */ public static int TableRowCount { get; }
/* 412 */ public static string TableIntegrity(int rows, string actualSha, string expectedSha)
/* 419 */ public static string ComputeTableSha(long[] et, byte[] kind, ushort[] flags, string[] refs, EventCo
rrection[] corr, long[] holes)
/* 427 */ public static string Sha256(byte[] b)
// --- (namespace)
/* 479 */ public static partial class EventRules
/* 1095 */ internal sealed class _EventCalendar
// --- _EventCalendar
/* 1097 */ public EventConfig config;
/* 1098 */ public string label = "EVENT";
/* 1099 */ public Action<string> journal, warnOnce;
/* 1101 */ public Action<string> error;
/* 1103 */ public TimeSpan barDuration = TimeSpan.Zero;
/* 1105 */ public DateTime firstBarUtc = DateTime.MinValue;
/* 1107 */ public List<EventRec> table;
/* 1108 */ public EventCorrection[] corrections;
/* 1111 */ public long[] coverageHoles;
/* 1113 */ public string tableShaExpected;
/* 1115 */ sealed class Year
// --- _EventCalendar.Year
/* 1117 */ public EventRec[] recs;
/* 1118 */ public Dictionary<DateTime, int[]> dayIdx;
// --- _EventCalendar
/* 1120 */ struct CsvRow
// --- _EventCalendar.CsvRow
/* 1122 */ public int line; public string action; public EEventKind kind; public string name; public
DateTime date; public TimeSpan? time;
/* 1123 */ public EEventTier? tier; public EEventFlags flags; public string refPeriod; public DateTime? annou
nced; public string source;
// --- _EventCalendar
/* 1125 */ sealed class DayCache
/* 1139 */ struct CovSpan
/* 1151 */ public TimeZoneInfo Eastern { get; }
/* 1152 */ public DateTime NowUtc { get; }
/* 1153 */ public DateTime BarOpenUtc { get; }
/* 1154 */ public DateTime TradingDay { get; }
/* 1155 */ public bool Blackout { get; }
/* 1156 */ public bool FlattenNow { get; }
/* 1157 */ public bool DailyBars { get; }
/* 1158 */ public int RowsLoadedCsv { get; }
/* 1160 */ public bool Disabled { get; }
/* 1168 */ public string Configure()
/* 1263 */ public DateTime ToEt(DateTime utc)
/* 1267 */ public string LoadCsv(string text)
/* 1461 */ public DateTime TradingDayOf(DateTime utc)
/* 1475 */ public DateTime SessionOpenUtc(DateTime day)
/* 1486 */ public void Step(DateTime barTime, DateTime barOpenUtc, DateTime barCloseUtc, bool rollSplice)
/* 1569 */ public EEventCoverage Coverage(EEventKind kind, DateTime day)
/* 1579 */ public bool CoverageBlocks(DateTime day, EEventClassMask mask, EEventTier minTier, out EEventKind
kind)
/* 1586 */ public bool CoverageBlocks(DateTime day, EEventClassMask mask)
/* 1611 */ public List<string> CoverageReport(DateTime from, DateTime to, EEventClassMask mask, out bool flag
ged)
/* 1613 */ public List<string> CoverageReport(DateTime from, DateTime to, EEventClassMask mask, EEventTier mi
nTier, out bool flagged)
/* 1633 */ public List<string> CoverageStats()
/* 1667 */ public bool NextEvent(DateTime nowUtc, EEventTier minTier, EEventClassMask mask, out EventRec e)
/* 1679 */ public bool PrevEvent(DateTime nowUtc, EEventTier minTier, EEventClassMask mask, out EventRec e)
/* 1691 */ public bool NextEvent(EEventTier minTier, out EventRec e)
/* 1692 */ public bool PrevEvent(EEventTier minTier, out EventRec e)
/* 1695 */ public double MinutesTo(EEventTier minTier, EEventClassMask mask = EEventClassMask.Macro)
```

```
/* 1697 */ public double MinutesSince(EEventTier minTier, EEventClassMask mask = EEventClassMask.Macro)
/* 1699 */ public double MinutesSinceAt(DateTime nowUtc, EEventTier minTier, EEventClassMask mask = EEventCla
ssMask.Macro)
/* 1704 */ public bool IsInWindow(EEventTier minTier, int preMin, int postMin, EEventClassMask mask = EEventC
lassMask.Macro)
/* 1706 */ public bool IsInWindowAt(DateTime openUtc, DateTime closeUtc, EEventTier minTier, int preMin, int
postMin, EEventClassMask mask, out EventRec e)
/* 1748 */ public bool EventBlackout { get; }
/* 1749 */ public bool EventFlattenNow { get; }
/* 1752 */ public List<EventRec> EventsOn(DateTime day, EEventClassMask mask = EEventClassMask.All)
/* 1774 */ public List<EventRec> EventsOnExPost(DateTime day, bool includeCanceled = false, EEventClassMask m
ask = EEventClassMask.All)
/* 1786 */ public List<EventRec> EventsInRange(DateTime fromDay, DateTime toDay, EEventClassMask mask = EEven
tClassMask.All)
/* 1788 */ public List<EventRec> EventsInRangeExPost(DateTime fromDay, DateTime toDay, bool includeCanceled
= false, EEventClassMask mask = EEventClassMask.All)
/* 1792 */ public EventDay Classify(DateTime day)
/* 1793 */ public EventDay Classify(DateTime day, DateTime asOfUtc)
/* 1794 */ public EventDay ClassifyExPost(DateTime day)
/* 1832 */ public bool IsRollWeek(DateTime day)
/* 1838 */ public bool IsRollWeek()
/* 1840 */ public bool IsRollDay()
/* 1842 */ public int SessionsToExpiry(DateTime day)
/* 1850 */ public int SessionsToExpiry()
/* 1858 */ public static DateTime RollDateFor(DateTime expiry)
/* 1860 */ public static DateTime RuleRollDate(DateTime day)
/* 1863 */ public bool InSpan(EEventKind kind, DateTime utc)
/* 1871 */ public bool InSpan(EEventKind kind)
/* 1874 */ public bool LastT1Instant(out DateTime et)
```

_PortMM.Function.CS

```
// --- (namespace)
/* 19 */ public static class _PortKeys
// --- _PortKeys
/* 21 */ public const int Version = 1;
/* 22 */ public const string Ver = "pm.ver", Id = "pm.id", Family = "pm.family", Cluster = "pm.cluster";
/* 23 */ public const string Eq = "pm.eq", Pnl = "pm.pnl", Ret = "pm.ret", URet = "pm.uRet", Vol = "pm.vol";
/* 24 */ public const string RDollar = "pm.R$", QtyCode = "pm.qtyCode", RiskNew = "pm.riskNew", RiskOpen = "p
m.riskOpen", OpenR = "pm.openR";
/* 25 */ public const string State = "pm.state", Side = "pm.side", Regime = "pm.regime", Score = "pm.score",
Stamp = "pm.stamp", Day = "pm.day", Pos = "pm.pos";
/* 28 */ public const string RegimeP = "pm.regimeP";
/* 32 */ public const string EvtTier = "pm.evtTier";
/* 33 */ public const string EvtMask = "pm.evtMask";
/* 34 */ public const string EvtWin = "pm.evtWin";
/* 35 */ public const string EvtNext = "pm.evtNext";
/* 37 */ public const string Mult = "pm.mult", Heat = "pm.heat", Dd = "pm.dd", Rho = "pm.rho";
// --- (namespace)
/* 40 */ public enum EPortState { Flat = 0, Armed = 1, Long = 2, Short = 3, Exiting = 4, Halted = 5 }
/* 41 */ public enum EPortSizing { CodeSize = 0, RiskParity = 1, VolTarget = 2 }
/* 44 */ public sealed class _PortReturns
// --- _PortReturns
/* 46 */ public const int DefaultCapacity = 260;
/* 48 */ public int Count { get; }
/* 49 */ public int Capacity { get; private set; }
/* 50 */ public DateTime LastDay { get; }
/* 51 */ public _PortReturns(int capacity = DefaultCapacity)
/* 53 */ public DateTime Day(int i)
/* 54 */ public double Value(int i)
/* 56 */ public _PortReturns Append(DateTime day, double value)
/* 65 */ public Dictionary<DateTime, double> ToMap()
// --- (namespace)
/* 68 */ public sealed class PortDdStep
/* 71 */ public sealed class PortMmConfig
// --- PortMmConfig
/* 73 */ public EPortSizing sizing = EPortSizing.RiskParity;
/* 74 */ public double bookVolTarget = 0.10;
/* 75 */ public double riskPerTrade = 0.005;
/* 76 */ public double heatCap = 0.04;
/* 77 */ public double clusterCap = 0.02;
/* 78 */ public double rhoMerge = 0.9;
/* 79 */ public int rhoWin = 60;
/* 80 */ public int rhoMinDays = 40;
```

```
/* 81 */ public int maxPositions = 6;
/* 82 */ public PortDdStep[] ddSteps = { new PortDdStep(0.05, 0.5), new PortDdStep(0.10, 0.25), new PortDdSte
p(0.15, 0.0) };
/* 83 */ public int ddRecoverHighDays = 20;
/* 84 */ public double dailyLossStop = 0.02;
/* 85 */ public double dailyLossFlatten = 0.03;
/* 86 */ public double stratKillR = -8;
/* 87 */ public int stratKillTrades = 20;
/* 88 */ public int staleBars = 3;
/* 89 */ public int minContracts = 1;
/* 90 */ public double idmCap = 2.5;
/* 92 */ public string Validate()
/* 116 */ public PortMmConfig Copy()
/* 123 */ public override string ToString()
// --- (namespace)
/* 138 */ public static class PortMmConfigLoader
// --- PortMmConfigLoader
/* 141 */ public static PortMmConfig LoadFile(string path, PortMmConfig defaults, List<string> warnings,
List<string> info)
/* 157 */ public static PortMmConfig Parse(string json, PortMmConfig defaults, List<string> warnings, string
source = "json")
/* 214 */ public static PortMmConfig Clone(PortMmConfig d)
// --- (namespace)
/* 268 */ public sealed class PortStratView
// --- PortStratView
/* 270 */ public int index;
/* 271 */ public string id = "";
/* 272 */ public bool publishes;
/* 273 */ public bool stale;
/* 274 */ public bool currencyOk = true;
/* 275 */ public bool paused;
/* 276 */ public EPortState state;
/* 277 */ public int side;
/* 278 */ public double rDollar = double.NaN;
/* 279 */ public int qtyCode;
/* 280 */ public double riskOpen;
/* 281 */ public double vol = double.NaN;
/* 282 */ public string cluster = "";
/* 283 */ public int position;
/* 284 */ public double pnl;
/* 285 */ public double score = double.NaN;
/* 286 */ public _PortReturns ret, uRet;
// --- (namespace)
/* 289 */ public sealed class PortDecision
// --- PortDecision
/* 291 */ public int index;
/* 292 */ public bool allowEntries, allowLong, allowShort;
/* 293 */ public int entryContracts = -1;
/* 294 */ public bool forceClose, pause;
/* 295 */ public string status = "";
/* 296 */ public string reason = "";
// --- (namespace)
/* 300 */ public static class _PortMath
// --- _PortMath
/* 303 */ public static double Pearson(_PortReturns a, _PortReturns b, int win, int minDays, Func<DateTime, b
ool> dayFilter = null)
/* 316 */ public static double Corr(IList<double> x, IList<double> y)
/* 325 */ public static int[] Clusters(string[] seeds, double[,] rho, double threshold)
/* 337 */ public static double Idm(int[] active, double[,] rho, int[] cluster, double cap)
/* 352 */ public static int DdStage(PortDdStep[] steps, double dd)
/* 358 */ public static int Size(PortMmConfig c, double mult, double book, double rDollar, int qtyCode, doubl
e vol, int k, double idm)
// --- (namespace)
/* 374 */ internal sealed class _PortMM
// --- _PortMM
/* 376 */ public PortMmConfig config = new PortMmConfig();
/* 377 */ public Action<string> journal;
/* 393 */ public double Mult { get; private set; }
/* 394 */ public double Heat { get; private set; }
/* 395 */ public double Drawdown { get; private set; }
/* 396 */ public double Book { get; private set; }
/* 397 */ public double[,] Rho { get; }
/* 398 */ public int DdStageNow { get; }
/* 399 */ public bool DayStopped { get; }
/* 401 */ public string Configure()
/* 407 */ public PortDecision[] Decide(DateTime day, double initialCapital, bool currencyOk, IList<PortStratV
iew> v)
```

_VolumeProfileBase.Strategy.CS

```
// --- (namespace)
/* 24 */ public abstract class _VolumeProfileBase : _CustomSignal
// --- _VolumeProfileBase
/* 27 */ public const double DEFAULT_HVN_PERCENTILE = 0.8;
/* 28 */ public const double DEFAULT_LVN_PERCENTILE = 0.2;
/* 29 */ public const int DEFAULT_SMOOTH_TICKS = 3;
/* 30 */ public const int DEFAULT_MIN_NODE_TICKS = 3;
/* 31 */ public const double PRICE_EPSILON = 1e-12;
/* 32 */ public const double DEFAULT_MAX_LVN_PCT_OF_RANGE = 0.12;
/* 33 */ public const int DEFAULT_VWAP_LENGTH = 20;
/* 34 */ public const int DEFAULT_CONFLUENCE_LOOKBACK = 4;
/* 35 */ public const int DEFAULT_CONFLUENCE_ALIGN_TICKS = 4;
/* 36 */ public const int DEFAULT_CONFLUENCE_TOP_COUNT = 5;
/* 37 */ public const int DEFAULT_CONFLUENCE_TARGET_CLEAR_TICKS = 1;
/* 38 */ public const double DEFAULT_CONFLUENCE_STREAM_WEIGHT_1 = 1.00;
/* 39 */ public const double DEFAULT_CONFLUENCE_STREAM_WEIGHT_2 = 1.25;
/* 40 */ public const double DEFAULT_CONFLUENCE_STREAM_WEIGHT_3 = 1.50;
/* 41 */ public const double DEFAULT_CONFLUENCE_NAKED_MULTIPLIER = 1.15;
/* 42 */ public const double DEFAULT_CONFLUENCE_REFINEMENT = 0.25;
/* 43 */ public const double DEFAULT_CONFLUENCE_OVERLAY_FACTOR = 0.50;
/* 44 */ public const double DEFAULT_CONFLUENCE_SHARE_PIVOT = 0.15;
/* 45 */ public const double DEFAULT_EXCEPTIONAL_VOLUME_PERCENTILE = 0.95;
/* 46 */ public const int MAX_DENSE_ROWS = 20000;
/* 48 */ internal const double FALLBACK_TICK = 0.01;
/* 49 */ internal const int CONFLUENCE_NODES_PER_PROFILE = 3;
/* 50 */ internal const double SHARE_FACTOR_MIN = 0.5;
/* 51 */ internal const double SHARE_FACTOR_MAX = 1.5;
/* 52 */ internal const double LEVEL_MATCH_TICK_FRACTION = 0.51;
/* 53 */ public const int DEFAULT_LVN_HVN_MAX_GAP_TICKS = 4;
/* 59 */ public enum EVwapType { Disabled = 0, Session = 1, Length = 2, AdaptiveFromDate = 3, Week = 4,
Month = 5, Quarter = 6, RollDate = 7 }
/* 75 */ internal struct Row
// --- _VolumeProfileBase.Row
/* 77 */ public double price;
/* 78 */ public double volume;
// --- _VolumeProfileBase
/* 84 */ public struct Node
// --- _VolumeProfileBase.Node
/* 86 */ public double low;
/* 87 */ public double high;
/* 88 */ public double peak;
/* 89 */ public double peakVol;
/* 90 */ public double strength;
/* 91 */ public bool isPoc;
// --- _VolumeProfileBase
/* 131 */ public class VpExtra
// --- _VolumeProfileBase.VpExtra
/* 133 */ public int dataStream;
/* 134 */ public int startBar;
/* 135 */ public int endBar;
/* 136 */ public double open;
/* 137 */ public double high;
/* 138 */ public double low;
/* 139 */ public double close;
/* 140 */ public double poc;
/* 141 */ public double pocVol;
/* 142 */ public bool pocComplete;
/* 143 */ public bool pocStale;
/* 144 */ public bool isDeveloping;
/* 145 */ public bool approximate;
/* 146 */ public bool isOwn;
/* 147 */ public bool isPartial;
/* 148 */ public bool crossesRoll;
/* 149 */ public bool pocNaked;
/* 150 */ public int pocTouchedBar;
/* 160 */ internal int lastNakedScanBar;
/* 161 */ internal bool nakedTracked;
/* 162 */ internal int ohlcStartBar;
/* 163 */ internal int ohlcFoldedThrough;
/* 164 */ internal int ohlcEndBar;
/* 165 */ internal bool nodesBuilt;
/* 166 */ internal int nodesBuiltForStart;
/* 167 */ internal int nodesBuiltGeneration = -1;
```

```
/* 168 */ internal int filledAtBar = -1;
/* 169 */ internal bool filledAtClose;
/* 170 */ internal int filledGeneration = -1;
/* 172 */ internal bool inClosedSet;
/* 173 */ internal int pctStampedFill = -1;
/* 174 */ internal int pctStampedClosedGen = -1;
/* 181 */ public bool IsNakedPoc(int asOfBar)
/* 186 */ public double vah;
/* 187 */ public double val;
/* 188 */ public double vwap1;
/* 189 */ public double vwap2;
/* 190 */ public double vwap3;
/* 191 */ public double vwap1Sigma;
/* 192 */ public double vwap2Sigma;
/* 193 */ public double vwap3Sigma;
/* 194 */ public int vwap1Stream;
/* 195 */ public int vwap2Stream;
/* 196 */ public int vwap3Stream;
/* 197 */ public readonly List<Node> hvns = new List<Node>();
/* 198 */ public readonly List<Node> lvns = new List<Node>();
/* 205 */ public readonly List<LevelVol> levels = new List<LevelVol>();
/* 206 */ public double maxTotalPercentile;
/* 207 */ public double maxAskPercentile;
/* 208 */ public double maxBidPercentile;
/* 210 */ public bool HasExceptionalVolume(double minPercentile)
/* 218 */ public bool HasExceptionalVolume()
/* 223 */ public List<LevelVol> ExceptionalLevels(double minPercentile)
/* 238 */ public LevelVol PeakTotalLevel()
// --- _VolumeProfileBase
/* 255 */ public struct LevelVol
// --- _VolumeProfileBase.LevelVol
/* 257 */ public double price;
/* 258 */ public double total;
/* 259 */ public double ask;
/* 260 */ public double bid;
/* 261 */ public double totalPercentile;
/* 262 */ public double askPercentile;
/* 263 */ public double bidPercentile;
/* 265 */ public bool IsExceptional(double minPercentile)
// --- _VolumeProfileBase
/* 277 */ public enum EConfluenceKind { Poc = 0, Vah = 1, Val = 2, Hvn = 3, Vwap1 = 4, Vwap2 = 5, Vwap3 = 6,
NakedPoc = 7, Lvn = 8 }
/* 294 */ public enum EConfluenceRole { Acceptance = 0, Rejection = 1, Magnet = 2, Mixed = 3 }
/* 305 */ public enum ESuggestedUse { Target = 0, Entry = 1, Conflict = 2 }
/* 315 */ public enum EConfluenceLocation { Unknown = 0, Premium = 1, Discount = 2, InsideValue = 3 }
/* 327 */ public enum EVpDirection { Bearish = -1, Bullish = 1 }
/* 336 */ public struct ConfluenceHit
// --- _VolumeProfileBase.ConfluenceHit
/* 338 */ public int dataStream;
/* 339 */ public int profileStartBar;
/* 340 */ public int profileAge;
/* 341 */ public EConfluenceKind kind;
/* 342 */ public double price;
/* 343 */ public double weight;
/* 344 */ public string sourceVolumeId;
/* 345 */ public string absorbedLabels;
/* 346 */ public double credit;
/* 347 */ public double volumeShare;
/* 348 */ public bool naked;
/* 349 */ public bool independent;
/* 350 */ public bool developing;
// --- _VolumeProfileBase
/* 356 */ public class ConfluenceDrop
// --- _VolumeProfileBase.ConfluenceDrop
/* 358 */ public int dataStream;
/* 359 */ public int profileStartBar;
/* 360 */ public EConfluenceKind kind;
/* 361 */ public double price;
/* 362 */ public string sourceVolumeId;
/* 363 */ public string reason;
// --- _VolumeProfileBase
/* 369 */ public class ConfluenceZone
// --- _VolumeProfileBase.ConfluenceZone
/* 371 */ public double low;
/* 372 */ public double high;
/* 373 */ public double center;
/* 374 */ public double rawScore;
```

```
/* 375 */ public double zScore;
/* 376 */ public double anchor;
/* 377 */ public double acceptanceScore;
/* 378 */ public double rejectionScore;
/* 379 */ public double contextScore;
/* 380 */ public int independentSources;
/* 381 */ public EConfluenceRole role;
/* 382 */ public EConfluenceLocation location;
/* 383 */ public ESuggestedUse suggestedUse;
/* 384 */ public double exitPrice;
/* 385 */ public double entryPrice;
/* 386 */ public double invalidationBelow;
/* 387 */ public double invalidationAbove;
/* 388 */ public int hitCount;
/* 389 */ public int streamCount;
/* 390 */ public int profileCount;
/* 391 */ public readonly List<ConfluenceHit> hits = new List<ConfluenceHit>();
/* 395 */ public List<ConfluenceHit> developingHits;
/* 396 */ public double developingScore;
/* 397 */ public bool HasDeveloping { get; }
/* 398 */ public readonly List<ConfluenceDrop> dropped = new List<ConfluenceDrop>();
/* 400 */ public bool HasStream(int dataStream)
/* 407 */ public bool HasKind(EConfluenceKind kind)
// --- _VolumeProfileBase
/* 420 */ public class ConfluenceTarget
// --- _VolumeProfileBase.ConfluenceTarget
/* 422 */ public double price;
/* 423 */ public double touch;
/* 424 */ public double distance;
/* 425 */ public EVpDirection direction;
/* 426 */ public ConfluenceZone zone;
// --- _VolumeProfileBase
/* 494 */ [Input] public double HvnPercentile { get; set; }
/* 499 */ [Input] public double LvnPercentile { get; set; }
/* 500 */ [Input] public int MinNodeTicks { get; set; }
/* 501 */ [Input] public bool TrackNakedPoc { get; set; }
/* 502 */ [Input] public int NakedPocTickTolerance { get; set; }
/* 505 */ [Input] public EVwapType Vwap1Type { get; set; }
/* 506 */ [Input] public int Vwap1Length { get; set; }
/* 507 */ [Input] public string Vwap1FromDate { get; set; }
/* 508 */ [Input] public int Vwap1DataStream { get; set; }
/* 510 */ [Input] public EVwapType Vwap2Type { get; set; }
/* 511 */ [Input] public int Vwap2Length { get; set; }
/* 512 */ [Input] public string Vwap2FromDate { get; set; }
/* 513 */ [Input] public int Vwap2DataStream { get; set; }
/* 515 */ [Input] public EVwapType Vwap3Type { get; set; }
/* 516 */ [Input] public int Vwap3Length { get; set; }
/* 517 */ [Input] public string Vwap3FromDate { get; set; }
/* 518 */ [Input] public int Vwap3DataStream { get; set; }
/* 541 */ public int ConfluenceDataStream1
/* 551 */ public int ConfluenceDataStream2
/* 560 */ public int ConfluenceDataStream3
/* 569 */ public double ConfluenceWeight1
/* 578 */ public double ConfluenceWeight2
/* 587 */ public double ConfluenceWeight3
/* 597 */ public int ConfluenceLookbackProfiles
/* 606 */ public int ConfluenceAlignTicks
/* 614 */ public int ConfluenceTopCount
/* 622 */ public bool ConfluenceIncludeHvns { get; set; }
/* 626 */ protected internal double ConfluenceSplitAtrFraction { get; set; }
/* 629 */ internal double confluenceAtr20d()
/* 636 */ protected bool UsesConfluenceZones { get; set; }
/* 640 */ internal void confluenceJournal(Func<string> text)
/* 647 */ public double MaxLvnFractionOfRange { get; set; }
/* 651 */ public bool ConfluenceIncludeVwapSlots { get; set; }
/* 655 */ public double ConfluenceTargetMinZScore { get; set; }
/* 660 */ public int ConfluenceTargetClearTicks
/* 668 */ public int ConfluenceTargetCount
/* 696 */ internal DateTime vwapSessionDate(DateTime barTime)
/* 698 */ internal bool vwapRollOn(int stream)
/* 715 */ internal _VpVwapEngine m_vwap1;
/* 716 */ internal _VpVwapEngine m_vwap2;
/* 717 */ internal _VpVwapEngine m_vwap3;
/* 718 */ internal readonly List<ConfluenceZone> m_confluenceZones = new List<ConfluenceZone>();
/* 737 */ public sealed class ProfileMath : _ProfileMath
/* 744 */ public enum EDevelopingSource { MultiCharts = 0, Own = 1 }
/* 746 */ public enum EDevProfileSourceMode { Auto = 0, VolumeAtPrice = 1, BarUniform = 2, BarTypical = 3 }
```

```
/* 748 */ public enum EValueAreaMethod { MultiCharts = 0, VolumeExpansion = 1, Sigma = 2 }
/* 750 */ public const double DEFAULT_VALUE_AREA_PERCENT = 0.70;
/* 751 */ public const double DEFAULT_VALUE_AREA_SIGMA = 1.0;
/* 756 */ [Input] public EDevelopingSource DevelopingSource { get; set; }
/* 758 */ [Input] public EDevProfileSourceMode DevProfileSourceMode { get; set; }
/* 760 */ [Input] public int DevProfileDataStream { get; set; }
/* 762 */ [Input] public EValueAreaMethod ValueAreaMethod { get; set; }
/* 764 */ [Input] public bool LookaheadAudit { get; set; }
/* 767 */ public double ValueAreaPercent { get; set; }
/* 769 */ public double ValueAreaSigma { get; set; }
/* 771 */ protected bool AuditPlantLeak { get; set; }
/* 817 */ protected virtual bool isNewProfileWindow(int stream, int barIndex, DateTime barTime, DateTime prev
BarTime)
/* 823 */ protected virtual bool includeBarInProfile(DateTime barTime)
/* 912 */ protected void fillFromHistogram(VpExtra e, double[] prices, double[] vols)
/* 936 */ public VpExtra DevelopingExtra(int stream)
/* 947 */ public List<VpExtra> OwnCompletedProfiles(int count)
/* 1081 */ public ProfileMath.ValueRel ValueRelationship(VpExtra prev, VpExtra cur)
/* 1088 */ public bool IsPoorHigh(VpExtra e, double fraction = 0.5, int minRows = 2)
/* 1095 */ public bool IsPoorLow(VpExtra e, double fraction = 0.5, int minRows = 2)
/* 1102 */ public bool HasUnfinishedAuction(VpExtra e, bool high)
/* 1115 */ public List<double[]> SinglePrints(VpExtra e, int minTicks, double pct = 0.2)
/* 1127 */ public ProfileMath.Shape ProfileShape(VpExtra e)
/* 1138 */ public bool IsLevelNaked(VpExtra e, double level, int asOfBar)
/* 1157 */ public bool IsNakedVah(VpExtra e, int asOfBar)
/* 1158 */ public bool IsNakedVal(VpExtra e, int asOfBar)
/* 1164 */ public int AcceptanceOutsideValue(VpExtra reference, bool above, int minCloses)
/* 1186 */ protected virtual List<VpExtra> migrationProfiles(int stream, int count)
/* 1198 */ public int? ValueMigrationScore(int stream, int n = 3, double minChangeAtrFraction = 0.10, int atr
Len = 20, bool includeDeveloping = true, double dailyAtr = 0)
/* 1232 */ public ProfileMath.MigrationState ValueMigrationState(int stream, int n = 3, double minChangeAtrFr
action = 0.10, int atrLen = 20, bool includeDeveloping = true, double dailyAtr = 0, int upAt = 2, int downAt
= 2)
/* 1239 */ protected virtual bool profileCrossesRoll(VpExtra e)
/* 1249 */ public VpExtra Composite(int stream, int n)
/* 1275 */ public bool IsBalanced(int stream, int n, double minVaOverlap = 0.6)
/* 1285 */ public bool BalanceRange(int stream, int n, bool union, out double high, out double low)
/* 1309 */ public int ValueMigrationScore(int stream, double atr20Daily, int n = 3, double minChangeFraction
= 0.10, bool includeDeveloping = true)
/* 1327 */ public _VolumeProfileBase(object _ctx)
/* 1329 */ protected override void Create()
/* 1380 */ protected override void Destroy()
/* 1386 */ protected override void StartCalc()
/* 1421 */ protected override void CalcBar()
/* 1442 */ public VpExtra ExtraVolumeProfileForBar(int barIndex)
/* 1453 */ public VpExtra ExtraVolumeProfileForBar(int datastream, int barIndex)
/* 1495 */ public List<VpExtra> RecentCompletedProfiles(int datastream, int count, int asOfBar)
/* 1513 */ public List<VpExtra> NakedPocs(int datastream)
/* 1524 */ public List<VpExtra> NakedPocs(int datastream, int asOfBar)
/* 1547 */ public bool IsNewVolumeProfile()
/* 1555 */ public bool IsNewVolumeProfile(int datastream)
/* 1567 */ public bool IsNewVolumeProfile(int datastream, int barIndex)
/* 1602 */ public List<ConfluenceZone> GetConfluenceZones()
/* 1614 */ public List<ConfluenceZone> GetTopConfluenceZones()
/* 1623 */ public List<ConfluenceZone> GetTopConfluenceZones(int count)
/* 1640 */ public List<ConfluenceTarget> GetConfluenceTargets(double price, EVpDirection direction)
/* 1651 */ public List<ConfluenceTarget> GetConfluenceTargets(double price, EVpDirection direction, int
count)
/* 1665 */ public List<ConfluenceZone> GetConfluenceTargetZones(double price, EVpDirection direction)
/* 1673 */ public List<ConfluenceZone> GetConfluenceTargetZones(double price, EVpDirection direction, int cou
nt)
/* 1685 */ public void InvalidateConfluenceZones()
/* 1696 */ public void RefreshVolumePercentiles()
/* 1708 */ public double CurrentVwap1 { get; }
/* 1710 */ public double CurrentVwap2 { get; }
/* 1712 */ public double CurrentVwap3 { get; }
/* 1714 */ public double CurrentVwapSigma1 { get; }
/* 1717 */ public double CurrentVwap1Sigma { get; }
/* 1718 */ public double CurrentVwap2Sigma { get; }
/* 1720 */ public double CurrentVwap3Sigma { get; }
/* 1722 */ public bool Vwap1Active { get; }
/* 1724 */ public bool Vwap2Active { get; }
/* 1726 */ public bool Vwap3Active { get; }
/* 1728 */ public int CurrentVwap1Stream { get; }
/* 1730 */ public int CurrentVwap2Stream { get; }
/* 1732 */ public int CurrentVwap3Stream { get; }
/* 1805 */ internal int streamItemBarIndex(int stream)
```

```
/* 1819 */ internal static int toNativeItemBar(int chartBar)
/* 2359 */ internal List<VpExtra> confluenceLookbackExtras(int datastream)
/* 2445 */ internal double helperTickSize()
/* 2447 */ internal DateTime barTime(int stream, int chartBar)
/* 2650 */ protected void buildNodesFromLevels(VpExtra extra)
/* 2982 */ protected RangeProfileConfig RangeProfiles { get; }
/* 2984 */ protected WeeklyProfileConfig WeeklyProfiles { get; }
/* 2986 */ protected ProfileStatsConfig ProfileStats { get; }
/* 2988 */ protected BarLadderConfig BarLadders { get; }
/* 3013 */ protected virtual bool sessionOpenAt(DateTime slotStart)
/* 3015 */ protected virtual _OrderFlow.FootprintBar footprintForLadder(int barNumber)
/* 3017 */ protected virtual bool ladderHasFlow { get; }
/* 3092 */ public VpExtra RangeProfile(int stream, DateTime t0, DateTime t1)
/* 3096 */ public VpExtra RangeProfileAfter(int stream, DateTime t0, DateTime t1)
/* 3126 */ public VpExtra RangeProfileBars(int stream, int barsBack, int count)
/* 3136 */ public RangeProfileHandle OpenRange(int stream, DateTime t0, string tag)
/* 3155 */ public void CloseRange(RangeProfileHandle h, DateTime t1)
/* 3163 */ public VpExtra RangeHandleProfile(RangeProfileHandle h)
/* 3173 */ public RangeStats RangeStatsOf(VpExtra e)
/* 3180 */ public List<LevelRef> RangeLevels(VpExtra e, DateTime t0, DateTime t1, string tag)
/* 3317 */ public VpExtra Week(int weeksAgo)
/* 3341 */ public VpExtra WeeklyComposite(int nWeeks)
/* 3347 */ public int CompletedWeeks { get; }
/* 3348 */ public DateTime WeekKey(int weeksAgo)
/* 3351 */ public List<LevelRef> WeekLevels(int weeksAgo)
/* 3391 */ public LadderBar Ladder(int ago)
/* 3393 */ public LadderBar Ladder(int ago, _OrderFlow.FlowQuality minQuality)
/* 3403 */ public LadderBar LadderAt(DateTime barTime)
/* 3412 */ public LadderBar HtfLadder(int htfStream, int ago)
/* 3437 */ public VpExtra LadderProfile(LadderBar b)
/* 3458 */ public EntropyResult ProfileEntropy(VpExtra e, double mmN)
/* 3468 */ public List<RingBar> SessionBars(DateTime tradingDay, bool rthOnly)
```

_VolumeProfileFuturesBase.Strategy.CS

```
// --- (namespace)
/* 49 */ public abstract class _VolumeProfileFuturesBase : _VolumeProfileBase
// --- _VolumeProfileFuturesBase
/* 52 */ public const string DEFAULT_ASIA_START = "18:00";
/* 53 */ public const string DEFAULT_ASIA_END = "03:00";
/* 54 */ public const string DEFAULT_EUROPE_START = "03:00";
/* 55 */ public const string DEFAULT_EUROPE_END = "11:30";
/* 56 */ public const string DEFAULT_NEWYORK_START = "08:00";
/* 57 */ public const string DEFAULT_NEWYORK_END = "17:00";
/* 58 */ public const string DEFAULT_RTH_START = "09:30";
/* 59 */ public const string DEFAULT_RTH_END = "16:15";
/* 60 */ public const string DEFAULT_HALT_START = "17:00";
/* 61 */ public const string DEFAULT_HALT_END = "18:00";
/* 62 */ public const string DEFAULT_GLOBEX_OPEN = "18:00";
/* 63 */ public const string DEFAULT_GLOBEX_CLOSE = "17:00";
/* 66 */ public static readonly TimeSpan DefaultAsiaStart = new TimeSpan(18, 0, 0);
/* 67 */ public static readonly TimeSpan DefaultAsiaEnd = new TimeSpan(3, 0, 0);
/* 68 */ public static readonly TimeSpan DefaultEuropeStart = new TimeSpan(3, 0, 0);
/* 69 */ public static readonly TimeSpan DefaultEuropeEnd = new TimeSpan(11, 30, 0);
/* 70 */ public static readonly TimeSpan DefaultNewYorkStart = new TimeSpan(8, 0, 0);
/* 71 */ public static readonly TimeSpan DefaultNewYorkEnd = new TimeSpan(17, 0, 0);
/* 72 */ public static readonly TimeSpan DefaultRthStart = new TimeSpan(9, 30, 0);
/* 73 */ public static readonly TimeSpan DefaultRthEnd = new TimeSpan(16, 15, 0);
/* 74 */ public static readonly TimeSpan DefaultHaltStart = new TimeSpan(17, 0, 0);
/* 75 */ public static readonly TimeSpan DefaultHaltEnd = new TimeSpan(18, 0, 0);
/* 76 */ public static readonly TimeSpan DefaultGlobexOpen = new TimeSpan(18, 0, 0);
/* 77 */ public static readonly TimeSpan DefaultGlobexClose = new TimeSpan(17, 0, 0);
/* 84 */ public enum MarketSession { None = 0, Asia = 1 << 0, Europe = 1 << 1, NewYork = 1 << 2, Rth = 1 <<
3, Premarket = 1 << 4, PostMarket = 1 << 5 }
/* 98 */ public enum GlobexState { Closed, Halt, Open }
/* 107 */ protected TimeSpan m_asiaStart, m_asiaEnd, m_euStart, m_euEnd, m_nyStart, m_nyEnd;
/* 108 */ protected TimeSpan m_rthStart, m_rthEnd, m_haltStart, m_haltEnd, m_globexOpen, m_globexClose;
/* 113 */ public _VolumeProfileFuturesBase(object _ctx)
/* 115 */ protected override void Create()
/* 125 */ protected override void presetExtraPriceAction(PriceActionConfig c)
/* 127 */ protected override void StartCalc()
/* 138 */ protected override void CalcBar()
/* 153 */ protected virtual void configureSessions()
/* 175 */ protected _CmeSessionClock SessionClock { get; }
```

```
/* 204 */ protected DateTime ToEastern(DateTime barTime)
/* 212 */ protected DateTime EasternNow()
/* 214 */ protected GlobexState GetGlobexState()
/* 217 */ protected GlobexState GetGlobexState(DateTime barTime)
/* 223 */ protected MarketSession GetSessions()
/* 226 */ protected MarketSession GetSessions(DateTime barTime)
/* 228 */ protected bool InSession(MarketSession session)
/* 233 */ protected bool InAny(MarketSession sessions)
/* 238 */ protected bool IsRth()
/* 239 */ protected bool IsAsia()
/* 240 */ protected bool IsEurope()
/* 241 */ protected bool IsNewYork()
/* 243 */ protected bool IsLondonNyOverlap()
/* 255 */ protected DateTime GlobexDate(DateTime barTime)
/* 261 */ protected bool IsNewGlobexDay()
/* 272 */ public MarketSession PrimarySession()
/* 278 */ public MarketSession PrimarySession(DateTime eastern)
/* 281 */ public TimeSpan TimeSinceOpen(MarketSession session)
/* 292 */ public TimeSpan TimeSinceOpen(DateTime eastern, MarketSession session)
/* 294 */ public int MinutesSinceOpen(MarketSession session)
/* 299 */ public int MinutesSinceOpen(DateTime eastern, MarketSession session)
/* 301 */ public TimeSpan TimeUntilClose(MarketSession session)
/* 306 */ public TimeSpan TimeUntilClose(DateTime eastern, MarketSession session)
/* 308 */ public int MinutesUntilClose(MarketSession session)
/* 313 */ public int MinutesUntilClose(DateTime eastern, MarketSession session)
/* 315 */ public TimeSpan SessionLength(MarketSession session)
/* 321 */ public bool IsOpeningRange(MarketSession session, int minutes)
/* 326 */ public bool IsOpeningRange(DateTime eastern, MarketSession session, int minutes)
/* 329 */ public bool IsRthAfterInitialBalance(int ibMinutes)
/* 334 */ public bool IsRthAfterInitialBalance(DateTime eastern, int ibMinutes)
/* 340 */ public bool TrySessionBounds( DateTime eastern, MarketSession session, out DateTime open, out DateT
ime close)
/* 356 */ protected override DateTime tradingDayKey(DateTime barTime)
/* 359 */ protected override TimeSpan sessionClock(DateTime barTime)
/* 369 */ public sealed class SessionStats
// --- _VolumeProfileFuturesBase.SessionStats
/* 371 */ public DateTime globexDate;
/* 372 */ public double globexHigh, globexLow, globexVolume;
/* 373 */ public double ethHigh, ethLow, ethVolume;
/* 374 */ public double rthOpen, rthHigh, rthLow, rthClose, rthVolume;
/* 375 */ public double ibHigh, ibLow;
/* 376 */ public bool ibComplete;
/* 377 */ public bool rthComplete;
/* 378 */ public double priorRthHigh, priorRthLow, priorRthClose;
/* 379 */ public double priorGlobexHigh, priorGlobexLow;
/* 380 */ public double IbRange { get; }
/* 382 */ public double IbExtensionUp { get; }
/* 384 */ public double IbExtensionDown { get; }
/* 386 */ public double Gap { get; }
// --- _VolumeProfileFuturesBase
/* 390 */ protected int IbMinutes { get; set; }
/* 392 */ protected TimeSpan m_rthValueEnd = DefaultRthValueEnd;
/* 393 */ public static readonly TimeSpan DefaultRthValueEnd = new TimeSpan(16, 0, 0);
/* 399 */ public SessionStats Session { get; }
/* 402 */ protected TimeSpan RthValueEndEt(DateTime globexDate)
/* 405 */ protected bool IsRthValueBarEt(DateTime et)
/* 467 */ public enum EOwnProfileWindow { MultiChartsProfile = 0, GlobexDay = 1, Rth = 2 }
/* 473 */ protected EOwnProfileWindow OwnProfileWindow { get; set; }
/* 475 */ protected override bool isNewProfileWindow(int stream, int barIndex, DateTime barTime, DateTime pre
vBarTime)
/* 484 */ protected override int todMinuteOfSession(DateTime barTime)
/* 491 */ protected override bool todHalfDay(DateTime sessionDate)
/* 498 */ protected override double todDelta(out int tier)
/* 512 */ protected override bool isRthBar(DateTime barTime)
/* 515 */ protected override bool rthKnown()
/* 517 */ protected override bool includeBarInProfile(DateTime barTime)
/* 524 */ public VpExtra RthProfile()
/* 530 */ public VpExtra GlobexProfile()
/* 536 */ protected override List<VpExtra> migrationProfiles(int stream, int count)
/* 548 */ public VpExtra PriorRthProfile()
/* 564 */ protected override bool profileCrossesRoll(VpExtra e)
/* 578 */ public enum EFlowTickSource { DataLoader = 0, Iog = 1, None = 2 }
/* 588 */ protected bool OrderFlowEnabled { get; set; }
/* 591 */ protected _OrderFlow.FlowConfig FlowCfg { get; }
/* 592 */ protected EFlowTickSource FlowTickSource { get; set; }
/* 594 */ protected bool IogMagnifierOneTick { get; set; }
/* 606 */ protected string McBidAskChartTypeName { get; set; }
```

```
/* 612 */ protected _OrderFlow.FlowEngine Flow { get; }
/* 627 */ protected override void StopCalc()
/* 856 */ protected override _OrderFlow.FootprintBar footprintForLadder(int barNumber)
/* 861 */ protected override bool ladderHasFlow { get; }
/* 863 */ protected override bool sessionOpenAt(DateTime slotStart)
/* 866 */ public _OrderFlow.FootprintBar Footprint(int barsAgo)
/* 874 */ public _OrderFlow.FlowValue BarDelta(int barsAgo)
/* 882 */ public _OrderFlow.FlowValue CumDelta(int barsAgo)
/* 890 */ public IList<_OrderFlow.Imbalance> DiagonalImbalances(int barsAgo, double ratio = 3.0, double minVo
l = 10)
/* 898 */ public _OrderFlow.StackedResult StackedImbalances(int barsAgo, double ratio = 3.0, double minVol =
10, int minStack = 3)
/* 906 */ public _OrderFlow.FlowValue Absorption(int barsAgo, _OrderFlow.Side side, int edgeTicks = 2,
double volMult = 2.0, int holdTicks = 1)
/* 922 */ protected bool UseCmeHolidayTable { get; set; }
/* 930 */ protected virtual bool IsGlobexClosedDay(DateTime globexDate)
/* 939 */ protected virtual bool IsNoRthDay(DateTime globexDate)
/* 949 */ protected virtual TimeSpan? HolidayHaltEt(DateTime globexDate)
/* 958 */ protected virtual TimeSpan? RthEarlyCloseEt(DateTime globexDate)
/* 964 */ protected override bool isTradingHoliday(DateTime date)
```

_RangeProfile.Function.CS

```
// --- (namespace)
/* 17 */ public enum ERangeRollPolicy { Reject = 0, Truncate = 1 }
/* 19 */ public sealed class RangeProfileConfig
// --- RangeProfileConfig
/* 21 */ public bool enabled = false;
/* 22 */ public int[] streams = { 1 };
/* 23 */ public int distribution = 2;
/* 24 */ public bool useTickLadder = false;
/* 25 */ public bool rthOnly = false;
/* 26 */ public ERangeRollPolicy rollPolicy = ERangeRollPolicy.Reject;
/* 27 */ public double valueAreaPercent = double.NaN;
/* 28 */ public int maxRows = 4000;
/* 29 */ public int cacheSize = 8;
/* 30 */ public int maxOpenHandles = 8;
/* 31 */ public int ringBars = 2880;
/* 33 */ public string Validate()
/* 47 */ public string Canon()
// --- (namespace)
/* 55 */ public sealed class RangeHistogram
// --- RangeHistogram
/* 57 */ public double tick;
/* 58 */ public double[] prices = new double[0], vols = new double[0];
/* 59 */ public double[] ask, bid;
/* 60 */ public int bars, ladderBars;
/* 61 */ public int startBar = -1, endBar = -1;
/* 62 */ public DateTime t0, t1, firstTime, lastTime;
/* 63 */ public double open, high, low, close, totalVol;
/* 64 */ public bool isPartial, crossesRoll, approximate = true, tooManyRows;
/* 65 */ public string tag;
/* 66 */ public int Rows { get; }
// --- (namespace)
/* 69 */ public sealed class RangeStats
// --- RangeStats
/* 71 */ public int bars, expectedBars, ladderBars, rows; public double coverage = double.NaN, totalVol;
/* 72 */ public override string ToString()
// --- (namespace)
/* 76 */ public sealed class RangeAccumulator
// --- RangeAccumulator
/* 78 */ public readonly _ProfileMath.Accumulator acc;
/* 80 */ public readonly RangeProfileConfig cfg;
/* 81 */ public int ladderBars, bidAskBars;
/* 82 */ public bool crossesRoll, isPartial;
/* 83 */ public DateTime t0, t1;
/* 85 */ public RangeAccumulator(double tick, RangeProfileConfig cfg)
/* 87 */ public int Bars { get; }
/* 89 */ public void Add(RingBar b, LadderBar lb)
/* 112 */ public RangeHistogram Build(string tag = null)
// --- (namespace)
/* 135 */ public static class RangeProfileBuilder
// --- RangeProfileBuilder
/* 145 */ public static RangeHistogram Build(_BarRing ring, DateTime t0, DateTime t1, double tick, RangeProfi
```

```
leConfig cfg, Func<RingBar, LadderBar> ladder, bool t0Exclusive = false, string tag = null)
/* 168 */ public static RangeHistogram BuildBars(_BarRing ring, int barsBack, int count, double tick, RangePr
ofileConfig cfg, Func<RingBar, LadderBar> ladder, string tag = null)
/* 183 */ public static int ExpectedSlots(DateTime t0, DateTime t1, TimeSpan res, Func<DateTime, bool>
isOpen, bool t0Exclusive = false)
/* 192 */ public static RangeStats Stats(RangeHistogram h, int expectedBars)
/* 199 */ public static string Tag(DateTime t0, DateTime t1)
// --- (namespace)
/* 209 */ public sealed class RangeProfileHandle
// --- RangeProfileHandle
/* 214 */ public readonly int stream;
/* 215 */ public readonly DateTime t0;
/* 216 */ public readonly string Tag;
/* 217 */ public bool IsClosed { get; private set; }
/* 218 */ public DateTime ClosedAt { get; private set; }
/* 219 */ public DateTime ClosedDay { get; private set; }
/* 220 */ public bool CrossesRoll { get; }
/* 221 */ public int Bars { get; }
/* 223 */ public int LastBar { get; }
/* 225 */ public Action<RangeProfileHandle> beforeRead;
/* 229 */ public RangeProfileHandle(int stream, DateTime t0, string tag, double tick, RangeProfileConfig cfg)
/* 236 */ public bool Step(RingBar b, LadderBar lb)
/* 251 */ public void Backfill(_BarRing ring, Func<RingBar, LadderBar> ladder)
/* 259 */ public void Close(DateTime t1)
/* 265 */ public RangeHistogram Profile
```

_WeeklyProfiles.Function.CS

```
// --- (namespace)
/* 19 */ public enum EWeeklySource { RthSessions = 0, GlobexSessions = 1 }
/* 20 */ public enum EWeeklyRollPolicy { Reset = 0, CashTermsReserved = 1, BackAdjustedReserved = 2 }
/* 22 */ public sealed class WeeklyProfileConfig
// --- WeeklyProfileConfig
/* 24 */ public bool enabled = false;
/* 25 */ public int dataStream = 1;
/* 26 */ public EWeeklySource source = EWeeklySource.RthSessions;
/* 27 */ public int keepWeeks = 8;
/* 28 */ public int minSessions = 3;
/* 29 */ public bool includeDeveloping = false;
/* 30 */ public EWeeklyRollPolicy rollPolicy = EWeeklyRollPolicy.Reset;
/* 31 */ public int distribution = 2;
/* 33 */ public string Validate()
/* 43 */ public string Canon()
// --- (namespace)
/* 47 */ public sealed class WeekSession
// --- WeekSession
/* 49 */ public DateTime day; public bool roll, partial, isRollWindow; public double volume; public int bars;
// --- (namespace)
/* 52 */ public sealed class WeekProfile
// --- WeekProfile
/* 54 */ public DateTime key;
/* 55 */ public double tick;
/* 56 */ public readonly SortedDictionary<long, double[]> rows = new SortedDictionary<long, double[]>();
/* 57 */ public readonly List<WeekSession> sessions = new List<WeekSession>();
/* 58 */ public bool isPartial, crossesRoll, complete, bidAskValid = true;
/* 59 */ public double high = double.MinValue, low = double.MaxValue, open = double.NaN, close = double.NaN;
/* 60 */ public int Sessions { get; }
/* 61 */ public bool IsRollWindow { get; }
/* 64 */ public bool Dense(out double[] prices, out double[] total, out double[] ask, out double[] bid)
/* 83 */ internal void merge(WeekProfile o)
/* 93 */ internal void addRow(long k, double total, double ask, double bid)
/* 100 */ public string Tag { get; }
// --- (namespace)
/* 103 */ public sealed class _WeeklyProfiles
// --- _WeeklyProfiles
/* 105 */ public readonly WeeklyProfileConfig config;
/* 106 */ public readonly double tick;
/* 107 */ public Action<string> journal;
/* 113 */ public _WeeklyProfiles(WeeklyProfileConfig cfg, double tick)
/* 116 */ public static DateTime WeekKeyOf(DateTime tradingDay)
/* 118 */ public int CompletedWeeks { get; }
/* 124 */ public void AddSession(DateTime day, IDictionary<long, double[]> rows, bool roll, bool partial, dou
ble open, double high, double low, double close, int bars, bool isRollWindow = false)
/* 158 */ public void RefreshRollWindow(Func<DateTime, bool> isRollWindow)
```

```
/* 166 */ public void Flush()
/* 182 */ public WeekProfile Week(int weeksAgo)
/* 190 */ public DateTime WeekKey(int weeksAgo)
/* 194 */ public WeekProfile Composite(int n)
```

_ProfileStats.Function.CS

```
// --- (namespace)
/* 27 */ public enum EEntropyRows { DenseRange = 0, TradedRows = 1 }
/* 28 */ public enum EMmN { Bars = 0, Trades = 1, Contracts = 2 }
/* 29 */ public enum EKdeVerdict { Unknown = 0, Stable, Flat, Bimodal }
/* 30 */ public enum ECvFolding { Contiguous = 0, Interleaved = 1 }
/* 32 */ public sealed class ProfileStatsConfig
// --- ProfileStatsConfig
/* 34 */ public bool enabled = false;
/* 35 */ public EEntropyRows entropyRows = EEntropyRows.DenseRange;
/* 36 */ public EMmN mmN = EMmN.Bars;
/* 37 */ public double klEpsilon = 1e-6;
/* 38 */ public int parkinsonMinBars = 60;
/* 39 */ public int ewmaSessions = 10;
/* 40 */ public int pctLookbackA229 = 250;
/* 41 */ public int pctLookbackA230 = 60;
/* 42 */ public double pctMinFill = 0.8;
/* 43 */ public int bootstrapBlock = 5, bootstrapDraws = 500;
/* 44 */ public double kdeMedFloorTicks = 1;
/* 45 */ public int kdeFolds = 5;
/* 46 */ public ECvFolding kdeFolding = ECvFolding.Contiguous;
/* 47 */ public int ringBars = 1440;
/* 49 */ public string Validate()
/* 64 */ public string Canon()
// --- (namespace)
/* 71 */ public struct EntropyResult
// --- EntropyResult
/* 73 */ public double H, Hnorm, Hmm, HnormMm, Neff, N;
/* 74 */ public int n, m;
/* 75 */ public override string ToString()
// --- (namespace)
/* 81 */ public sealed class BootstrapResult
// --- BootstrapResult
/* 83 */ public double medianPoc = double.NaN, ci90Ticks = double.NaN, stability = double.NaN, zoneLo = doubl
e.NaN, zoneHi = double.NaN;
/* 84 */ public double[] pocs = new double[0];
/* 85 */ public int bars, draws, block; public ulong seed;
// --- (namespace)
/* 88 */ public sealed class KdeResult
// --- KdeResult
/* 90 */ public EKdeVerdict verdict = EKdeVerdict.Unknown;
/* 91 */ public double sigma = double.NaN, maxJ = double.NaN, medJ = double.NaN, range = double.NaN, hCv = do
uble.NaN, cvPoc = double.NaN;
/* 92 */ public ECvFolding folding = ECvFolding.Contiguous;
/* 93 */ public double hCvAlt = double.NaN;
/* 94 */ public double HcvContiguous { get; }
/* 95 */ public double HcvInterleaved { get; }
/* 97 */ public string Journal()
/* 102 */ public bool disagree;
/* 103 */ public double[] h = new double[0], poc = new double[0];
/* 104 */ public double[] cvH = new double[0], cvLL = new double[0];
// --- (namespace)
/* 107 */ public sealed class VtResult
// --- VtResult
/* 109 */ public double volPoc = double.NaN, tpoPoc = double.NaN, atr = double.NaN;
/* 110 */ public readonly List<double> prices = new List<double>(), ratios = new List<double>();
// --- (namespace)
/* 117 */ public enum EGmmNeff { Bars = 0, Rows = 1, Contracts = 2 }
/* 119 */ public sealed class ProfileMixtureResult
// --- ProfileMixtureResult
/* 121 */ public GmmModel model;
/* 122 */ public double[] mu = new double[0], sd = new double[0], pi = new double[0], voidPrice = new
double[0], voidDepth = new double[0];
/* 123 */ public double nEff = double.NaN, tick = double.NaN;
/* 124 */ public ulong seed;
/* 125 */ public int K { get; }
/* 128 */ public string Journal(DateTime date)
/* 137 */ internal static string join(double[] a, string fmt)
```

```
// --- (namespace)
/* 146 */ public sealed class GmmSession
// --- GmmSession
/* 148 */ public DateTime day, closeTime;
/* 149 */ public string contract;
/* 150 */ public bool roll;
/* 151 */ public double[] prices = new double[0], vols = new double[0];
/* 152 */ public int bars;
/* 153 */ public double sVwap = double.NaN, sigma = double.NaN, totalVol;
/* 156 */ public static GmmSession FromBars(DateTime tradingDay, IList<RingBar> rth1m, double tick, bool
roll, string contract, int minBars = 60)
// --- (namespace)
/* 172 */ public sealed class SigmaMixtureResult
// --- SigmaMixtureResult
/* 174 */ public bool ok;
/* 175 */ public string reason;
/* 176 */ public ProfileMixtureResult mix;
/* 177 */ public double priorVwap = double.NaN, sigmaHat = double.NaN;
/* 178 */ public double[] levels = new double[0];
/* 179 */ public int sessions;
/* 180 */ public string contract;
/* 181 */ public List<DateTime> days = new List<DateTime>();
// --- (namespace)
/* 184 */ public static class _ProfileStats
// --- _ProfileStats
/* 193 */ public static EntropyResult Entropy(double[] vols, EEntropyRows rows, double mmN)
/* 217 */ public static double MmN(EMmN mode, int bars, double trades, double contracts)
/* 233 */ public static double KL(double[] pPrices, double[] pVols, double[] qPrices, double[] qVols, double
tick, double eps)
/* 247 */ public static double KLKdePrior(double[] pPrices, double[] pVols, double[] qPrices, double[]
qVols, double tick, double priorHcv, double eps)
/* 256 */ public static double KLKde(double[] pPrices, double[] pVols, double[] qPrices, double[] qVols, doub
le tick, double h, double eps)
/* 303 */ public static double[] KdeOnGrid(double[] prices, double[] vols, double gridLo, int gridN, double t
ick, double h, double trunc)
/* 325 */ public static double ParkinsonPerBar(IList<RingBar> bars, out int n)
/* 337 */ public static double ParkinsonPerBar(IList<RingBar> bars)
/* 346 */ public static double ParkinsonSessionPrice(IList<RingBar> bars, double refPrice, int minBars)
/* 354 */ public static double EwmaForecast(IList<double> valuesOldestFirst, int span)
/* 375 */ public static List<RingBar> AggregateBars(IList<RingBar> bars, TimeSpan size)
/* 406 */ public static double WilderAtr(IList<RingBar> bars, int len)
/* 423 */ public static void BarRows(RingBar b, double tick, int mode, Action<long, double> add)
/* 437 */ public static bool Histogram(IList<RingBar> bars, IList<int> idx, double tick, int mode, out
double[] prices, out double[] vols)
/* 447 */ public static double WeightedSd(double[] prices, double[] vols)
/* 459 */ public static double Percentile(IList<double> values, double q)
/* 476 */ public static double MidRank(IList<double> values, double x)
/* 486 */ public const int BOOTSTRAP_BLOCK = 5, BOOTSTRAP_DRAWS = 500;
/* 489 */ public static BootstrapResult BootstrapPoc(IList<RingBar> bars, double tick, int distribution, ulon
g seed)
/* 495 */ public static ulong BootstrapSeed(DateTime tradingDay, int block, int draws)
/* 503 */ public static BootstrapResult BootstrapPoc(IList<RingBar> bars, double tick, int distribution, int
block, int draws, ulong seed)
/* 540 */ public const int KDE_SWEEP_POINTS = 30, KDE_CV_POINTS = 25;
/* 541 */ public const double KDE_CV_LO = 0.15, KDE_CV_HI = 1.2;
/* 544 */ public static double[] KdeCvGrid(double sigma)
/* 559 */ public static KdeResult KdeSweep(IList<RingBar> bars, double tick, int distribution, double mcPoc
= double.NaN, double medFloorTicks = 1, int folds = 5, ECvFolding folding = ECvFolding.Contiguous)
/* 588 */ public static KdeResult KdeSweep(double[] prices, double[] vols, double tick, double medFloorTicks
= 1)
/* 626 */ public static double[] CvLogLik(IList<RingBar> bars, double tick, int distribution, double[] hs, in
t folds, ECvFolding folding)
/* 672 */ public static VtResult VtPrices(IList<RingBar> bars, double tick, int distribution, double atr30, i
nt top = 3)
/* 700 */ public static List<LevelRef> BootstrapLevels(BootstrapResult b, DateTime created, DateTime t0, Date
Time t1, int stream, string tag)
/* 709 */ public static List<LevelRef> VtLevels(VtResult vt, DateTime created, DateTime t0, DateTime t1, int
stream, string tag)
/* 719 */ public const int GMMP_COMPONENT = 27;
/* 720 */ public const int A232_WINDOW = 20, A232_EWMA_SPAN = 10, A232_MIN_BARS = 60, A232_KMAX = 4;
/* 721 */ public const double A232_BIN = 0.05;
/* 724 */ public static ulong MixtureSeed(DateTime tradingDay)
/* 727 */ public static double MixtureNeff(EGmmNeff mode, int bars, double[] vols)
/* 742 */ public static ProfileMixtureResult ProfileMixture(double[] prices, double[] vols, int kMax, ulong s
eed, EGmmNeff neff, int bars, double tick, GmmFitConfig cfg = null)
/* 763 */ public static void MixtureVoids(GmmModel m, double tick, out double[] voidPrice, out double[] voidD
epth)
```

```
/* 790 */ public static List<LevelRef> MixtureLevels(ProfileMixtureResult r, double edgeC, DateTime created,
DateTime t0, DateTime t1, int stream, string tag)
/* 812 */ public static SigmaMixtureResult SigmaMixture(IList<GmmSession> sessions, ulong seed, int window =
A232_WINDOW, int ewmaSpan = A232_EWMA_SPAN, double bin = A232_BIN, int kMax = A232_KMAX, GmmFitConfig cfg = n
ull)
/* 851 */ public static List<LevelRef> SigmaLevels(SigmaMixtureResult r, DateTime created, DateTime t0, DateT
ime t1, int stream, string tag)
// --- (namespace)
/* 861 */ public sealed class _PriorDayValue
// --- _PriorDayValue
/* 864 */ public int keep = 30;
/* 865 */ public void Set(DateTime day, double x)
/* 872 */ public double Before(DateTime day)
// --- (namespace)
/* 884 */ public sealed class _EntropyTracker
// --- _EntropyTracker
/* 886 */ public readonly int lookback; public readonly double minFill;
/* 890 */ public _EntropyTracker(int lookback, double minFill, IEnumerable<int> slots)
/* 896 */ public bool HasSlot(int slot)
/* 897 */ public int Count(int slot)
/* 900 */ public bool Push(int slot, DateTime day, double x)
/* 912 */ public double Percentile(int slot, double x)
/* 920 */ public static int Slot(DateTime t)
// --- (namespace)
/* 928 */ public sealed class _GmmProfileTracker
// --- _GmmProfileTracker
/* 930 */ public double tick;
/* 931 */ public int stream = 1;
/* 932 */ public int kMaxPrice = 3, window = _ProfileStats.A232_WINDOW, ewmaSpan = _ProfileStats.A232_EWMA_SP
AN, minBars = _ProfileStats.A232_MIN_BARS, kMaxSigma = _ProfileStats.A232_KMAX;
/* 933 */ public double bin = _ProfileStats.A232_BIN, edgeC = 1.0;
/* 934 */ public EGmmNeff neff = EGmmNeff.Bars;
/* 935 */ public bool priceMixture = true, sigmaMixture = true;
/* 936 */ public int keepSessions = 40;
/* 937 */ public Action<string> journal;
/* 944 */ public ProfileMixtureResult LastPrice { get; private set; }
/* 945 */ public SigmaMixtureResult LastSigma { get; private set; }
/* 946 */ public int Fits { get; private set; }
/* 948 */ public _GmmProfileTracker(double tick)
/* 951 */ public void OnSessionClose(DateTime tradingDay, IList<RingBar> rth1m, bool roll, string contract =
null)
/* 969 */ public int SkippedSessions(string contract)
/* 971 */ public IDictionary<string, int> SkippedByContract { get; }
/* 974 */ public List<LevelRef> Levels()
```

_ProfileMath.Function.CS

```
// --- (namespace)
/* 14 */ public class _ProfileMath
// --- _ProfileMath
/* 16 */ protected _ProfileMath()
/* 18 */ public enum ValueRel { Unknown = 0, Higher, Lower, OverlappingHigher, OverlappingLower, Inside, Outs
ide }
/* 19 */ public enum Shape { Unknown = 0, D, P, b, Thin, Double }
/* 22 */ public const double SHAPE_SKEW_SHARE = 0.50;
/* 23 */ public const double SHAPE_THIN_PEAK_TO_MEAN = 1.5;
/* 24 */ public const double SHAPE_DOUBLE_PEAK = 0.60;
/* 25 */ public const double SHAPE_DOUBLE_VALLEY = 0.50;
/* 26 */ public const double SHAPE_DOUBLE_SEPARATION = 0.25;
/* 28 */ public static long Key(double price, double tick)
/* 31 */ public static bool ToDense(IList<double> prices, IList<double> vols, double tick, out double[] dense
Prices, out double[] denseVols)
/* 53 */ public static int PocIndex(double[] vols)
/* 71 */ public static bool ValueAreaExpansion(double[] vols, double percent, out int lo, out int hi, out
int poc)
/* 93 */ public static bool ValueAreaSigma(double[] prices, double[] vols, double k, out double vah, out doub
le val)
/* 110 */ public static ValueRel Relationship(double prevVah, double prevVal, double curVah, double curVal)
/* 124 */ public static bool IsPoorExtreme(double[] vols, bool top, double fraction, int minRows)
/* 144 */ public static List<int[]> SinglePrints(double[] vols, int vaLo, int vaHi, double pct, int minRows)
/* 164 */ public static Shape ProfileShape(double[] vols, double vaPercent)
/* 191 */ public enum MigrationState { Unknown = 0, Down, Neutral, Up }
/* 194 */ public static int MigrationStep(double a, double b, double d)
/* 206 */ public static int? MigrationScore(IList<double> pocsOldestToNewest, int n, double d, double?
```

```
devPoc)
/* 217 */ public static MigrationState ClassifyMigration(int? v, int upAt = 2, int downAt = 2)
/* 226 */ public static bool IsBalanced(IList<double> vahs, IList<double> vals, double minOverlap)
/* 247 */ public static bool ValueMigrationScore(IList<double> completedPocs, int n, double developingPoc, do
uble minChange, out int score)
/* 279 */ public static bool Prominence(double[] d, double prom, List<double[]> hvn, List<double[]> lvn)
/* 313 */ public sealed class Accumulator
// --- _ProfileMath.Accumulator
/* 315 */ public readonly SortedDictionary<long, double> rows = new SortedDictionary<long, double>();
/* 316 */ public double tick;
/* 317 */ public int bars;
/* 318 */ public double open, high, low, close;
/* 319 */ public int firstBar = -1, lastBar = -1;
/* 320 */ public DateTime firstTime, lastTime;
/* 322 */ public Accumulator(double tick)
/* 324 */ public void Reset()
/* 326 */ public void Add(int barIndex, DateTime time, double o, double h, double l, double c, double
volume, int mode)
/* 345 */ public void AddRow(long k, double v)
/* 355 */ public void Dense(out double[] prices, out double[] vols)
```

_BarLadder.Function.CS

```
// --- (namespace)
/* 16 */ public sealed class LadderBar
// --- LadderBar
/* 18 */ public int bar; public DateTime time; public double tick;
/* 19 */ public _OrderFlow.FlowSource source = _OrderFlow.FlowSource.Unavailable;
/* 20 */ public _OrderFlow.FlowQuality quality = _OrderFlow.FlowQuality.Unknown;
/* 21 */ public long[] rows = new long[0]; public double[] bid = new double[0], ask = new double[0], total =
new double[0];
/* 22 */ public double High, Low, Buy, Sell, Volume, coverage = 1;
/* 23 */ public bool crossesRoll;
/* 24 */ public bool HasBidAsk { get; }
/* 25 */ public bool HasRows { get; }
/* 26 */ public double Delta { get; }
// --- (namespace)
/* 29 */ public sealed class BarLadderConfig
// --- BarLadderConfig
/* 31 */ public bool enabled = false;
/* 32 */ public int keepBars = 2000;
/* 33 */ public int[] htfStreams = new int[0];
/* 34 */ public _OrderFlow.FlowQuality minQuality = _OrderFlow.FlowQuality.Low;
/* 36 */ public string Validate()
/* 43 */ public string Canon()
// --- (namespace)
/* 46 */ public sealed class _BarLadderStore
// --- _BarLadderStore
/* 48 */ public readonly BarLadderConfig config;
/* 49 */ public readonly double tick;
/* 53 */ public _BarLadderStore(BarLadderConfig cfg, double tick)
/* 55 */ public int Count { get; }
/* 59 */ public LadderBar Get(RingBar rb, _OrderFlow.FootprintBar fp, bool final = true)
/* 70 */ public bool TryGet(int bar, out LadderBar lb)
/* 71 */ public void Clear()
/* 74 */ public static LadderBar Build(RingBar rb, _OrderFlow.FootprintBar fp, double tick)
/* 92 */ public static LadderBar BarUpDown(RingBar rb, double tick)
/* 111 */ public static LadderBar Merge(IList<LadderBar> parts, IList<RingBar> ringBars, int expectedSlots, i
nt htfBar, DateTime htfTime, double tick)
/* 181 */ public static bool Dense(LadderBar lb, out double[] prices, out double[] vols)
```

_Gmm.Function.CS

```
// --- (namespace)
/* 18 */ public enum EGmmTolMode { Relative = 0, PerSample = 1 }
/* 20 */ public sealed class GmmModel
// --- GmmModel
/* 22 */ public int K, D;
/* 23 */ public double[] w;
/* 24 */ public double[][] mu;
/* 25 */ public double[][,] cov;
/* 26 */ public double logLik = double.NaN, bic = double.NaN;
```

```
/* 27 */ public int iters;
/* 28 */ public bool converged;
/* 29 */ public int reseeds;
/* 31 */ public double nEff = double.NaN;
/* 32 */ public int restart = -1;
/* 33 */ public int cholRetries;
/* 34 */ public bool reseedInFinalIter;
/* 35 */ public GmmModel[] perK;
/* 36 */ public GmmModel init;
/* 38 */ public int Params { get; }
/* 41 */ public double Sd(int k)
/* 43 */ public GmmModel Clone()
// --- (namespace)
/* 55 */ public sealed class GmmFitConfig
// --- GmmFitConfig
/* 57 */ public int kMin = 1, kMax = 3, restarts = 10, maxIter = 200;
/* 58 */ public double tol = 1e-6;
/* 59 */ public EGmmTolMode tolMode = EGmmTolMode.Relative;
/* 60 */ public double regCovar = 1e-6, minWeight = 1e-3, bicDelta = 10;
/* 61 */ public ulong seed;
/* 62 */ public int lloydIters = 10;
/* 63 */ public int cholRetries = 3;
/* 65 */ public string Validate()
/* 78 */ public string Canon()
/* 84 */ public GmmFitConfig Copy()
// --- (namespace)
/* 87 */ public static class _Gmm
// --- _Gmm
/* 89 */ public const int MaxD = 4;
/* 92 */ public static int ParamCount(int K, int D)
/* 99 */ public static GmmModel Fit(double[][] x, double[] weights, double nEff, GmmFitConfig cfg, int orderF
eature = 0, int tieFeature = -1)
/* 125 */ public static GmmModel Fit1D(IList<double> x, IList<double> weights, double nEff, GmmFitConfig cfg)
/* 158 */ public static GmmModel FitK(double[][] x, double[] om, double nEff, int K, GmmFitConfig cfg, int or
derFeature = 0, int tieFeature = -1)
/* 172 */ public static GmmModel FitOnce(double[][] x, double[] om, double nEff, int K, GmmFitConfig cfg,
int restart)
/* 365 */ public static bool CholeskyWithRetry(double[,] cov, int D, double regCovar, int retries, double[,]
L, ref int retriesUsed)
/* 380 */ public static bool Cholesky(double[,] a, int D, double[,] L)
/* 404 */ public static double LogPdf(double[] x, double[] mu, double[,] L, int D)
/* 418 */ public static double LogDensity(GmmModel m, double[] x)
/* 433 */ public static double Density1D(GmmModel m, double x)
/* 441 */ public static void Order(GmmModel m, int orderFeature, int tieFeature)
```

_Rng.Function.CS

```
// --- (namespace)
/* 12 */ public sealed class _Rng
// --- _Rng
/* 16 */ public _Rng(ulong seed)
/* 18 */ public ulong State { get; }
/* 21 */ public ulong Next()
/* 30 */ public static ulong Mix(ulong z)
/* 41 */ public double NextDouble()
/* 44 */ public int NextInt(int n)
/* 68 */ public static ulong Seed(params long[] parts)
/* 80 */ public static long Hash(string s)
```

_LevelStudy.Function.CS

```
// --- (namespace)
/* 21 */ public sealed class LevelStudyConfig
// --- LevelStudyConfig
/* 23 */ public bool enabled = false;
/* 24 */ public int controls = 20;
/* 25 */ public double distLo = 0.9, distHi = 1.1;
/* 26 */ public int reactionBars = 60;
/* 27 */ public int sigmaBars = 60;
/* 28 */ public double touchTicks = 0;
/* 29 */ public int maxAge = 15;
/* 30 */ public double deathR = 1.0;
```

```
/* 31 */ public int seed = 33;
/* 32 */ public DateTime from = DateTime.MinValue, to = DateTime.MaxValue;
/* 33 */ public string csvPath = "";
/* 34 */ public bool controlsEitherSide = false;
/* 35 */ public bool reactionAfterTouch = false;
/* 37 */ public string Validate()
// --- (namespace)
/* 53 */ public sealed class LevelEvent
// --- LevelEvent
/* 55 */ public DateTime day; public string type = ""; public string key = ""; public int age; public double
quality = double.NaN;
/* 56 */ public bool control; public int levelNo; public double price, refPrice, sigma;
/* 57 */ public bool touched; public int touchBar = -1; public int fromAbove;
/* 58 */ public double reaction = double.NaN, asymmetry = double.NaN, volAttraction = double.NaN; public int
barsSeen;
/* 59 */ internal double maxBounce, maxThrough, volSum; internal int volBars;
// --- (namespace)
/* 62 */ public sealed class LevelTypeResult
// --- LevelTypeResult
/* 64 */ public string type = ""; public int levels, touchedLevels;
/* 65 */ public double touchEdge = double.NaN, touchT = double.NaN;
/* 66 */ public double edge = double.NaN, t = double.NaN, p = double.NaN, pHolm = double.NaN;
/* 67 */ public double asymEdge = double.NaN, volEdge = double.NaN;
/* 68 */ public double[] edgeByAge = new double[0]; public int[] nByAge = new int[0]; public double[] tByAge
= new double[0];
/* 69 */ public double halfLife = double.NaN;
/* 70 */ public double[] edgeByTercile = { double.NaN, double.NaN, double.NaN }; public double[] tByTercile
= { double.NaN, double.NaN, double.NaN };
/* 71 */ public double[] kmSurvival = new double[0];
/* 72 */ public double kmMedianAge = double.NaN;
/* 73 */ public string verdict = "";
// --- (namespace)
/* 76 */ internal sealed class _LevelStudy
// --- _LevelStudy
/* 78 */ public LevelStudyConfig config = new LevelStudyConfig();
/* 79 */ public double tickSize = 0.25;
/* 80 */ public Action<string> journal;
/* 90 */ public IReadOnlyList<LevelEvent> Events { get; }
/* 91 */ public string Configure()
/* 94 */ public void BeginSession(DateTime day)
/* 104 */ public void SetSessionReference(double refPrice, double sigma)
/* 107 */ public bool AddLevel(string type, double price, int ageSessions, double quality = double.NaN, strin
g key = null)
/* 134 */ public void Bar(double o, double h, double l, double c, double v)
/* 184 */ public List<LevelTypeResult> Finish()
/* 188 */ public static List<LevelTypeResult> Analyse(IList<LevelEvent> events, LevelStudyConfig cfg)
/* 245 */ public static void meanT(IList<double> x, out double mean, out double t)
/* 254 */ public static double Phi(double z)
/* 262 */ public static void Holm(List<LevelTypeResult> r)
/* 269 */ public static double HalfLife(double[] edgeByAge, int[] n)
/* 283 */ public static double[] KaplanMeier(IList<KeyValuePair<double, bool>> obs, int maxT)
/* 296 */ public static string Verdict(LevelTypeResult r)
/* 309 */ public static string Summary(LevelTypeResult r)
/* 317 */ public void WriteCsv(TextWriter w)
```

_Gate0.Function.CS

```
// --- (namespace)
/* 22 */ public enum EGate0Mode { Strategy = 0, EventStudy = 1, ControlRT = 2, ControlFL = 3, Canary = 4 }
/* 25 */ public enum EGate0Family { All = 0, PriorProfile = 1, NakedPoc = 2, IbEdge = 3, Overnight = 4,
Avwap = 5, Vwap = 6, Weekly = 7, CompositeVa = 8, SinglePrint = 9, DvSource = 10, HvnLvn = 11, Mixture = 12,
SigmaMixture = 13 }
/* 28 */ public enum EGate0Canary { VW08 = 0, A2_24_i = 1, A2_24_ii = 2, A2_24_iii = 3, DV10 = 4, A2_53 = 5,
DV01 = 6, PS03 = 7 }
/* 30 */ public enum EGate0Entry { None = 0, Market = 1, Limit = 2 }
/* 40 */ public static class Gate0Data1
// --- Gate0Data1
/* 42 */ public static string Check(string chartType, string type, long size)
// --- (namespace)
/* 52 */ public struct Gate0Bar
// --- Gate0Bar
/* 54 */ public DateTime et;
/* 55 */ public DateTime day;
/* 56 */ public double o, h, l, c, v;
```

```
/* 57 */ public bool roll;
/* 58 */ public Gate0Bar(DateTime et, DateTime day, double o, double h, double l, double c, double v, bool ro
ll = false)
// --- (namespace)
/* 63 */ public sealed class Gate0Intent
// --- Gate0Intent
/* 65 */ public int side;
/* 66 */ public EGate0Entry entry;
/* 67 */ public double entryPrice = double.NaN;
/* 68 */ public double stop = double.NaN, target = double.NaN;
/* 69 */ public double reverseStop = double.NaN;
/* 70 */ public bool flatten;
/* 71 */ public string tag = "";
/* 72 */ public void Clear()
// --- (namespace)
/* 76 */ public sealed class Gate0Profile
// --- Gate0Profile
/* 78 */ public readonly SortedDictionary<long, double> rows = new SortedDictionary<long, double>();
/* 79 */ public double tick;
/* 80 */ public double total;
/* 81 */ public Gate0Profile(double tick)
/* 82 */ public void Add(double h, double l, double v)
/* 91 */ public void Merge(Gate0Profile p)
/* 92 */ public bool Empty { get; }
/* 95 */ public bool ValueArea(double pct, out double poc, out double vah, out double val)
/* 113 */ public long FirstRow { get; }
/* 114 */ public long LastRow { get; }
/* 118 */ public static List<double[]> InteriorRuns(long first, long last, Func<long, bool> thin, int
minRows, double tick)
/* 131 */ public List<double[]> SinglePrintRuns(double frac, int minRows)
/* 140 */ public static List<double[]> TpoSinglePrints(IDictionary<long, int> tpo, int minRows, double tick)
/* 150 */ public void Nodes(double bwTicks, double prom, out List<double[]> hvn, out List<double[]> lvn)
// --- (namespace)
/* 167 */ public sealed class Gate0Vwap
// --- Gate0Vwap
/* 169 */ public double pv, p2v, v;
/* 170 */ public void Reset()
/* 171 */ public void Add(double price, double vol)
/* 172 */ public bool Ready { get; }
/* 173 */ public double Value { get; }
/* 174 */ public double Sigma { get; }
// --- (namespace)
/* 178 */ public sealed class Gate0Atr
// --- Gate0Atr
/* 181 */ public double Value = double.NaN;
/* 182 */ public Gate0Atr(int len)
/* 184 */ public void ResetPrev()
/* 185 */ public void Add(double h, double l, double c)
// --- (namespace)
/* 196 */ public sealed class Gate0Bucket
// --- Gate0Bucket
/* 198 */ public readonly int minutes;
/* 199 */ public bool open; public DateTime end; public double o, h, l, c, v;
/* 200 */ public Gate0Bucket(int minutes)
/* 201 */ public static DateTime EndOf(DateTime t, int minutes)
/* 206 */ public bool PreBar(DateTime t, out DateTime endOut, out double oo, out double hh, out double ll, ou
t double cc, out double vv)
/* 212 */ public void Add(DateTime t, double bo, double bh, double bl, double bc, double bv)
/* 218 */ public bool PostBar(DateTime t)
// --- (namespace)
/* 222 */ public sealed class Gate0Session
// --- Gate0Session
/* 224 */ public DateTime day; public bool roll;
/* 225 */ public double rthO = double.NaN, rthH = double.MinValue, rthL = double.MaxValue, rthC = double.NaN;
/* 226 */ public double ibH = double.MinValue, ibL = double.MaxValue, onH = double.MinValue, onL = double.Max
Value;
/* 227 */ public Gate0Profile profile;
/* 228 */ public readonly Dictionary<long, int> tpo = new Dictionary<long, int>();
/* 229 */ internal readonly Dictionary<long, int> tpoLast = new Dictionary<long, int>();
/* 230 */ public Gate0Vwap vwap = new Gate0Vwap(), avHi = new Gate0Vwap(), avLo = new Gate0Vwap();
/* 231 */ public double poc = double.NaN, vah = double.NaN, val = double.NaN;
/* 232 */ public List<double> closes1m = new List<double>();
/* 233 */ public int bars1m;
/* 234 */ public List<RingBar> rth1m;
/* 235 */ public double hCv = double.NaN;
/* 236 */ public double pkSum; public int pkN;
/* 237 */ public bool pocNaked = true;
```

```
/* 238 */ public bool HasRth { get; }
/* 240 */ public double SigmaPrice { get; }
/* 241 */ public Gate0Session(DateTime day, double tick)
// --- (namespace)
/* 245 */ public sealed class Gate0DvLevel
// --- Gate0DvLevel
/* 247 */ public string type = ""; public double price; public int dir; public DateTime validFrom; public
int expirySessions; public bool touched;
// --- (namespace)
/* 251 */ public sealed class Gate0Result
// --- Gate0Result
/* 253 */ public string type = ""; public int levels, touches, sessions;
/* 254 */ public int pairs; public double edge = double.NaN, t = double.NaN, p = double.NaN, pHolm = double.N
aN, ciLo = double.NaN, ciHi = double.NaN;
/* 255 */ public double halfLife = double.NaN; public double[] tByAge = new double[0]; public int[] nByAge =
new int[0]; public double[] tByTercile = new double[3];
/* 256 */ public string a2Verdict = "";
/* 257 */ public string verdict = "";
/* 258 */ public bool verdictRun = true;
/* 259 */ public string AgeList()
/* 265 */ public string Line()
// --- (namespace)
/* 273 */ public sealed class Gate0Engine
// --- Gate0Engine
/* 275 */ public static readonly DateTime DevEnd = new DateTime(2025, 1, 1);
/* 276 */ public static readonly DateTime CanaryA224From = new DateTime(2019, 1, 1);
/* 277 */ public const int BootstrapDraws = 500, BootstrapBlock = 5;
/* 278 */ public const int Controls = 20, ReactionBars5m = 12, MaxAge = 15;
/* 279 */ public const int PriorProfileMaxAge = 15, NakedPocMaxAge = 15;
/* 281 */ public const int CompositeSessions = 5;
/* 282 */ public const int SigmaCompositeSessions = 20, SigmaEwmaSpan = 10, ParkinsonMinBars = 60, GmmMaxK =
4;
/* 283 */ public const int HistoryCap = 26;
/* 284 */ public const double KdeBandwidthTicks = 6, KdeProminence = 0.10;
/* 287 */ public const int HcvDistribution = 2;
/* 290 */ public static string PreRegistration()
/* 298 */ public const int SinglePrintMinRows = 4; public const double SinglePrintVolFrac = 0.05;
/* 299 */ public const double DvRotW = 1.5, DvBreakAtr = 1.5, DvTrendLegAtr = 3.0, DvHvnMedianX = 2.0;
/* 300 */ public const int DvRotExpirySessions = 5, DvTrendExpirySessions = 10, DvTrendArmBars = 2, DvTrendLo
okback = 20, DvPendingBars = 26;
/* 301 */ public static readonly int[] DvRotBars = { 6, 12 };
/* 306 */ public static readonly string[] AllTypes = { "POC", "VAH", "VAL", "NPOC", "IBH", "IBL", "ONH", "ONL
", "AV_PH", "AV_PL", "AV_WK", "AV_MO", "AV_PC",
/* 311 */ public EGate0Mode mode = EGate0Mode.EventStudy;
/* 312 */ public int variant;
/* 313 */ public double tick = 0.25;
/* 314 */ public long seed = 33;
/* 315 */ public double vaPct = 0.70;
/* 316 */ public Action<string> journal;
/* 318 */ public double a253Gap = 0.005; public int a253Bar = 1, a253Minutes = 30; public bool a253Filter = t
rue;
/* 319 */ public double dv01F = 0.15, dv01K = 1.0;
/* 320 */ public double dv10S = 0.5, dv10F = 0.20;
/* 322 */ internal readonly _LevelStudy study = new _LevelStudy();
/* 323 */ public readonly Gate0Intent intent = new Gate0Intent();
/* 324 */ public readonly List<DateTime> rollDays = new List<DateTime>();
/* 325 */ public readonly List<DateTime> sessionDays = new List<DateTime>();
/* 326 */ public int Trades { get; private set; }
/* 329 */ public int RollSkips { get; private set; }
/* 331 */ public bool VerdictRun { get; }
/* 355 */ public IList<Gate0Session> History { get; }
/* 356 */ public Gate0Session Current { get; }
/* 357 */ public IList<Gate0DvLevel> DvLevels { get; }
/* 358 */ public double SessionVwap { get; }
/* 359 */ public double Atr20d { get; }
/* 360 */ public double Atr5 { get; }
/* 361 */ public double Atr30 { get; }
/* 365 */ public static bool DecodeA253(int v, out double gap, out int j, out int minutes, out bool filter)
/* 375 */ public string CanaryName { get; }
/* 377 */ public string Configure()
/* 399 */ public void Bar(Gate0Bar b, int mp = 0)
/* 637 */ public List<KeyValuePair<string, double[]>> Candidates()
/* 701 */ public static GmmSession ToGmmSession(Gate0Session s, double tick)
/* 712 */ public static SigmaMixtureResult SigmaCompositeFit(IList<Gate0Session> hist, double tick)
/* 762 */ public bool InRollWindow(DateTime day)
/* 771 */ public static void HolmFixed(List<LevelTypeResult> r, int m)
/* 779 */ public List<Gate0Result> Finish()
```

```
/* 821 */ public static double PairedReaction(LevelEvent lv, IList<LevelEvent> ctl)
/* 831 */ public static void ScoreEdge(Gate0Result r, IList<KeyValuePair<DateTime, double>> diffs, LevelTypeR
esult a2, long seed)
/* 868 */ public static string Verdict(int n, double t, string a2)
/* 915 */ public static bool InsideDay(double h, double l, double h1, double l1)
/* 923 */ public double Hv(int n)
```

_DvpEngine.Function.CS

```
// --- (namespace)
/* 17 */ public enum EDvpSrc { HL2, Close }
/* 18 */ public enum EDvpDegMode { DevCoverageFrozen, Fixed1, Calibrate }
/* 19 */ public enum EDvpConeMode { Prediction, Confidence }
/* 20 */ public enum EDvpFibAnchor { MostRecentPivot, ExtremeInLookback }
/* 23 */ public static class DvpConst
// --- DvpConst
/* 25 */ public const string ConfMode = "FamilyCount";
/* 26 */ public const int MinFamilies = 3;
/* 27 */ public const double ReportWeightVa = 1.00, ReportWeightPoc = 0.77;
/* 28 */ public const int RowFloorTicks = 1;
/* 29 */ public const string RollAdjust = "None";
/* 30 */ public const int MinBars = 12;
/* 31 */ public const int AtrLen = 20;
/* 32 */ public const int HoldBars = 40;
/* 33 */ public const int MaxDegree = 3;
/* 34 */ public const double SseEps = 1e-12;
/* 35 */ public const double BreakdownRel = 1e-13;
/* 38 */ public const bool P2_PASSED = false;
// --- (namespace)
/* 42 */ public sealed class DvpConfig
// --- DvpConfig
/* 44 */ public bool enabled = false;
/* 45 */ public int[] streams = { 1 };
/* 46 */ public EDvpSrc src = EDvpSrc.HL2;
/* 47 */ public int winLen = 240;
/* 48 */ public EDvpDegMode degMode = EDvpDegMode.DevCoverageFrozen;
/* 49 */ public int degMax = 3;
/* 50 */ public bool weightVol = false;
/* 51 */ public double minR2 = 0.35;
/* 52 */ public int manRows = 64;
/* 53 */ public double manSpan = 3;
/* 54 */ public double vaPct = 70;
/* 55 */ public double nodeProm = 0.12;
/* 56 */ public int maxNodes = 3;
/* 57 */ public double[] sigmaList = { 1, 2, 3 };
/* 58 */ public int projBars = 60;
/* 59 */ public EDvpConeMode coneMode = EDvpConeMode.Prediction;
/* 60 */ public double coneMult = 2;
/* 61 */ public bool useFib = true;
/* 62 */ public int fibRanges = 3;
/* 63 */ public int fibPivot = 20;
/* 64 */ public int fibLookMult = 8;
/* 65 */ public EDvpFibAnchor fibAnchor = EDvpFibAnchor.MostRecentPivot;
/* 66 */ public double fibMerge = 0.40;
/* 67 */ public double fibNear = 14;
/* 68 */ public double confTol = 0.30;
/* 69 */ public bool splitFlow = false;
/* 70 */ public int refitEvery = 1;
/* 71 */ public bool journalFits = true;
/* 72 */ public bool journalRails = true;
/* 74 */ public double VaFraction { get; }
/* 79 */ public List<string> Validate()
/* 116 */ public string Canon()
// --- (namespace)
/* 142 */ public static class DvpFrozenDegree
// --- DvpFrozenDegree
/* 144 */ public struct Entry
// --- DvpFrozenDegree.Entry
/* 146 */ public string root; public int stream, winLen, degree;
/* 147 */ public Entry(string root, int stream, int winLen, int degree)
// --- DvpFrozenDegree
/* 150 */ public static readonly Entry[] Table = new Entry[0];
/* 152 */ public static bool TryGet(string root, int stream, int winLen, out int degree)
/* 154 */ public static bool TryGet(Entry[] table, string root, int stream, int winLen, out int degree)
```

```
/* 161 */ public static string Canon(Entry[] table)
// --- (namespace)
/* 174 */ public static class DvpDegree
// --- DvpDegree
/* 177 */ public static int Cap(int n, int degMax)
/* 188 */ public static bool Select(EDvpDegMode mode, int n, int degMax, bool hasFrozen, int dStar, ref bool
warnedLatch, out int d, out bool calibrate, out bool warn)
// --- (namespace)
/* 208 */ public sealed class DvpFit
// --- DvpFit
/* 210 */ public const int MaxDeg = DvpConst.MaxDegree;
/* 214 */ public readonly double[] A = new double[MaxDeg + 1], B = new double[MaxDeg + 1], N = new double[Max
Deg + 1], C = new double[MaxDeg + 1];
/* 215 */ public readonly double[] Sse = new double[MaxDeg + 1], Sigma = new double[MaxDeg + 1], R2 = new dou
ble[MaxDeg + 1], Bic = new double[MaxDeg + 1];
/* 217 */ public bool Valid { get; private set; }
/* 218 */ public int n { get; private set; }
/* 219 */ public int DTop { get; private set; }
/* 220 */ public int MaxValidDeg { get; private set; }
/* 221 */ public int Deg { get; private set; }
/* 222 */ public bool Calibrating { get; private set; }
/* 223 */ public double YBar { get; private set; }
/* 224 */ public double Ctr { get; private set; }
/* 225 */ public double Scl { get; private set; }
/* 226 */ public double SumW { get; private set; }
/* 227 */ public double Swy2 { get; private set; }
/* 228 */ public bool UsedFallback { get; private set; }
/* 229 */ public bool WeightsFellBack { get; private set; }
/* 230 */ public int FallbackCount { get; private set; }
/* 232 */ public DvpFit(int capacity)
/* 240 */ public int Capacity { get; }
/* 243 */ public bool Fit(double[] y, double[] v, int n, int degMax, bool weightVol)
/* 251 */ public bool FitRaw(double[] y, double[] v, int n, int dTop, bool weightVol)
/* 329 */ public void Choose(int d, bool calibrate)
/* 338 */ public double U(double i)
/* 340 */ public double UAhead(int h)
/* 341 */ public double UT { get; }
/* 344 */ public double Eval(double u, int d, out double lev, out double dYdu)
/* 360 */ public double Yhat(double u, int d)
/* 361 */ public double Yhat(double u)
/* 362 */ public double Leverage(double u, int d)
/* 363 */ public double Leverage(double u)
/* 364 */ public double SigmaAt(int d)
/* 365 */ public double SigmaD { get; }
/* 368 */ public double SlopeEnd(int d)
/* 371 */ public void Cone(int hAhead, double coneMult, EDvpConeMode mode, out double lo, out double hi)
/* 380 */ public double Resid(int i, int d)
/* 383 */ public double CovIn(int d, double k)
/* 395 */ public double CovFwd(double[] yFwd, int H, int d, double k, bool withLeverage)
/* 411 */ public void ResidualStats(int d, out double dw, out double skew)
// --- (namespace)
/* 437 */ public struct DvpNode
// --- DvpNode
/* 439 */ public int row; public double z, prom; public int side;
// --- (namespace)
/* 443 */ public struct DvpRail
// --- DvpRail
/* 445 */ public ELevelKind kind; public double z, price;
// --- (namespace)
/* 448 */ public sealed class DvpManifold
// --- DvpManifold
/* 457 */ public readonly DvpNode[] Hvn, Lvn;
/* 458 */ public int HvnCount { get; private set; }
/* 459 */ public int LvnCount { get; private set; }
/* 461 */ public bool Valid { get; private set; }
/* 462 */ public int RowsEff { get; private set; }
/* 463 */ public double Span { get; private set; }
/* 464 */ public double Wz { get; private set; }
/* 465 */ public double Sigma { get; private set; }
/* 466 */ public double TailVol { get; private set; }
/* 467 */ public double TotalVol { get; private set; }
/* 468 */ public double UpTail { get; private set; }
/* 469 */ public double DownTail { get; private set; }
/* 470 */ public int Poc { get; private set; }
/* 471 */ public int VaLo { get; private set; }
/* 472 */ public int VaHi { get; private set; }
/* 473 */ public double PocZ { get; private set; }
```

```
/* 474 */ public double VaLoZ { get; private set; }
/* 475 */ public double VaHiZ { get; private set; }
/* 476 */ public double Entropy { get; private set; }
/* 477 */ public double Concentration { get; private set; }
/* 478 */ public double ConcentrationOcc { get; private set; }
/* 479 */ public double InSampleCov2 { get; private set; }
/* 481 */ public DvpManifold(int manRows, bool splitFlow)
/* 501 */ public int ManRows { get; }
/* 503 */ public double[] Rows { get; }
/* 504 */ public double[] UpRows { get; }
/* 505 */ public double[] DownRows { get; }
/* 507 */ public double RowZ(int r)
/* 517 */ public static int RowsEffFor(int manRows, double span, double sigma, double tick)
/* 533 */ public static double Deposit(double[] rows, int R, double span, double wz, double loZ, double hiZ,
double v)
/* 565 */ public bool Build(DvpFit fit, double[] high, double[] low, double[] vol, double[] up, double[]
down, double manSpan, double tick, double vaFraction, double nodeProm, int maxNodes)
/* 606 */ public static void Shape(double[] rows, int poc, out double concentration, out double concentration
Occ)
/* 623 */ public void SelectNodes(double[] rows, int poc, double nodeProm, int maxNodes, double span)
// --- (namespace)
/* 680 */ public sealed class DvpRails
// --- DvpRails
/* 682 */ public readonly DvpRail[] Items;
/* 683 */ public int Count { get; private set; }
/* 684 */ public double YhatT { get; private set; }
/* 685 */ public double Sigma { get; private set; }
/* 686 */ public int Deg { get; private set; }
/* 688 */ public DvpRails(int maxSigmaRails)
/* 690 */ public bool Build(DvpFit fit, DvpManifold man, double[] sigmaList)
/* 696 */ public bool Build(DvpFit fit, DvpManifold man, double[] sigmaList, double uT)
/* 725 */ public double Price(double z)
/* 727 */ public double Z(double price)
/* 730 */ public double RailPrice(ELevelKind kind, double z)
/* 739 */ public int NextRail(double fromZ, int dir)
/* 747 */ public static string Tag(int ds, DateTime tPrev, int deg, ELevelKind kind, double z)
/* 758 */ public void Publish(List<LevelRef> into, List<LevelRef> pool, int ds, DateTime tPrev, DateTime t0,
bool isRollWindow)
// --- (namespace)
/* 780 */ public sealed class DvpSnapshot
// --- DvpSnapshot
/* 782 */ public int ds, endBar, anchorBar, sinceRefit, n, deg, dCap, manRowsEff;
/* 783 */ public DateTime tPrev, t0;
/* 784 */ public bool valid, crossesRoll, lowR2, refitBar, calibrating, manValid, diagnosticOnly;
/* 785 */ public double atr, sigma, sigmaAtr, r2, dw, skew, yhatEnd, slopeEnd, slope1, slope1SigmaPerBar, end
pointDrift, uT, yhatT,
/* 788 */ internal void Clear(int ds, int endBar, DateTime tPrev)
// --- (namespace)
/* 799 */ public sealed class DvpCoef
// --- DvpCoef
/* 801 */ public readonly double[] A = new double[DvpFit.MaxDeg + 1], B = new double[DvpFit.MaxDeg + 1], N =
new double[DvpFit.MaxDeg + 1], C = new double[DvpFit.MaxDeg + 1];
/* 802 */ public double yBar, sigma; public int n, deg;
/* 803 */ public long startPos; public int seen, hits, hitsFlat;
/* 805 */ public void From(DvpFit f, long startPos)
/* 813 */ public double Eval(double u, out double lev)
// --- (namespace)
/* 832 */ public sealed class DvpStream
// --- DvpStream
/* 834 */ sealed class Slot
// --- DvpStream.Slot
/* 836 */ public readonly DvpFit fit; public readonly DvpManifold man; public readonly DvpRails rails;
public readonly DvpSnapshot snap = new DvpSnapshot();
/* 837 */ public int key = int.MinValue, fitAnchor = int.MinValue;
/* 838 */ public bool fitOk;
/* 839 */ public double dw, skew, rthShare;
/* 840 */ public Slot(DvpConfig c)
// --- DvpStream
/* 843 */ public readonly int Ds;
/* 844 */ public readonly DvpConfig Config;
/* 845 */ public readonly string Root;
/* 846 */ public readonly bool RollDetect, RthKnown;
/* 860 */ public Action<string> warnOnce;
/* 863 */ public DateTime SessionDay { get; private set; }
/* 864 */ public bool SessionHalfDay { get; private set; }
/* 865 */ public bool NewSession { get; private set; }
/* 866 */ public int Sessions { get; private set; }
```

```
/* 867 */ public bool LastHalfDay { get; private set; }
/* 870 */ public long BarsSeen { get; }
/* 871 */ public long Eligible { get; private set; }
/* 872 */ public long Skipped { get; private set; }
/* 873 */ public long FitsComputed { get; private set; }
/* 874 */ public long FitLines { get; private set; }
/* 875 */ public long SkipLines { get; private set; }
/* 878 */ public DvpStream(int ds, DvpConfig cfg, string root, double tick, bool rollDetect, bool rthKnown)
/* 903 */ public readonly DvpLattice Lattice; public readonly DvpPivots[] Pivots;
/* 905 */ public const int PivotCapacity = 128;
/* 909 */ public DvpLattice LatticeFor(_BarRing ring, int endAgo)
/* 919 */ public static int RingNeed(DvpConfig c)
/* 921 */ public bool DiagnosticOnly { get; }
/* 922 */ public double SkipShare { get; }
/* 923 */ public double MeanCovIn { get; }
/* 924 */ public double MeanCovFwd { get; }
/* 925 */ public double MeanCovFwdFlat { get; }
/* 926 */ public int FallbackCount { get; }
/* 937 */ public bool OnBar(_BarRing ring, bool halfDay, Action<string> journal, string runId, string configH
ash)
/* 977 */ public DvpSnapshot Snapshot(_BarRing ring, int endAgo)
/* 989 */ public readonly DvpRailHistory History; public readonly DvpConfluence Confluence; public readonly D
vpRailJournal RailJournal;
/* 996 */ public DvpSnapshot HistSnapshot(_BarRing ring, int endAgo, out DvpRails rails)
/* 1008 */ public DvpRails RailsFor(DvpSnapshot snap)
/* 1016 */ public DvpFit FitFor(DvpSnapshot snap)
/* 1127 */ public long FwdDropped { get; private set; }
/* 1165 */ public static string FitLine(DvpSnapshot s, DvpFit f, string runId, string configHash)
// --- (namespace)
/* 1222 */ internal sealed class _DvpEngine
// --- _DvpEngine
/* 1224 */ public readonly DvpConfig Config;
/* 1225 */ public readonly string RunId, ConfigHash;
/* 1226 */ public readonly bool RollDetect;
/* 1230 */ public Action<string> journal, warnOnce;
/* 1233 */ public _DvpEngine(DvpConfig cfg, string runId, string root, double tick, bool rollDetect, bool rth
Known, Action<string> journal, Action<string> warnOnce, string paLabels = null)
/* 1244 */ public int StreamCount { get; }
/* 1249 */ public bool Tradeable(int ds)
/* 1260 */ public DvpStream Stream(int ds)
/* 1262 */ public static ulong Fnv1a64(string s)
/* 1290 */ public bool OnBar(int ds, _BarRing ring, bool halfDay)
/* 1297 */ public DvpSnapshot Snapshot(int ds, _BarRing ring, int endAgo)
/* 1305 */ public DvpSnapshot SnapshotRaw(int ds, _BarRing ring, int endAgo)
/* 1311 */ public DvpLattice LatticeOf(int ds, _BarRing ring, int endAgo)
/* 1317 */ public DvpRails RailsOf(int ds, _BarRing ring, int endAgo)
/* 1325 */ public DvpRailHistory HistoryOf(int ds, _BarRing ring, ELevelKind kind, double z)
/* 1334 */ public DvpConfluence ConfluenceOf(int ds, DvpSnapshot snap, DvpRails rails, DvpLattice lat,
double cPrev)
/* 1344 */ public void Finish()
// --- (namespace)
/* 1361 */ public sealed class DvpPivots
// --- DvpPivots
/* 1363 */ public readonly double[] Price; public readonly int[] ExtremeBar, ConfirmBar; public readonly
bool[] IsHigh;
/* 1364 */ public int Count;
/* 1365 */ public DvpPivots(int capacity)
/* 1366 */ public int Capacity { get; }
/* 1367 */ public void Clear()
/* 1368 */ public bool Add(bool isHigh, double price, int extremeBar, int confirmBar)
// --- (namespace)
/* 1385 */ public sealed class DvpLattice
// --- DvpLattice
/* 1388 */ public static double FibLevel(int i)
/* 1399 */ public const int NFib = 24;
/* 1401 */ public readonly int Ranges;
/* 1402 */ public readonly double[] Price, F; public readonly int[] Range;
/* 1403 */ public int Count;
/* 1404 */ public readonly double[] AnchorA, AnchorB; public readonly int[] AnchorABar, AnchorBBar; public re
adonly bool[] RangeOk;
/* 1407 */ public DvpLattice(int ranges)
/* 1414 */ public static int Strength(DvpConfig c, int j)
/* 1417 */ public static bool Anchor(DvpPivots p, int endBar, int look, out double a, out double b, out int a
Bar, out int bBar)
/* 1441 */ public void Build(DvpConfig c, DvpPivots[] pivots, int endBar, double atr, double close)
/* 1477 */ public static string Tag(int ds, DateTime tPrev, int deg, double f, int range)
/* 1485 */ public void Publish(List<LevelRef> into, List<LevelRef> pool, int ds, DateTime tPrev, DateTime
```

```
t0, int deg, bool isRollWindow)
// --- (namespace)
/* 1513 */ public sealed class DvpConfluence
// --- DvpConfluence
/* 1515 */ public const int FamManifold = 0, FamSigma = 1, FamLattice = 2;
/* 1516 */ public readonly double[] CandPrice; public readonly int[] CandFam, CandRange, CandCluster; public
readonly ELevelKind[] CandKind;
/* 1517 */ public int CandCount;
/* 1519 */ public int RailCount { get; private set; }
/* 1520 */ public readonly double[] ClusterPrice, ClusterScore077; public readonly int[] ClusterFamilies, Clu
sterRanges, ClusterSize, ClusterSeed;
/* 1521 */ public int ClusterCount;
/* 1522 */ public double Atr { get; private set; }
/* 1523 */ public double Tol { get; private set; }
/* 1526 */ public DvpConfluence(int capacity)
/* 1535 */ public static int Family(ELevelKind k)
/* 1543 */ public void Clear()
/* 1546 */ public bool Add(ELevelKind kind, double price, int range)
/* 1556 */ public void Load(DvpRails rails, DvpLattice lat)
/* 1570 */ public void Cluster(double atr, double cPrev, double confTol)
/* 1573 */ public void Cluster(double atr, double cPrev, double confTol, double wVa, double wPoc)
/* 1612 */ public bool Qualifies(int cl)
/* 1613 */ public int QualifyingCount { get; }
/* 1616 */ public int CandFamilies(int i)
/* 1617 */ public int CandRanges(int i)
/* 1618 */ public double CandScore077(int i)
/* 1622 */ public void FamiliesAt(double price, double tolAtr, out int families, out int nRanges)
/* 1638 */ public static string Tag(int ds, DateTime tPrev, int deg, double z)
/* 1642 */ public void Publish(List<LevelRef> into, List<LevelRef> pool, int ds, DateTime tPrev, DateTime
t0, DvpRails rails, bool isRollWindow)
// --- (namespace)
/* 1665 */ public static class DvpAccept
// --- DvpAccept
/* 1667 */ public const double Delta = 0.10;
/* 1668 */ public const int MaxN = 6;
/* 1673 */ public static bool Beyond(double[] c, double[] p, double[] a, int len, int k, int dir, double delt
aAtr)
/* 1679 */ public static bool Triggered(double[] c, double[] p, double[] a, int len, int k, int dir)
/* 1686 */ public static int RunBeyond(double[] c, double[] p, double[] a, int len, int dir, double deltaAtr)
/* 1695 */ public static bool BackInside(double[] c, double[] p, double[] a, int len, int dir)
/* 1711 */ public static bool Accepted(double[] c, double[] p, double[] a, int len, int dir, int N, bool coun
tTrigger, double deltaAtr)
// --- (namespace)
/* 1728 */ public sealed class DvpRailHistory
// --- DvpRailHistory
/* 1730 */ public const int Depth = 8;
/* 1733 */ public readonly double[] C = new double[Depth], P = new double[Depth], A = new double[Depth];
/* 1734 */ public int Len;
/* 1735 */ public long Copied { get; private set; }
/* 1737 */ public DvpRailHistory(DvpStream s, int maxRails)
/* 1745 */ public void Reset()
/* 1750 */ public int Entry(_BarRing ring, int k)
/* 1775 */ public const double NodeZTol = 0.25;
/* 1776 */ public static int Match(DvpRail[] it, int n, ELevelKind kind, double z)
/* 1793 */ public double PriceIn(int slot, ELevelKind kind, double z)
/* 1801 */ public int Load(_BarRing ring, ELevelKind kind, double z)
/* 1817 */ public double ZClose(_BarRing ring, int k)
// --- (namespace)
/* 1825 */ public struct DvpRailEvent
// --- DvpRailEvent
/* 1827 */ public ELevelKind kind; public double z, price, close, atr, distAtr, score077, sigmaAtr, mfe, mae;
/* 1828 */ public int dir, families, nRanges, heldBars, deg, elapsed;
/* 1829 */ public bool trigger, closedThrough, acc2, acc3, acc5, isRollWindow, halfDay, active, accepted41, b
ack41;
/* 1830 */ public DateTime time, tPrev;
// --- (namespace)
/* 1842 */ public sealed class DvpRailJournal
// --- DvpRailJournal
/* 1844 */ public const int MaxTracked = 512;
/* 1846 */ public readonly DvpRailEvent[] Events;
/* 1847 */ public int EventCount;
/* 1848 */ public long Written { get; private set; }
/* 1849 */ public long Dropped { get; private set; }
/* 1852 */ public DvpRailJournal(int maxRails)
/* 1857 */ public static string Line(DvpRailEvent e, int ds, string runId, bool complete)
/* 1876 */ public int Step(DvpStream s, _BarRing ring, DvpSnapshot snap, DvpRails rails, DvpConfluence conf,
bool isRollWindow, bool halfDay, Action<string> journal, string runId)
```

```
/* 1935 */ public void Finish(int ds, Action<string> journal, string runId)
// --- (namespace)
/* 1947 */ public static class DvpA233
// --- DvpA233
/* 1950 */ public static string Line(string runId, DateTime day, bool halfDay, int rails, int conf, int rollS
kipped, int fibSkipped)
/* 1957 */ public static bool IsG0dType(ELevelKind k)
/* 1966 */ public static int Sample(IReadOnlyList<LevelRef> src, Func<string, double, int, double, string, bo
ol> add, ref int rollSkipped, ref int fibSkipped)
// --- (namespace)
/* 1983 */ public static class DvpHtf
// --- DvpHtf
/* 1989 */ public static int EndAgo(bool barZeroClosed, int ringCount, int ringNewestBar, int currentBar)
/* 1995 */ public static bool ShouldStep(bool barZeroClosed, int lastSteppedBar, int currentBar)
/* 1999 */ public static string RootError(int ds, string data1Name, string data1Root, string dsName, string d
sRoot)
/* 2010 */ public static double NextRailDistAtr(DvpRails rails, DvpLattice lat, double c, int dir, double
atr)
// --- (namespace)
/* 2033 */ internal static class DvpPaBridge
// --- DvpPaBridge
/* 2035 */ public static void Fill(_PriceActionEngine pa, int barOffset, int endBar, DvpPivots into)
```

_SignalJournal.Function.CS

```
// --- (namespace)
/* 13 */ internal sealed class _SignalJournal
// --- _SignalJournal
/* 16 */ public const string PROBE_ID = "CS-0";
/* 19 */ public static string Describe()
```

_VpVwapEngine.Function.CS

```
// --- (namespace)
/* 22 */ internal sealed class _VpVwapEngine
// --- _VpVwapEngine
/* 24 */ public _VolumeProfileBase parent;
/* 25 */ public EVwapType type;
/* 26 */ public int length;
/* 27 */ public DateTime fromDate;
/* 28 */ public int datastream;
/* 29 */ public string label;
/* 31 */ public double value;
/* 32 */ public double sigma;
/* 33 */ public bool valid;
/* 56 */ public void Reset()
/* 75 */ public void Step(IInstrument bars)
```

_VpConfluenceEngine.Function.CS

```
// --- (namespace)
/* 24 */ public interface IVpConfAnchor
/* 26 */ public interface IVpConfSurvivor
/* 28 */ public static class VpConfluenceRules
// --- VpConfluenceRules
/* 30 */ public const double SplitTolMultiple = 4.0;
/* 31 */ public const int MinSplitCount = 3;
/* 32 */ public const double DefaultSplitAtrFraction = 0.03;
/* 35 */ public static bool AtrReady(double atr20d)
/* 38 */ public static double SplitWidth(double tol, double atr20d, double atrFraction)
/* 47 */ public static int SplitAt(IList<double> prices, int lo, int hi, double tol, double atr20d, double at
rFraction)
/* 62 */ public static int DistinctCount(IEnumerable<string> ids)
/* 71 */ public static void Partition(IList<double> prices, int lo, int hi, double tol, double atr20d,
double atrFraction, List<int[]> zones, List<int[]> splits)
/* 82 */ public static string SourceId(string sourceVolumeId, string profileId)
/* 89 */ public static void ZoneSourceCounts(IReadOnlyList<IVpConfSurvivor> survivors, out int independentSou
rces, out int profileCount)
/* 106 */ public static int EmitZones(IList<double> prices, int lo, int hi, double tol, Func<double> hostAtr2
```

```
0d, double atrFraction, Action<int, int> emitZone, Action<int, int, int, double> onSplit)
/* 121 */ public static double DevelopingHits<THit>(IReadOnlyList<IVpConfAnchor> cluster, int lo, int hi, IRe
adOnlyList<IVpConfSurvivor> survivors, Func<IVpConfAnchor, IVpConfSurvivor, double, THit> makeHit, ref List<T
Hit> hits)
/* 143 */ public static bool NeedsAtr20d(bool usesConfluenceZones, double atrFraction, int ds1, int ds2, int
ds3)
/* 149 */ public static double HostAtr20d(bool needsDaily, Func<double> dailyAtr)
/* 155 */ public static void ValidateStreams(int ds1, int ds2, int ds3)
/* 162 */ public static string DuplicateStreamError(int ds1, int ds2, int ds3)
// --- (namespace)
/* 189 */ internal sealed class _VpConfluenceEngine
// --- _VpConfluenceEngine
/* 249 */ public _VpConfluenceEngine(_VolumeProfileBase owner)
/* 254 */ public void Rebuild()
```

_VpHistogram.Function.CS

```
// --- (namespace)
/* 15 */ internal static class _VpHistogram
// --- _VpHistogram
/* 17 */ internal static void clearHistogram(VpExtra extra)
/* 27 */ internal static void readLevels(IProfile vp, List<LevelVol> dest)
/* 50 */ internal static List<Row> rowsFromLevels(List<LevelVol> levels)
/* 64 */ internal static List<Row> denseRowsFromLevels(List<LevelVol> levels, double tick, out bool[] traded)
/* 95 */ internal static double[] mergeSorted(double[] a, int count, List<LevelVol> levels, int kind, int n)
/* 110 */ internal static void stampLevelPercentiles(VpExtra extra, double[] streamTotal, double[]
streamAsk, double[] streamBid, int count)
/* 134 */ internal static double rankPercentile(double[] sortedAscending, int count, double x)
/* 149 */ internal static Node makeNode(List<Row> rows, double[] vols, int a, int b, int peak, double
pocVol, bool isPoc)
/* 163 */ internal static bool isLocalMin(double[] v, int i)
/* 170 */ internal static bool hasHvnAdjacent(List<Node> hvns, double edge, bool lookingDown, double maxGap)
/* 188 */ internal static double[] smooth(double[] src, int window)
/* 206 */ internal static double percentile(double[] src, double pct)
/* 227 */ internal static double priceForBar(IProfile vp, int chartBar, int kind)
/* 232 */ internal static double priceForBarNative(IProfile vp, int bar, int kind)
/* 246 */ internal static double firstPositive(params double[] xs)
/* 254 */ internal static double volumeAtPrice(VpExtra extra, double price)
/* 273 */ internal static double levelPrice(ILevel lvl)
/* 283 */ internal static double vaExtreme(IProfile vp, bool high)
```

_Holidays.Function.CS

```
// --- (namespace)
/* 20 */ public abstract class _Holidays : FunctionSimple<int>
// --- _Holidays
/* 26 */ protected enum WeekOfMonth { First = 1, Second = 2, Third = 3, Fourth = 4, Last = 5, }
/* 39 */ protected List<DateTime> m_holidays;
/* 42 */ public _Holidays(CStudyControl _master)
/* 43 */ public _Holidays(CStudyControl _master, int _ds)
/* 45 */ public bool isHoliday(DateTime dateTime)
/* 51 */ protected override void StartCalc()
/* 56 */ protected override int CalcBar()
/* 77 */ protected abstract void refreshHolidays(int year);
/* 87 */ protected DateTime getNthDayOfNthWeek(DateTime date, DayOfWeek dayOfWeek, WeekOfMonth weekOfMonth)
```

E2. Feature × example coverage matrix

Direct use = the example's own code references the feature's API (comments excluded). Inherited = used through the host class of a module example.

Feature	Direct use (examples)	Inherited
F-DEV	21, 22, 26, 27, 28, 29, 30, 31, 32, 34	35
F-PROF	6, 11, 21, 22, 26, 27, 28, 30, 32, 41	35
F-COMP	21, 26, 27, 28, 29, 31, 34	—

Feature	Direct use (examples)	Inherited
F-NPOC	6, 41	—
F-HVN	29, 33, 34	—
F-SESS	19, 20, 22, 25, 26, 27, 28, 29, 30, 31, 32, 36, 37, 38, 39	35, 40
F-ROLL	1, 3, 4, 5, 6, 7, 8, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 22, 23, 25, 26, 27, 28, 29, 30, 31, 32, 33, 34, 36, 37, 38, 39, 41, 42	21, 24, 35, 40
F-CAL	1, 2, 3, 4, 5, 6, 7, 8, 10, 11, 12, 13, 14, 15, 17, 18, 19, 20, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 36, 37, 38, 39, 41	21, 35, 40
F-OF	38	—
F-VWAP	3, 14, 20	—
F-CONF	—	—
F-TM	12, 13, 17, 18, 20, 22, 23, 25, 26, 27, 28, 29, 30, 31, 32, 36, 37, 38, 40	24, 35
F-JRNL	1, 2, 3, 5, 6, 7, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 22, 23, 25, 26, 27, 28, 29, 30, 31, 32, 33, 34, 36, 37, 38	21, 24, 35, 40
F-EVT	1, 2, 3, 5, 6, 7, 10, 11, 12, 13, 14, 15, 16, 20, 24, 29, 32	21, 35
M-AVWAP	16	—
M-SWING	19, 26, 28, 31, 34	—
M-MTF	2, 5, 6, 7, 10, 11, 13, 14, 36	—
M-TOD	4, 19, 25	—
M-DAILY	9, 10, 17, 20, 27, 30, 31, 32, 33	35
M-EVENT	2, 4, 24	—
M-TICK	37	40
M-XMKT	39	—
M-PORT	37, 40	—
M-COST	—	—
M-GMM	8, 27, 35, 36	—
M-GEX	—	—
VP helpers	7, 8, 10, 24, 33, 34, 35, 36	—
A2-33	41	—
Gate0	—	—
_RegimeEngine	9	—
M-DVP	42	—

Features with no example: F-CONF, M-COST, M-GEX, Gate0. Each still has a snippet in §5 (except M-GEX, which has no code).

F. Open VERIFY items and discrepancies

Still VERIFY (could not be settled from the code; most need MultiCharts itself):

1. **TraceLevel override.** The default `None` is set in the `_CustomSignal` constructor (PR #49). Whether MC applies the Format Signal dialog value of `TraceLevel` after a value set in code in the constructor.
2. **Example trading day.** `_Test6` is settled (PR #46: Globex key via `_CmeSessionClock.GlobexTradingDayEt`). The M-DAILY examples 9, 10, 17, 20, 27, 30, 31, 32, 33 (and 35 via 32) that derive from a plain `_CustomSignal` still do not override `tradingDayKey` (and leave `TradingDayRoll` at Calendar). On a Globex template their M-DAILY days are calendar days, not Globex days. Decide if that is intended.

3. **EventBlackout timing.** `EventBlackout` is computed on the bar close for `[open, close)`. Whether an order sent on that bar (filled next bar) should also check the next bar's window.
4. **M-TICK reflection.** The DataLoader member names that `_TickStream` reaches by reflection (they only resolve inside MC).
5. **HTF bar-end stamping (MC Probe P2).** Examples 2, 5, 6, 7, 10, 11, 13, 14 and 36 (all M-MTF users, E2) assume HTF and 1m bars are stamped at their end. Run Probe P2 first, as the catalogue says.
6. **G0-a cost model.** The A2-40 cost model used by Gate 0 (including the 2-tick slippage for 09:30–09:35 fills) is VERIFY against the catalogue's A2-40/A2-59 text (§2.5).
7. **Example 10 zone break.** After the touch and before the 60m trigger, one RTH 1m close below the pullback zone kills the week's setup. That is a very strict coding choice; the catalogue is silent.
8. **MC API stubs.** The examples compile against stubs I wrote for the MC API. A member MC names differently only shows in the PowerLanguage editor.
9. **v4 examples (11–42).** Same caveat as item 8: compiled against my stubs only. PLACEHOLDER values (e.g. example 38's minimum imbalance volume, standing in for the catalogue's P70) must be calibrated.
10. **Newest-first order of `RecentCompletedProfiles`.** Example 41 ages naked POCs by their index in that list; the order is assumed newest first.

Discrepancies found (code vs catalogue vs v1 manual):

1. **M-DAILY at rolls — partly RESOLVED by PR #46.** v1 said "roll gaps are removed" from the ATR. On `main`, a roll reseeds the ATR by default: `Atr20d` is NaN for `atrLength` sessions after each roll (only when RollDetection is on). PR #46 added `DailyConfig.atrRestartOnRoll = false`: the ATR carries on and only the splice day's true range is skipped, which matches the catalogue's A2-63 "Atr20d, roll-gap removed"; `_Test6` uses it with `Table` detection. Examples that keep RollDetection Off (e.g. example 9) still let the gap enter the ATR.
2. **PS-13's 200-day ATR.** The catalogue's M-DAILY `atrLength = 200` never becomes valid with RollDetection on and the default `atrRestartOnRoll = true`, because @ES rolls about every 63 sessions. Example 7 computes its own ATR200d instead; `atrRestartOnRoll = false` (PR #46) would now also make the M-DAILY value usable (example 7 not changed).
3. **DV-03 follow-through** (≥ 1 box width before the retest) is not coded in example 7 (the PS-13 version).
4. **A2-29 gating.** The G0-d table lists the A2-25…A2-32 range as G0-d-gated, but A2-29's own entry says "Gate (v2): none (classifier)".
5. **`_Test1` vs `_TestGate0`.** The catalogue (§3.0: "A2-33 and A2-39 run in `_Test1`"; also the Batch-1 slot table) names `_Test1` as the host, and so does the `LevelStudyConfig` comment in `_CustomSignal` ("e.g. `_Test1`"). `_LevelStudy` itself doesn't name a host. `_Test1` is a plain `SignalObject`; Gate 0 is hosted by `_TestGate0`.
6. **KD-02 canary.** The catalogue's G0-b canary list includes KD-02; `EGate0Canary` does not have it.
7. **Cost per round trip (G0-a conflict) — RESOLVED by PR #30.** The `_TestGate0` header's MC settings are now $2.25 commission and 0.5 tick slippage ($6.25), each per side, so an ES round trip costs $4.50 + $12.50 = **$17.00**, matching the catalogue's cost floor (§2.1) assumption of **$16.98** (IBKR round trip + 1 tick in total). At $17.00 the 14-tick floor ($175) keeps costs at about 0.097 R, within ≤ 0.10 R. Earlier canary runs at the old setting were over-costed.
8. **Example file names — RESOLVED in v4.** Example numbers are now the file numbers; the §3 table gives the reading order.
9. **Cost floor: max or skip — RESOLVED** (decided by Ettienne 2026-10-23: widen, then skip only on a budget breach; size is never shrunk). Catalogue §2.1 said both "every idea's stop is max(structural stop, cost floor)" and "trades whose structural stop sits under the floor are logged as skipped rather than resized". Ruling: the stop distance is max(structural, floor), so a short structural stop is widened to the 14-tick floor; "skipped rather than resized" is about position size: if the widened stop breaches the per-trade risk budget, the trade is skipped rather than taken at a smaller size. v3.1 changed examples 5, 6, 7 and 10 (current file numbers) to widen-then-skip-on-budget-breach; all v4 stop-based examples follow the same rule; the budget is `riskBudgetFraction`

= 0.01, 1% of equity (decided by Ettienne 2026-10-23; PF-04's 0.5% cap (§3.11) is not the default for the floor rule). The catalogue §2.1 text is reworded to match.

10. **AV-08 at rolls (example 3).** The catalogue says multi-day anchors reset at rolls; the Week/Month VWAP slots don't (`_VpVwapEngine` resets them only at a new week or month). Example 3 turns the layer off until the next clean period (`wtdClean`/`mtdClean`). Re-anchoring at `rollOn(1)` is the alternative.
11. **A2-57 at rolls (example 5).** The catalogue is silent on the 60m EMA at a roll. Example 5 re-seeds it at `rollOn(1)` (3 × 13 closed 60m bars of warm-up); ignoring turns for that long without re-seeding is the alternative.
12. **TS-01 market (example 2).** The catalogue tests MES on 30m/60m bars (kill $2.47). Example 2 is written for @ES on 1m.
13. **Example 10 zone break.** See VERIFY item 7: one 1m close below the zone after the touch cancels the week, which is stricter than the catalogue's text.
14. **M-DVP row.** The catalogue's §2.4 says M-DVP is "not yet coded". Main a07257a has spec rev 5.3 PR-D0a–D7: the fit core, manifold and rails, Data1 host wiring with the journal and cov_fwd (PR #36), the Fib lattice with the PA bridge (PR #37), D5 confluence and acceptance (PRs #39, #40: `dvpConfluence`, `dvpFamiliesAt`, `dvpTriggered`/`dvpRunBeyond`/`dvpBackInside`/`dvpAccepted`, `DvpConf` rails, `DVP RAIL`), and D6 HTF streams (PR #41: same-root Data2/3, `dvpTradeable` false for ds ≥ 2 until Probe P2, `dvpHtfAccepted`, `dvpHtfNextRailDistAtr`), and D7 the A2-33 hook (PR #42: `dvpLevelStudyStep`, `DvpA233`). `fibAnchor` other than `MostRecentPivot` is rejected. **Still pending:** D8 the canary/Calibrate study; D9 the frozen d*; Probe P2; `rollMode`. Example 42 uses the D3/D4 host API only and remains a diagnostic until D8/D9.
15. **A2-33 row.** Kaplan–Meier is on main; Cox regression is not.
16. **Example 36 histogram.** A2-27 asks for the mixture on the session profile; the example builds the histogram from the Data1 5m ring (`SessionBars`), a coarser input.
17. **Example 40 vs A2-78.** The catalogue's kill switch is book-level; the strategy-level `DailyLossStop*` rules count one strategy's closed trades. The book-level rule is `_PortfolioRiskManager` / `PortMmConfig`.
18. **Example 39 vs PF-03.** PF-03 measures sleeve-P&L correlation; the example journals the bar-return correlation of the two instruments as a proxy.

Known issues (open on main a07257a):

1. **RollDetection.Series over-detects on intraday charts (open; fix in progress).** On 1-minute @ES 2016–2024, `Series` flags 72 rolls against about 33 real quarterly rolls. The extras are on weekend opens around roll week. The fix ("RollDetection.Series: key by Globex trading day, one splice per contract") is in PR #51, not yet merged. Until it is merged, use `RollDetection = Table` with `RollDates = EventRules.RollDate(y, 3/6/9/12)` on intraday charts (as `_Test6` does), and treat `Series`-based `rollOn`, Daily `OnRoll` and AVWAP/PA roll resets as unreliable. See §1.7 and §4.13.

G. EV-4.6b — event-table coverage (PRs #28/#29)

What changed. Before EV-4.6b, any coverage gap in the event table disabled the whole engine. Now:

- The engine is **disabled only** when the compiled table is missing or its hash differs from `EventRules.ExpectedTableSha` (`"228cdee716a6eafa"` on a07257a, unchanged since 628542f). It writes an Error line ending `; engine disabled`, does not throw, and every query answers neutrally (no events, no blackout).
- Known holes in the table (`EventRules.CoverageHoles()`, part of the hash) become a third state per kind and day: `EEventCoverage { NoEvent = 0, Event = 1, Unknown = 2 }`, read with `Coverage(kind, day)`.

Using it.

```
// in CalcBar, before acting on an event filter
_EventCalendar ev = EventCalendar;
if (ev == null || ev.Disabled) { /* fail closed: a disabled engine says "no event" */ return; }
EEventKind hole;
```

```
if (ev.CoverageBlocks(day, EEventClassMask.Macro, EEventTier.T1, out hole)) { /* unknown coverage: skip the day */ return; }
```

- `CoverageBlocks(day, mask, minTier, out kind)` returns true if some kind in `mask` is Unknown on that day and its (effective) tier passes `minTier`. It returns false when the engine is disabled or `EventConfig.allowPartialCoverage` is true. `CoverageBlocks(day, mask)` uses `EEventTier.None`.
- **PR #29:** an Unknown kind whose tier is below the query's `minTier` never blocks, and the tier test uses the **effective** tier (after `EventConfig.tierOverride`).
- `CoverageReport(from, to, mask[, minTier], out flagged)` reports, per kind, the share of Unknown-blocked trading days in `[from, to]`. `CoverageStats()` returns one `COVSHARE run {Kind} blocked {n}/{days} {p}%` line per kind, with `FLAG` above 5%.

Journal. Once at start: `COVGAP {Kind} {from}..{to} Unknown …`. Then, per affected trading day: `COVGAP day {yyyy-MM-dd} Unknown Kind(blocks|belowMinTier|NoEvent)`.

Where the manual uses it. Example 2 (TS-01) skips Unknown T1 macro days and fails closed when the engine is disabled. Example 4 (A2-50) tags Unknown days in its journal instead of calling them report or non-report days. The catalogue's TS-01 caveat about GDP/PCE holes in 2016–24 is the case this handles.

H. Changelog

v4 (2026-10-24), source `origin/main` @ a07257a (drafted on 628542f; references moved to b2685dd in round 5, to 800bce7 in round 6 and to a07257a in round 7).

- §3: 32 new examples (11–42); all 42 ordered by difficulty; example numbers now equal file numbers (v3.2 example 6 → 9, 7 → 6, 9 → 7). Examples 1–10 keep their v3.2 text; their code blocks are the current files. Example 32 gained a virtual `pocGate` hook (used by example 35); example 37 calls `portfolioArm`/`portfolioStop`.
- §2.6: gating notes for the new examples (from the catalogue's Gate lines only).
- §5 (new): feature reference for all 31 rows of catalogue §2.4, with compiled snippets.
- Appendix E regenerated from 628542f; E2 coverage matrix added; Appendix F items 9–10 and 14–18 added, item 8 resolved.
- Rebased on `origin/main` 628542f (fetched 2026-10-24 14:49 EDT; the first-parent merges since 7be55bf are #36 and #37 only). §5.31 M-DVP was rewritten for the host API, the Fib lattice and the PA bridge, with a new `_Sn31` snippet. Example 42 now uses the host API (`DvpConfig`/`dvpSnapshot`/`dvpZ`/`dvpRailPrice`/`dvpRails`) in place of a hand-driven `DvpFit`. The A2-33 status, the §2.6 example 21 note and Appendix F item 14 were updated. Appendix E and E2 were regenerated.
- Analyst 2 review (main still 628542f, re-fetched 2026-10-24 ~15:00 EDT): `fibAnchor = ExtremeInLookback` documented as accepted silently (falls back to `MostRecentPivot`; rejection with D5/#39); the stop floor marked as Ettienne's rule implemented per strategy (library `CostConfig` pending, IP-3.2/IP-8.1) with the 1%-sizing consequence; Gate 0 slippage corrected to 0.5 tick ($6.25) per side; example 2 gained the floor and risk check, example 3 the risk check, examples 32 and 33 now widen the stop to the floor instead of skipping (target unchanged); §5.31 pending list (D5–D9, `Accepted`/countTrigger, no 240m sweep, rollMode None, `minBreakoutLayer = 0`); §1.7 stop-example list, §3 house rules, §4.8, §5.13 (`equityAvailable` percent overload, floor note), §5.16 (Data2), §5.24 (0.097 R arithmetic), lifecycle wording, App F VERIFY #2/#5 and #9 updated; App F items 14–18 moved back under Discrepancies.
- §4 rework (Ettienne's feedback; checked against `origin/main` b2685dd, merge of PR #41, 2026-10-24 21:03 EDT): every §4 chapter now opens with **Why it exists** and **When to use it** (with example numbers), API tables give a plain meaning per entry, and each snippet has a lead-in saying what it does and what the result means. Snippets are compiled classes `_S4_01`–`_S4_14`, split by lifecycle with `// Create`, `// StartCalc (before/after base.StartCalc)`, `// CalcBar` and `// class fields` labels, chained one-liners split with inline comments. §4.14 rewritten concept-first (rings → FVG → `LevelRef`). Fixes: §4.15 base note (31 of 42 examples run on `_VolumeProfileFuturesBase`; 11 on `_CustomSignal`), OnRoll at `_CustomSignal` L958, TOD bucket at L860. §5: all 30 snippets relabelled by lifecycle and split the same way, each feature gained a **Why it exists** line and a

snippet lead-in. `fibAnchor` statements updated (rejected on b2685dd); §5.31 pending list and App F #14 note that D5/D6 merged. Other line numbers in §5 and App E remain those of 628542f.

- Round 5 (main still b2685dd, re-fetched 2026-10-24 ~21:55 EDT): every file:line reference moved from 628542f to b2685dd. That is 90 §5 signature comments, 408 Appendix E line numbers (Appendix E regenerated: +82 D5/D6 entries), and 12 prose references (§5.31 Where, `minBreakoutLayer` L625, `requireDataStream` L742/L2293). §5.31 was re-verified against D5 (#39, #40) and D6 (#41): the status, Where, signatures (+21) and pitfalls now cover confluence, acceptance semantics, HTF streams, `dvpTradeable` and the P2 gate, `dvpHtfAccepted`, `dvpHtfNextRailDistAtr`, and the `fibAnchor` rejection. D7–D9, Probe P2 and `rollMode` stay pending. The §5 status lines and the compile-status notes now name b2685dd. The quoted file header and the code of example 42 are unchanged (listing byte-match).
- Round 6 (main 800bce7, merge of PR #42 / M-DVP D7): Analyst 2's Round 4/5 review applied after checking each item against the code. B1 `ValueMigrationScore(1, 3)` needs 4 completed sessions (20 without a daily ATR); B2 the §4.1 snippet now sizes at 1% with the 14-tick floor, skip-on-breach and no resize, and `riskPerTrade 0.005` is marked as the library default. Also: config changes after `base.StartCalc()` are unsupported, with `requireRing` as the after-StartCalc exception; ring capacity is the sum of requests; 10 + 1 `_CustomSignal` examples (24 via 18); the `entryAllowed` list no longer includes 39; order flow in 38 directly and in 34 via `BarLadders` (VPFB L133); `_ProfileStats` KDE/bootstrap/mixture members added; the KDE verdict rules in §5.25. §5.31 now reflects rev 5.3 with D7 on main, and `diagnosticOnly` is a by-design pitfall. References moved to 800bce7: 78 §4–§5 signature comments, 170 Appendix E line numbers (+4 `DvpA233` entries) and 25 prose references.
- Round 7 (main a07257a, merges of PRs #43–#49): AV_OPEN retired (PR #44; §2.2 row now a retirement note, App E `AllTypes` now lists 36 types) and the Holm family is m = 36; ATR restart at a roll splice (PR #43) and `DailyConfig.atrRestartOnRoll` (PR #46; §4.3 paragraph and snippet, §5.19 pitfall, App F discrepancy #1 partly resolved); Data1 setup check `Gate0Data1.Check` in StartCalc (PR #45); bounded order-flow window `FlowCfg.MaxBars`, `DroppedBars`/`BarCap` and truncated cumulative delta (PR #48, §5.9); confluence zone width `ConfluenceSplitAtrFraction`, `UsesConfluenceZones` and the developing-profile score (PR #47, §5.11); trace lines via `AppendAllText` with per-signal trace-log path and `TraceLevel` default None set in the constructor (PR #49; §1.5, §4.1, App B); `_Test6` uses a roll Table and overrides `tradingDayKey` (§1.6, §1.7, §4.13, App F VERIFY #2). The §4.1 snippet now calls `portfolioStop(stop)` after the widened stop passes the budget check. References moved to a07257a: 125 §4–§5 signature comments, Appendix E regenerated, and 30 prose line references.
- Round 7b (main a07257a): Analyst 2's Round 7 review. E1 the `TraceLevel` default is set in the constructor (PR #49), not `Create()` (§1 listing, §1 Input note, App F VERIFY #1). E2 `portfolioStop` only sets the stop behind `pm.riskOpen`; `pm.R$`/`pm.riskNew` stay structural and `pm.R$w` is draft-only, so pending risk is understated (§4.1 prose and snippet, §4.16). E3 `Flow.DroppedBars` is a whole-run total (§5.9); the two stop-floor notes now cite a07257a. E4 new App F Known issues: `RollDetection.Series` over-detects on intraday charts (72 vs ~33 rolls, 1-minute @ES 2016–2024), fix pending in PR #51; use `Table` meanwhile (cross-referenced from §1.7 and §4.13).
- Nothing in this manual reports trading results.

v3.2 and earlier: see `engines-manual-v3.2.md`.